

The Backend Engineer's Library

Reliability, Observability, and SRE

Volume 11

Contents

SLIs, SLOs, and Error Budgets

Metrics and the Golden Signals

Logging at Scale

Distributed Tracing and OpenTelemetry

Incident Response and On-Call

Blameless Postmortems

Load Testing and Capacity Planning

Chaos Engineering

Deployment Strategies: Blue/Green, Canary, and Feature Flags

Resilience Patterns in Production

Chapter 1 — SLIs, SLOs, and Error Budgets

What this chapter covers. Reliability without a definition is a wish. Service Level Indicators (SLIs), Service Level Objectives (SLOs), and Error Budgets turn "the service should be reliable" into a measurable, enforceable contract between a service and its consumers — and between engineering and product. This chapter builds the full framework from first principles: how to choose SLIs that reflect user experience, how to set SLOs that balance reliability and velocity, and how to use error budgets to make shipping decisions objectively rather than emotionally. Every concept is grounded in production configuration — OpenSLO / Sloth definitions, Prometheus recording and alerting rules, and multi-window burn-rate alerts — so you can apply it directly.

Learning goals — after this chapter you should be able to:

- Define SLIs precisely for availability, latency, throughput, and correctness, and explain why server-side metrics must approximate the user experience, not the server's convenience.
- Choose SLO targets using the "happy users" method and justify them with data rather than round numbers.
- Compute error budgets, reason about budget consumption, and connect budget state to release policy.
- Write SLOs as code (OpenSLO, Sloth, Pyrra) and wire them to Prometheus recording rules and multi-window, multi-burn-rate alerts.
- Explain the distributed-systems consequences: how per-service SLOs compose (or fail to compose) into user-journey SLOs, and how dependency SLOs propagate.

Why SLOs exist

Every service has two competing pressures: ship features fast and keep the service reliable. Without a shared, quantitative definition of "reliable enough," these pressures resolve through politics — the loudest voice wins, release freezes are declared after outages and quietly lifted under

deadline pressure, and reliability work is perpetually deprioritized until the next incident.

SLOs break this cycle by making reliability a budgeted resource, like CPU or money.

The vocabulary, inherited from Google's SRE tradition, is precise:

Term	Definition	Example
SLI — Service Level Indicator	A measured ratio or percentile over a window: <code>good_events / valid_events</code>	"Proportion of HTTP requests that returned 2xx within 300 ms over the last 28 days"
SLO — Service Level Objective	A target value for an SLI over a rolling window	"99.9% of valid requests succeed within 300 ms over any 28-day window"
SLA — Service Level Agreement	A contractual commitment with consequences (credits, penalties)	"99.9% monthly uptime or 10% service credit" — negotiated with customers
Error budget	$1 - \text{SLO}$ — the allowed fraction of bad events	For a 99.9% SLO, the error budget is 0.1% of valid events

The critical distinction: the **SLI is the measurement**, the **SLO is the target**, the **SLA is the contract**, and the **error budget is the consequence**. Teams confuse these constantly. An SLI without an SLO is a dashboard. An SLO without an error budget policy is a poster. An SLA without an SLO underneath it is a promise you cannot verify.

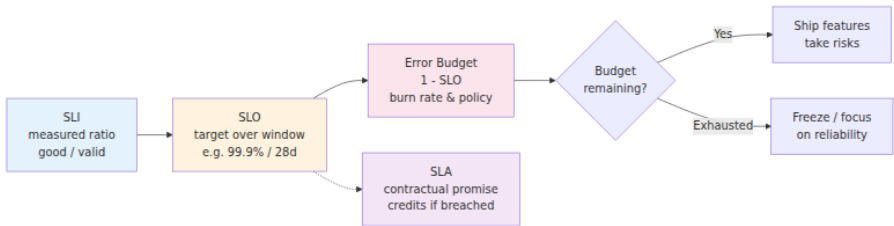


Figure 1-1: The SLO chain. SLIs are measured, SLOs set the target, error budgets drive decisions, and SLAs are the outward contract built on top.

Boundary note. This chapter defines reliability targets and how to enforce them. How those targets are *observed* — metrics pipelines, Prometheus internals, log aggregation, distributed tracing — is covered in Chapters 2-4. How budget exhaustion translates into *operational* response — incident management, on-call, and postmortems — is covered in Chapters 5-6. Deployment safety mechanisms that consume error budgets are covered in Chapter 9.

SLIs: measuring what matters to users

The cardinal rule

An SLI must measure the user's experience, not the server's internal state. CPU utilization, queue depth, and replica count are useful signals for capacity planning and alerting, but they are not SLIs — a service can have 40% CPU and still serve 100% errors, or 95% CPU and serve every request correctly. The SLI answers: "of the valid interactions a user attempted, how many were good?"

The canonical form is a ratio:

$$\text{SLI} = (\text{good events}) / (\text{valid events}) \quad \text{over window } W$$

Both numerator and denominator must be defined precisely. Vagueness here produces SLIs that look green while users suffer.

Choosing the right SLI type

SLI type	Good event definition	When to use	Pitfall
Availability	Request returned a non-error (2xx/3xx, or application-level success)	Every user-facing service	Counting 4xx as failures punishes the service for client bugs; exclude or separate them

SLI type	Good event definition	When to use	Pitfall
Latency	Request completed within threshold T	Latency-sensitive paths (search, checkout, feed)	Averages hide tail suffering; always use percentiles
Throughput / freshness	Data was processed within freshness bound	Pipelines, replication, search indexing	"Throughput" as raw QPS is not an SLI; freshness or lag is
Correctness	Response was semantically correct	Where wrong answers are worse than errors (payments, recommendations)	Hard to measure automatically; often requires probing or auditing
Durability	Written data survives failures	Storage systems	Measured over long windows; loss events are rare but catastrophic

Most services need **two SLIs**: an availability SLI and a latency SLI. Throughput and correctness SLIs are added where the domain demands them. More than four SLIs per service is usually a sign that the service boundary is too broad.

Availability: what counts as "valid" and "good"

The denominator — valid events — is where most SLI bugs hide. Consider an HTTP API:

- Should health-check probes count? No — they inflate the denominator and mask real errors.
- Should requests that never reached the service (client timeout, DNS failure) count? Only if you can measure them — often you cannot, so be explicit about the measurement point.
- Should 4xx responses count as failures? Generally no for the availability SLI. A 400 Bad Request or 404 Not Found is the service

behaving correctly given invalid input. But `429 Too Many Requests` is ambiguous — if your rate limiter is too aggressive, users experience it as unavailability. Many teams track `4xx`-excluding-`429` as valid-but-not-failures and alert on `429` rate separately.

A practical HTTP availability SLI:

```
valid_events = count of requests where response code is not synthetic
              (exclude health checks, exclude canary probes if
              separately tagged)
good_events  = valid_events where status is 2xx or 3xx
              (or: status < 500, depending on whether you count 4xx
              as good)
SLI          = good_events / valid_events
```

For gRPC, the mapping uses status codes: `OK` is good, `INVALID_ARGUMENT` / `NOT_FOUND` / `UNAUTHENTICATED` are valid-but-not-failures, `UNAVAILABLE` / `DEADLINE_EXCEEDED` / `INTERNAL` are failures.

Latency: percentiles, not averages

Latency SLIs must use percentiles. The mean is dominated by the common case and hides the experience of the slowest users — who are often the most valuable (large accounts with more data, users on slow networks, requests that hit cold caches).

A typical latency SLI:

```
good_events = count of valid requests where duration <= threshold T
              (e.g., T = 300 ms for an API, T = 100 ms for an internal
              RPC)
SLI         = good_events / valid_events
```

Many teams define two latency SLOs — a strict threshold for the median and a relaxed threshold for the tail:

- 90% of requests complete within 100 ms (p90 SLI).
- 99% of requests complete within 500 ms (p99 SLI).

The implementation matters: Prometheus histograms and summaries compute percentiles differently (see Chapter 2 for the full mechanics). For SLO purposes, prefer histograms with well-chosen buckets so the percentile is computed server-side from bucket counts, not estimated from a sampled summary.

Data freshness and correctness SLIs

For asynchronous systems — replication pipelines, search indexes, materialized views — the user-visible property is not request latency but **freshness**: how stale is the data the user reads?

```
freshness SLI = proportion of reads where data age < bound B
                (e.g., 99% of reads return data written within 5 seconds)
```

This is measured by injecting synthetic writes with known timestamps and measuring the time until they appear at the read path (a pattern sometimes called "canary documents" or "watermark probes").

Correctness SLIs are the hardest to automate. For a payment service, "the correct amount was charged" requires reconciliation against an expected value. For a search service, "results were relevant" requires human judgment or a proxy metric. Where direct correctness measurement is infeasible, teams use **prober-based SLIs**: synthetic transactions with known expected outcomes, run continuously, whose success rate is the SLI.

SLOs: setting the target

The SLO is a product decision, not an engineering preference

Setting an SLO at 99.9% because "three nines sounds right" is cargo culting. The correct method is to work backwards from user happiness and business constraints:

1. **Determine the user tolerance.** Through user research, support ticket analysis, or experimentation, find the reliability level below which users notice and complain, churn, or lose trust. For many consumer services this is around 99.5–99.9%. For infrastructure services consumed by other services, it may be 99.95–99.99%.
2. **Check what you actually achieve.** Measure the current SLI over the last 1–3 months. If you are already at 99.95% with no special effort, setting the SLO at 99.9% ratifies reality and gives you budget to move faster. Setting it at 99.99% when you have never exceeded 99.9% creates permanent budget deficit and alert fatigue.

3. **Consider the cost of each additional nine.** Each nine is roughly an order of magnitude more engineering investment. Moving from 99.9% to 99.99% typically requires multi-region failover, automated remediation, and significant architectural changes. The business must decide whether the revenue retained justifies the cost.
4. **Set the window.** The SLO window determines how quickly the budget recovers and how sensitive the SLO is to brief incidents. Common choices:

Window	Budget recovery	Sensitivity	Use when
7 days	Fast — one bad day clears in a week	High — a single incident can exhaust the budget	Fast-moving services that want tight feedback
28/30 days	Moderate — the industry default	Balanced	Most user-facing services
90 days	Slow — incidents linger for a quarter	Low — smooths over brief spikes	Infrastructure with infrequent but impactful failures

Google's practice is 28-day rolling windows. Many teams use 30 days for calendar simplicity. The window should be **rolling** (continuously evaluated), not calendar-aligned — a calendar month SLO that resets on the 1st creates perverse incentives to ship recklessly on the 2nd and freeze on the 29th.

How many nines do you actually need?

SLO	Allowed downtime/error	Typical consumer	Architectural implications
99% (two nines)	7.2 h/month, 3.65 d/year	Internal batch jobs, best-effort features	Single zone, manual remediation acceptable
99.9% (three nines)	43 m/month, 8.76 h/year	Most user-facing web services	Multi-zone, automated failover, on-call

SLO	Allowed downtime/error	Typical consumer	Architectural implications
99.95%	21 m/month, 4.38 h/year	Payment processing, critical APIs	Multi-zone, fast detection, runbooks tested
99.99% (four nines)	4.3 m/month, 52 m/year	Infrastructure primitives (DNS, load balancer)	Multi-region, automated remediation, chaos tested
99.999% (five nines)	26 s/month, 5.26 m/year	Telephony, medical devices	Active-active, formal verification territory

The key insight: **100% is the wrong SLO for almost everything**. It implies zero error budget, which means no deploys, no experiments, and no changes can ever be made without violating the objective. Worse, it forces engineers to optimize for reliability beyond the point where users benefit, at the expense of features users actually want. An SLO deliberately acknowledges that some unreliability is acceptable — and that the budget for it should be spent intentionally.

SLOs for internal and dependency services

A user-facing service depends on many internal services. Its availability cannot exceed the joint availability of its critical dependencies. If service A depends on B and C in series (both must succeed for A to succeed):

$$A_availability \leq B_availability * C_availability$$

If B is 99.9% and C is 99.9%, A cannot exceed 99.8% even if A itself never fails. This has two consequences:

- Internal services that sit on many critical paths need **tighter SLOs** than the user-facing services that depend on them. A common pattern is to set internal SLOs one nine tighter than the external SLO (external 99.9%, internal 99.95%).
- Dependency budgets must account for **fan-out**. A frontend that calls 10 microservices, each at 99.99%, still has a theoretical ceiling of 99.9% if any single dependency failure fails the request. Reducing critical-path fan-out, adding fallbacks, and making dependencies non-

blocking are reliability strategies that directly improve the achievable SLO.

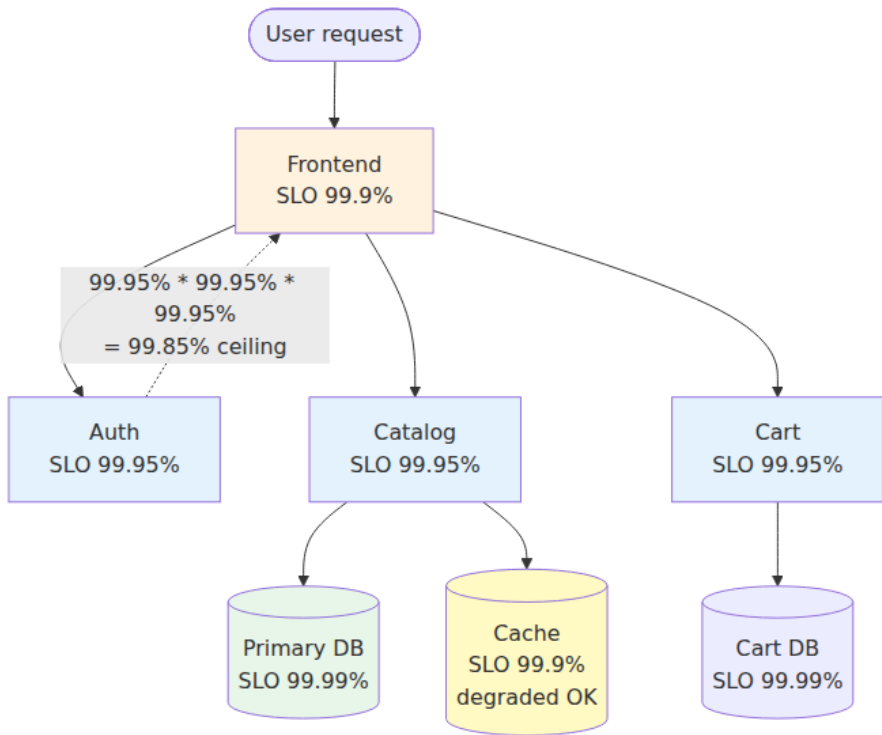


Figure 1-2: SLO composition. Internal dependencies need tighter SLOs than the frontend they support. A non-critical dependency like the cache can have a looser SLO if the service degrades gracefully without it.

Error budgets: the mechanism that makes SLOs actionable

Computing the budget

The error budget is the complement of the SLO:

```
error_budget = 1 - SLO_target
```

For a 99.9% SLO over 28 days with 10 million valid requests in the window:

```
budget = 0.001 * 10,000,000 = 10,000 bad requests allowed per 28 days
```

For time-based SLIs (common for availability measured in seconds):

```
budget_seconds = (1 - 0.999) * 28 * 24 * 3600 = 2,419 seconds (~40 minutes) of downtime per 28 days
```

Budget is consumed by bad events. When a 5-minute outage causes 2% of requests to fail, and the service handles ~250 requests/second:

```
consumed = 0.02 * 250 * 300 = 1,500 requests
budget_remaining = 10,000 - 1,500 = 8,500
burn_rate = consumed / elapsed_budget (are we burning faster than the SLO allows?)
```

Burn rate

Burn rate compares the current error rate to the budgeted error rate:

```
burn_rate = (current_error_rate) / (budgeted_error_rate)
           = (current_error_rate) / (1 - SLO)
```

- `burn_rate = 1` — burning exactly at the rate the SLO allows. You will exhaust the budget precisely at the end of the window if this continues.
- `burn_rate = 10` — burning 10x faster than allowed. At this rate the 28-day budget is consumed in 2.8 days.
- `burn_rate < 1` — under budget, accumulating headroom.

Burn rate is the correct quantity to alert on, not raw error rate, because it normalizes across SLO tightness. A 1% error rate is catastrophic for a 99.99% SLO (burn rate 100x) but tolerable for a 99% SLO (burn rate 1x).

Error budget policy

An SLO without a policy is unenforceable. The policy connects budget state to concrete actions:

Budget state	Policy
> 50% remaining	Normal operations. Ship features, run experiments, accept risk.
25-50% remaining	Caution. Require additional review for risky changes. Prioritize reliability work in the next sprint.
0-25% remaining	Restrict deploys to fixes and low-risk changes. Reliability work takes priority over features.
Exhausted ($\leq$ 0%)	Freeze non-essential releases. All effort goes to reliability until the window rolls forward and budget recovers.

The freeze is not a punishment — it is the mechanism that makes the SLO credible. If the team ships through budget exhaustion without consequence, the SLO was never real. Conversely, if the budget is consistently under-consumed (e.g., 80% remaining every window), the SLO is too loose — tighten it or acknowledge that the service is more reliable than required and reallocate effort.

Practical nuance. Most organizations implement budget policy as a **conversation**, not an automated gate. A hard deploy freeze triggered by budget exhaustion can block a security fix or a reliability improvement. The policy should require explicit, documented approval to ship while over budget — with the approver accepting the risk — rather than an unbreakable lock.

Budget visibility

Every team should have a dashboard that shows, at a glance:

- Current SLI value over the SLO window (are we above or below the target?).
- Budget remaining (absolute count and percentage).
- Budget burn rate over the last 1 hour and 6 hours (are we currently burning fast?).
- Projected exhaustion time at the current burn rate.

This is not optional instrumentation — without it, error budgets are numbers in a document that nobody checks until after an incident.

SLOs as code: from YAML to Prometheus

SLOs must be version-controlled, reviewed, and applied declaratively — just like any other infrastructure. Three widely used approaches are OpenSLO, Sloth, and Pyrra. All three generate Prometheus recording and alerting rules from a higher-level SLO definition.

OpenSLO

OpenSLO is a vendor-neutral specification (v2alpha1 as of 2024) for declaring SLOs in YAML. It separates the SLI definition, the SLO target, and the alerting configuration:

```

# openslo.yaml - OpenSLO v2alpha1 definition
apiVersion: openslo/v1alpha
kind: SLO
metadata:
  name: http-availability
  displayName: HTTP Availability
spec:
  description: 99.9% of valid HTTP requests return non-5xx over 28 days
  service: web-api
  budgetingMethod: Occurrences # vs. Timeslices
  objectives:
    - displayName: availability-28d
      target: 0.999
      op: gte
      timeWindow:
        duration: 28d
        isRolling: true
      indicator:
        metadata:
          name: http-availability-sli
        spec:
          ratioMetric:
            counter: true
            good:
              source: prometheus
              queryType: promql
              query: |
                sum(rate(http_requests_total{service="web-
api",code=~"2..|3.."}[5m]))
            total:
              source: prometheus
              queryType: promql
              query: |
                sum(rate(http_requests_total{service="web-api",code!
~"5..",job!~"healthcheck"}[5m]))
          alertPolicies:
            - name: availability-burn-rate
              alertName: HighErrorBudgetBurn
              alertDesc: "Burn rate {{ .burnRate }}x - budget will exhaust in {
{ .timeToExhaustion }}"

```

For latency, the SLI uses histogram buckets — the good events are those within the threshold:

```
# Latency SLI fragment – 90% of requests under 300 ms
indicator:
  spec:
    ratioMetric:
      counter: true
    good:
      source: prometheus
      queryType: promql
      query: |
        sum(rate(http_request_duration_seconds_bucket{
          service="web-api",le="0.3"}[5m]))
    total:
      source: prometheus
      queryType: promql
      query: |
        sum(rate(http_request_duration_seconds_count{
          service="web-api"}[5m]))
```

Sloth

Sloth generates Prometheus recording and alerting rules from a simpler SLO manifest. It is widely used where OpenSLO's generality is more than needed:


```

# sloth.yaml – Sloth SLO manifest
version: "prometheus/v1"
service: "web-api"
labels:
  team: platform
  tier: "1"
slos:
  - name: "http-availability"
    objective: 99.9
    description: "99.9% of valid HTTP requests succeed"
    sli:
      events:
        error_query: |
          sum(rate(http_requests_total{service="web-api",code=~"5.."}
[5m]))
        total_query: |
          sum(rate(http_requests_total{service="web-api",code!
~"healthcheck"}[5m]))
      alerting:
        name: WebApiAvailability
        labels:
          severity: critical
          team: platform
        annotations:
          summary: "High error budget burn for web-api availability"

  - name: "http-latency-p99"
    objective: 99
    description: "99% of requests complete within 300 ms"
    sli:
      events:
        # Latency errors are requests that exceed the threshold: total
- good.
        error_query: |
          sum(rate(http_request_duration_seconds_count{service="web-
api"}[5m]))
          -
          sum(rate(http_request_duration_seconds_bucket{service="web-
api",le="0.3"}[5m]))
        total_query: |
          sum(rate(http_request_duration_seconds_count{service="web-
api"}[5m]))
      alerting:
        name: WebApiLatency
        labels:
          severity: warning

  # Canonical raw form (equivalent to the events form above) – prefer
one or the other:
  - name: "http-latency-p99-raw"

```

```

objective: 99
description: "99% of requests under 300 ms (raw form – same SLI as
above)"
sl:
  raw:
    error_ratio_query: |
      (
        sum(rate(http_request_duration_seconds_count{service="web-
api"}[5m]))
        -
        sum(rate(http_request_duration_seconds_bucket{service="web-
api",le="0.3"}[5m]))
      )
      /
      sum(rate(http_request_duration_seconds_count{service="web-
api"}[5m]))
    alerting:
      name: WebApiLatencyRaw
      labels:
        severity: warning

```

Applied with:

```

sloth generate -i ./sloth.yaml -o ./prometheus-rules/
# Produces prometheus-rules/web-api.yml with recording + alerting rules

```

What Sloth / OpenSLO generate: Prometheus rules

Whether authored via OpenSLO, Sloth, or hand-written, the generated Prometheus rules follow a consistent pattern — recording rules that precompute SLI values and burn rates, and alerting rules that fire on multi-window burn conditions:

```

# prometheus-rules/web-api-slos.yml – generated recording + alerting
rules
groups:
- name: slos-web-api-availability
  interval: 30s
  rules:
    # --- Recording rules: precompute SLI components ---

    # Total valid request rate (denominator)
    - record: sli:http_requests:valid:rate5m
      expr: |
        sum(rate(http_requests_total{service="web-api",code!
~"healthcheck"}[5m]))

    # Good request rate (numerator) – non-5xx
    - record: sli:http_requests:good:rate5m
      expr: |
        sum(rate(http_requests_total{service="web-api",code=~"2..|
3..|4.."}[5m]))

    # Instantaneous error ratio
    - record: sli:http_availability:error_ratio:rate5m
      expr: |
        1 - (sli:http_requests:good:rate5m /
sli:http_requests:valid:rate5m)

# SLI over the full SLO window (28 days) – for dashboard + budget calc
    - record: sli:http_availability:sli_28d
      expr: |
        1 - (
          sum(increase(http_requests_total{service="web-
api",code=~"5.."}[28d]))
          /
          sum(increase(http_requests_total{service="web-api",code!
~"healthcheck"}[28d]))
        )

    # Budget remaining (0 = exhausted, 1 = full)
    - record: sli:http_availability:budget_remaining
      expr: |
        (sli:http_availability:sli_28d - 0.999) / (1 - 0.999)

    # Burn rates at two horizons – short (fast detection) and long
    (confirmation)
    - record: sli:http_availability:burn_rate_1h
      expr: |
        sli:http_availability:error_ratio:rate5m / (1 - 0.999)
    - record: sli:http_availability:burn_rate_6h
      expr: |

```

```

(
    sum(increase(http_requests_total{service="web-
api",code=~"5.."}[6h]))
    /
    sum(increase(http_requests_total{service="web-api",code!
~"healthcheck"}[6h]))
    ) / (1 - 0.999)

- name: slos-web-api-availability-alerts
  interval: 30s
  rules:
    # Multi-window, multi-burn-rate alerts (SRE Workbook pattern)
    # Page only when BOTH short and long windows confirm high burn.

    # Critical: burning at 14x – 28d budget gone in 2 days
    - alert: HighErrorBudgetBurnCritical
      expr: |
        sli:http_availability:burn_rate_1h > 14
        and
        sli:http_availability:burn_rate_6h > 14
      for: 5m
      labels:
        severity: critical
      annotations:
        summary: "Critical burn rate {{ $value | humanize }}x for web
-api availability"
        description: "At this rate the 28-day error budget is consume
d in ~2 days."

    # Warning: burning at 6x – budget gone in ~4.7 days
    - alert: HighErrorBudgetBurnWarning
      expr: |
        sli:http_availability:burn_rate_1h > 6
        and
        sli:http_availability:burn_rate_6h > 6
      for: 15m
      labels:
        severity: warning
      annotations:
        summary: "Elevated burn rate {{ $value | humanize }}x for web
-api availability"

    # Exhaustion warning: budget nearly gone regardless of burn rate
    - alert: ErrorBudgetExhausted
      expr: |
        sli:http_availability:budget_remaining < 0.1
      for: 10m
      labels:
        severity: warning
      annotations:
        summary: "Less than 10% error budget remaining for web-api"

```

Multi-window, multi-burn-rate alerting

The pattern above — alerting only when both a short window (1 hour) and a long window (6 hours) agree that burn rate is high — is the standard technique from the SRE Workbook (Chapter 5). It solves a fundamental tension:

- Alerting on a short window alone is **fast but noisy** — a brief spike pages someone even though the 28-day budget is barely affected.
- Alerting on a long window alone is **accurate but slow** — by the time the 6-hour average confirms the problem, significant budget has already been consumed.

Requiring both windows to exceed the threshold gives fast detection (the 1-hour window) with confirmation (the 6-hour window), dramatically reducing false positives while preserving responsiveness.

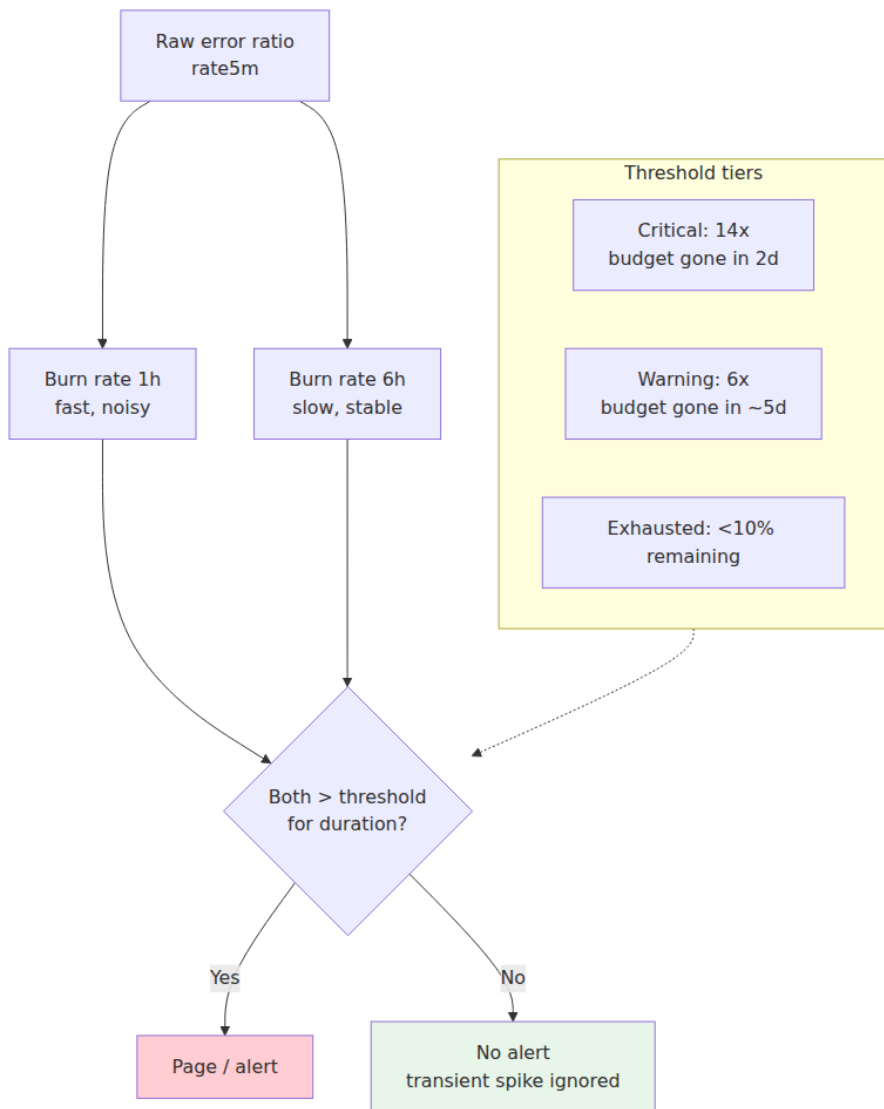


Figure 1-3: Multi-window burn-rate alerting. Both the short and long window must agree before paging, suppressing transient spikes without sacrificing detection speed.

Pyrra (Kubernetes-native alternative)

Pyrra manages SLOs as Kubernetes custom resources and auto-generates both Prometheus rules and Grafana dashboards:

```
# pyrra ServiceLevelObjective CR
apiVersion: pyrra.dev/v1alpha1
kind: ServiceLevelObjective
metadata:
  name: web-api-availability
  namespace: monitoring
  labels:
    prometheus: k8s
    role: alert-rules
spec:
  target: "99.9"
  window: 28d
  indicator:
    ratio:
      errors:
        metric: http_requests_total{code=~"5..",service="web-api"}
      total:
        metric: http_requests_total{service="web-api"}
  alerting:
    burnrates: true
```

```
kubectl apply -f slo-availability.yaml
# Pyrra operator generates PrometheusRule + Grafana dashboard
automatically
kubectl get prometheusrule -n monitoring
```

Choose based on your platform: OpenSLO for vendor-neutral portability, Sloth for Prometheus-centric simplicity, Pyrra for Kubernetes-native GitOps.

SLOs in a distributed system

User-journey SLOs vs. service SLOs

Service-level SLOs (one per microservice) are necessary but not sufficient. Users experience **journeys** — sequences of service calls that together accomplish a goal (search → view product → add to cart → checkout). A journey SLO is measured at the edge (the frontend or API gateway) and reflects the actual user experience across all dependencies.

```
journey SLI = good_journeys / valid_journeys
  where a journey is good only if every step succeeded within its
  latency bound
```

Journey SLOs are harder to instrument — they require either end-to-end synthetic probes or careful propagation of journey context through the call graph — but they are the SLOs that correlate most directly with user satisfaction and business metrics.

A common structure is two tiers:

- **Journey SLOs** (2–4 per product, e.g., "99.9% of checkout journeys succeed within 3 seconds") — owned by product teams, measured at the edge.
- **Service SLOs** (one or two per service) — owned by service teams, measured at the service boundary, with targets derived from the journey SLOs they contribute to.

Error budget distribution

When a user-facing journey has a 99.9% SLO (0.1% budget) and depends on 5 services in series, the budget must be **allocated** across those services. Naive equal allocation gives each service a 0.02% budget (99.98% SLO), which may be unachievable for some and wastefully loose for others.

A better approach allocates budget proportionally to each service's **contribution to journey failure** — services with higher historical error rates or greater architectural complexity receive more budget, while inherently reliable services (e.g., a cache that can be bypassed) receive less. This allocation is revisited periodically as architecture and reliability evolve.

For services behind fan-out (the frontend calls 10 services in parallel, and any failure fails the journey), the math is harsher:

```
P(journey succeeds) = P(all parallel calls succeed)  (if no fallbacks)
                    = product of individual availabilities
```

Adding fallbacks, graceful degradation, and making non-critical dependencies non-blocking are the primary tools for improving journey SLOs without requiring every dependency to achieve extreme reliability.

The distributed tracing connection

Accurate journey SLIs require knowing which service caused a journey to fail — and that requires distributed tracing (Chapter 4). Without traces, an SLO dashboard can tell you *that* the checkout journey is failing but not *which* service is responsible, forcing incident responders to check every dependency dashboard sequentially. With traces, the journey SLI can be broken down by contributing service, and error budget attribution becomes automatic.

Common failure modes

Too many SLOs. A service with 8 SLOs has 8 error budgets to track, 8 sets of alerts, and 8 policies to enforce. Nobody can reason about that. Two SLOs (availability + latency) per service is the right default. Add a third only when a distinct user-visible property (freshness, correctness, durability) demands it.

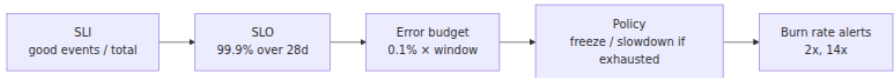
SLIs that measure the wrong thing. An SLI based on server-side success rate will look healthy even when users experience failures caused by client-side issues (DNS, CDN, TLS errors) or by load balancer rejections that never reach the server. Measure as close to the user as possible — at the edge load balancer or via Real User Monitoring (RUM) — and reconcile with server-side SLIs to identify the gap.

Alerting on SLOs without burn-rate context. Firing an alert every time the instantaneous error rate exceeds the SLO threshold creates noise during brief spikes that barely affect the 28-day budget. Always alert on burn rate with multi-window confirmation.

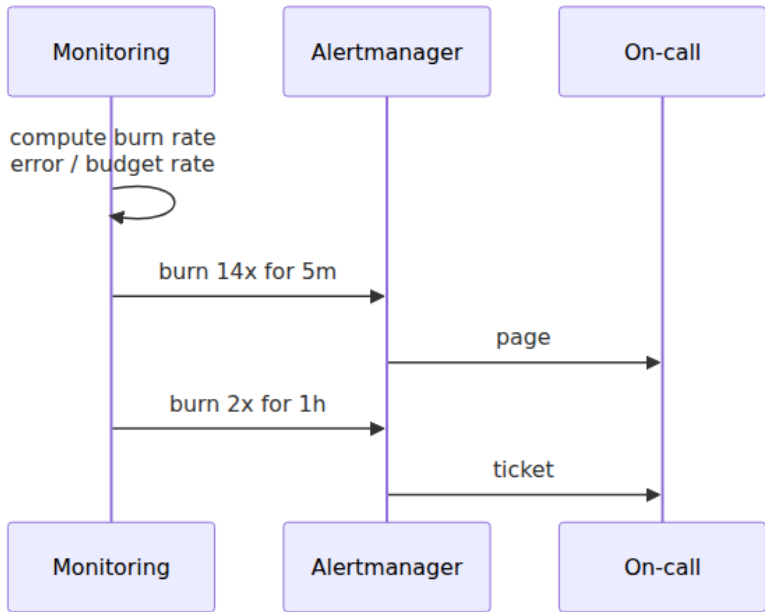
Treating the SLO as a target to hit rather than a lower bound. Teams that consistently achieve 99.99% against a 99.9% SLO sometimes relax their practices because "we have budget." The SLO is a floor, not a ceiling — consistently exceeding it is good, and the surplus budget is an opportunity to take well-judged risks, not to become complacent.

Setting SLOs before measuring. An SLO set without historical data is a guess. Measure the current SLI for at least one full window before setting the target. The initial SLO should usually be set at or slightly below the current measured value, then tightened as reliability improves.

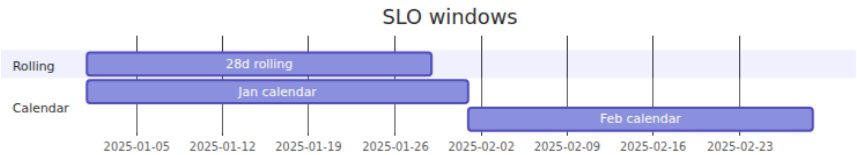
SLI to Error Budget



Burn Rate Alerting



SLO Window Types



Key takeaways

- An SLI measures the user's experience as `good_events / valid_events` over a window; an SLO is the target for that ratio; the

error budget ($1 - \text{SLO}$) is the allowed fraction of bad events; an SLA is the contractual promise built on top.

- Availability and latency (using percentiles, never averages) cover most services; freshness and correctness SLIs are added for pipelines and systems where wrong answers are worse than errors.
- SLO targets should be set from user tolerance and measured reality, not round numbers. 100% is almost never the right target — it implies zero budget for change.
- Error budgets make reliability actionable: they connect budget state to release policy, and burn rate ($\text{current_error_rate} / \text{budgeted_error_rate}$) is the correct quantity to alert on.
- Multi-window, multi-burn-rate alerting (requiring both a short and long window to agree) gives fast detection with low noise — the standard pattern from the SRE Workbook.
- SLOs should be declared as code (OpenSLO, Sloth, or Pyrra), version-controlled, and compiled into Prometheus recording and alerting rules.
- In a distributed architecture, internal services on critical paths need tighter SLOs than the user-facing journeys that depend on them; journey SLOs measured at the edge are the ground truth for user experience.

Further reading

- Google SRE Book, Chapter 4 — *Service Level Objectives* (<https://sre.google/sre-book/service-level-objectives/>) — the foundational treatment of SLIs, SLOs, and error budgets.
- Google SRE Workbook, Chapter 5 — *Alerting on SLOs* (<https://sre.google/workbook/alerting-on-slos/>) — the definitive guide to multi-window burn-rate alerting, with worked examples.
- OpenSLO Specification v2alpha1 (<https://openslo.cloud/>) — vendor-neutral SLO-as-code spec.
- Sloth — Easy SLO generation for Prometheus (<https://sloth.dev/>) — practical SLO-as-code for Prometheus-centric stacks.
- Pyrra — SLOs with Prometheus (<https://pyrra.dev/>) — Kubernetes-native SLO management.

- optimizer: SLO optimizer notebook (<https://github.com/metalmatze/slo-libsonnet>, <https://github.com/google/slo-burn>) — tools for analyzing historical data to choose SLO targets.
- Niall Murphy et al., *Site Reliability Engineering* (O'Reilly, 2016) and *The Site Reliability Workbook* (O'Reilly, 2018) — Chapters 4–6 of the SRE Book and Chapters 5–6 of the Workbook are the canonical references for this chapter's material.

Chapter 2 — Metrics and the Golden Signals

What this chapter covers. Metrics are the primary sensory system of any production backend — the numerical time series that tell you whether the system is healthy, where it is struggling, and how it behaved during the last incident. This chapter covers the full metrics pipeline from instrumentation to collection to storage to alerting, grounded in the Prometheus ecosystem that dominates cloud-native observability. You will learn the four metric types and when to use each, why histograms and summaries are not interchangeable, how the four Golden Signals (latency, traffic, errors, saturation) organize dashboards and alerts, and how to build recording rules, alerting rules, and PromQL queries that remain accurate at scale. Every abstraction is connected to running configuration — Prometheus scrape configs, recording and alerting rules, Grafana dashboards, and Pushgateway and federation patterns for distributed deployments.

Learning goals — after this chapter you should be able to:

- Distinguish counter, gauge, histogram, and summary metric types and choose correctly for any signal, explaining the monotonicity, aggregation, and cardinality implications of each.
- Explain the difference between histograms and summaries for latency measurement, including bucket selection, quantile computation, and why summaries cannot be aggregated across instances.
- Apply the four Golden Signals (and the complementary USE and RED methods) to design dashboards and alerts for any service.

- Write PromQL queries for SLI computation, burn-rate alerting, capacity forecasting, and anomaly detection, and explain range vectors, instant vectors, and subquery evaluation.
 - Configure Prometheus for production — scrape targets, service discovery, relabeling, federation, remote write, and high-availability — and reason about storage, retention, and cardinality costs.
 - Diagnose common metrics failure modes: cardinality explosion, stale series, counter resets, histogram bucket misconfiguration, and scrape aliasing.
-

Why metrics

A backend system produces three kinds of observability data: metrics (numbers over time), logs (discrete events), and traces (request-scoped causality). Metrics are the coarsest but most universally available signal. Every production system — from a single binary on a VM to a thousand microservices on Kubernetes — can expose a `/metrics` endpoint and be scraped. Metrics answer the first question in any incident — "is something wrong, and roughly where?" — before anyone opens a log viewer or trace waterfall.

The defining properties of metrics are:

- **Numerical and temporal.** Each data point is a float (or integer) at a timestamp. This makes metrics amenable to mathematical operations: rates, percentiles, aggregations, derivatives, forecasting.
- **Pre-aggregated and lossy.** Unlike logs and traces, which record individual events, metrics collapse many events into summary statistics (counts, sums, bucket distributions). You gain efficiency and lose detail — a counter tells you 1,000 requests failed but not which ones or why.
- **Cheap to store and fast to query.** A well-designed metrics pipeline retains months of data at second or minute granularity and answers queries in milliseconds, precisely because individual events have been aggregated away.

The trade-off is fundamental: metrics tell you *that* something is wrong and *how much*, but not *why* for a specific request. That "why" requires

logs (Chapter 3) and traces (Chapter 4). The three pillars are complementary, not competing.

Metric types: the four primitives

Prometheus defines four metric types. Choosing the wrong type produces metrics that look correct on a dashboard but give wrong answers when aggregated or alerted on. The type determines the wire format, the query semantics, and the aggregation behavior.

Counter

A counter is a monotonically increasing cumulative value — it only goes up (and resets to zero on process restart). It represents the total count or sum of events since the process started.

```
# HELP http_requests_total Total HTTP requests by status and method.
# TYPE http_requests_total counter
http_requests_total{method="GET",code="200"} 48213
http_requests_total{method="GET",code="500"} 127
http_requests_total{method="POST",code="200"} 8934
```

Use a counter for anything you want to count or sum: requests, errors, bytes transferred, tasks completed. The key property is that the **rate of increase** is the useful signal, not the absolute value. You query counters with `rate()` or `increase()`:

```
# Request rate per second over the last 5 minutes, by status code
sum by (code) (rate(http_requests_total[5m]))

# Error rate as a fraction of total traffic
sum(rate(http_requests_total{code=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m]))
```

Counter resets. When a process restarts, its counters reset to zero. `rate()` and `increase()` handle this automatically by detecting decreases and compensating — but only if the reset is visible within the range window. A range window shorter than the scrape interval can miss resets and produce incorrect rates. Always use a range window of at least `4 * scrape_interval`.

Gauge

A gauge is a value that can go up or down arbitrarily. It represents a point-in-time measurement.

```
# HELP process_resident_memory_bytes Resident memory size.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 134217728

# HELP http_requests_in_flight Current in-flight HTTP requests.
# TYPE http_requests_in_flight gauge
http_requests_in_flight{handler="/api/search"} 42

# HELP node_cpu_seconds_total Seconds of CPU time by mode (actually a
counter, despite the name)
```

Use a gauge for: current resource utilization (memory, CPU, disk), queue depth, number of in-flight requests, pool sizes, temperatures, feature-flag states. Gauges are queried directly — no `rate()` needed:

```
# Memory usage per instance
process_resident_memory_bytes / 1024 / 1024 # in MiB

# 95th percentile of in-flight requests over 10 minutes
quantile_over_time(0.95, http_requests_in_flight[10m])
```

The critical distinction: **a counter measures events, a gauge measures state**. If you can meaningfully ask "how many have happened so far?" it is a counter. If you ask "what is the current value?" it is a gauge. Confusing the two — for example, using a gauge to count requests by incrementing it — breaks `rate()` and makes the metric unaggregatable.

Histogram

A histogram samples observations (typically request durations or response sizes) and counts them into configurable buckets. It exposes multiple time series per metric: one counter per bucket boundary, plus `_sum` and `_count`.

```
# HELP http_request_duration_seconds HTTP request latency.
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{le="0.05"} 24054
http_request_duration_seconds_bucket{le="0.1"} 33444
http_request_duration_seconds_bucket{le="0.2"} 38034
http_request_duration_seconds_bucket{le="0.5"} 39204
http_request_duration_seconds_bucket{le="1.0"} 39830
http_request_duration_seconds_bucket{le="+Inf"} 40000
http_request_duration_seconds_sum 4123.53
http_request_duration_seconds_count 40000
```

Each `le` (less-than-or-equal) bucket is **cumulative** — `le="0.5"` counts all observations ≤ 500 ms, including those already counted in `le="0.05"`, `le="0.1"`, and `le="0.2"`. The `+Inf` bucket always equals `_count`.

Histograms are the correct type for measuring **distributions** — latency, request size, queue wait time — where you need to compute quantiles. The quantile is computed server-side via `histogram_quantile()`:

```
# p99 latency over the last 5 minutes
histogram_quantile(0.99,
  sum by (le) (rate(http_request_duration_seconds_bucket[5m]))
)

# p50 latency per handler
histogram_quantile(0.50,
  sum by (le, handler) (rate(http_request_duration_seconds_bucket[5m]))
)
```

Bucket selection is the most important histogram design decision.

Buckets that are too coarse lose resolution where it matters; buckets that are too fine create cardinality explosion. Choose buckets that are dense around your SLO thresholds and expected latency range:

```
// Go – histogram with buckets tuned for an API with a 300 ms SLO
var httpDuration = prometheus.NewHistogramVec(
  prometheus.HistogramOpts{
    Name:    "http_request_duration_seconds",
    Help:    "HTTP request latency by handler and method.",
    Buckets: []float64{0.005, 0.01, 0.025, 0.05, 0.1, 0.2, 0.3,
0.5, 1.0, 2.5, 5.0},
    //           ^^^^^ fine near SLO threshold (0.3s) ^^^^^
  },
  []string{"method", "handler", "code"},
)
```


For workloads with wide dynamic range (caches that serve in microseconds and fall back to seconds), consider **exponential buckets** or Prometheus native histograms (see below).

Summary

A summary also samples observations but computes quantiles **client-side** (inside the instrumented process) using a streaming quantile estimator. It exposes precomputed quantiles directly:

```
# HELP rpc_duration_seconds RPC latency summary.
# TYPE rpc_duration_seconds summary
rpc_duration_seconds{quantile="0.5"} 0.045
rpc_duration_seconds{quantile="0.9"} 0.112
rpc_duration_seconds{quantile="0.99"} 0.342
rpc_duration_seconds_sum 4123.53
rpc_duration_seconds_count 40000
```

Summaries have one advantage: no bucket configuration needed, and quantiles are computed with higher accuracy for a single instance. But they have a critical limitation that makes them unsuitable for most distributed systems:

Summaries cannot be aggregated across instances. The p99 of instance A and the p99 of instance B cannot be combined to produce the p99 of A+B. Averaging quantiles is mathematically incorrect, and no PromQL function can merge summary quantiles. If you run 10 replicas, you get 10 independent p99 values with no way to compute the fleet-wide p99.

Histograms, by contrast, **can** be aggregated — you sum the bucket counters across instances and then compute the quantile from the merged distribution. This is why **histograms are the default choice for latency in any multi-instance service**, and summaries should only be used for single-instance measurements or where client-side quantile accuracy is more important than aggregatability.

Property	Histogram	Summary
Quantile computed	Server-side (<code>histogram_quantile</code>)	Client-side (streaming estimator)

Property	Histogram	Summary
Aggregatable across instances	Yes — sum buckets, then quantile	No — quantiles cannot be merged
Configuration	Bucket boundaries (chosen by operator)	Quantile objectives + error tolerance
Accuracy	Depends on bucket density near the quantile	Higher for single-instance quantiles
Cardinality	Buckets × label combinations	Quantiles × label combinations
Recommendation	Default for distributed services	Single-instance or leader-only metrics

Native histograms (exponential histograms)

Traditional histograms require manually chosen buckets. Prometheus native histograms (stable since Prometheus 2.50+, OpenTelemetry exponential histograms) solve this by using an **exponential bucket scheme** that automatically adapts to the data distribution. They provide higher resolution with lower cardinality and more accurate quantile estimation, especially for wide-dynamic-range signals.

```
# prometheus.yml – enable native histograms
scrape_configs:
  - job_name: 'web-api'
    scrape_interval: 15s
    scrape_protocols: ["OpenMetricsText1.0.0", "OpenMetricsText0.0.4",
"PrometheusText0.0.4"]
    enable_native_histograms: true
```

```
// Go – native histogram (requires prometheus client >= 1.19)
var httpDurationNative = prometheus.NewHistogramVec(
    prometheus.HistogramOpts{
        Name: "http_request_duration_seconds",
        Help: "HTTP request latency (native histogram).",
        NativeHistogramBucketFactor: 1.1, // ~10% bucket width growth
        NativeHistogramMaxBuckets: 160,
    },
    []string{"method", "handler"},
)
```

Native histograms are the future for latency measurement but require compatible visualization (Grafana ≥ 10.4) and careful migration — you cannot mix classic and native histogram queries without handling both.

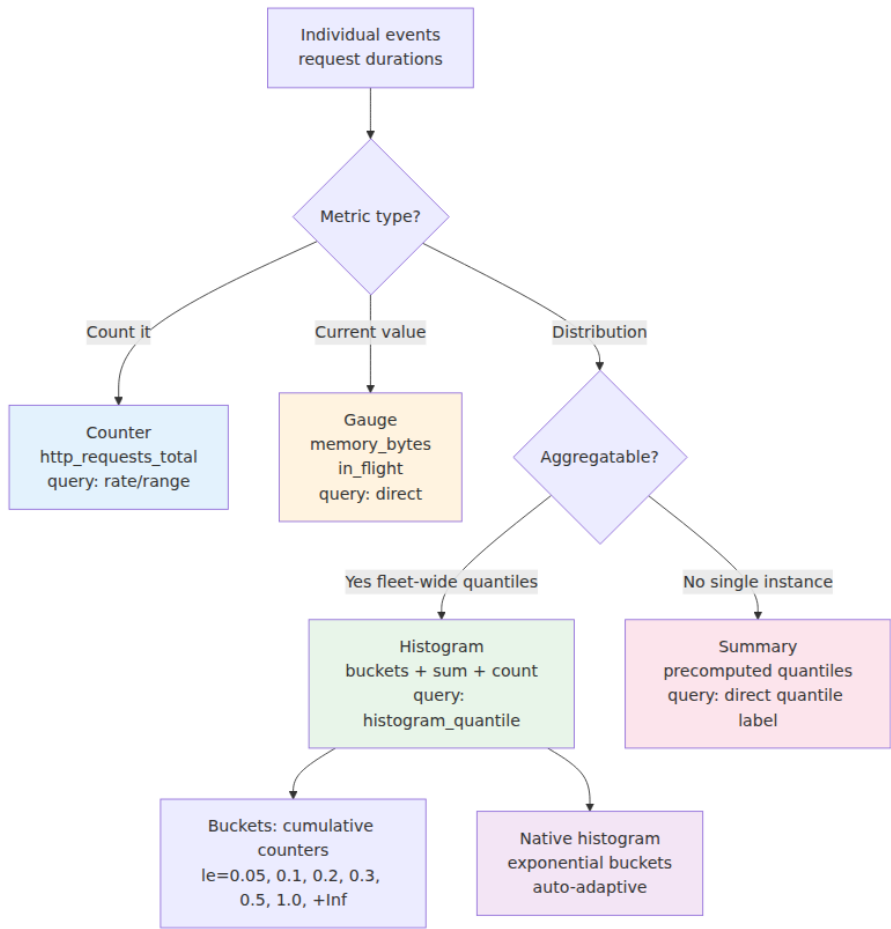


Figure 2-1: Choosing the metric type. Counters for events, gauges for state, histograms for distributions that need fleet-wide quantiles, summaries only when aggregation is not needed.

The Golden Signals

Google's SRE discipline organizes monitoring around four Golden Signals — the minimum set of metrics that, if healthy, mean the service is healthy, and if unhealthy, tell you where to look next. They are not the only metrics you need, but they are the first dashboard you open and the first alerts you configure.

The four signals

Signal	What it measures	Typical metric	Alert condition
Latency	Time to process a request, by percentile	Histogram: <code>http_request_duration_seconds</code>	p99 exceeds SLO threshold; or p50 degrades (indicates systemic slowdown)
Traffic	Demand on the service — request rate, data rate	Counter: <code>http_requests_total</code>	Sudden drop (upstream failure) or spike (abuse, retry storm)
Errors	Rate of failed requests	Counter: <code>http_requests_total{code=~"5.."} or application error counter</code>	Error rate or burn rate exceeds threshold
Saturation	How "full" the service is — resource exhaustion	Gauges: CPU, memory, disk, thread pools, queue depth, connection pools	Saturation approaching 100% — predicts imminent failure

Two complementary frameworks organize the same signals differently:

- **USE** (Utilization, Saturation, Errors) — Brendan Gregg's method for **resources** (CPU, memory, disk, network). For each resource: is it utilized, is it saturated (queued work), are there errors?
- **RED** (Rate, Errors, Duration) — Tom Wilkie's method for **request-driven services**. For each service endpoint: what is the request rate, error rate, and duration distribution?

In practice you need all three lenses: Golden Signals for service health, USE for infrastructure, RED for per-endpoint service behavior.

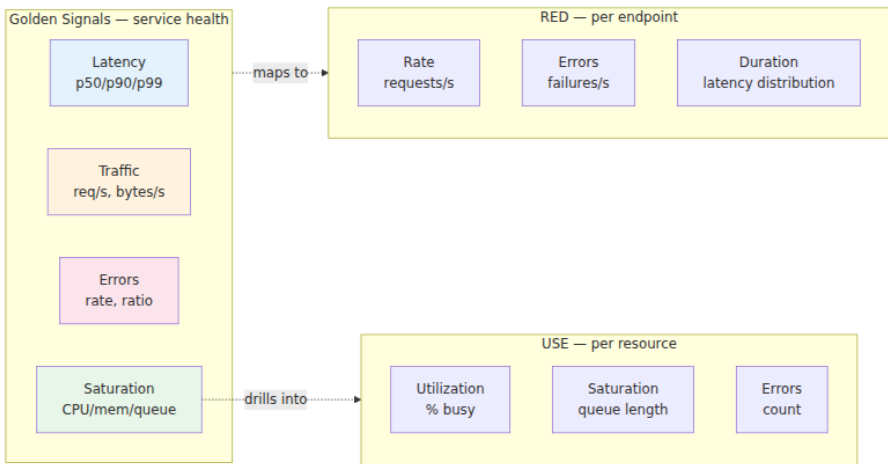


Figure 2-2: Three complementary lenses. Golden Signals give the service-level health check, RED breaks it down per endpoint, and USE explains saturation by drilling into each resource.

Latency in detail

Latency is the most nuanced Golden Signal because a single number cannot represent a distribution. At minimum, track three percentiles:

- **p50 (median)** — the typical experience. A p50 regression indicates a systemic slowdown affecting most users.
- **p90 or p95** — the experience of users having a noticeably slow interaction.

- **p99 (or p99.9)** — the tail. Tail latency is often dominated by a different cause than median latency (GC pauses, cold caches, retry amplification, noisy neighbors) and requires separate investigation.

```
# p50, p90, p99 latency – the standard latency panel
histogram_quantile(0.50, sum by (le) (rate(http_request_duration_second
s_bucket[5m])))
histogram_quantile(0.90, sum by (le) (rate(http_request_duration_second
s_bucket[5m])))
histogram_quantile(0.99, sum by (le) (rate(http_request_duration_second
s_bucket[5m])))

# Latency per handler – identifies which endpoint is slow
histogram_quantile(0.99,
  sum by (le, handler) (rate(http_request_duration_seconds_bucket[5m]))
)

# Latency heatmap – bucket fill rate over time (for Grafana heatmap
panel)
sum by (le) (rate(http_request_duration_seconds_bucket[5m]))
```

Bimodal distributions hide behind percentiles. If your p50 is 20 ms and your p99 is 2 seconds, the mean and even p90 may look healthy. Always visualize the full distribution (heatmap of bucket fill rates over time) alongside percentile lines. A bimodal latency distribution — fast cache hits and slow cache misses — appears as two distinct bands in a heatmap but is invisible in a single percentile.

Traffic

Traffic measures demand. For an HTTP service it is requests per second; for a streaming service it is messages or bytes per second; for a database it is queries per second. Traffic is essential context for the other three signals — an error rate of 10/second means something very different at 100 requests/second (10% failure) versus 10,000 requests/second (0.1% failure).

```
# Total request rate
sum(rate(http_requests_total[5m]))

# Request rate per handler – identifies hot endpoints
sum by (handler) (rate(http_requests_total[5m]))

# Traffic composition – for capacity planning
sum by (method) (rate(http_requests_total[5m]))
```

Traffic anomalies are often the first sign of trouble: a sudden drop may mean an upstream service stopped sending traffic (or DNS broke); a sudden spike may mean a retry storm, a scrape loop, or an abuse pattern.

Errors

Errors are failed requests. The definition of "failed" depends on the protocol and the SLI (see Chapter 1):

```
# Error rate – 5xx responses per second
sum(rate(http_requests_total{code=~"5.."}[5m]))

# Error ratio – the SLI error fraction
sum(rate(http_requests_total{code=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m]))

# Error ratio per handler – pinpoints the failing endpoint
sum by (handler) (rate(http_requests_total{code=~"5.."}[5m]))
/
sum by (handler) (rate(http_requests_total[5m]))

# Application-level errors (business logic failures that return 200)
sum(rate(app_operation_errors_total[5m]))
/
sum(rate(app_operations_total[5m]))
```

Application errors that return HTTP 200. Many services return 200 OK with an error body for business logic failures (e.g., `{"status": "insufficient_funds"}`). These are invisible to `code=~"5.."` filters. Instrument a separate application-level error counter or use response-body-aware metrics to capture them. Otherwise your error SLI will look green while users experience failures.

Saturation

Saturation is the most predictive signal — it tells you the service is *about* to fail, while the other three tell you it *is* failing. Common saturation metrics:

Resource	Saturation metric	Prometheus source
CPU	Utilization approaching limits; throttling	<code>container_cpu_usage_seconds_total</code> , <code>container_cpu_cfs_throttled_seconds_total</code>
Memory	Usage vs. limit; OOM kills	<code>container_memory_working_set_bytes</code> , <code>container_oom_events_total</code>
Disk	Space and inode exhaustion	<code>node_filesystem_avail_bytes</code> , <code>node_filesystem_files_free</code>
Thread / goroutine pools	Active vs. max; queue depth	<code>http_server_threads_busy</code> , <code>go_goroutines</code> , custom pool gauges
Connection pools	Active/idle/pending	<code>db_pool_in_use_connections</code> , <code>db_pool_pending_requests</code>
Network	Bandwidth, conntrack, file descriptors	<code>node_network_receive_bytes_total</code> , <code>process_open_fds</code>

```
# CPU saturation – throttling ratio (cgroup CFS)
sum(rate(container_cpu_cfs_throttled_seconds_total[5m]))
/
sum(rate(container_cpu_usage_seconds_total[5m]))

# Memory pressure – working set as fraction of limit
container_memory_working_set_bytes / container_spec_memory_limit_bytes

# Thread pool saturation – busy threads / max threads
http_server_threads_busy / http_server_threads_max

# Connection pool exhaustion – pending requests (queued, waiting for a
connection)
db_pool_pending_requests # gauge – any sustained > 0 means saturation
```


PromQL: the query language

PromQL is the functional query language for Prometheus. Understanding its data model is essential for writing correct queries — many production PromQL bugs produce results that look plausible but are numerically wrong.

Data model

Every Prometheus time series is identified by a **metric name** plus a set of **labels**:

```
http_requests_total{method="GET", handler="/api/search", code="200",
instance="web-3:8080"}
```

metric name	label
set	one time series

A PromQL expression evaluates to one of:

- **Instant vector** — a set of time series, each with a single value at one timestamp.
- **Range vector** — a set of time series, each with values over a time range (used as input to aggregation functions).
- **Scalar** — a single float.
- **String** — rarely used.

http_requests_total	# instant vector – current
counter values	
http_requests_total[5m]	# range vector – last 5
minutes of samples	
rate(http_requests_total[5m])	# instant vector – per-
second rate over 5 min	
sum(rate(http_requests_total[5m]))	# scalar (if no by/
without) or instant vector grouped	

Essential PromQL patterns

```

# --- Rates and ratios ---

# Request rate per second (use rate for counters)
sum(rate(http_requests_total[5m]))

# Error ratio (fraction of requests that are 5xx)
sum(rate(http_requests_total{code=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m]))

# Apex-like satisfaction ratio (fast + tolerable/2) / total
(
  sum(rate(http_request_duration_seconds_bucket{le="0.1"}[5m])) #
  fast: < 100 ms
  + sum(rate(http_request_duration_seconds_bucket{le="0.5"}[5m])) / 2
  # tolerable: 100-500 ms (half credit)
) / sum(rate(http_request_duration_seconds_count[5m]))

# --- Percentiles ---

# p99 latency (histogram)
histogram_quantile(0.99, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))

# p99 latency per handler
histogram_quantile(0.99, sum by (le, handler) (rate(http_request_duration_seconds_bucket[5m])))

# --- Saturation ---

# CPU usage per instance (irate for fast-moving gauges used as counters)
sum by (instance) (rate(container_cpu_usage_seconds_total[5m]))

# Memory usage ratio
container_memory_working_set_bytes / container_spec_memory_limit_bytes

# --- Availability / SLI ---

# Availability SLI over 30 days (for SLO dashboard)
1 - (
  sum(increase(http_requests_total{code=~"5.."}[30d]))
  /
  sum(increase(http_requests_total[30d]))
)

# --- Capacity / forecasting ---

# Predict when disk will fill (linear regression over last 6 hours,
predict 4 hours ahead)

```

```

predict_linear(node_filesystem_free_bytes[6h], 4 * 3600) < 0

# Days until disk full
(
    node_filesystem_avail_bytes / node_filesystem_size_bytes
) < 0.1 # alert when < 10% free

# --- Aggregation ---

# Top 5 handlers by request rate
topk(5, sum by (handler) (rate(http_requests_total[5m])))

# Average CPU across fleet (not correct for latency – see histogram
section)
avg(container_cpu_usage_seconds_total)

# Count of instances per job
count by (job) (up)

```

Common PromQL pitfalls

Staleness and lookback. Prometheus marks a series as stale 5 minutes after the last scrape. A `rate(metric[5m])` query requires at least two samples in the window to compute a rate. If the scrape interval is 30 seconds, a 1-minute range window may contain only one or two samples and produce no data or inaccurate rates. **Rule of thumb:** range window $\geq 4 * \text{scrape_interval}$.

Counter resets and extrapolation. `rate()` and `increase()` extrapolate to the window boundaries and handle counter resets. But `increase(metric[1h])` on a counter that resets within the hour will undercount if the window does not capture the reset. For SLO calculations over long windows (28 days), prefer `increase(...[28d])` with recording rules that precompute daily increments.

irate vs rate. `irate` uses only the last two samples — it is fast-reacting but noisy. `rate` averages over the full window — it is smooth but slow. Use `rate` for alerting and dashboards; use `irate` only for highly volatile signals where you need instant feedback (and accept the noise).

Label aggregation traps. `sum(metric)` without `by` collapses all labels. `sum by (handler) (metric)` preserves the `handler` dimension. Forgetting `by` when you need per-handler breakdown, or including it when you need a fleet total, are among the most common PromQL errors. Always verify which labels survive the aggregation.

Prometheus architecture: scrape, store, query

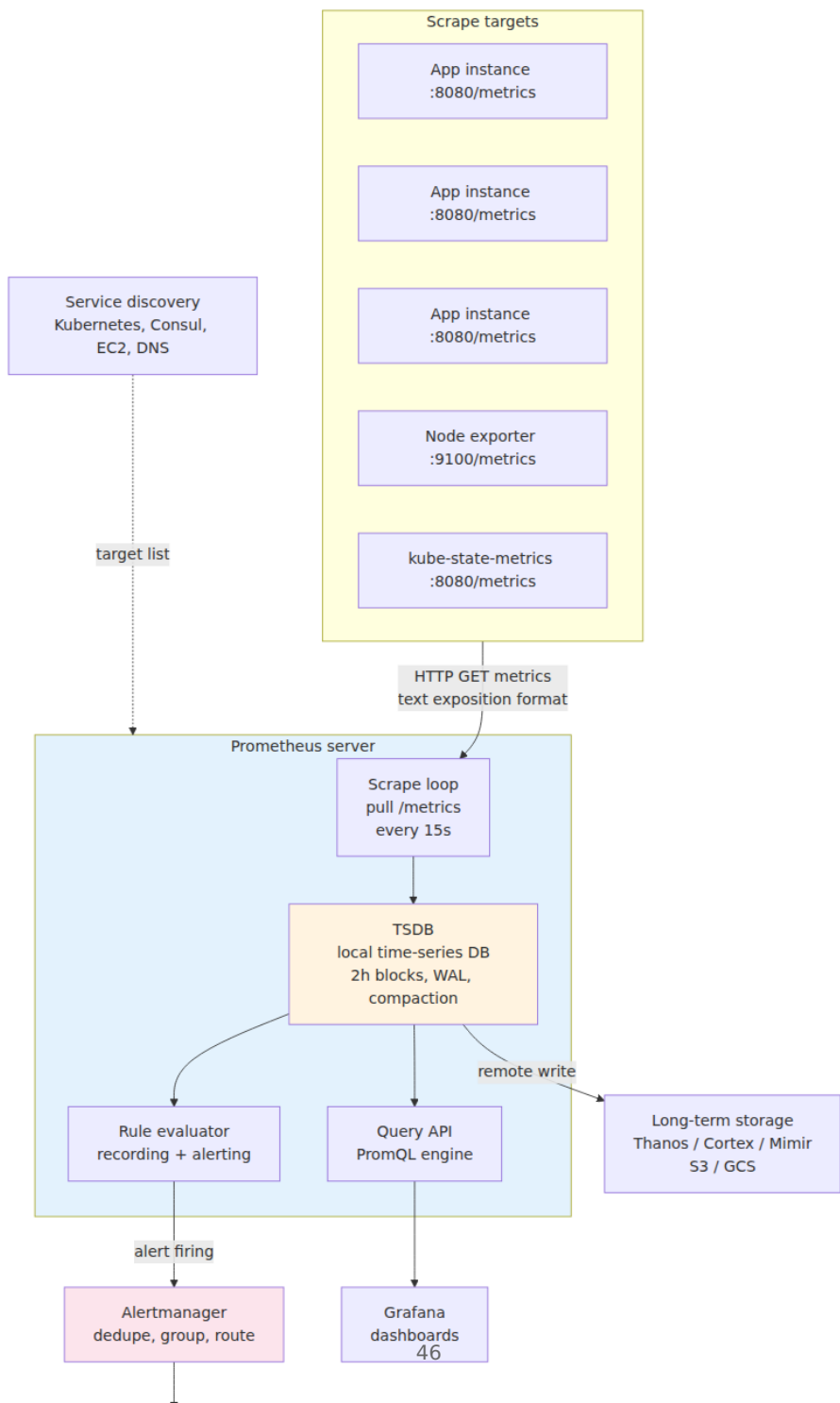


Figure 2-3: Prometheus pull architecture. Targets expose metrics, Prometheus scrapes them, evaluates rules, and fans out to Alertmanager and long-term storage. Service discovery automates target management.

Scrape configuration

Prometheus **pulls** metrics over HTTP. Each target exposes a text endpoint (default `/metrics`) in the Prometheus exposition format. The scrape configuration controls what to scrape, how often, and how to discover targets:

```

# prometheus.yml – production scrape configuration
global:
  scrape_interval: 15s           # default scrape frequency
  evaluation_interval: 15s       # how often to evaluate recording/
    alerting rules
  external_labels:
    cluster: prod-us-central1
    replica: prom-a              # identifies this Prometheus in HA /
    federation

# Alertmanager – where firing alerts are sent
alerting:
  alertmanagers:
    - static_configs:
      - targets: ["alertmanager:9093"]

# Rule files – recording + alerting rules
rule_files:
  - /etc/prometheus/rules/*.yaml
  - /etc/prometheus/slos/*.yaml

scrape_configs:
  # Kubernetes pod discovery – scrape any pod annotated with
  # prometheus.io/scrape=true
  - job_name: kubernetes-pods
    kubernetes_sd_configs:
      - role: pod
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_
scrape]
        action: keep
        regex: "true"
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_
path]
        action: replace
        target_label: __metrics_path__
        regex: (.+)
      - source_labels: [__address__, __meta_kubernetes_pod_annotation_p
rometheus_io_port]
        action: replace
        regex: "([^:]+)(?:[\\d+]?;([\\d+])?"
        replacement: "${1}:${2}"
        target_label: __address__
      - source_labels: [__meta_kubernetes_namespace, __meta_kubernetes_
pod_name]
        separator: "/"
        target_label: pod
      - source_labels: [__meta_kubernetes_pod_label_app]
        target_label: app
    metric_relabel_configs:

```



```

# Drop high-cardinality labels that cause storage explosion
- source_labels: [user_id]
  action: drop
  regex: ".+"
# Keep only needed metrics from verbose exporters
- source_labels: [__name__]
  regex: "go_gc.*"
  action: drop

# Static targets – node exporters, external services
- job_name: node-exporter
  static_configs:
    - targets: ["node-1:9100", "node-2:9100", "node-3:9100"]
  relabel_configs:
    - source_labels: [__address__]
      regex: "([^:]+):.*"
      target_label: instance
      replacement: "${1}"

# Federated scrape – pull aggregated metrics from edge Prometheus
- job_name: federate
  honor_labels: true
  metrics_path: /federate
  params:
    match[]:
      - '{job="web-api"}'
      - '{__name__=~"sl:.*."}'
  static_configs:
    - targets: ["prometheus-edge:9090"]

# Blackbox probing – synthetic checks for SLI validation
- job_name: blackbox-http
  metrics_path: /probe
  params:
    module: [http_2xx]
  static_configs:
    - targets:
        - https://api.example.com/health
        - https://api.example.com/v1/search
  relabel_configs:
    - source_labels: [__address__]
      target_label: __param_target
    - source_labels: [__param_target]
      target_label: instance
    - target_label: __address__
      replacement: "blackbox-exporter:9115"

```

Recording and alerting rules

Recording rules precompute expensive queries into new time series.
Alerting rules evaluate conditions and fire alerts to Alertmanager:

```

# rules/golden-signals.yml
groups:
- name: golden-signals
  interval: 30s
  rules:
    # --- Recording rules: precompute Golden Signal components ---

    # Traffic: request rate per service
    - record: golden:traffic:rate5m
      expr: sum by (service) (rate(http_requests_total[5m]))

    # Error ratio per service
    - record: golden:errors:ratio_rate5m
      expr: |
        sum by (service) (rate(http_requests_total{code=~"5.."}[5m]))
        /
        sum by (service) (rate(http_requests_total[5m]))

    # Latency p99 per service
    - record: golden:latency:p99_rate5m
      expr: |
        histogram_quantile(0.99,
          sum by (le, service)
            (rate(http_request_duration_seconds_bucket[5m]))
        )

    # Saturation: CPU throttling ratio
    - record: golden:saturation:cpu_throttle_ratio
      expr: |
        sum by (pod)
          (rate(container_cpu_cfs_throttled_seconds_total[5m]))
        /
        sum by (pod) (rate(container_cpu_usage_seconds_total[5m]))

    # --- Alerting rules ---

    # Latency SLO burn (also see Chapter 1 for full burn-rate
    pattern)
    - alert: HighLatencyP99
      expr: golden:latency:p99_rate5m > 0.5
      for: 10m
      labels:
        severity: warning
        signal: latency
      annotations:
        summary: "p99 latency {{ $value }}s exceeds 500 ms for {{ $la
        bels.service }}"
        runbook: "https://runbooks.example.com/high-latency"

    # Error rate

```

```

- alert: HighErrorRate
expr: golden:errors:ratio_rate5m > 0.01
for: 5m
labels:
  severity: critical
  signal: errors
annotations:
  summary: "Error rate {{ $value | humanizePercentage }} for
{{ $labels.service }}"

# Saturation – predict exhaustion before it happens
- alert: DiskFillingUp
expr: predict_linear(node_filesystem_free_bytes[6h], 4 * 3600)
< 0
for: 30m
labels:
  severity: warning
  signal: saturation
annotations:
  summary: "Disk on {{ $labels.instance }} predicted to fill
in 4 hours"

# Absence – traffic drop (upstream failure or scrape failure)
- alert: TrafficDropAnomaly
expr: |
(
  golden:traffic:rate5m
  / ignoring(service) group_left
  avg_over_time(golden:traffic:rate5m[1w] offset 1w)
) < 0.5
for: 10m
labels:
  severity: warning
  signal: traffic
annotations:
  summary: "Traffic for {{ $labels.service }} dropped below
50% of last week's average"

# Target down
- alert: PrometheusTargetDown
expr: up == 0
for: 5m
labels:
  severity: warning
annotations:
  summary: "Prometheus cannot scrape {{ $labels.job }}/{{ $label
ls.instance }}"

```

Cardinality: the cost model

Every unique combination of label values creates a new time series in the TSDB. Cardinality is the dominant cost driver for Prometheus — in storage, memory, and query latency. A single metric with 3 labels, each with 100 values, can produce up to 1,000,000 series.

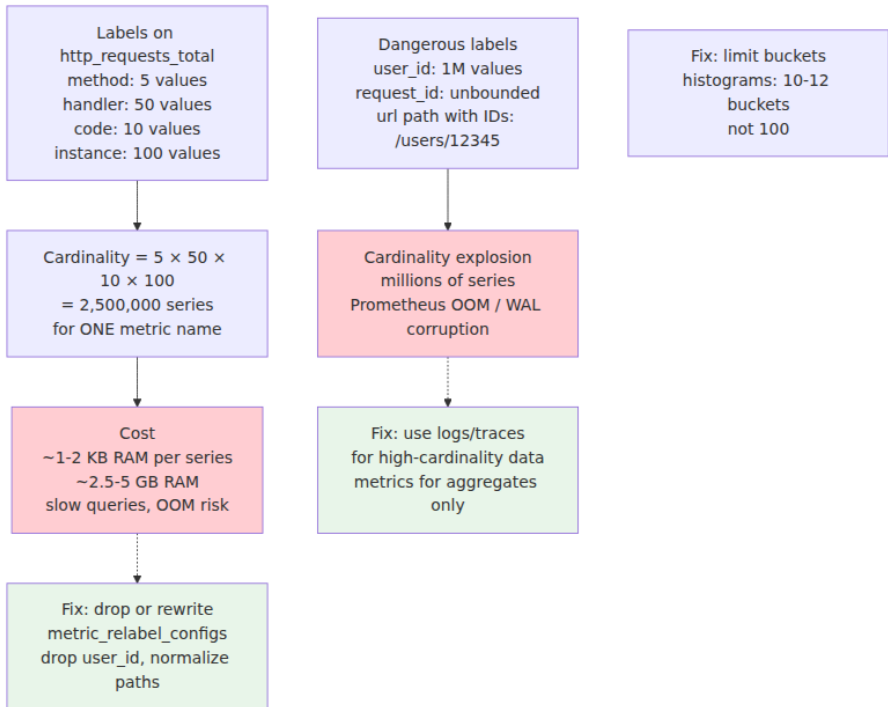


Figure 2-4: Cardinality is the cost model. Each label value multiplies the series count. High-cardinality labels like `user_id` or `request_id` must never appear on metrics.

Rules for cardinality hygiene:

- **Never put unbounded values in labels** — user IDs, email addresses, request IDs, raw URL paths with embedded IDs. Use logs or traces for per-request identity.
- **Normalize URL paths** — `/users/{id}` not `/users/12345`. Rewrite at instrumentation time or via `metric_relabel_configs`.

- **Limit histogram buckets** — 10-12 buckets is sufficient for most latency distributions. 50 buckets with 10 label combinations is 500 series per histogram.
- **Drop verbose metrics at scrape time** — use `metric_relabel_configs` to discard `go_*` or `process_*` metrics you do not need.
- **Monitor cardinality directly:**

```
# Top 10 metrics by series count
topk(10, count by (__name__) ({__name__=~".+"}))

# Total series per job
count by (job) ({__name__=~".+"})

# Series count for a specific metric
count({__name__="http_requests_total"})
```

High availability and long-term storage

A single Prometheus server is a single point of failure and has finite retention (typically 15 days local). Production deployments address both:

HA pair with deduplication. Run two identical Prometheus servers scraping the same targets. Both evaluate the same alerting rules, but Alertmanager deduplicates via grouping. For queries, Thanos Query or Cortex deduplicates by `replica` label:

```
# prometheus-ha.yml – two replicas with distinct external_labels
global:
  external_labels:
    replica: prom-a # second instance uses replica: prom-b
```

```
# thanos-query.yml – deduplicates across replicas
query:
  replicaLabels: ["replica"]
```

Remote write for long-term retention. Prometheus streams samples to a remote system for durable, long-term storage:

```
# prometheus.yml - remote write to Mimir / Thanos / Cortex
remote_write:
- url: "http://mimir:9009/api/v1/push"
  queue_config:
    capacity: 10000
    max_shards: 200
    max_samples_per_send: 5000
  write_relabel_configs:
  - source_labels: [ __name__ ]
    regex: "go_.*"
    action: drop
```

Thanos / Mimir / Cortex provide global query, downsampling, and object-storage-backed retention (S3/GCS) so that dashboards can query months of data without keeping it all in Prometheus local TSDB.

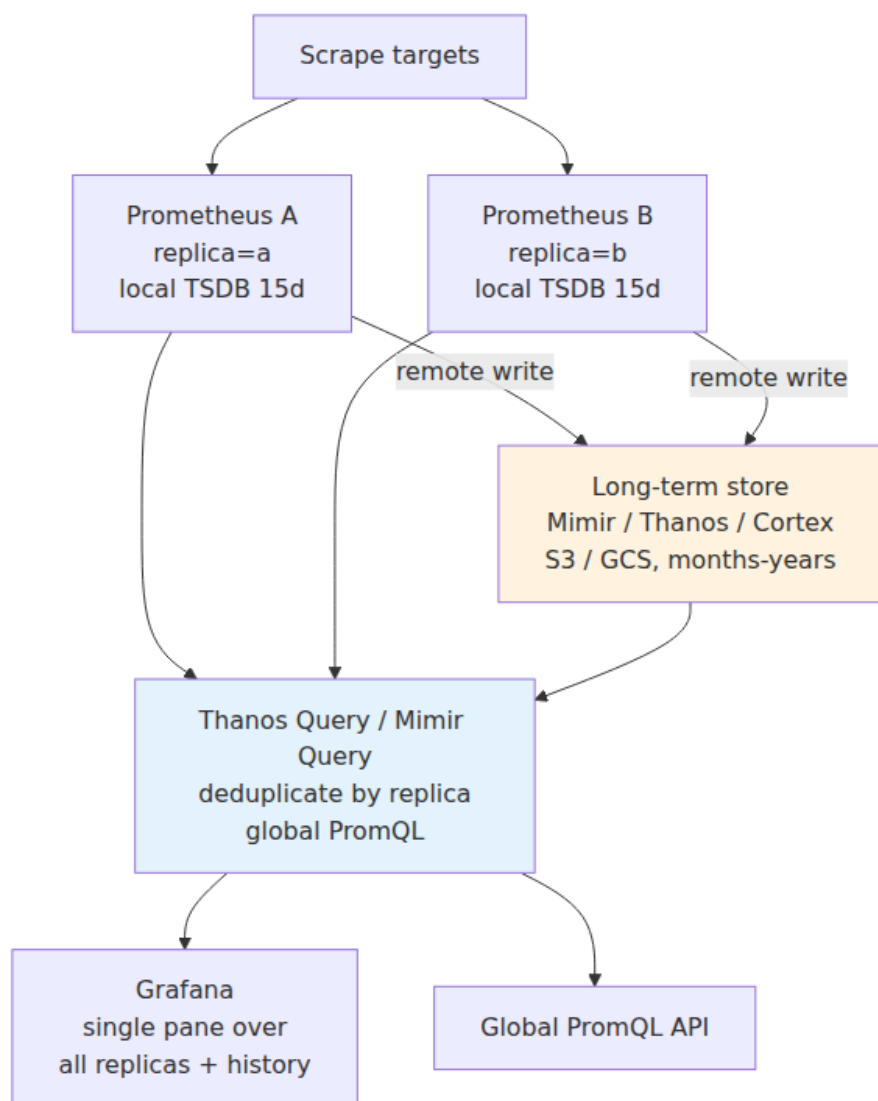


Figure 2-5: HA Prometheus with long-term storage. Two replicas scrape identically, remote-write to durable storage, and a global query layer deduplicates and fans out.

Instrumentation: exposing metrics from application code

Metrics are only as good as the instrumentation that produces them. The Prometheus client libraries (Go, Java, Python, Rust, Node.js) follow a consistent pattern:

```

// Go - complete instrumentation for an HTTP service
package main

import (
    "net/http"
    "time"

    "github.com/prometheus/client_golang/prometheus"
    "github.com/prometheus/client_golang/prometheus/promauto"
    "github.com/prometheus/client_golang/prometheus/promhttp"
)

var (
    httpRequestsTotal = promauto.NewCounterVec(
        prometheus.CounterOpts{
            Name: "http_requests_total",
            Help: "Total HTTP requests by method, handler, and code.",
        },
        []string{"method", "handler", "code"},
    )
    httpDuration = promauto.NewHistogramVec(
        prometheus.HistogramOpts{
            Name: "http_request_duration_seconds",
            Help: "HTTP request latency by method and handler.",
            Buckets: []float64{0.005, 0.01, 0.025, 0.05, 0.1, 0.2,
0.3, 0.5, 1.0, 2.5, 5.0},
        },
        []string{"method", "handler"},
    )
    httpInFlight = promauto.NewGaugeVec(
        prometheus.GaugeOpts{
            Name: "http_requests_in_flight",
            Help: "Current in-flight HTTP requests by handler.",
        },
        []string{"handler"},
    )
)

func instrumentedHandler(handler string, h http.HandlerFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        httpInFlight.WithLabelValues(handler).Inc()
        defer httpInFlight.WithLabelValues(handler).Dec()

        start := time.Now()
        rw := &responseWriter{ResponseWriter: w, code: 200}
        h(rw, r)
        duration := time.Since(start).Seconds()

        code := http.StatusText(rw.code)
    }
}

```

```

        // Normalize to class for cardinality control if needed: 2xx,
        4xx, 5xx
        httpRequestTotal.WithLabelValues(r.Method, handler, rw.code.St
ring()).Inc()
        httpDuration.WithLabelValues(r.Method, handler).Observe(duratio
n)
    }
}

type responseWriter struct {
    http.ResponseWriter
    code int
}

func (rw *responseWriter) WriteHeader(code int) {
    rw.code = code
    rw.ResponseWriter.WriteHeader(code)
}

func main() {
    http.Handle("/metrics", promhttp.Handler())
    http.HandleFunc("/api/search", instrumentedHandler("/api/search", s
earchHandler))
    http.HandleFunc("/api/checkout", instrumentedHandler("/api/
checkout", checkoutHandler))
    http.ListenAndServe(":8080", nil)
}

```

```

# Python – equivalent instrumentation with prometheus_client
from prometheus_client import Counter, Histogram, Gauge, start_http_server
import time

http_requests_total = Counter(
    "http_requests_total", "Total HTTP requests",
    labelnames=["method", "handler", "code"],
)
http_duration = Histogram(
    "http_request_duration_seconds", "HTTP request latency",
    labelnames=["method", "handler"],
    buckets=[0.005, 0.01, 0.025, 0.05, 0.1, 0.2, 0.3, 0.5, 1.0, 2.5, 5.0],
)
http_in_flight = Gauge(
    "http_requests_in_flight", "In-flight HTTP requests",
    labelnames=["handler"],
)

def handle_request(method: str, handler: str, func, *args, **kwargs):
    http_in_flight.labels(handler=handler).inc()
    start = time.monotonic()
    try:
        status_code = func(*args, **kwargs)
    finally:
        duration = time.monotonic() - start
        http_in_flight.labels(handler=handler).dec()
        http_requests_total.labels(method=method, handler=handler,
code=str(status_code)).inc()
        http_duration.labels(method=method, handler=handler).observe(duration)

```

Push vs. pull and the Pushgateway

Prometheus pulls metrics. Short-lived jobs (batch tasks, CronJobs, CI pipelines) may exit before the next scrape. The Pushgateway is a bridge — jobs push their final metric values there, and Prometheus scrapes the gateway:

```
# Batch job pushes its metrics on completion
cat <<EOF | curl --data-binary @- http://pushgateway:9091/metrics/job/
batch/instance/worker-1
# TYPE batch_job_duration_seconds gauge
batch_job_duration_seconds 42.5
# TYPE batch_job_last_success gauge
batch_job_last_success 1713628800
# TYPE batch_job_records_processed counter
batch_job_records_processed 150000
EOF
```

```
# prometheus.yml – scrape the Pushgateway
scrape_configs:
- job_name: pushgateway
  honor_labels: true
  static_configs:
  - targets: ["pushgateway:9091"]
```

Pushgateway is not a general metrics pipeline. It is a workaround for jobs that cannot be scraped. Long-lived services should always expose `/metrics` directly. Metrics pushed to the gateway persist until overwritten — a stale metric from a dead job looks identical to a current one unless you add timestamp or staleness handling.

For Kubernetes CronJobs, the modern alternative is to expose metrics via a sidecar or to use OpenTelemetry with push-based export (OTLP) alongside Prometheus scraping.

Metrics in a distributed system

Service discovery and label consistency

In a microservices architecture with hundreds of services, consistent labeling is what makes global queries possible. Every metric should carry at minimum:

- `service` or `job` — which service produced the metric.
- `instance` — which replica (host:port or pod name).
- `env` / `cluster` — which environment (prod, staging).

Kubernetes service discovery with relabeling automates this, but label naming must be standardized across teams. A service that labels its

HTTP metric `service="checkout"` and another that uses `app="checkout"` cannot be queried together without relabeling normalization.

Distributed aggregation

Fleet-wide queries (total QPS, global p99, error budget across all services) require aggregating metrics from many instances. Prometheus handles this via `sum by` and `histogram_quantile` over aggregated buckets — but only if the metric types support it (histograms yes, summaries no). For cross-cluster aggregation, federation or remote-write with a global query layer (Thanos/Mimir) is required.

Metrics and the observability triangle

Metrics are the coarsest signal. When a Golden Signal alert fires, the next step is to narrow down:

1. **Metrics** tell you *which* service and *which* signal is unhealthy (Golden Signal dashboard).
2. **Logs** (Chapter 3) tell you *what* happened for specific requests — error messages, stack traces, state transitions.
3. **Traces** (Chapter 4) tell you *where* in the call graph the latency or error originated — which downstream dependency is responsible.

No single pillar is sufficient. A metrics-only observability strategy can detect that p99 latency doubled but cannot tell you whether the cause is a slow database query, a saturated thread pool, or a downstream service regression — that requires correlating metrics with logs and traces via shared identifiers (trace IDs, request IDs).

Common failure modes

Cardinality explosion. The most common Prometheus operational failure. A single high-cardinality label (user ID, URL path with IDs, container ID) can create millions of series, exhaust memory, and crash the TSDB. Enforce cardinality budgets via `metric_relabel_configs`, code review of new metrics, and automated cardinality monitoring.

Histogram bucket misconfiguration. Buckets that are too coarse around the SLO threshold make `histogram_quantile` inaccurate precisely where accuracy matters most. Buckets that are too fine waste cardinality.

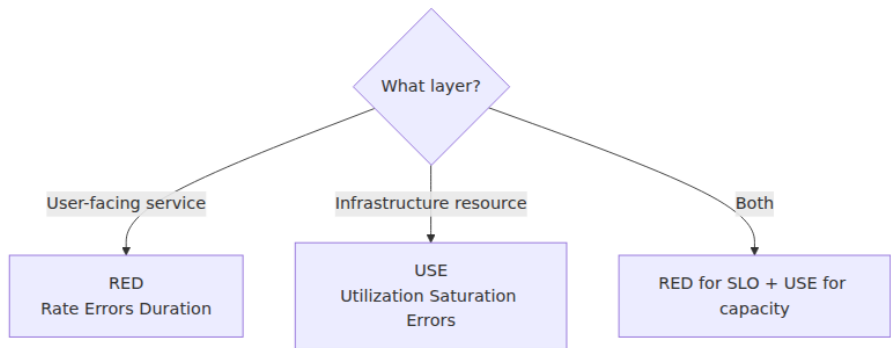
Review bucket boundaries against actual latency distributions (use the heatmap) and SLO thresholds.

Stale series and phantom alerts. When a target disappears (pod evicted, instance terminated), its last value persists as stale for 5 minutes. Queries like `sum(metric)` include stale series, and `up == 0` alerts may fire late. Use `absent()` or `absent_over_time()` for presence checks on critical metrics.

Scrape interval aliasing. A scrape interval that is too long relative to the phenomenon being measured aliases short spikes. A 30-second scrape interval cannot reliably detect a 5-second latency spike. For latency SLOs with tight thresholds, 10-15 second scrape intervals are recommended, with recording rules that smooth appropriately.

Pushgateway staleness. Metrics pushed to the Pushgateway never expire automatically. A batch job that stops running leaves its last push visible forever, making dashboards and alerts believe the job is still succeeding. Always include a `push_time_seconds` gauge and alert when it goes stale.

RED vs USE Decision



Metrics Pipeline



Key takeaways

- Four metric types cover all use cases: **counters** for events (monotonically increasing, queried via `rate / increase`), **gauges** for state (up/down, queried directly), **histograms** for distributions that need fleet-wide aggregation, and **summaries** only for single-instance quantiles.
 - Histograms expose cumulative bucket counters plus `_sum` and `_count`; quantiles are computed server-side via `histogram_quantile()` after summing buckets across instances. Summaries compute quantiles client-side and cannot be aggregated — prefer histograms for distributed services.
 - The four Golden Signals — **latency** (percentiles, not averages), **traffic** (request rate), **errors** (rate and ratio), and **saturation** (resource exhaustion) — are the minimum dashboard for any service; USE (per resource) and RED (per endpoint) provide complementary drill-down.
 - PromQL's data model (instant vectors, range vectors, labels) determines query correctness; always use a range window $\geq 4 \times$ the scrape interval for `rate()`, and verify which labels survive each aggregation.
 - Cardinality is the cost model: every unique label-value combination creates a new time series. Never put unbounded values (user IDs, request IDs, raw paths) in metric labels.
 - Prometheus pulls metrics via HTTP scraping with service discovery, relabeling, recording/alerting rules, and optional remote-write to long-term storage (Thanos/Mimir/Cortex) for HA and retention.
 - Metrics detect *that* something is wrong and *how much*; logs and traces explain *why* for specific requests — the three pillars are complementary and correlated via shared identifiers.
-

Further reading

- Prometheus Documentation — *Metric Types, Querying Basics, Histogram and Summary* (https://prometheus.io/docs/concepts/metric_types/, <https://prometheus.io/docs/practices/histograms/>)

- Prometheus Histograms — Past, Present, and Future (<https://www.robustperception.io/how-does-a-prometheus-histogram-work>) — Björn Rabenstein's detailed explanation of histogram mechanics.
- Native Histograms in Prometheus (https://prometheus.io/docs/specs/native_histograms/) — exponential histogram specification.
- Google SRE Book, Chapter 10 — *Practical Alerting* and Chapter 6 — *Monitoring Distributed Systems* (<https://sre.google/sre-book/monitoring-distributed-systems/>) — the Golden Signals framework and alerting philosophy.
- Brendan Gregg, *USE Method* (<https://www.brendangregg.com/usemethod.html>) and Tom Wilkie, *RED Method* (<https://grafana.com/blog/2018/08/02/the-red-method-how-to-instrument-your-services/>) — complementary monitoring methodologies.
- Robust Perception Blog (<https://www.robustperception.io/blog/>) — Brian Brazil's Prometheus best practices, cardinality management, and PromQL pitfalls.
- *Prometheus: Up & Running* (O'Reilly, 2nd ed.) — comprehensive operational guide for Prometheus at scale.

Chapter 3 — Logging at Scale

What this chapter covers. Metrics tell you *that* something is wrong; logs tell you *what happened* for a specific request, task, or state transition. At scale — thousands of services, millions of log lines per second, terabytes per day — logging becomes a distributed systems problem in its own right: collection must be lossless under backpressure, storage must balance cost against query speed, and querying must remain fast over weeks of retention without drowning in cardinality. This chapter covers the full logging pipeline from structured emission in application code to collection, transport, storage, indexing, and query, with production configurations for the two dominant stacks — the ELK/EFK (Elasticsearch, Logstash/Fluent Bit, Kibana) and Grafana Loki — plus the emerging OpenTelemetry Logs path that unifies them. Every concept is grounded in running configuration: Filebeat and Fluent Bit collectors, Logstash and Vector pipelines, Elasticsearch index lifecycle management, Loki compactor and ruler configs, and Promtail/Alloy scrapers.

Learning goals — after this chapter you should be able to:

- Explain why structured (JSON) logging is the only viable format at scale and what fields every log line must carry for distributed correlation.
- Compare the ELK/EFK and Loki architectures on indexing strategy, storage cost, query model, and operational complexity, and choose between them for a given workload.
- Configure collection agents (Filebeat, Fluent Bit, Vector, Alloy/Promtail) to tail container logs, enrich with Kubernetes metadata, and ship reliably under backpressure.
- Configure Elasticsearch index templates, lifecycle policies, and Kibana data views, and configure Loki tenants, retention, and LogQL queries for production use.
- Design a retention, sampling, and redaction strategy that controls cost without losing the logs you need during incidents.
- Correlate logs with metrics and traces via trace IDs, request IDs, and exemplars, and explain how OpenTelemetry Logs bridges the three pillars.

Why logging, and why it is hard at scale

What logs are for

Every observability signal has a distinct role:

Signal	Granularity	Question it answers	Example
Metrics (Ch 2)	Aggregated over many events	"Is the service healthy? How much is failing?"	Error rate spiked to 5%
Logs (this chapter)	One line per discrete event	"What happened for this specific request/task?"	<code>ERROR request_id=abc123 user_id=42 payment failed: insufficient_funds</code>

Signal	Granularity	Question it answers	Example
Traces (Ch 4)	One record per request, spanning services	"Where in the call graph did the latency/error originate?"	Trace waterfall showing 2 s spent in <code>payment-service</code>

Logs are the most *detailed* signal — they carry arbitrary context (user IDs, error messages, stack traces, business state) that metrics and traces cannot. But that detail is also what makes them expensive: a service handling 10,000 requests/second that logs 5 lines per request produces 50,000 log lines/second. At ~500 bytes per JSON line, that is ~2 TB/day from a single service.

The scale challenges

At the scale of a modern backend — hundreds of services, thousands of instances, tens of terabytes per day — five problems dominate:

1. **Volume and cost.** Storing every log line verbatim in a full-text index (ELK) is powerful but expensive. Storage, indexing CPU, and query latency all scale with volume. Without retention and sampling discipline, logging costs exceed compute costs.
2. **Collection reliability.** Logs must be collected from every instance without loss, even when the central pipeline is temporarily overloaded or down. Agents must handle backpressure, disk buffering, and retransmission — otherwise the most critical logs (those emitted during an outage) are the ones most likely to be dropped.
3. **Query performance over time.** During an incident you need to search across hours or days of logs from dozens of services in seconds. A naive `grep` over flat files is unusable; an index must exist. But indexing every field of every log line is what drives cost — the tension between query speed and storage cost is the central trade-off in logging architecture.
4. **Cardinality and structure.** Logs with inconsistent formats, missing correlation IDs, or unstructured messages ("something went wrong") are unqueryable. At scale, log quality must be enforced by

convention and validated by tooling, not left to individual developer discipline.

5. **Security and compliance.** Logs routinely contain PII, credentials, and business-sensitive data. At scale, manual review is impossible — redaction, access control, and retention enforcement must be automated.

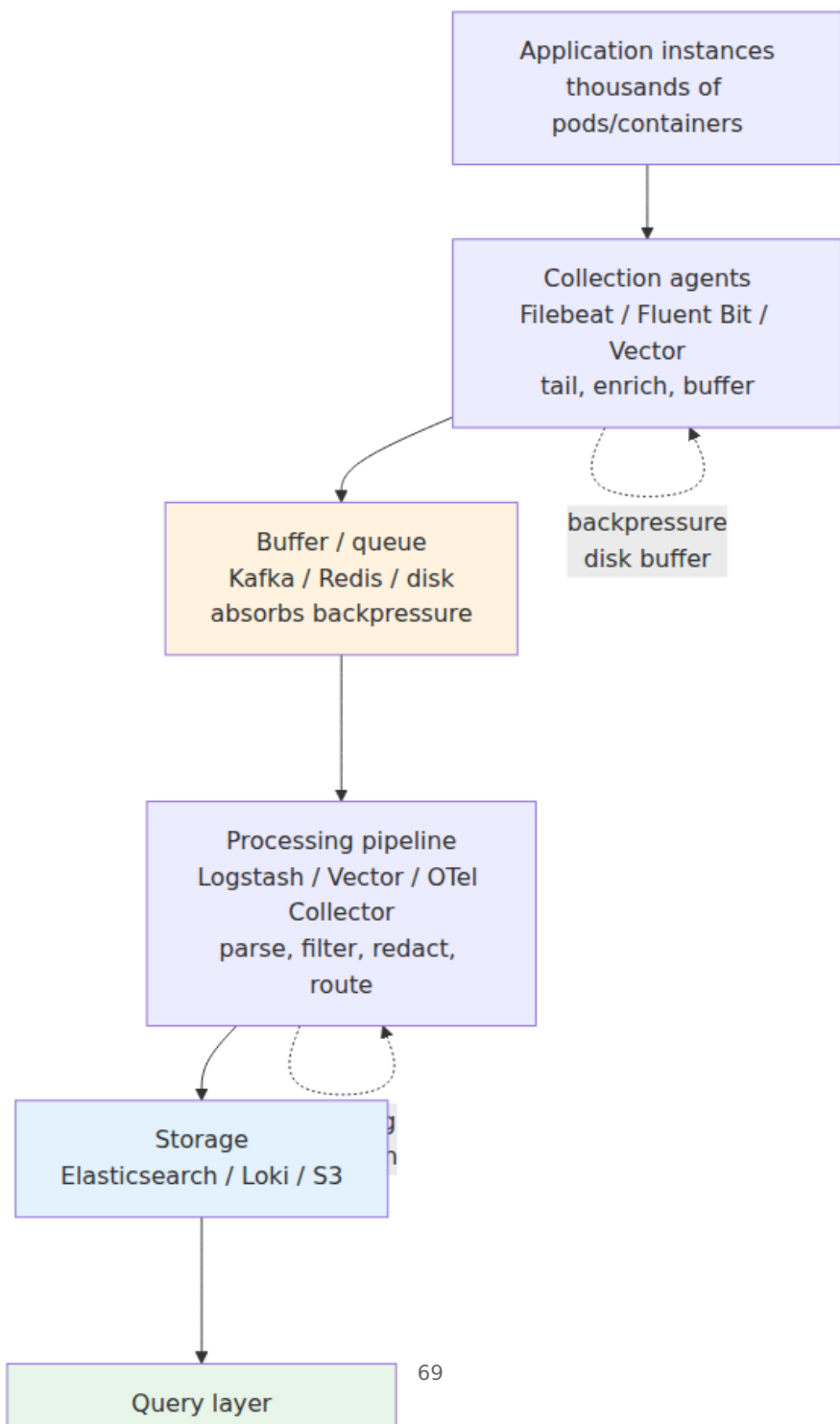


Figure 3-1: The logging pipeline. Every stage — collection, buffering, processing, storage, query — must handle scale, backpressure, and failure independently.

Structured logging: the foundation

Unstructured logs do not scale

Consider two log lines that record the same event:

```
# Unstructured – human-readable, machine-hostile
2026-01-15 14:32:11 ERROR payment failed for user 42:
insufficient_funds (request abc123, took 234ms)

# Structured – JSON, every field is a queryable attribute
{"timestamp":"2026-01-15T14:32:11.234Z","level":"error","service":"payment",
"handler":"charge","request_id":"abc123","trace_id":"4bf92f3577b34
da6a3ce929d0e0e4736","user_id":"42","error":"insufficient_funds","duration_ms":234}
```

The unstructured line requires regex parsing to extract any field, breaks when the message format changes, and cannot be filtered or aggregated without fragile pattern matching. The structured line is trivially queryable: `level="error" AND service="payment" AND error="insufficient_funds"` — no parsing needed, no regex, no ambiguity.

At scale, every log line must be structured JSON (or equivalent) with a consistent schema. This is not a style preference — it is a prerequisite for every downstream stage to function without per-service parsing logic.

The required field set

Every log line emitted by any service should carry at minimum:

Field	Purpose	Example
timestamp	When the event occurred (UTC, ISO 8601 / RFC 3339, millisecond precision)	2026-01-15T14:32:11.234Z

Field	Purpose	Example
<code>level</code>	Severity — <code>debug</code> , <code>info</code> , <code>warn</code> , <code>error</code> , <code>fatal</code>	<code>error</code>
<code>service</code>	Which service emitted the log	<code>payment</code>
<code>message</code>	Human-readable description (still useful for quick scanning)	<code>payment charge failed</code>
<code>request_id</code>	Unique ID for the request/operation (propagated via header)	<code>abc123</code>
<code>trace_id</code>	Distributed trace ID (W3C <code>traceparent</code> or B3) for correlation with traces	<code>4bf92f3577b34da6a...</code>
<code>span_id</code>	Span within the trace	<code>00f067aa0ba902b7</code>
<code>duration_ms</code>	How long the operation took (for latency-relevant events)	<code>234</code>

Additional fields that should be present on relevant log lines:

- `user_id` / `tenant_id` — for per-customer investigation (subject to PII handling).
- `error` / `error.type` / `error.stack` — structured error classification, not just a string.
- `http.method`, `http.path`, `http.status_code` — for HTTP request logs.
- `k8s.pod`, `k8s.namespace`, `k8s.node` — injected by the collection agent, not the application.

Levels and when to use them

Level	When to emit	Production volume	Retention
<code>debug</code>	Detailed diagnostic state — variable values, branch decisions	Very high — disabled in production by default, enabled per-service during investigation	Hours to 1 day, or sampled

Level	When to emit	Production volume	Retention
info	Normal operational events — request completed, task started/finished	High — the bulk of production logs	Days to weeks
warn	Unexpected but handled — retry, fallback, deprecated usage, slow path	Low	Weeks
error	Operation failed — request errored, task failed, invariant violated	Low — but the most queried during incidents	Weeks to months
fatal	Process cannot continue — crash, unrecoverable state	Very low	Months

The most common logging mistake is logging at `info` what should be a metric. "Request completed in 234 ms" as a log line at 10,000 requests/second produces 10,000 log lines/second that must be stored and indexed. The same data as a Prometheus histogram costs a handful of time series and supports quantile queries that log aggregation cannot. Rule of thumb: if you will aggregate it (count, average, percentile), it is a metric. If you need the individual event with full context, it is a log.

Instrumentation: structured logging in application code

```

// Go – structured logging with slog (stdlib since Go 1.21) + trace
correlation
package main

import (
    "context"
    "log/slog"
    "net/http"
    "os"
    "time"

    "go.opentelemetry.io/otel/trace"
)

var logger = slog.New(slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{
    Level:      slog.LevelInfo,
    AddSource:  false, // set true for debug; adds file:line
}))

func loggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
        start := time.Now()
        traceID :=
trace.SpanContextFromContext(r.Context()).TraceID().String()
        spanID := trace.SpanContextFromContext(r.Context()).SpanID().St
ring()

        requestID := r.Header.Get("X-Request-ID")

        rw := &responseWriter{ResponseWriter: w, statusCode: 200}
        next.ServeHTTP(rw, r)

        duration := time.Since(start)

        attrs := []slog.Attr{
            slog.String("service", "payment"),
            slog.String("handler", r.URL.Path),
            slog.String("method", r.Method),
            slog.Int("status_code", rw.statusCode),
            slog.Int64("duration_ms", duration.Milliseconds()),
            slog.String("request_id", requestID),
            slog.String("trace_id", traceID),
            slog.String("span_id", spanID),
        }

        switch {
        case rw.statusCode >= 500:
            logger.LogAttrs(r.Context(), slog.LevelError, "request
failed",

```

```

        append(attrs, slog.String("error", http.StatusText(rw.s
tatusCode)))...)
        case rw.statusCode >= 400:
            logger.LogAttrs(r.Context(), slog.LevelWarn, "request
client error", attrs...)
        case duration > 500*time.Millisecond:
            logger.LogAttrs(r.Context(), slog.LevelWarn, "request
slow", attrs...)
        default:
            logger.LogAttrs(r.Context(), slog.LevelInfo, "request
completed", attrs...)
    }
})
}

// Structured error logging with full context
func chargeCard(ctx context.Context, userID string, amount int64)
error {
    traceID := trace.SpanContextFromContext(ctx).TraceID().String()
    err := doCharge(ctx, userID, amount)
    if err != nil {
        logger.ErrorContext(ctx, "payment charge failed",
            slog.String("user_id", userID),
            slog.Int64("amount_cents", amount),
            slog.String("error", err.Error()),
            slog.String("error_type", errorType(err)),
            slog.String("trace_id", traceID),
        )
        return err
    }
    logger.InfoContext(ctx, "payment charge succeeded",
        slog.String("user_id", userID),
        slog.Int64("amount_cents", amount),
        slog.String("trace_id", traceID),
    )
    return nil
}

```

```

# Python - structured logging with structlog + trace correlation
import structlog
import logging
import time
from opentelemetry import trace

structlog.configure(
    processors=[
        structlog.contextvars.merge_contextvars,
        structlog.processors.add_log_level,
        structlog.processors.TimeStamper(fmt="iso"), # ISO 8601 UTC
        structlog.processors.JSONRenderer(),
    ],
    wrapper_class=structlog.make_filtering_bound_logger(logging.INFO),
    context_class=dict,
    logger_factory=structlog.PrintLoggerFactory(),
)

logger = structlog.get_logger(service="payment")

def handle_charge(request_id: str, user_id: str, amount_cents: int):
    start = time.monotonic()
    span = trace.get_current_span()
    ctx = span.get_span_context()
    trace_id = format(ctx.trace_id, "032x") if ctx.is_valid else "none"

    log = logger.bind(
        request_id=request_id,
        trace_id=trace_id,
        user_id=user_id,
        amount_cents=amount_cents,
    )
    try:
        do_charge(user_id, amount_cents)
        log.info("payment charge succeeded", duration_ms=int((time.monotonic() - start) * 1000))
    except InsufficientFundsError as e:
        log.error("payment charge failed",
            error="insufficient funds",
            error_type="InsufficientFundsError",
            duration_ms=int((time.monotonic() - start) * 1000),
        )
        raise
    except Exception as e:
        log.error("payment charge failed",
            error=str(e),
            error_type=type(e).__name__,
            duration_ms=int((time.monotonic() - start) * 1000),
            exc_info=True, # includes stack trace

```

```
)  
raise
```

```
// Output – every line is parseable JSON with consistent fields  
{  
  "time": "2026-01-15T14:32:11.234Z",  
  "level": "error",  
  "service": "payment",  
  "handler": "/api/charge",  
  "method": "POST",  
  "status_code": 402,  
  "duration_ms": 234,  
  "request_id": "abc123",  
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",  
  "span_id": "00f067aa0ba902b7",  
  "msg": "payment charge failed",  
  "error": "insufficient_funds",  
  "error_type": "InsufficientFundsError",  
  "user_id": "42",  
  "amount_cents": 1999  
}  
{  
  "time": "2026-01-15T14:32:12.001Z",  
  "level": "info",  
  "service": "payment",  
  "handler": "/api/charge",  
  "method": "POST",  
  "status_code": 200,  
  "duration_ms": 45,  
  "request_id": "def456",  
  "completed": true  
}
```

Collection: getting logs off the host

Logs are written to stdout/stderr (the 12-factor and Kubernetes convention) or to files. Collection agents tail these outputs, enrich them with metadata, and ship them to the pipeline. The agent is the most reliability-critical component — if it drops logs during an outage, the evidence needed to diagnose that outage is gone.

Kubernetes log collection model

In Kubernetes, every container's stdout/stderr is captured by the kubelet into files on the node:

```
/var/log/pods/<namespace>_<pod>_<uid>/<container>/0.log   
kubelet-managed, JSON per line  
/var/log/containers/<pod>_<namespace>_<container>-<id>.log  symlink  f  
or convenience
```

Each line is a JSON object with `log` (the raw application output), `stream` (stdout/stderr), and `time`. The collection agent runs as a **DaemonSet** (one pod per node) that tails these files.

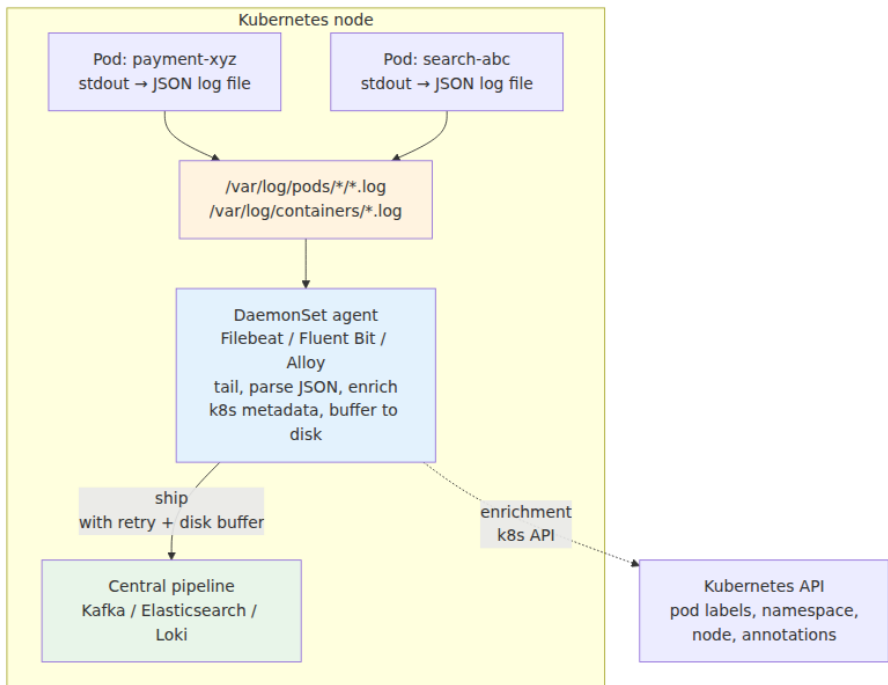


Figure 3-2: Kubernetes log collection. The DaemonSet agent tails kubelet-managed log files, enriches with Kubernetes metadata, and ships with disk buffering for reliability.

Filebeat (Elastic stack)

Filebeat is the Elastic-recommended lightweight shipper for the ELK stack. It tails files, handles rotation, maintains a registry of read offsets, and ships to Logstash or directly to Elasticsearch:

```

# filebeat.yml – production DaemonSet configuration
filebeat.inputs:
- type: container
  paths:
    - /var/log/containers/*.log
    # Parse the kubelet's JSON wrapper (log, stream, time)
  json.keys_under_root: false
  json.add_error_key: true
  # Multiline: combine stack traces into a single event
  multiline:
    type: pattern
    pattern: '^\s+at |\s+... \d+ more|^\s*Caused by: '
    negate: false
    match: after

processors:
  # Enrich with Kubernetes metadata from the API server
  - add_kubernetes_metadata:
      host: ${NODE_NAME}
      matchers:
        - logs_path:
            logs_path: /var/log/containers/
    # Drop health-check noise
  - drop_event:
      when:
        contains:
          log: 'GET /health'
    # Parse application JSON logs (the inner "log" field)
  - decode_json_fields:
      fields: ["log"]
      target: ""
      overwrite_keys: true
      add_error_key: true
    # Redact PII – drop or hash sensitive fields
  - script:
      lang: javascript
      source: |
        function process(event) {
          var email = event.Get("user_email");
          if (email) { event.Put("user_email", "****REDACTED****"); }
          var card = event.Get("card_number");
          if (card) { event.Put("card_number", "****REDACTED****"); }
        }

# Backpressure handling – spool to disk when downstream is slow
queue:
  spool:
    file:
      path: /var/lib/filebeat/spool.dat
      size: 512MiB

```

```

    page_size: 16KiB
  write:
    buffer_size: 10MiB
    flush.timeout: 5s
    flush.events: 2048

output.elasticsearch:
  hosts: ["https://elasticsearch:9200"]
  index: "logs-%{[kubernetes.namespace]}-%{+yyyy.MM.dd}"
  # Or use data streams (recommended for ILM):
  # index: "logs-generic-default"
  ssl.certificate_authorities: ["/etc/certs/ca.crt"]
  username: "${ES_USERNAME}"
  password: "${ES_PASSWORD}"
  # Retry with backoff
  backoff:
    init: 1s
    max: 60s
  worker: 2
  bulk_max_size: 2048

# Alternative: ship to Logstash for heavier processing
# output.logstash:
#   hosts: ["logstash:5044"]
#   ssl.certificate_authorities: ["/etc/certs/ca.crt"]

logging:
  level: info
  json: true

monitoring.enabled: true

```

Fluent Bit (lightweight, widely used in Kubernetes)

Fluent Bit is a CNCF project with lower resource overhead than Filebeat or Logstash, making it popular for resource-constrained DaemonSets:


```
# fluent-bit.conf – Kubernetes DaemonSet configuration
```

[SERVICE]

```
Flush          5
Daemon         Off
Log_Level      info
Parsers_File   parsers.conf
storage.path    /var/log/flb-storage
storage.sync    normal
storage.checksum Off
storage.backlog.mem_limit 50M
HTTP_Server    On
HTTP_Listen    0.0.0.0
HTTP_Port      2020
```

[INPUT]

```
Name          tail
Path          /var/log/containers/*.log
Parser        docker          # parses kubelet JSON wrapper
Tag           kube.*
Refresh_Interval 5
Mem_Buf_Limit 50M
Skip_Long_Lines On
DB            /var/log/flb_kube.db      # offset registry
DB.Sync       Normal
storage.type   filesystem             # disk buffer for
backpressure
```

[FILTER]

```
Name          kubernetes
Match         kube.*
Kube_URL      https://kubernetes.default.svc:443
Kube_CA_File   /var/run/secrets/kubernetes.io/serviceaccount/
ca.crt
Kube_Token_File /var/run/secrets/kubernetes.io/serviceaccount/
token
Merge_Log     On                # merge JSON log field into record
Keep_Log      Off
K8S-Logging.Parser On
K8S-Logging.Exclude On
```

[FILTER]

```
Name          multiline
Match         kube.*
multiline.key_content log
multiline.parser java, python, go # combine stack traces
```

[FILTER]

```
Name          modify
Match         kube.*
Condition     Key_value_equals    stream stderr
```

```

Add                                level                                error

[FILTER]
Name                               grep
Match                             kube.*
Exclude                           log      GET /health  # drop health checks

[OUTPUT]
Name                               es
Match                             kube.*
Host                              elasticsearch
Port                              9200
Index                             logs
Type                              _doc
Logstash_Format                   On
Logstash_Prefix                   logs
Replace_Dots                      On
Suppress_Type_Name                On
Trace_Output                      Off
Trace_Error                       On
Suppress_Type_Name                On
tls                               On
tls.verify                        On
HTTP_User                         ${ES_USERNAME}
HTTP_Passwd                       ${ES_PASSWORD}
# Retry with exponential backoff
Retry_Limit                       5
storage.total_limit_size          128M

# Alternative output – Loki
# [OUTPUT]
#   Name                           loki
#   Match                         kube.*
#   Host                          loki-gateway
#   Port                          3100
#   Labels                        job=kube,
namespace=$kubernetes['namespace_name'], app=$kubernetes['labels.app'],
pod=$kubernetes['pod_name']
#   Auto_Kubernetes_Labels        On

```

```
# parsers.conf - multiline parsers for stack traces
[PARSER]
Name          docker
Format        json
Time_Key      time
Time_Format   %Y-%m-%dT%H:%M:%S.%L
Time_Keep     On

[MULTILINE_PARSER]
name          java
type          regex
flush_timeout 1000
rule          "start_state"  "/^(Dec \d+|20\d{2}-\d{2}-
\d{2}).*at /"      "cont"
rule          "cont"         "/^\s+at.*|^\s+... \d+ more|^\s*Caused
by:/"      "cont"

[MULTILINE_PARSER]
name          python
type          regex
flush_timeout 1000
rule          "start_state"  "/^Traceback.*:$/"      "cont"
rule          "cont"         "/^\s+.*$/"      "cont"

[MULTILINE_PARSER]
name          go
type          regex
flush_timeout 1000
rule          "start_state"  "/^(panic|goroutine \d+).*/"      "cont"
rule          "cont"         "/^(.*|\\t.*)$/"      "cont"
```

Vector (high-performance, Rust-based)

Vector is a newer alternative that handles both collection and transformation with strong backpressure and exactly-once delivery semantics:

```

# vector.yaml - DaemonSet + aggregator topology
sources:
  kubernetes_logs:
    type: kubernetes_logs
    self_node_name: "${VECTOR_SELF_NODE_NAME}"
    extra_label_selector: ""
    exclude_paths_glob_patterns:
      - "**/_kube-system*"

transforms:
  # Parse JSON application logs
  parse_json:
    type: remap
    inputs: [kubernetes_logs]
    source: |
      . = parse_json!(.message)
      # Normalize level field
      .level = lowercase(string(.level) ?? "info")
      # Redact PII
      if exists(.user_email) { .user_email = "****REDACTED****" }
      if exists(.card_number) { .card_number = "****REDACTED****" }
      # Enrich: add cluster label
      .cluster = "prod-us-central1"

  # Route: errors to P1 index, everything else to default
  route_errors:
    type: route
    inputs: [parse_json]
    route:
      errors: '.level == "error" || .level == "fatal"'
      default: 'true'

  # Sample debug logs - keep 10% to control volume
  sample_debug:
    type: sample
    inputs: [route_errors.default]
    rate: 10
    condition: '.level == "debug"'

sinks:
  es_errors:
    type: elasticsearch
    inputs: [route_errors.errors]
    endpoints: ["https://elasticsearch:9200"]
    index: "logs-errors-%Y-%m-%d"
    auth:
      strategy: basic
      user: "${ES_USERNAME}"
      password: "${ES_PASSWORD}"
    batch:

```

```

    max_bytes: 10485760
    timeout_secs: 5
  request:
    retry_attempts: 5
    retry_initial_backoff_secs: 1
  buffer:
    type: disk
    max_size: 536870912 # 512 MiB disk buffer per sink
    when_full: block # backpressure – block instead of dropping

es_default:
  type: elasticsearch
  inputs: [sample_debug, "route_errors.default"]
  endpoints: ["https://elasticsearch:9200"]
  index: "logs-%Y-%m-%d"
  auth:
    strategy: basic
    user: "${ES_USERNAME}"
    password: "${ES_PASSWORD}"
  batch:
    max_bytes: 10485760
    timeout_secs: 5
  buffer:
    type: disk
    max_size: 536870912
    when_full: block

# Fan-out to Loki for cheap long-term retention
loki:
  type: loki
  inputs: [parse_json]
  endpoint: "http://loki:3100"
  labels:
    service: "{{ service }}"
    level: "{{ level }}"
    namespace: "{{ kubernetes.pod_namespace }}"
  encoding:
    codec: json
  batch:
    max_bytes: 1048576
    timeout_secs: 2

```

Storage and indexing: ELK vs. Loki

The choice between ELK (Elasticsearch) and Loki is the most consequential architectural decision in logging at scale. They represent

fundamentally different trade-offs between indexing cost and query power.

Elasticsearch (ELK/EFK): full-text indexing

Elasticsearch indexes **every field** of every log line into an inverted index. Any field can be searched, filtered, aggregated, and visualized with sub-second latency over large time ranges. The cost is storage and indexing overhead — typically 1.5–2× the raw log volume for the index, plus significant CPU for indexing.

```
Raw log JSON (500 bytes)
  ↳ Elasticsearch document
    ↳ Inverted index (per field tokenization + posting lists)
    ↳ Doc values (columnar, for aggregations)
    ↳ Stored fields (for retrieval)
Total: ~750-1000 bytes indexed per 500-byte log line
```

When to choose ELK:

- You need ad-hoc full-text search across any field ("find all logs mentioning `insufficient_funds` regardless of field").
- You need aggregations and analytics on log data (top error types, error rate over time, cardinality analysis).
- You need Kibana's visualization and alerting (Lens, Discover, Watcher/Alerting).
- Your log volume is moderate (< 1 TB/day) or your budget supports the indexing cost.
- You need cross-cluster search and SQL-like query (ES|QL).

Elasticsearch production configuration:

```
// elasticsearch-templates/logs-template.json – index template for log
data streams
{
  "index_patterns": ["logs-*"],
  "data_stream": {},
  "template": {
    "settings": {
      "number_of_shards": 3,
      "number_of_replicas": 1,
      "refresh_interval": "5s",
      "index.lifecycle.name": "logs-ilm-policy",
      "index.codec": "best_compression",
      "index.mapping.total_fields.limit": 2000,
      "index.mapping.ignore_malformed": true
    },
    "mappings": {
      "dynamic": "strict",
      "properties": {
        "@timestamp": { "type": "date", "format": "strict_date_optio
nal_time||epoch_millis" },
        "level": { "type": "keyword" },
        "service": { "type": "keyword" },
        "handler": { "type": "keyword" },
        "message": { "type": "text", "analyzer": "standard", "fie
lds": { "keyword": { "type": "keyword", "ignore_above": 256 } } },
        "request_id": { "type": "keyword" },
        "trace_id": { "type": "keyword" },
        "span_id": { "type": "keyword" },
        "user_id": { "type": "keyword" },
        "error": { "type": "keyword" },
        "error_type": { "type": "keyword" },
        "duration_ms": { "type": "long" },
        "status_code": { "type": "integer" },
        "kubernetes": {
          "properties": {
            "namespace": { "type": "keyword" },
            "pod_name": { "type": "keyword" },
            "labels": { "type": "object", "dynamic": true }
          }
        }
      }
    }
  }
}
```

// elasticsearch-ilm/logs-ilm-policy.json – lifecycle management: hot → warm → cold → delete

```
{
  "policy": {
    "phases": {
      "hot": {
        "min_age": "0ms",
        "actions": {
          "rollover": {
            "max_primary_shard_size": "50GB",
            "max_age": "1d"
          },
          "set_priority": { "priority": 100 }
        }
      },
      "warm": {
        "min_age": "7d",
        "actions": {
          "forcemerge": { "max_num_segments": 1 },
          "shrink": { "number_of_shards": 1 },
          "set_priority": { "priority": 50 },
          "allocate": { "number_of_replicas": 1 }
        }
      },
      "cold": {
        "min_age": "30d",
        "actions": {
          "searchable_snapshot": { "snapshot_repository": "s3-
snapshots" },
          "set_priority": { "priority": 0 }
        }
      },
      "frozen": {
        "min_age": "60d",
        "actions": {
          "searchable_snapshot": { "snapshot_repository": "s3-
snapshots" }
        }
      },
      "delete": {
        "min_age": "90d",
        "actions": { "delete": {} }
      }
    }
  }
}
```



```

# logstash-pipeline/logstash.conf – central processing pipeline
(alternative to Vector)
input {
  beats {
    port => 5044
    ssl => true
    ssl_certificate_authorities => ["/etc/certs/ca.crt"]
    ssl_certificate => "/etc/certs/logstash.crt"
    ssl_key => "/etc/certs/logstash.key"
  }
}

filter {
  # Parse application JSON (already partially parsed by Filebeat)
  if [log] =~ /^\/\{ {
    json {
      source => "log"
      target => "app"
      skip_on_invalid_json => true
    }
    if "_jsonparsefailure" not in [tags] {
      mutate { rename => { "[app][level]" => "level" } }
      mutate { rename => { "[app][trace_id]" => "trace_id" } }
      # ... additional field promotion
    }
  }

  # GeoIP enrichment for access logs
  if [service] == "edge-proxy" and [client_ip] {
    geoip { source => "client_ip" }
  }

  # Drop health checks that slipped through
  if [handler] == "/health" or [handler] == "/readyz" {
    drop {}
  }

  # Redact sensitive fields
  mutate {
    gsub => ["user_email", ".", "****REDACTED****"]
    gsub => ["card_number", ".", "****REDACTED****"]
  }

  date {
    match => ["timestamp", "ISO8601"]
    target => "@timestamp"
  }
}

output {

```

```

elasticsearch {
  hosts => ["https://elasticsearch:9200"]
  data_stream => true
  data_stream_type => "logs"
  data_stream_dataset => "generic"
  data_stream_namespace => "default"
  ssl => true
  cacert => "/etc/certs/ca.crt"
  user => "${ES_USERNAME}"
  password => "${ES_PASSWORD}"
  # Dead letter queue for indexing failures
}
# Fan-out to S3 for long-term archive (cheaper than keeping in ES)
# s3 {
#   bucket => "logs-archive-prod"
#   prefix => "%{+YYYY}/{+MM}/{+dd}/"
#   codec => "json_lines"
# }
}

```

Loki: label-indexed, content-unindexed

Loki takes the opposite approach from Elasticsearch. It indexes **only labels** (small, low-cardinality key-value pairs like `service`, `level`, `namespace`) and stores log content as **compressed chunks** (gzipped log lines grouped by label set and time). Queries first filter by labels to find relevant chunks, then scan chunk content with grep-like filters.

```

Raw log lines grouped by label set {service="payment", level="error"}
→ Compressed chunks (gzip, ~10:1 compression)
→ Index: only the label → chunk pointer mapping
Total: ~50-100 bytes indexed per 500-byte log line (10-20x less than ES)

```

This makes Loki dramatically cheaper — roughly 10× lower storage cost than Elasticsearch for the same log volume — but with a query trade-off: filtering by label is fast (indexed), while searching within log content requires scanning decompressed chunks (slower, especially over large time ranges without selective labels).

When to choose Loki:

- Cost is a primary concern and log volume is high (> 500 GB/day).
- Your query pattern is predominantly label-filtered ("all `error` logs from `payment` in the last hour") rather than full-text search.

- You already use Grafana and Prometheus — Loki integrates natively with the same label model and query UX.
- You need long retention (weeks to months) at manageable cost — Loki's S3-backed chunk storage with compaction is designed for this.

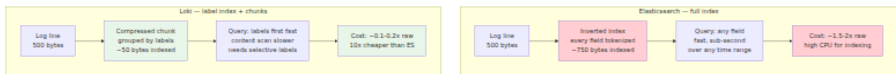


Figure 3-3: Indexing trade-off. Elasticsearch indexes everything for fast arbitrary queries at high cost; Loki indexes only labels and scans chunk content, trading query flexibility for 10× lower cost.

Loki / Grafana Alloy production configuration:

```

# loki-config.yaml - production Loki deployment (monolithic or
microservices)
auth_enabled: true

server:
  http_listen_port: 3100
  grpc_listen_port: 9096
  log_level: info

common:
  path_prefix: /loki
  storage:
    filesystem:
      chunks_directory: /loki/chunks
      rules_directory: /loki/rules
  replication_factor: 1
  ring:
    instance_addr: 127.0.0.1
    kvstore:
      store: inmemory

# For production with S3/GCS long-term storage:
storage:
  type: s3
  s3:
    endpoint: s3.amazonaws.com
    bucketnames: loki-chunks-prod
    region: us-central1
    s3forcepathstyle: false
  boltdb_shipper:
    active_index_directory: /loki/index
    cache_location: /loki/index_cache
    cache_ttl: 24h
    shared_store: s3

# Compactor - deduplicates and merges index + manages retention
compactor:
  working_directory: /loki/compactor
  shared_store: s3
  compaction_interval: 10m
  retention_enabled: true
  retention_delete_delay: 2h
  retention_delete_worker_count: 150

# Retention - per-tenant + global defaults
limits_config:
  retention_period: 744h # 31 days default
  retention_stream:
    - selector: '{level="debug"}'
      period: 24h # debug logs: 1 day only

```

```

    - selector: '{level="error"}'
      period: 2160h      # error logs: 90 days
    ingestion_rate_mb: 16
    ingestion_burst_size_mb: 32
    max_label_name_length: 1024
    max_label_value_length: 2048
    max_label_names_per_series: 15
    reject_old_samples: true
    reject_old_samples_max_age: 168h
    # Cardinality protection – the most important Loki safeguard
    max_global_streams_per_user: 10000
    ingestion_rate_strategy: local

# Ruler – alerting on log content
ruler:
  alertmanager_url: http://alertmanager:9093
  storage:
    type: local
    local:
      directory: /loki/rules
  ring:
    kvstore:
      store: inmemory
  enable_api: true

# Query performance
querier:
  max_concurrent: 8
  query_range:
    align_queries_with_step: true
    max_retries: 5
    cache_results: true
    results_cache:
      cache:
        embedded_cache:
          enabled: true
          max_size_mb: 100

# Ingester – where chunks are built before flushing to storage
ingester:
  lifecycler:
    ring:
      kvstore:
        store: inmemory
      replication_factor: 1
    chunk_idle_period: 1h
    max_chunk_age: 2h
    chunk_target_size: 1572864 # 1.5 MB – target compressed chunk size
    chunk_retain_period: 30s

# Structured metadata (Grafana Loki 2.8+ – attach arbitrary fields

```

```
without indexing)  
allow_structured_metadata: true  
volume_enabled: true
```

```

# alloy-config.alloy – Grafana Alloy (successor to Promtail) as
DaemonSet collector
logging {
  level = "info"
  format = "json"
}

loki.source.kubernetes "pods" {
  targets = discovery.kubernetes.pods.targets
  forward_to = [loki.process.enrich.receiver]
}

discovery.kubernetes "pods" {
  role = "pod"
}

discovery.relabel "pods" {
  targets = discovery.kubernetes.pods.targets

  // Keep only pods with logs
  rule {
    source_labels = ["__meta_kubernetes_pod_phase"]
    regex         = "Running"
    action        = "keep"
  }

  // Map Kubernetes metadata to Loki labels – LOW cardinality only
  rule {
    source_labels = ["__meta_kubernetes_namespace"]
    target_label  = "namespace"
  }
  rule {
    source_labels = ["__meta_kubernetes_pod_label_app"]
    target_label  = "app"
  }
  rule {
    source_labels = ["__meta_kubernetes_pod_container_name"]
    target_label  = "container"
  }
  // Do NOT add pod name, trace_id, or user_id as labels – cardinality
  explosion
}

loki.process "enrich" {
  stage.json {
    expressions = {
      level      = "level",
      service    = "service",
      trace_id   = "trace_id",
      msg        = "msg",
    }
  }
}

```

```

    }
  }

  // Promote structured fields as Loki structured metadata (not labels
  - queryable without cardinality cost)
  stage.structured_metadata {
    values = {
      trace_id = "trace_id",
      level     = "level",
    }
  }

  // Drop health-check noise before shipping
  stage.drop {
    source = "msg"
    expression = "GET /health"
  }

  // Multiline: reassemble stack traces
  stage.multiline {
    firstline   = "^\\{.*level.*\\}"
    max_wait_time = "3s"
  }

  forward_to = [loki.write.default.receiver]
}

loki.write "default" {
  endpoint {
    url = "http://loki-gateway:3100/loki/api/v1/push"
    tenant_id = "prod"
  }
  external_labels = {
    cluster = "prod-us-central1",
  }
}

```

Choosing between them — or both

Many organizations run **both**: Loki for the bulk of application logs (cheap, good enough for label-filtered incident queries) and Elasticsearch for specific high-value log types that need full-text search (audit logs, security events, access logs that feed analytics). Vector and Alloy both support fanning out to multiple sinks, making a dual-stack deployment straightforward.

Dimension	Elasticsearch (ELK)	Loki	Dual-stack
Index cost	~1.5-2× raw volume	~0.1-0.2× raw volume	Loki for bulk, ES for critical
Query: label filter	Fast	Fast (indexed)	—
Query: full-text / any field	Fast (inverted index)	Slow (chunk scan)	ES for ad-hoc search
Query: aggregation	Powerful (ES)	QL, aggregations)	Limited (LogQL metric queries)
Retention cost	High — index on hot storage	Low — chunks on S3/GCS	Loki retains longer cheaply
Operational complexity	High — shard management, mapping, ILM	Lower — fewer moving parts, S3-native	Highest — two systems
Ecosystem	Kibana, Beats, Logstash, ES	QL	Grafana, LogQL, Prometheus-compatible labels

Querying: LogQL and Kibana

LogQL (Loki)

LogQL is Loki's query language, deliberately modeled on PromQL. Every query starts with a **label filter** (which chunks to read) followed by **line filters** and **parsers** (what to do with the content):

```

# Basic: all error logs from the payment service in the last hour
{service="payment", level="error"}

# Line filter: errors mentioning a specific failure mode
{service="payment", level="error"} |= "insufficient_funds"

# Exclusion: errors that are NOT insufficient_funds (to find other failure modes)
{service="payment", level="error"} != "insufficient_funds"

# JSON parsing: extract fields from JSON log lines, then filter on the m
{service="payment"} | json | level="error" and duration_ms > 500

# Trace correlation: all logs for a specific trace
{service=~"payment|checkout|inventory"} | json | trace_id="4bf92f3577b34da6a3ce929d0e0e4736"

# Regex: extract structured data from semi-structured lines
{app="nginx"} | regexp (?P<method>\w+) (?P<path>/[^\s]+) (?P<status>\d+)

# Pattern parser (faster than regexp for simple patterns)
{app="nginx"} | pattern <method> <path> <status> <duration>

# Metric query: error rate per service over time (for dashboards/alerts)
sum by (service) (rate({level="error"}[5m]))

# Metric query: p99 request duration from log-derived durations
quantile_over_time(0.99, {service="payment"} | json | unwrap duration_ms [5m]) by (service)

# Alerting: error rate exceeds threshold (used in Loki ruler)
sum(rate({service="payment", level="error"}[5m])) > 10

```

The cardinal rule of LogQL performance: always start with the most selective label filter. `{service="payment", level="error"}` reads only chunks for that label set. `{service=~".+"} |= "error"` reads every chunk and scans every line — orders of magnitude slower. Label selectivity is the primary determinant of query latency in Loki.

Kibana / ES|QL (Elasticsearch)

```
// KQL (Kibana Query Language) – Discover view
service: payment and level: error and error: insufficient_funds
trace_id: "4bf92f3577b34da6a3ce929d0e0e4736"

// Lucene syntax – more expressive
service:payment AND level:error AND duration_ms:>500

// ES|QL – SQL-like analytics on log data (Elasticsearch 8.11+)
FROM logs-*
| WHERE service == "payment" AND level == "error"
| STATS error_count = COUNT(*), p99_duration = PERCENTILE(duration_ms,
99) BY error
| SORT error_count DESC
| LIMIT 10

// ES|QL – time-bucketed error rate for dashboard
FROM logs-*
| WHERE @timestamp >= NOW() - 1 hour
| EVAL bucket = DATE_TRUNC(5 minutes, @timestamp)
| STATS errors = COUNT(*) WHERE level == "error" BY bucket, service
| SORT bucket ASC
```

Correlating logs, metrics, and traces

The most powerful incident workflow connects all three pillars via shared identifiers:

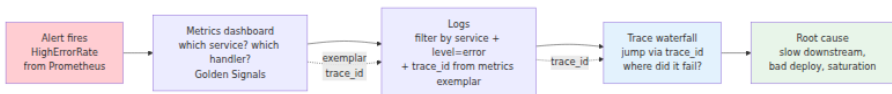


Figure 3-4: The incident workflow across pillars. An alert on a metric leads to log investigation via trace_id correlation, then to a trace waterfall that pinpoints the failing span.

Grafana exemplars make this automatic — a Prometheus histogram can carry trace IDs as exemplars, and Grafana renders them as clickable links from the metrics dashboard directly to the relevant trace and logs:

```
# prometheus.yml – enable exemplars
storage:
  exemplars:
    max_exemplars: 100000
```

```
// Go – attach trace ID as exemplar on histogram observation
import "go.opentelemetry.io/otel/trace"

func handleRequest(w http.ResponseWriter, r *http.Request) {
    start := time.Now()
    // ... handle request ...
    duration := time.Since(start).Seconds()
    traceID := trace.SpanContextFromContext(r.Context()).TraceID().String()
    httpDuration.WithLabelValues(r.Method,
    r.URL.Path).Observe(duration)
    // With OpenTelemetry + Prometheus exemplars, the trace_id is auto-attached
    // via the exemplar storage – no manual wiring needed when using
    OTel SDK
}
```

Retention, sampling, and cost control

The retention problem

Storing every log line from every service at full fidelity for 90 days is rarely affordable or necessary. Different log levels and services have different value over time:

Log category	Value during incident	Value after 7 days	Recommended retention
error / fatal	Critical — primary evidence	High — needed for trend analysis	60-90 days
warn	Useful — shows degraded behavior	Moderate	30 days
info (request logs)	Useful for recent incidents	Low — superseded by metrics	7-14 days
debug	Very useful if enabled during incident	Negligible	1-3 days, or sampled 10%

Log category	Value during incident	Value after 7 days	Recommended retention
Audit / security logs	Critical for compliance	Critical — regulatory requirement	1–7 years (often on S3/GCS archive)
Access logs (edge proxy)	Useful for traffic analysis	Low (metrics cover it)	7 days, or sampled 1–10%

Sampling strategies

Head sampling (decide at log-emission time whether to keep a line) is simple but loses the errors you most need:

```
// Naive: sample all info logs at 10% – but this drops errors too if not careful
if level == slog.LevelInfo && rand.Float64() > 0.1 {
    return // drop 90% of info logs
}
// Never sample error/fatal – always keep them
if level >= slog.LevelError {
    // always emit
}
```

Tail sampling (decide after seeing the outcome whether to keep the full trace/log set) is more powerful — keep all logs for failed requests, sample successful ones:

```
# OpenTelemetry Collector – tail sampling processor
processors:
  tail_sampling:
    decision_wait: 10s
    num_traces: 100000
    expected_new_traces_per_sec: 1000
    policies:
      # Always keep errors
      - name: errors
        type: status_code
        status_code: { status_codes: [ERROR] }
      # Always keep slow requests
      - name: slow
        type: latency
        latency: { threshold_ms: 1000 }
      # Sample 10% of everything else
      - name: probabilistic
        type: probabilistic
        probabilistic: { sampling_percentage: 10 }
```

ILM and retention enforcement

Both ELK and Loki enforce retention via lifecycle policies (shown in full above for both stacks). The key principle: **retention must be automatic and enforced by the storage layer**, not by manual deletion or application logic. An ILM policy that moves data from hot (fast SSD, fully indexed) to warm (fewer replicas, force-merged) to cold (searchable snapshots on S3) to delete ensures cost decreases over time without operator intervention.

For audit and compliance logs that must survive beyond normal retention, add an S3/GCS archival sink that copies every log line to cheap object storage before it ages out of the indexed store:

```
# Vector – archive all logs to S3 before they expire from
Elasticsearch/Loki
sinks:
  s3_archive:
    type: aws_s3
    inputs: [parse_json]
    bucket: "logs-archive-prod"
    key_prefix: "year=%Y/month=%m/day=%d/"
    compression: gzip
    encoding:
      codec: json
    batch:
      max_bytes: 104857600 # 100 MB objects
      timeout_secs: 300
```

Security: redaction, access control, and compliance

Never log secrets

This is the most violated logging rule in production systems. Common leaks:

- Passwords and tokens logged in request bodies or headers
(`Authorization: Bearer eyJ...`).
- PII (email, phone, government IDs) logged verbatim for debugging and never removed.
- Full SQL queries with embedded parameter values.
- Stack traces that include environment variables or configuration dumps.

Enforce redaction at **two layers**: the application (never emit the secret) and the pipeline (catch anything the application missed):

```
// Go – application-layer redaction: never log the raw value
func logAuthAttempt(userID string, token string) {
  // BAD: logger.Info("auth attempt", slog.String("token", token))
  // GOOD: log only the token's presence and prefix for debugging
  logger.Info("auth attempt",
    slog.String("user_id", userID),
    slog.String("token_prefix", token[:4]+"..."),
    slog.Bool("token_present", token != ""),
  )
}
```

```
# Vector/Fluent Bit pipeline – catch-all redaction for anything that
slipped through
transforms:
  redact_secrets:
    type: remap
    inputs: [kubernetes_logs]
    source: |
      # Redact known secret patterns in any field
      .message = replace(string(.message) ?? "", r'Bearer\s+[A-Za-
z0-9\-_\.\.]+\s+', "Bearer ***REDACTED***")
      .message = replace(string(.message) ?? "", r'password["\s:=]+
[^"\s,}]+\s+', "password=***REDACTED***")
      # Hash or drop PII fields
      if exists(.user_email) { .user_email = "***REDACTED***" }
      if exists(.card_number) { .card_number = "***REDACTED***" }
```

Access control

Logs containing PII or business-sensitive data must be access-controlled at the storage layer:

- **Elasticsearch:** index-level and field-level security via the Elastic Stack Security feature (roles that restrict which indices and fields a user can query).
- **Loki:** tenant isolation via `X-Scope-OrgID` header + per-tenant retention and query limits; Grafana data source permissions control which tenants a team can query.
- **Audit:** log every query against sensitive indices — who searched for what, when — to satisfy compliance requirements.

OpenTelemetry Logs: the emerging unified path

OpenTelemetry Logs (stable since 2024) provides a vendor-neutral log data model and collection pipeline that can ship to any backend — Elasticsearch, Loki, or OTLP-native stores:


```

# otel-collector-config.yaml – unified collection for metrics, traces,
and logs
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318
  filelog:
    include: [/var/log/pods/*/*/*.log]
    operators:
      - type: json_parser
        parse_from: body.log
      - type: add
        field: resource["k8s.pod.name"]
        value: EXPR(attributes["k8s.pod.name"])

processors:
  batch:
    timeout: 5s
    send_batch_size: 8192
  k8sattributes:
    auth_type: serviceAccount
    passthrough: false
    extract:
      metadata: [k8s.namespace.name, k8s.pod.name, k8s.container.name]
      labels:
        - tag_name: app
          key: app
          from: pod
  filter/drop_health:
    logs:
      exclude:
        bodies: ["*GET /health*"]
  transform/redact:
    log_statements:
      - context: log
        statements:
          - replace_pattern(body, "Bearer\\s+[A-Za-z0-9\\-_\\.]+",
"Bearer ***REDACTED***")

exporters:
  # Ship to Loki via OTLP (Loki 3.0+ supports OTLP ingestion)
  otlphttp/loki:
    endpoint: http://loki:3100/otlp
    headers:
      X-Scope-OrgID: prod
  # Ship to Elasticsearch via OTLP or elasticsearch exporter
  elasticsearch:

```

```

endpoints: [https://elasticsearch:9200]
user: "${ES_USERNAME}"
password: "${ES_PASSWORD}"
logs_index: logs-otel-default
mapping:
  mode: bodymap

service:
  pipelines:
    logs:
      receivers: [otlp, filelog]
      processors: [k8sattributes, filter/drop_health, transform/
redact, batch]
      exporters: [otlphttp/loki, elasticsearch]
    traces:
      receivers: [otlp]
      processors: [k8sattributes, batch]
      exporters: [otlphttp/loki] # Tempo/Jaeger for traces
    metrics:
      receivers: [otlp]
      processors: [batch]
      exporters: [prometheusremotewrite]

```

The strategic value of OTel Logs is not that it replaces ELK or Loki — it is that it **decouples instrumentation from storage**. Applications instrument once with the OTel SDK, and operators choose (or change) the backend without touching application code.

Logging in a distributed system

The request-scoped correlation problem

In a monolith, a single log file contains all events for a request. In a microservices architecture, a single user action fans out into dozens of log streams across different services, hosts, and storage shards. Reconstructing the full story requires correlation:

1. **Generate a trace ID and request ID at the edge** (API gateway or frontend) and propagate via headers (`traceparent` per W3C Trace Context, `X-Request-ID` for request correlation).
2. **Every service includes both IDs in every log line** — the application logger reads them from context (as shown in the Go/Python examples above).

3. **The query layer joins on trace_id** — `trace_id="4bf92f..."` across all services returns the complete distributed log set for one request.

Without this, incident responders must guess which log lines from which services correspond to the failing request — correlating by timestamp alone is unreliable when clocks skew and request rates are high.

Clock synchronization

Log timestamps are only comparable across services if clocks are synchronized. In Kubernetes, `chrony` or `systemd-timesyncd` with NTP/PTP keeps node clocks within milliseconds. For stricter ordering, use the trace's span timestamps (which share a single clock per trace) rather than log timestamps to sequence cross-service events.

Structured logging and schema governance

At scale, log schema drift — one team renames `user_id` to `userId`, another adds `user_id` as an integer while the rest use strings — breaks dashboards and alerts silently. Governance mechanisms:

- **Shared logging library** — a single internal package that enforces field names, types, and required fields. All services import it.
- **Schema validation in the pipeline** — Vector/OTel transform that rejects or coerces non-conforming log lines, with a dead-letter queue for violations.
- **Elasticsearch `dynamic: strict` mapping** — rejects documents with unexpected fields rather than silently creating new mappings that fragment the index.
- **Linting in CI** — static analysis that flags `log.Print` / `console.log` with unstructured messages and requires structured logger usage.

Common failure modes

Logging synchronously on the hot path. Writing logs synchronously (blocking the request handler until the log line is flushed) adds latency and can deadlock under backpressure. Always log asynchronously — write to a buffered channel or ring buffer, and let a background goroutine/thread flush to stdout. Most structured logging libraries handle this internally, but verify the behavior under load.

Cardinality explosion in Loki labels. Adding `trace_id`, `user_id`, or `request_id` as Loki labels creates one stream per unique value — millions of streams that exhaust memory and make queries unusably slow. These high-cardinality identifiers belong in **structured metadata** (Loki 2.8+) or as **LogQL line filters**, never as labels. Labels should be low-cardinality: `service`, `level`, `namespace`, `app` — values with tens to hundreds of distinct values, not millions.

Index mapping explosion in Elasticsearch. Dynamic mapping that creates a new field for every unique JSON key — especially when application logs contain unbounded keys (e.g., `{"custom_field_12345": "value"}`) — can create thousands of fields per index, exhausting heap and slowing queries. Use `dynamic: strict` or `dynamic: runtime` for untrusted fields, and set `index.mapping.total_fields.limit` to catch runaway mapping growth.

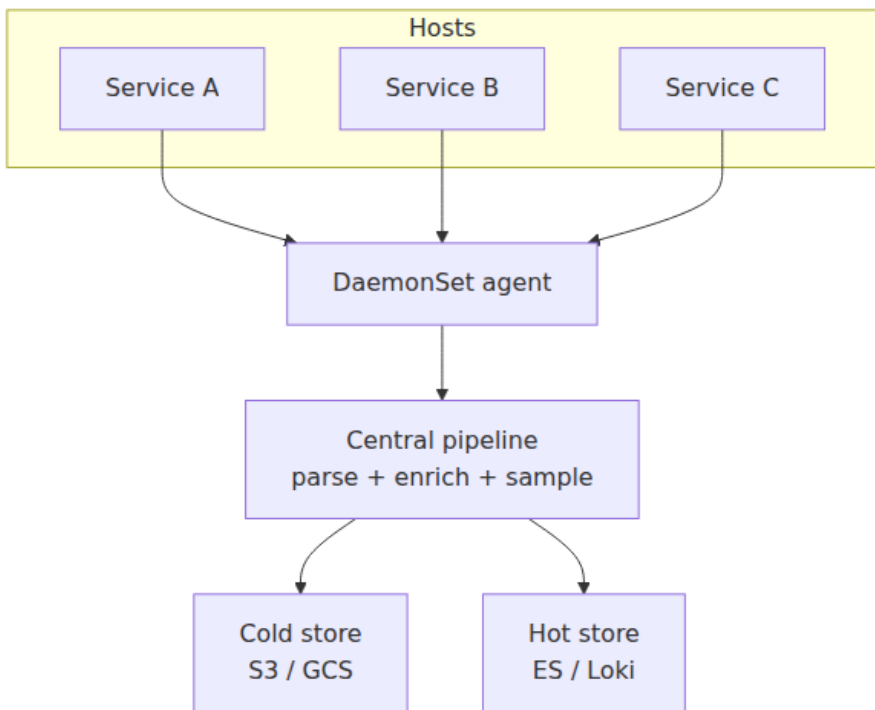
Dropping logs under backpressure. When the central pipeline is overloaded, agents must buffer to disk and retry — not drop. Verify this by load-testing the pipeline: produce logs faster than the pipeline can index, and confirm that no log lines are lost (compare emitted count vs. indexed count). Filebeat's `queue.spool`, Fluent Bit's `storage.type: filesystem`, and Vector's `buffer.type: disk` with `when_full: block` all provide this — but only if configured.

Retaining everything forever. The default should be short retention (7–14 days) for bulk logs with selective long retention for errors and audit logs. Teams that retain all logs at full fidelity for 90 days without sampling or tiering routinely discover that logging is their largest infrastructure cost.

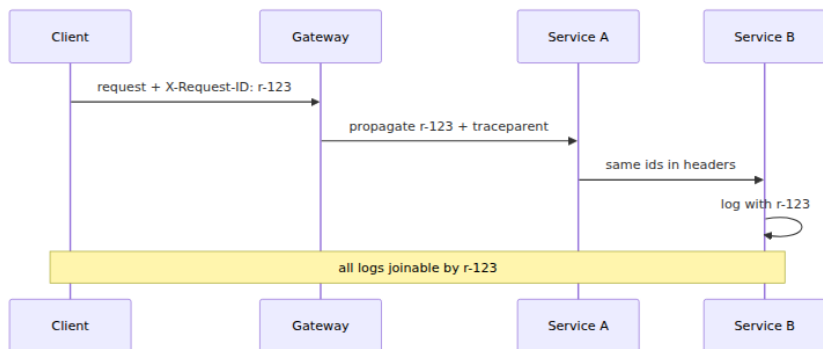
Structured Logging Pipeline



Log Aggregation Architecture



Correlation ID Propagation



Key takeaways

- Every log line at scale must be **structured JSON** with a consistent schema — `timestamp`, `level`, `service`, `message`, `request_id`,

`trace_id`, and relevant domain fields — so that downstream stages can query without per-service parsing.

- Log **levels** control volume: `debug` is off by default, `info` is the bulk, `warn / error` are low-volume but high-value, and sampling should never drop `error / fatal`.
- Collection agents (Filebeat, Fluent Bit, Vector, Alloy) run as **DaemonSets** tailing kubelet-managed files, enriching with Kubernetes metadata, handling multiline stack traces, and **buffering to disk** under backpressure — lossless collection is non-negotiable.
- **Elasticsearch (ELK)** indexes every field for fast arbitrary queries at $\sim 1.5\text{--}2\times$ raw storage cost; **Loki** indexes only labels and stores content as compressed chunks at $\sim 0.1\text{--}0.2\times$ raw cost, trading query flexibility for $10\times$ lower cost — many organizations run both, with Loki for bulk and Elasticsearch for high-value logs needing full-text search.
- **LogQL** queries must start with selective label filters (`{service="payment", level="error"}`) before content scanning; **KQL/ES|QL** queries can filter on any indexed field directly.
- **Trace ID propagation** (W3C `traceparent`) is what makes distributed log correlation possible — every service includes `trace_id` in every log line, and the query layer joins on it to reconstruct the full distributed story for one request.
- **Retention must be automatic and tiered** — hot/warm/cold/delete via ILM (Elasticsearch) or `retention_stream` (Loki) — with short retention for bulk `info / debug` and long retention for `error / audit` logs; S3/GCS archival preserves compliance logs beyond indexed retention.
- **Never log secrets** — enforce redaction at both the application layer (never emit) and the pipeline layer (catch what slipped through) — and never put high-cardinality values (`trace_id`, `user_id`, `request_id`) in Loki labels or Elasticsearch mappings without bounds.
- **OpenTelemetry Logs** decouples instrumentation from storage — instrument once with the OTEL SDK, ship to any backend (ELK, Loki, or both) without changing application code.

Further reading

- Elasticsearch Guide — *Index Lifecycle Management, Data Streams, Index Templates* (<https://www.elastic.co/guide/en/elasticsearch/reference/current/index-lifecycle-management.html>)
- Grafana Loki Documentation — *Architecture, Labels, LogQL, Retention* (<https://grafana.com/docs/loki/latest/fundamentals/overview/>, <https://grafana.com/docs/loki/latest/query/>)
- Grafana Alloy Documentation (<https://grafana.com/docs/alloy/latest/>) — successor to Promtail, OTel-compatible collector for Loki.
- Fluent Bit Documentation — *Kubernetes Filter, Multiline Parser* (<https://docs.fluentbit.io/manual/pipeline/filters/kubernetes>, <https://docs.fluentbit.io/manual/pipeline/parsers>)
- Vector Documentation (<https://vector.dev/docs/>) — high-performance log/metrics pipeline with remap language (VRL).
- OpenTelemetry Logs — *Specification, Collector* (<https://opentelemetry.io/docs/specs/otel/logs/>, <https://opentelemetry.io/docs/collector/>)
- Google SRE Workbook, Chapter 10 — *Monitoring and Alerting* discussion of log volume management.
- Nate Smalley, *The Log Levels You Should Know* and the structured logging literature — practical guidance on level assignment (widely referenced in SRE practice).
- Brice Fernandes, *Logging Best Practices* (<https://betterstack.com/community/guides/logging/>) — comprehensive survey of structured logging, sampling, and retention at scale.

Chapter 4 — Distributed Tracing and OpenTelemetry

What this chapter covers. In a system of dozens or hundreds of services, a single user request fans out into a tree of RPCs, database calls, cache lookups, and async jobs — any of which can be the source of latency or failure. Metrics tell you *that* latency spiked; logs tell you *what each service did*; traces tell you *where time went and which path failed* by stitching the entire request journey into one causal graph. This

chapter covers the full tracing stack from first principles to production operation: the trace/span model, W3C context propagation, the OpenTelemetry project (API, SDK, and Collector), instrumentation patterns in Go, Java and Python, Collector pipelines with real YAML, sampling strategies that control cost without losing the traces you need, and the storage and query layer that makes traces useful during incidents. Every concept is grounded in runnable configuration.

Learning goals — after this chapter you should be able to:

- Explain the trace/span model, parent-child relationships, and why distributed context propagation is the central mechanism that makes tracing work.
 - Describe the W3C Trace Context (`traceparent` / `tracestate`) and Baggage specifications and how they flow through HTTP, gRPC, and async messaging.
 - Instrument services with OpenTelemetry SDKs (manual and automatic) in Go, Java, and Python, adding custom attributes, events, and span links correctly.
 - Write a production OpenTelemetry Collector configuration — receivers, processors (batch, attributes, tail sampling), exporters, and pipelines — for both sidecar/agent and gateway deployment models.
 - Design a sampling strategy (head, tail, adaptive) that keeps cost bounded while preserving error and high-latency traces.
 - Correlate traces with metrics (exemplars) and logs (trace ID injection) and explain how that correlation accelerates incident investigation.
 - Evaluate trace storage backends (Jaeger, Grafana Tempo, vendor SaaS) and query patterns (trace lookup, service map, latency histogram) for a given scale.
-

Why tracing

The gap that metrics and logs leave

Chapter 2 (metrics) and Chapter 3 (logging) give you two powerful lenses:

Signal	Answers	Fails when
Metrics	"Is the service healthy? How many requests are failing?"	You need to know <i>which</i> downstream call in a 12-service fan-out is slow
Logs	"What did this service do for request X?"	Request X touched 8 services — you must manually join 8 log streams by request ID

Tracing fills the gap by recording the **causal structure** of a request. A single trace captures:

- Every service the request visited and how they called each other (the call graph).
- How long each operation took and where time was actually spent (latency breakdown).
- Which operation returned an error and how that error propagated (failure path).

Without tracing, debugging a p99 latency regression in a microservice architecture is a manual exercise in correlating timestamps across dashboards. With tracing, it is a single query: show me the slowest spans in traces where `http.route = /checkout` over the last hour.

When tracing pays for itself

Tracing is not free — instrumentation adds latency, the Collector pipeline consumes resources, and trace storage is significant. It pays for itself when:

- **Request fan-out is deep.** A request that touches 3 services can be debugged with logs alone. A request that fans out to 15 services with parallel branches cannot.
- **Latency is a product requirement.** If p99 latency has an SLO (see Chapter 1), you need traces to attribute latency to the correct component.
- **Ownership is distributed.** When each service is owned by a different team, a trace is the shared artifact that lets teams collaborate without passing log snippets back and forth.

- **Async work is involved.** Queues, background jobs, and event-driven workflows break the request/response model that logs assume. Traces with span links connect the async pieces.

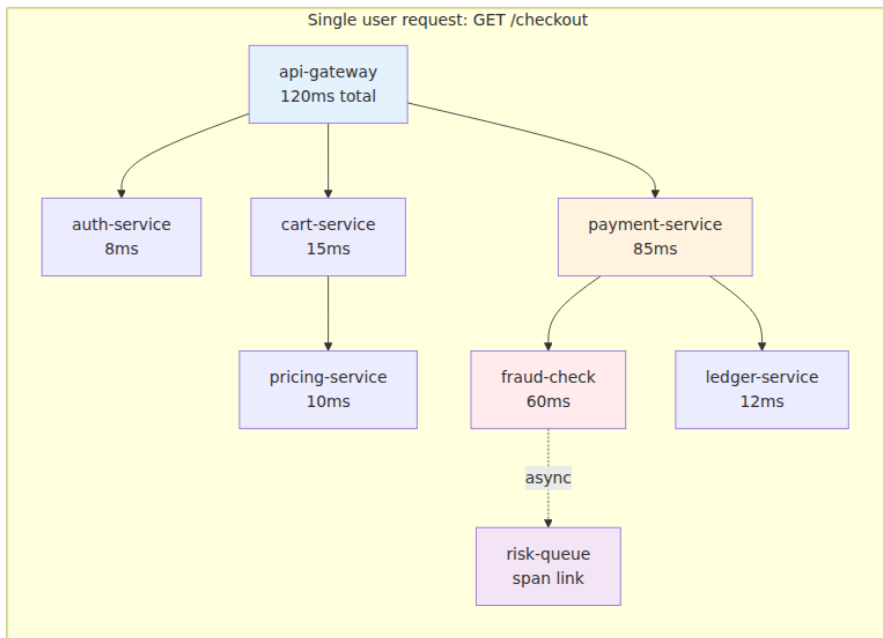


Figure 4-1: A single checkout request as a trace. The trace reveals that 60 ms of the 85 ms in payment-service was spent in fraud-check — the obvious place to investigate a latency regression. Without the trace you would see only that payment-service is slow.

The trace data model

Spans, traces, and relationships

The OpenTelemetry data model (inheriting from Dapper and OpenTracing) has three core concepts:

Span — a named, timed operation. Every span carries:

- **name** — operation name (e.g., `GET /checkout`, `SELECT orders`, `fraud.evaluate`).

- `trace_id` — 128-bit identifier shared by all spans in one trace (32 hex chars, e.g., `4bf92f3577b34da6a3ce929d0e0e4736`).
- `span_id` — 64-bit identifier unique within the trace (16 hex chars).
- `parent_span_id` — the span that caused this one. The root span has no parent.
- `start_time` / `end_time` — nanosecond timestamps. Duration is derived.
- `attributes` — key-value pairs (e.g., `http.method=GET` , `db.system=postgresql` , `user.id=42`).
- `status` — `Unset` , `Ok` , or `Error` (with optional description).
- `events` — timestamped annotations within a span (e.g., `cache miss` , `retry attempt 2`).
- `links` — references to other spans/traces (used for async and batch patterns).

Trace — the tree (or DAG, when links are used) of all spans sharing a `trace_id`. The trace is the unit of storage and query — you fetch an entire trace to understand one request.

Span context — the triple (`trace_id`, `span_id`, `trace_flags`) that must be propagated from caller to callee so the callee can create a child span linked to its parent.

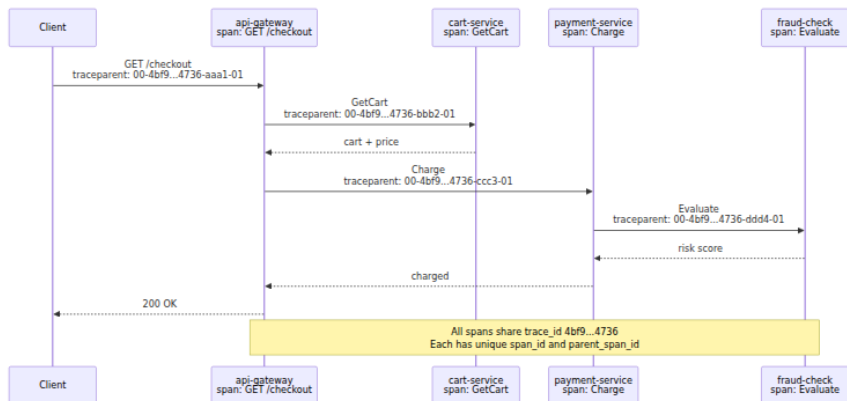


Figure 4-2: Context propagation across a request. Each hop forwards the W3C traceparent header carrying `trace_id`, `parent span_id`, and sampling flags so the callee can join the same trace.

Span kinds

OTel defines five span kinds that give structure to the trace graph:

Kind	Meaning	Example
SERVER	Handling an incoming request	HTTP handler, gRPC service method
CLIENT	Making an outgoing request	HTTP client call, gRPC stub invocation
PRODUCER	Sending a message to a broker	Publishing to Kafka, SQS SendMessage
CONSUMER	Receiving/processing a message	Kafka consumer poll, SQS handler
INTERNAL	Work inside a service	Function call, DB query wrapper

Correct span kinds matter because visualization tools (Jaeger, Grafana) use them to render the service map and to distinguish synchronous RPCs from async messaging.

Semantic conventions

OTel defines [Semantic Conventions](#) — standard attribute names that make traces queryable across services regardless of language:

```
// HTTP server span attributes (semconv)
http.method = "GET"
http.route = "/checkout"
http.status_code = 200
http.scheme = "https"
net.host.name = "api-gateway-7df9b"
net.peer.ip = "10.2.3.44"

// Database span
db.system = "postgresql"
db.name = "orders"
db.statement = "SELECT * FROM orders WHERE id = $1"
db.operation = "SELECT"

// Messaging span
messaging.system = "kafka"
messaging.destination = "risk-events"
messaging.operation = "publish"
messaging.kafka.partition = 3
```

Using `semconv` means a query like `db.system = "postgresql" AND db.statement CONTAINS "orders"` works across every service that touches Postgres, even if they are written in different languages by different teams.

Context propagation

Context propagation is the mechanism that turns isolated per-service spans into a distributed trace. Without it, each service creates disconnected spans that cannot be correlated.

W3C Trace Context

The [W3C Trace Context](#) specification defines two HTTP headers:

traceparent — carries the span context:

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
              | |                                     | |
              | trace_id (128-bit, 32 hex)           | span_id (64-bit)
trace_flags (sampled=1)
              └─ version (00)
```

- **version** — always `00` today; future versions may add fields after the flags.

- `trace_flags` — bit 0 is `sampled` (1 = this trace should be recorded). Other bits reserved.

`tracestate` — vendor-specific trace data (e.g., tenant, sampling decisions) carried alongside `traceparent`:

```
tracestate: rojo=00f067aa0ba902b7,congo=t61rcWkgMzE
```

`tracestate` is how tail-sampling decisions and multi-tenant routing hints propagate without polluting application headers. Each vendor uses a key (e.g., `rojo`) with an opaque value.

`baggage` — separate from trace context, [W3C_Baggage](#) carries application-level key-value pairs that travel with the trace but are not part of the trace identity:

```
baggage: user_id=42,tenant=acme,feature_flag=checkout_v2
```

Baggage is powerful (it lets downstream services read `tenant` or `feature_flag` without an extra lookup) but must be used carefully — every baggage key is propagated to every downstream service, so unbounded baggage bloats every request.

Propagation through different transports

Transport	How traceparent is carried	Notes
HTTP/1.1, HTTP/2	<code>traceparent</code> / <code>tracestate</code> headers	Automatic with OTel HTTP instrumentation
gRPC	Same headers as HTTP/2 (gRPC is HTTP/2)	Automatic with OTel gRPC interceptors
Kafka / AMQP	Message headers (record headers)	Requires explicit propagator; not automatic
Async jobs / queues	Injected into job payload or message headers	Application must propagate manually or via SDK helper
Databases	Not propagated (DB is leaf span)	No downstream to propagate to

For async messaging the propagation pattern looks like:

```

# Producer: inject context into message headers
from opentelemetry import propagate

headers = {}
propagate.inject(headers) # writes traceparent + tracestate + baggage
kafka_producer.send("risk-events", value=payload, headers=headers)

# Consumer: extract context and create linked span
from opentelemetry import trace, propagate
from opentelemetry.trace.propagation import get_current_span

ctx = propagate.extract(headers) # reconstructs span context
# Option 1: child span (if consumer is direct continuation)
with trace.get_tracer(__name__).start_as_current_span(
    "process risk-event", context=ctx, kind=trace.SpanKind.CONSUMER
):
    handle_event(payload)

# Option 2: span link (if consumer is async / batch – preferred for
# queues)
with trace.get_tracer(__name__).start_as_current_span("process risk-
event") as span:
    span.add_link(get_current_span(ctx).get_span_context())
    handle_event(payload)

```

The distinction between **child span** and **span link** matters: a child implies the producer causally triggered this specific consumer execution. A link says "this consumer processing is related to that producer trace" without claiming direct parentage — correct for batch consumers that process many producer traces in one poll.

B3 and other propagators

Some systems still use the [B3 propagation](#) format from Zipkin:

```

X-B3-TraceId: 4bf92f3577b34da6a3ce929d0e0e4736
X-B3-SpanId: 00f067aa0ba902b7
X-B3-Sampled: 1

```

OTel supports multiple propagators simultaneously via the composite propagator — essential during migration:

```
# otel-collector.yaml – accept both W3C and B3
receivers:
  otlp:
    protocols:
      http:
        include_metadata: true
```

```
// Go: accept W3C + B3 on incoming requests
import (
  "go.opentelemetry.io/contrib/propagators/b3"
  "go.opentelemetry.io/otel/propagation"
)
propagation.NewCompositeTextMapPropagator(
  propagation.TraceContext{}, // W3C
  propagation.Baggage{},
  b3.New(),                  // Zipkin B3 (both single and multi-
// header)
)
```

In SDK configuration, always propagate W3C as the primary format and add B3 only for compatibility with legacy services. New services should emit W3C exclusively.

OpenTelemetry: architecture

OpenTelemetry (OTel) is the CNCF project that unifies tracing, metrics, and logs under one set of APIs, SDKs, and a collector. It is the successor to OpenTracing and OpenCensus (both now deprecated) and is the industry standard for instrumentation.

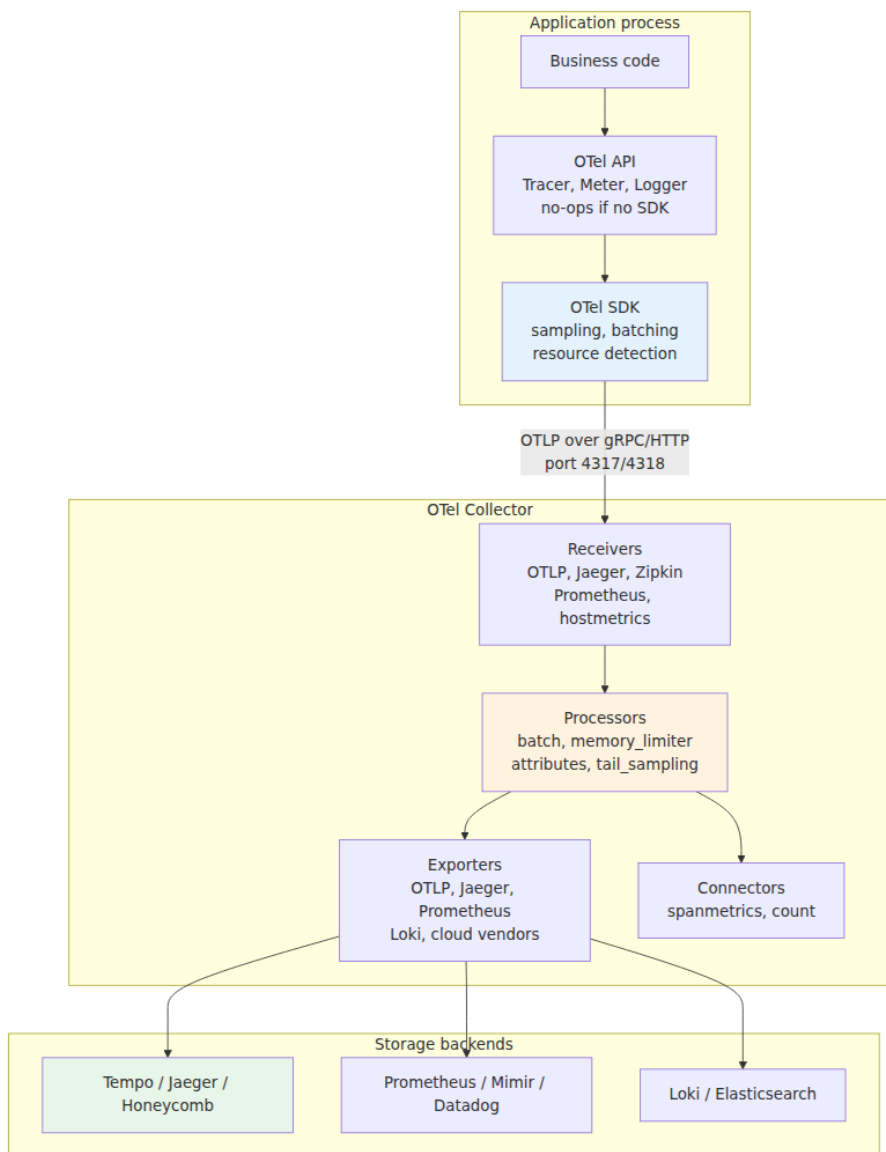


Figure 4-3: OpenTelemetry end-to-end architecture. Application code calls the API, the SDK batches and samples, the Collector receives/processes/exports, and storage backends serve queries. The API/SDK split means uninstrumented code has near-zero overhead.

API vs SDK separation

This split is deliberate and important:

- **API** — interfaces (`Tracer` , `Meter` , `Logger`) that library and application code calls. The API package has no implementation — if no SDK is registered, every call is a no-op. This means libraries can instrument themselves without forcing a dependency on a specific SDK or backend.
- **SDK** — the implementation that does real work: sampling decisions, span batching, attribute processing, resource detection, and export. Applications register the SDK at startup; libraries never do.

This means you can add OTel to a shared library (`import "go.opentelemetry.io/otel"`) without worrying about which tracing backend the final application uses.

The Collector

The [OTel Collector](#) is a standalone binary (often deployed as a sidecar, DaemonSet, or gateway) that receives, processes, and exports telemetry. It is the central control plane for observability data:

- **Decouples applications from backends.** Applications always export OTLP to the Collector; the Collector fans out to whatever backends you use. Switching from Jaeger to Tempo requires changing Collector config, not application code.
- **Centralizes processing.** Sampling, attribute enrichment, filtering, and redaction happen in one place with one configuration — not scattered across every service.
- **Handles backpressure and reliability.** The Collector buffers, retries, and applies backpressure so a slow backend does not block application threads.

Collector deployment models:

Model	Where it runs	Use case
Agent (sidecar/ DaemonSet)	Alongside each application pod or per-node	Lightweight processing, host resource detection, local batching

Model	Where it runs	Use case
Gateway	Central cluster (often scaled horizontally)	Heavy processing (tail sampling), fan-out to multiple backends, cross-service correlation
Agent + Gateway	Both (recommended at scale)	Agents handle local concerns, gateways handle global concerns

At scale, the agent+gateway pattern is standard: agents do cheap per-span work (batching, resource detection), gateways do expensive cross-trace work (tail sampling).

Instrumentation

Automatic instrumentation

Most OTel SDKs offer auto-instrumentation that wraps common frameworks without code changes:

Java — the most mature auto-instrumentation, via a `-javaagent`:

```
# No code changes – add the agent jar
java -javaagent:opentelemetry-javaagent.jar \
-Dotel.service.name=payment-service \
-Dotel.exporter.otlp.endpoint=http://otel-collector:4317 \
-Dotel.traces.sampler=parentbased_traceidratio \
-Dotel.traces.sampler.arg=0.1 \
-jar payment-service.jar
```

The Java agent instruments Servlet, Spring MVC/WebFlux, gRPC, JDBC, Redis (Lettuce/Jedis), Kafka, and more automatically.

Python — via `opentelemetry-instrument`:

```
opentelemetry-bootstrap -a install # auto-detect libraries
opentelemetry-instrument \
--traces_exporter otlp \
--exporter_otlp_endpoint http://otel-collector:4317 \
--service_name cart-service \
python -m myapp
```

Auto-instruments Flask, Django, FastAPI, requests, psycpg2, pymongo, Celery, and others.

Go — no runtime auto-instrumentation (Go lacks bytecode manipulation). Instrumentation is manual or via compile-time wrapping with `otelhttp`, `otelgrpc`, `otelsql`:

```
import (
    "go.opentelemetry.io/contrib/instrumentation/net/http/otelhttp"
    "go.opentelemetry.io/contrib/instrumentation/google.golang.org/
    grpc/otelgrpc"
)

// HTTP server – one line wraps the handler
handler := otelhttp.NewHandler(http.HandlerFunc(checkoutHandler), "GET
/checkout")
http.Handle("/checkout", handler)

// HTTP client – one line wraps the transport
client := &http.Client{Transport: otelhttp.NewTransport(http.DefaultTra
nsport)}

// gRPC – interceptor on dial and server
conn, _ := grpc.Dial(target,
    grpc.WithStatsHandler(otelgrpc.NewClientHandler()),
)
grpc.NewServer(grpc.StatsHandler(otelgrpc.NewServerHandler()))
```

Manual instrumentation

Auto-instrumentation covers framework boundaries. Custom business logic needs manual spans:

Go:

```

import (
    "context"
    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/attribute"
    "go.opentelemetry.io/otel/codes"
    "go.opentelemetry.io/otel/trace"
)

var tracer = otel.Tracer("payment-service/charge")

func Charge(ctx context.Context, req ChargeRequest) error {
    ctx, span := tracer.Start(ctx, "payment.Charge",
        trace.WithAttributes(
            attribute.String("payment.method", req.Method),
            attribute.Int64("payment.amount_cents", req.AmountCents),
        ),
    )
    defer span.End()

    // Add events for notable moments within the span
    span.AddEvent("validating payment", trace.WithAttributes(
        attribute.String("card.last4", req.CardLast4),
    ))

    result, err := fraudCheck(ctx, req) // ctx carries span context
    if err != nil {
        span.RecordError(err)
        span.SetStatus(codes.Error, err.Error())
        return err
    }

    span.SetAttributes(attribute.String("fraud.verdict",
result.Verdict))
    span.SetStatus(codes.Ok, "")
    return nil
}

func fraudCheck(ctx context.Context, req ChargeRequest) (*FraudResult,
error) {
    // Child span – automatically parented via ctx
    _, span := tracer.Start(ctx, "fraud.Evaluate")
    defer span.End()
    // ... call fraud service
    return result, nil
}

```

Python:

```

from opentelemetry import trace
from opentelemetry.trace import Status, StatusCode

tracer = trace.get_tracer("payment.charge")

def charge(req: ChargeRequest) -> None:
    with tracer.start_as_current_span(
        "payment.Charge",
        attributes={
            "payment.method": req.method,
            "payment.amount_cents": req.amount_cents,
        },
    ) as span:
        span.add_event("validating payment", {"card.last4": req.card_la
st4})

        try:
            result = fraud_check(req) # context propagated implicitly
            span.set_attribute("fraud.verdict", result.verdict)
            span.set_status(Status(StatusCode.OK))
        except Exception as exc:
            span.record_exception(exc)
            span.set_status(Status(StatusCode.ERROR, str(exc)))
            raise

```

Instrumentation guidelines

- **Span names should be low-cardinality.** Use `GET /checkout` or `payment.Charge`, not `GET /checkout?user=42`. High-cardinality span names make trace storage and indexing expensive and queries useless.
- **Attributes carry the high-cardinality data.** Put `user.id`, `order.id`, `http.target` in span attributes where they are queryable but do not affect span name indexing.
- **Record errors with `RecordError` / `record_exception`, not just status.** The exception event captures stack trace and type; status alone loses that.
- **Do not create spans for trivial operations.** A span per function call in a tight loop creates millions of spans per trace. Span per RPC, per DB query, per meaningful business step.
- **Propagate context through async boundaries explicitly.** Thread pools, goroutines, and queue consumers do not automatically carry trace context — you must pass `ctx` or inject/extract headers.

The Collector in depth

Pipeline model

Every Collector pipeline has three stages:

```
Receivers → Processors → Exporters
(in)      (transform)   (out)
```

Data flows through pipelines of a single signal type (traces, metrics, or logs). Connectors bridge signals (e.g., `spanmetrics` generates metrics from trace spans).

Production Collector configuration

Below is a production-grade Collector configuration covering the agent+gateway pattern. The agent config is lightweight; the gateway handles tail sampling and multi-backend export.

Agent (DaemonSet / sidecar) — `otel-agent.yaml` :

```

receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

    # Collect host metrics for resource correlation
  hostmetrics:
    collection_interval: 30s
    scrapers:
      cpu: {}
      memory: {}
      load: {}

processors:
  # Detect k8s pod / node / deployment from env
  resourcedetection:
    detectors: [env, system, gcp, eks, aks]
    timeout: 5s

  k8sattributes:
    auth_type: serviceAccount
    passthrough: false
    extract:
      metadata:
        - k8s.pod.name
        - k8s.pod.uid
        - k8s.deployment.name
        - k8s.namespace.name
        - k8s.node.name
    pod_association:
      - sources:
          - from: resource_attribute
            name: k8s.pod.ip
        - sources:
          - from: resource_attribute
            name: k8s.pod.uid
      - sources:
          - from: connection

  # Enrich every span with cluster-level attributes
  resource:
    attributes:
      - key: deployment.environment
        value: production
        action: upsert
      - key: service.version
        from_attribute: app.version

```



```

    action: upsert

memory_limiter:
  check_interval: 1s
  limit_mib: 512
  spike_limit_mib: 128

batch:
  timeout: 5s
  send_batch_size: 512
  send_batch_max_size: 1024

# Drop health-check spans that add noise
filter:
  traces:
    exclude:
      match_type: strict
      bodies:
        - '.*"http.route": "/healthz".*'
        # Better: filter on span attributes (OTTL)
    spans:
      - 'attributes["http.route"] == "/healthz" or attributes["http.r
oute"] == "/readyz"'

exporters:
  otlp/gateway:
    endpoint: otel-gateway.observability.svc.cluster.local:4317
    tls:
      insecure: false
      ca_file: /etc/otel/certs/ca.crt
      cert_file: /etc/otel/certs/tls.crt
      key_file: /etc/otel/certs/tls.key
    sending_queue:
      enabled: true
      num_consumers: 4
      queue_size: 5000
    retry_on_failure:
      enabled: true
      initial_interval: 5s
      max_interval: 30s
      max_elapsed_time: 300s

# Also expose Prometheus metrics from the agent itself
prometheus:
  endpoint: 0.0.0.0:8889

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, k8sattributes, resourcedetection, re

```

```

source, filter, batch]
  exporters: [otlp/gateway]
metrics:
  receivers: [otlp, hostmetrics]
  processors: [memory_limiter, resourcedetection, resource, batch]
  exporters: [otlp/gateway]

telemetry:
  logs:
    level: info
  metrics:
    readers:
      - pull:
          exporter:
            prometheus:
              host: 0.0.0.0
              port: 8889

```

Gateway — otel-gateway.yaml :

```

receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
        max_recv_msg_size_mib: 16
      http:
        endpoint: 0.0.0.0:4318

processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 4096
    spike_limit_mib: 512

  batch:
    timeout: 10s
    send_batch_size: 1024

# Tail sampling – the key cost control for tracing
tail_sampling:
  decision_wait: 10s # how long to wait for all spans in a
trace
  num_traces: 100000 # traces held in memory for decision
  expected_new_traces_per_sec: 5000
  policies:
    # Always keep errors and slow traces
    - name: errors
      type: status_code
      status_code: {status_codes: [ERROR]}
    - name: slow-traces
      type: latency
      latency: {threshold_ms: 1000}
    # Keep a sample of health + probe traces for baseline
    - name: probabilistic
      type: probabilistic
      probabilistic: {sampling_percentage: 5}
    # Keep traces for a specific tenant under investigation
    - name: debug-tenant
      type: string_attribute
      string_attribute: {key: tenant, values: [acme-debug]}
    # Composite: slow traces filtered further by attributes
    - name: slow-checkout
      type: composite
      composite:
        max_total_spans_per_second: 1000
        policy_order: [is_checkout, is_slow]
        composite_sub_policy:
          - name: is_checkout
            type: string_attribute

```

```

        string_attribute: {key: http.route, values: ["/checkout"]}
    }

    - name: is_slow
      type: latency
      latency: {threshold_ms: 500}

# Redact sensitive attributes before export
attributes/redact:
  actions:
    - key: user.email
      action: delete
    - key: http.request.header.authorization
      action: delete
    - key: db.statement
      action: hash # keep cardinality signal without leaking query
text

# Add cluster-level resource attributes
resource:
  attributes:
    - key: collector.gateway
      value: otel-gateway
      action: upsert

# Generate RED metrics from spans – no separate instrumentation
needed
# (configured as a connector, not processor, in newer Collector
versions)

connectors:
  spanmetrics:
    namespace: traces_spanmetrics
    dimensions:
      - name: http.method
      - name: http.route
      - name: service.name
    aggregation_temporality: DELTA
    latency_histogram_bins: [2ms, 10ms, 50ms, 100ms, 250ms, 500ms, 1000
ms, 2500ms]

count:
  spans:
    payment.errors:
      description: "Count of error spans in payment-service"
      conditions:
        - 'attributes["service.name"] == "payment-service" and status
.code == STATUS_CODE_ERROR'

exporters:
  # Primary trace store – Grafana Tempo
  otlp/tempo:

```

```

    endpoint: tempo.observability.svc.cluster.local:4317
    tls: {insecure: true}
    sending_queue: {enabled: true, queue_size: 5000}
    retry_on_failure: {enabled: true, initial_interval: 5s,
max_interval: 30s}

# Fallback / secondary – keep a copy in Jaeger for real-time
debugging
otlp/jaeger:
    endpoint: jaeger-collector.observability.svc.cluster.local:4317
    tls: {insecure: true}

# Span-derived metrics to Prometheus / Mimir
prometheusremotewrite:
    endpoint: http://mimir.observability.svc.cluster.local:9009/api/v1/
push
    tls: {insecure: true}

# Debug exporter for troubleshooting (disable in production)
# debug:
#   verbosity: basic
#   sampling_initial: 5
#   sampling_thereafter: 100

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, tail_sampling, attributes/redact, re
source, batch]
      exporters: [otlp/tempo, otlp/jaeger]
    traces/spanmetrics:
      receivers: [otlp]
      processors: [batch]
      exporters: [spanmetrics] # connector: traces → metrics
    metrics:
      receivers: [spanmetrics]
      processors: [batch]
      exporters: [prometheusremotewrite]
    metrics/count:
      receivers: [count]
      processors: [batch]
      exporters: [prometheusremotewrite]

```

Key design decisions in this configuration:

- **Health-check filtering at the agent** prevents `/healthz` spans from ever reaching the gateway — at 10 probes/second per pod across 500 pods, that is 5,000 spans/second of pure noise.

- **Tail sampling at the gateway** makes the keep/drop decision after seeing the entire trace. Head sampling (decided at the SDK) would have to decide at trace start, before knowing whether the trace contains an error. Tail sampling keeps 100% of error traces regardless of the base sampling rate.
- **Attribute redaction** (`user.email` , `Authorization` header) happens before export — sensitive data never reaches storage.
- **spanmetrics connector** generates RED metrics (rate, error, duration) from spans automatically. This means you get service-level latency histograms and error rates without separate metric instrumentation — though dedicated metric instrumentation (Chapter 2) is still preferred for accuracy.

Sampling

Tracing every request at full fidelity is rarely affordable. A service handling 10,000 req/s that creates 5 spans per request produces 50,000 spans/s. At ~1 KB per span, that is ~4 TB/day for one service. Sampling is how you bound that cost.

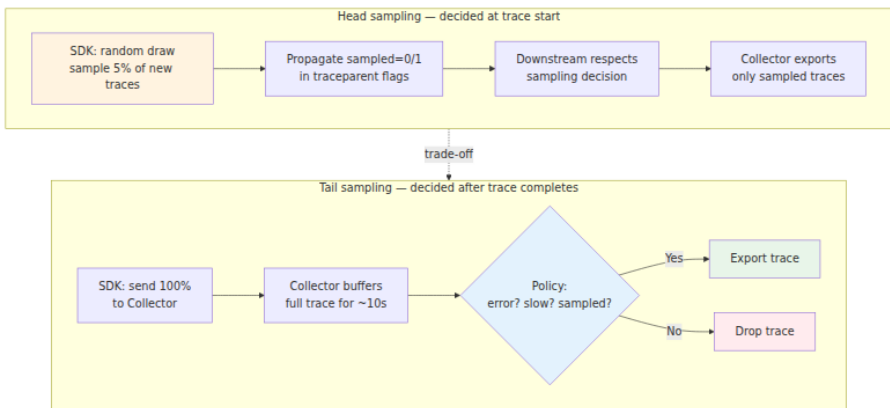


Figure 4-4: Head sampling (cheap, early decision, may drop errors) vs tail sampling (expensive, late decision, keeps all interesting traces). In practice most production systems use tail sampling at the gateway with a head-sampling fallback.

Sampling strategies compared

Strategy	Where decided	Cost	Keeps errors?	Keeps slow traces?	Complexity
Always on	SDK	Highest — 100% of spans	Yes	Yes	Trivial
Probabilistic head (e.g., 5%)	SDK, at trace start	Low — 5% of spans	No — errors in unsampled traces are lost	No	Low
Parent-based	SDK, inherits from parent	Low	Depends on root decision	Depends on root	Low
Rate limiting	SDK or Collector	Bounded — N traces/s	No guarantee	No guarantee	Low
Tail sampling	Collector, after trace completes	Higher — all spans reach Collector, memory for buffering	Yes — policy keeps all errors	Yes — latency policy	Medium — needs gateway memory
Adaptive	Collector, adjusts rate dynamically	Bounded + smart	Yes, with priority	Yes, with priority	High — needs feedback loop

Head sampling configuration

Head sampling is configured in the SDK. It is the fallback when the Collector is unavailable or when you need hard cost bounds:

```
// Go – parent-based with 5% probabilistic fallback for root spans
import (
    sdktrace "go.opentelemetry.io/otel/sdk/trace"
)

tp := sdktrace.NewTracerProvider(
    sdktrace.WithSampler(sdktrace.ParentBased(
        sdktrace.TraceIDRatioBased(0.05), // 5% for new traces
    )),
    sdktrace.WithBatcher(exporter),
)
```

```
# Collector – equivalent head sampling (applied at receive time)
processors:
    probabilistic_sampler:
        sampling_percentage: 5
```

Tail sampling — the production choice

Tail sampling buffers every trace for a window (typically 10–30 seconds) then applies policies to decide which traces to keep. The policies shown in the gateway config above illustrate the pattern:

- **status_code: ERROR** — keep every trace containing an error span. Non-negotiable — you never want to lose the trace that explains an outage.
- **latency: threshold_ms: 1000** — keep every trace where any span exceeds 1 second. Captures latency regressions even when no error occurred.
- **probabilistic: 5** — keep 5% of remaining traces as a baseline for normal behavior comparison.
- **Composite policies** — combine conditions for precise targeting (e.g., slow /checkout traces specifically).

The cost of tail sampling is memory: the gateway must hold `num_traces` traces in RAM for `decision_wait` seconds. For a system doing 5,000 traces/second with 10-second `decision_wait`, that is 50,000 traces in memory simultaneously. At ~10 KB per trace (multiple spans), that is ~500 MB — which is why the gateway `memory_limiter` is critical.

Adaptive and intelligent sampling

At very large scale, static tail sampling is not enough. Adaptive approaches include:

- **Rate-adaptive sampling** — the Collector monitors export throughput and adjusts the probabilistic percentage to stay within a target (e.g., 100 traces/second per service).
- **Error-biased sampling** — always keep errors, sample successes at a lower rate, sample slow successes at a higher rate.
- **Span-level filtering** — drop noisy child spans (e.g., cache lookups that succeed) even within a kept trace to reduce storage without losing the important spans.

The principle: **spend your trace budget on traces that teach you something** — errors, latency outliers, and traces for services under active investigation. Uniform random sampling wastes budget on healthy traces that look the same.

Correlating traces with metrics and logs

Traces in isolation are useful; traces correlated with metrics and logs are transformative during incidents.

Trace → metrics: exemplars and spanmetrics

Exemplars attach a trace ID to a metric data point, so a latency histogram bucket can link directly to the traces that fell in that bucket:

```
# Prometheus histogram with exemplar
http_server_duration_seconds_bucket{le="0.25",route="/checkout"} 4821
# {trace_id="4bf92f3577b34da6a3ce929d0e0e4736"} 0.234

http_server_duration_seconds_bucket{le="0.5",route="/checkout"} 5103
# {trace_id="0813a1f72d2e4a1b8c9d0e7f6a5b4c3d2"} 0.487
```

In Grafana, clicking the exemplar dot in a latency panel jumps directly to the trace. The `spanmetrics` connector in the gateway config generates these automatically from spans — every span's duration becomes a histogram observation tagged with the trace that produced it.

Span-derived metrics (via the `spanmetrics` connector) generate RED metrics without separate instrumentation:

```
# Error rate derived from spans – no separate metric instrumentation
sum(rate(traces_spanmetrics_calls_total{status_code="STATUS_CODE_ERROR"}[5m]))
/
sum(rate(traces_spanmetrics_calls_total[5m]))

# p99 latency derived from spans
histogram_quantile(0.99,
  sum(rate(traces_spanmetrics_latency_bucket[5m])) by (le,
    service_name)
)
```

These are convenient but less accurate than dedicated metric instrumentation (Chapter 2) because they inherit sampling — if you sample at 5%, your span-derived metrics have 20× less data than real metrics.

Trace → logs: trace ID injection

Every log line should carry `trace_id` and `span_id` so that searching for `trace_id=4bf9...4736` across all services reconstructs the full log story for one trace. Most OTel SDKs and log bridges do this automatically:

```
// Go – slog with trace correlation
import (
    "log/slog"
    "go.opentelemetry.io/contrib/bridges/otelslog"
    "go.opentelemetry.io/otel/trace"
)

// otelslog bridge injects trace_id/span_id automatically
logger := otelslog.NewLogger("payment-service")

// Or manual injection:
func logWithTrace(ctx context.Context, msg string) {
    sc := trace.SpanContextFromContext(ctx)
    slog.Info(msg,
        "trace_id", sc.TraceID().String(),
        "span_id", sc.SpanID().String(),
        "trace_sampled", sc.IsSampled(),
    )
}
```

```
# Python – structlog with trace correlation
import structlog
from opentelemetry import trace

def add_trace_info(_, __, event_dict):
    span = trace.get_current_span()
    ctx = span.get_span_context()
    if ctx.is_valid:
        event_dict["trace_id"] = format(ctx.trace_id, "032x")
        event_dict["span_id"] = format(ctx.span_id, "016x")
        event_dict["trace_sampled"] = ctx.trace_flags.sampled
    return event_dict

structlog.configure(processors=[add_trace_info, structlog.processors.JSONRenderer()])
```

```
# Collector – ensure trace_id flows into log records (OTel Logs)
processors:
  transform/logs:
    log_statements:
      - context: log
        statements:
          - set(attributes["trace_id"], trace_id.string) where
            trace_id != nil
          -
            set(attributes["span_id"], span_id.string) where span_id != nil
```

In the query layer, this enables the golden incident workflow:

1. Alert fires on a metric threshold (Chapter 2).
2. Click exemplar → jump to a representative trace.
3. From the trace, click any span → jump to logs for that `trace_id + span_id` in that service.
4. Read the error log line that explains the failure.

This three-way correlation (metrics → traces → logs) is why OTel unifies all three signals under one context propagation mechanism.

Storage and query

Backend options

Backend	Architecture	Query model	Strengths	Trade-offs
Jaeger	Cassandra / Elasticsearch / Badger	Trace ID lookup, service/operation search, duration filter	Mature, simple, good for small-medium scale	Elasticsearch cost at large scale; limited TraceQL
Grafana Tempo	Object storage (S3/GCS) + optionalTraceQL	TraceQL — powerful trace query language	Extremely cost-efficient (object storage), scales to massive trace volume	Newer, fewer operational guides
Honeycomb	Columnar SaaS	BubbleUp, derived columns, SLO heatmaps	Best-in-class query UX, designed for high-cardinality	Vendor lock-in, cost at scale
Datadog / New Relic	SaaS	Integrated APM (traces + metrics + logs)	Single pane of glass, zero storage ops	Highest cost, deepest lock-in
ClickHouse	ClickHouse	SQL over trace tables	Flexible, cost-efficient if you already run ClickHouse	You operate it

For most teams, **Tempo + Grafana** is the recommended starting point for self-hosted, and a SaaS APM for teams that prefer not to operate storage.

TraceQL and Jaeger queries

TraceQL (Tempo) lets you search traces by span attributes, duration, and structure — far beyond simple trace ID lookup:

```
# Find slow checkout traces that hit payment-service
{ resource.service.name = "payment-service" && span.http.route = "/checkout" && duration > 500ms }

# Find traces with an error in any span, grouped by root service
{ status = error } | by(resource.service.name) | count() > 10

# Find traces where fraud-check was called and took > 100ms
{ span.name = "fraud.Evaluate" && duration > 100ms }

# Structural query: traces where cart-service called pricing-service
{ resource.service.name = "cart-service" } >> { resource.service.name = "pricing-service" }
```

Jaeger queries are more limited but still effective:

```
# Jaeger UI search
Service: payment-service
Operation: Charge
Tags: http.route="/checkout" error=true
Min Duration: 500ms
Lookback: 1h
Limit: 20
```

Service map

Both Jaeger and Grafana derive a **service map** from traces — the directed graph of which services call which:

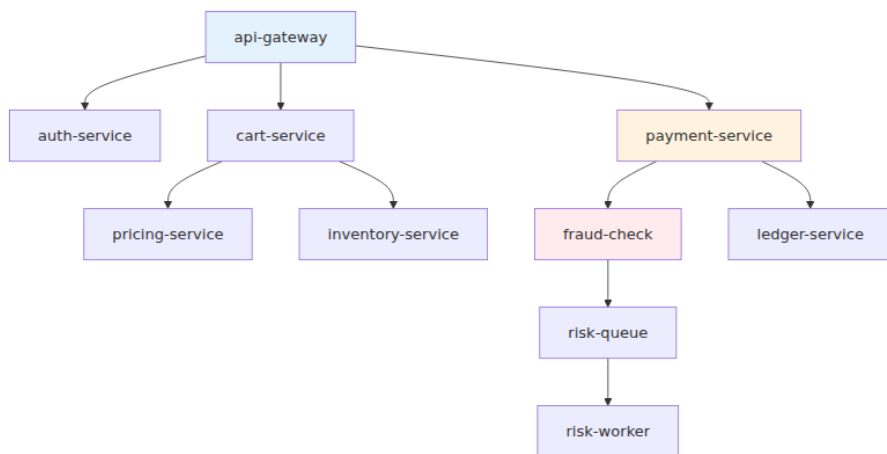


Figure 4-5: Service map derived from traces. The Collector's `servicegraph` connector or Tempo's `metrics-generator` builds this automatically — no manual topology definition needed.

Operating tracing in production

Overhead and performance

Tracing overhead comes from three places:

Source	Typical overhead	Mitigation
Span creation (allocation, timestamps)	~1-5 μ s per span	Do not create spans in tight loops; batch small operations into one span
Context propagation (header serialization)	Negligible (a few bytes per request)	No action needed
Export (serialization, network)	Batch exporter amortizes; ~1-2% CPU at 5% sampling	Tune <code>BatchSpanProcessor</code> queue size and batch timeout

Source	Typical overhead	Mitigation
Collector (processing, tail sampling memory)	Gateway memory scales with trace rate $\times$ decision_wait	Set <code>memory_limiter</code> and <code>num_traces</code> appropriately; scale gateways horizontally

Measured overhead for a typical Go service at 5% head sampling + tail sampling at the gateway is under 2% CPU and under 5 MB heap for the SDK. Auto-instrumented Java services see slightly higher overhead (3–5% CPU) due to bytecode instrumentation.

Resource and deployment

Collector sizing depends on trace throughput:

Trace throughput	Agent (per node)	Gateway (per replica)	Storage (Tempo, per day)
100 traces/s	0.2 CPU / 256 MB	0.5 CPU / 512 MB	~10 GB (with compression)
1,000 traces/s	0.5 CPU / 512 MB	1 CPU / 2 GB	~100 GB
10,000 traces/s	1 CPU / 1 GB	4 CPU / 8 GB ($\times$ N replicas)	~1 TB

Scale gateways horizontally with a consistent hash on `trace_id` so all spans for one trace land on the same gateway instance (required for tail sampling). Most Collector distributions support this via the `loadbalancing` exporter or a hash-ring sidecar.

Security considerations

- **Do not put PII or secrets in span attributes or names.** Span data is stored in trace backends that may have broader access than application databases. Use the `attributes/redact` processor to strip sensitive fields before export.
- **Trace IDs are not secrets but can leak information.** A `trace_id` in a URL or error message tells an attacker that tracing is active and may reveal internal service topology via error details.

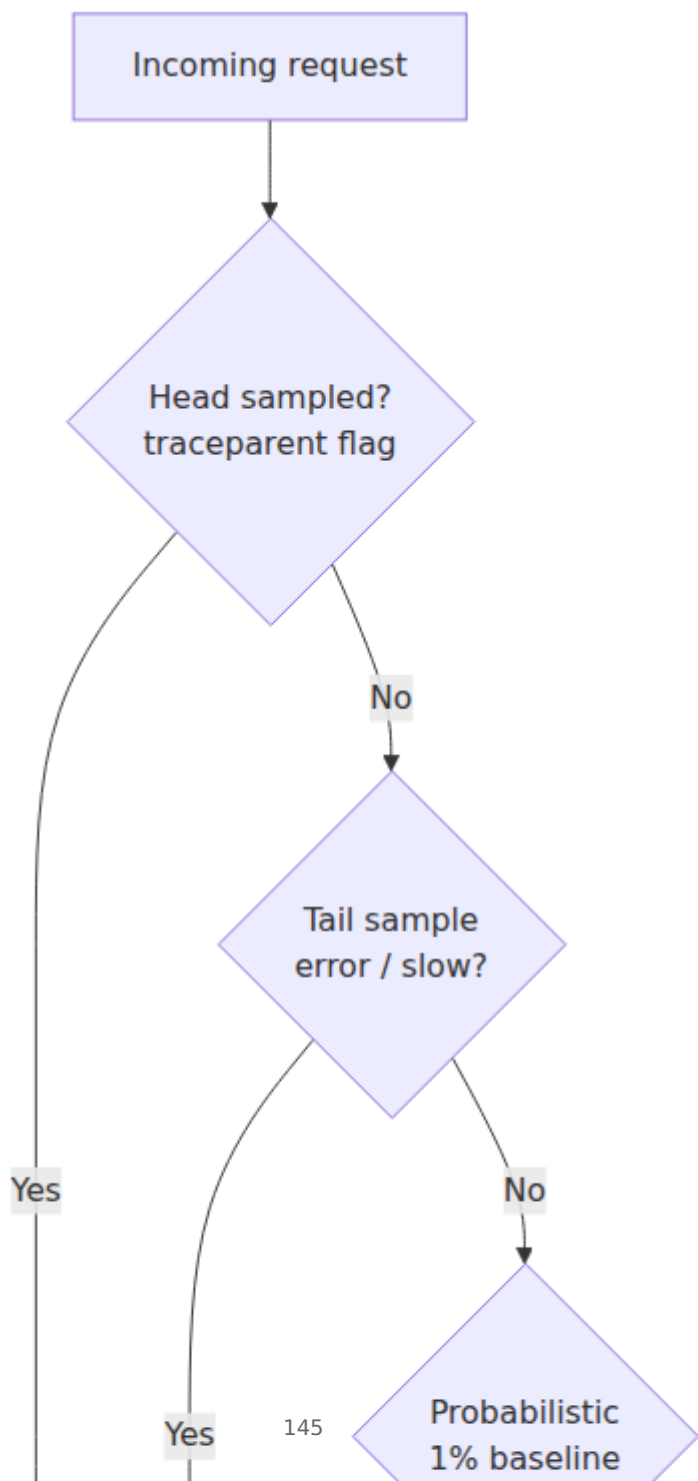
- **Baggage propagation can leak tenant or user data** to every downstream service. Restrict baggage keys to non-sensitive, low-cardinality values.

Distributed-systems lens

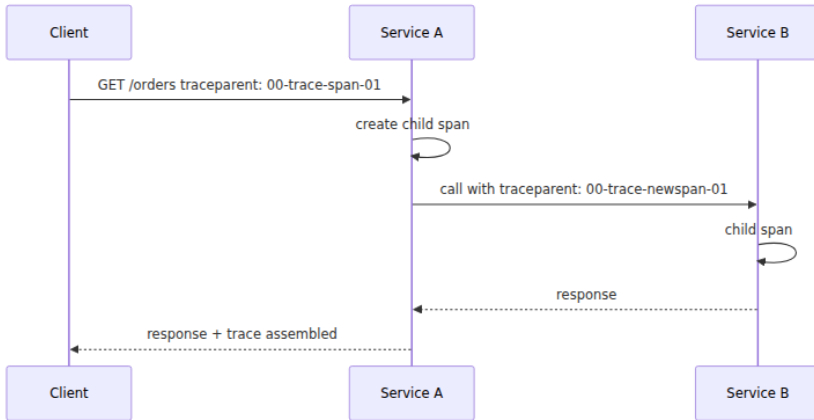
In a distributed backend, tracing is not optional tooling — it is the only signal that captures **causality across service boundaries**. Consider what breaks without it:

- **Debugging tail latency.** A p99 regression from 200 ms to 800 ms could be caused by any of 15 services. Metrics show *which* service's latency histogram shifted; traces show *which span inside that service* (a specific DB query, a downstream call, lock contention) is the root cause. Without traces, each team checks their own dashboards and the investigation stalls at team boundaries.
- **Understanding failure propagation.** When `payment-service` returns 500 to `api-gateway`, the trace shows whether the error originated in `payment-service` itself, in `fraud-check` downstream, or in a timeout between them — three very different remediation paths.
- **Validating async workflows.** Event-driven architectures (Chapter 10 — Messaging) break the synchronous request model. Span links are the only mechanism that connects a Kafka publish in one service to its consumption in another, minutes later, possibly in a different availability zone.
- **Capacity and dependency reasoning.** The service map derived from traces is the ground truth of runtime dependencies — not what the architecture diagram says, but what actually calls what in production, with real latency and error rate annotations. It reveals unexpected dependencies (a service you thought was isolated actually calls a shared database) and unused ones (a declared dependency that never appears in traces).

Sampling Decision



Trace Context Propagation



Key takeaways

- A **trace** is the causal graph of one request across all services; a **span** is one timed operation within that graph. Parent-child relationships and W3C `traceparent` propagation stitch isolated per-service spans into a single distributed trace.
- **W3C Trace Context** (`traceparent` + `tracestate`) is the standard propagation format; carry it on HTTP/gRPC headers automatically and on message headers explicitly. Use **span links** (not parent-child) for async/batch consumers where one consumer handles many producer traces.
- **OpenTelemetry** separates a no-op **API** (what libraries call) from a real **SDK** (what applications configure) and a **Collector** (the pipeline that receives, processes, and exports). The Collector's agent+gateway deployment is standard at scale — agents handle local batching and enrichment, gateways handle tail sampling and multi-backend fan-out.
- **Auto-instrumentation** (Java agent, Python `opentelemetry-instrument`, Go `otelhttp/otelgrpc`) covers framework boundaries; **manual spans** cover business logic. Keep span names low-cardinality and put high-cardinality data in attributes.
- A production **Collector configuration** uses `memory_limiter`, `k8sattributes`, `resourcedetection`, `filter` (drop health checks),

`batch`, `tail_sampling` (keep all errors + slow traces, sample the rest), `attributes/redact` (strip PII), and `spanmetrics / count` connectors to generate metrics from traces.

- **Head sampling** (probabilistic at the SDK) is cheap but may drop the error trace you need; **tail sampling** (at the gateway, after seeing the full trace) keeps all errors and slow traces by policy at the cost of gateway memory. Use tail sampling as the primary strategy with head sampling as a safety valve.
 - **Three-way correlation** — metrics (exemplars) → traces → logs (trace ID injection) — is the fastest path from alert to root cause. Every log line should carry `trace_id / span_id`, and every latency histogram should carry exemplars that link back to traces.
 - **Trace storage** choice (Jaeger, Tempo, Honeycomb, vendor SaaS) depends on scale, cost tolerance, and operational appetite; **TraceQL** (Tempo) is the most expressive query language, supporting duration, attribute, and structural queries.
-

Further reading

- OpenTelemetry Documentation — *Concepts, Instrumentation, Collector* (<https://opentelemetry.io/docs/concepts/signals/traces/>, <https://opentelemetry.io/docs/collector/>)
- W3C Trace Context Specification (<https://www.w3.org/TR/trace-context/>) and W3C Baggage Specification (<https://www.w3.org/TR/baggage/>)
- OpenTelemetry Semantic Conventions — *Trace* (<https://opentelemetry.io/docs/specs/semconv/general/trace/>)
- OpenTelemetry Collector — *Tail Sampling Processor, Transform Processor, Connectors* (<https://opentelemetry.io/docs/collector/configuration/>)
- Grafana Tempo Documentation — *TraceQL* (<https://grafana.com/docs/tempo/latest/traceql/>)
- Jaeger Documentation — *Architecture, Sampling, Deployment* (<https://www.jaegertracing.io/docs/>)
- Cindy Sridharan, *Distributed Systems Observability* (O'Reilly, 2018) — still the best conceptual framing of metrics/traces/logs as a unified system.

- Ben Sigelman et al., *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure* (Google, 2010) — the paper that defined the trace/span model (<https://research.google/pubs/pub36356/>).
- Yuri Shkuro, *Mastering Distributed Tracing* (Packt, 2019) — practical guide to Jaeger and trace data modeling.
- Honeycomb — *Distributed Tracing Concepts* (<https://docs.honeycomb.io/concepts/distributed-tracing/>)

Chapter 5 — Incident Response and On-Call

What this chapter covers. No matter how well you design for reliability, production will break — at 3 AM, during a deploy, under unexpected load, or from a dependency you did not know you had. What separates teams that recover in minutes from teams that flail for hours is not heroics but process: a practiced incident response framework, clear roles, reliable detection and paging, and an on-call culture that is sustainable rather than burning people out. This chapter covers the full incident lifecycle from detection to resolution to handoff, the roles and coordination mechanisms that make response effective, on-call design including rotations, paging policies, and alert quality, and the operational tooling (status pages, runbooks, incident management platforms) that supports response at scale. Every concept is grounded in production configurations: PagerDuty and Grafana OnCall schedules, Alertmanager routing, and executable runbooks.

Learning goals — after this chapter you should be able to:

- Describe the incident lifecycle — detection, triage, coordination, mitigation, resolution, and handoff — and the goal of each phase.
- Define and operate the core incident roles (Incident Commander, Communications Lead, Operations Lead, Subject Matter Experts) and explain why role separation matters under stress.
- Design an on-call rotation that balances coverage, fairness, and sustainability, including primary/secondary shadows, follow-the-sun, and compensation.

- Configure alert routing (Alertmanager, Grafana Alerting, PagerDuty) so that the right person is paged for the right severity with minimal noise.
 - Write executable runbooks that guide responders through diagnosis and mitigation without requiring tribal knowledge.
 - Manage incident communications — internal stakeholder updates, customer-facing status pages, and executive briefings — without slowing down the technical response.
 - Explain how incident response interacts with SLIs/SLOs (Chapter 1), observability (Chapters 2-4), and postmortems (Chapter 6) as a unified reliability system.
-

Why incident response

Incidents are inevitable

Three facts guarantee that every production system will have incidents:

1. **Distributed systems have failure modes you cannot fully anticipate.** Network partitions, cascading overload, subtle consistency bugs, and emergent interactions between services that were correct in isolation but fail in combination. Testing and review reduce the rate but never eliminate it.
2. **Change is constant.** Hundreds of deploys per week, dependency updates, configuration changes, and infrastructure operations each carry a small probability of causing an incident. At high deploy frequency, even a 0.1% incident-per-deploy rate produces regular incidents.
3. **Humans are in the loop.** Operational mistakes — wrong config pushed, query run against production, secret rotated without updating consumers — are not moral failures but predictable consequences of complex systems operated under time pressure.

The question is not whether you will have incidents but **how quickly and calmly you recover**. That is what incident response discipline determines.

The cost of poor response

Failure mode	What happens	Cost
No clear commander	Multiple people attempt conflicting mitigations simultaneously	Longer MTTR, risk of making things worse
Alerts paged the wrong person	On-call for service A is woken for a failure in service B they cannot fix	Wasted attention, slower response, burnout
No communication structure	Stakeholders flood the response channel asking for updates	Responders context-switch away from fixing the problem
No runbooks	Responder must reverse-engineer the system under time pressure	Longer MTTR, inconsistent response quality
No severity framework	Every anomaly becomes a Sev1; responders cannot prioritize	Alert fatigue, true emergencies drowned in noise
Unsustainable on-call	Same people always on call, frequent night pages, no recovery time	Burnout, attrition, declining response quality

Good incident response is not about preventing incidents (that is Chapters 7-10) but about **minimizing their duration and blast radius** when they happen — and doing so in a way that does not burn out the people responsible for responding.

The incident lifecycle

Every incident moves through the same phases, though the boundaries blur and phases may overlap or repeat:

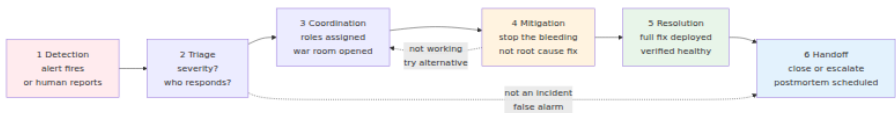


Figure 5-1: The incident lifecycle. Mitigation and coordination loop until the bleeding stops; resolution follows only when the fix is verified. Many teams merge mitigation and resolution — the critical distinction is that mitigation restores service even if the root cause is not yet fixed.

Phase 1: Detection

Detection is either **automated** (an alert fires when an SLI breaches its SLO, see Chapter 1) or **human-reported** (a customer, support engineer, or internal user notices something wrong).

Automated detection quality determines everything downstream. Good detection has:

- **Low time-to-detect (TTD).** The gap between "service is broken" and "someone knows" should be seconds to low minutes, not the time until a customer reports it. This requires SLI-based alerting (Chapter 1), not just infrastructure alerts.
- **Low false positive rate.** If detection fires frequently when nothing is wrong, responders learn to ignore it. The alert that cried wolf is worse than no alert — it trains people to distrust the system.
- **Clear signal.** The alert should say *what* is wrong and *where*, not just that something somewhere is unhappy. `payment-service p99 latency > 2s for 5m` is actionable; `something is slow` is not.

Detection gaps are common. Typical blind spots: async job failures (no user-facing latency signal), data corruption (no error rate signal), and partial degradation (error rate within SLO but affecting a specific tenant or feature).

Phase 2: Triage

Triage answers three questions:

1. **Is this a real incident?** Not every alert is an incident. A brief latency spike that self-recovers, a single pod crash that Kubernetes rescheduled, or a canary deploy that was automatically rolled back may not need a coordinated response. Triage filters signal from noise.
2. **How severe is it?** Severity drives how many people are mobilized and how urgently. Most organizations use a 3-4 level scale (detailed below).

3. **Who should respond?** The on-call for the affected service is the default, but triage may determine that the issue is actually in a dependency — routing to the wrong team wastes critical minutes.

Triage should take **minutes, not tens of minutes**. If triage is slow, severity defaults upward (treat it as more severe until proven otherwise) — it is cheaper to mobilize and stand down than to under-respond to a real Sev1.

Phase 3: Coordination

Once an incident is declared, coordination begins:

- **A war room is opened** — a dedicated Slack channel, Zoom bridge, or incident management tool session where all response activity happens. One place, not scattered DMs.
- **Roles are assigned** — Incident Commander, Communications Lead, Operations Lead (detailed below). Roles are claimed explicitly ("I am IC"), not assumed.
- **Status is tracked** — a shared document or incident tool records timeline, actions taken, and current hypothesis. Without a written timeline, nobody remembers what was tried and when.

The coordination phase is where untrained teams fail most visibly. Without a designated commander, everyone talks at once, nobody has the full picture, and contradictory mitigations are attempted simultaneously.

Phase 4: Mitigation

Mitigation means **stopping the bleeding** — restoring service to users, even if the root cause is not yet understood or fixed. Mitigation options in rough order of preference:

Mitigation	When to use	Example
Rollback	Recent deploy or config change is the likely cause	<code>kubectl rollout undo</code> , feature flag off, revert config
Failover	One zone/region is affected	DNS cutover, traffic shift via load balancer

Mitigation	When to use	Example
Load shedding / degradation	System is overloaded	Disable non-critical features, shed low-priority traffic
Scale up	Resource exhaustion (CPU, connections, queue depth)	HPA bump, manual replica increase, DB connection pool increase
Kill switch	A specific code path or integration is causing the failure	Feature flag, circuit breaker forced open

The key principle: **mitigate first, diagnose second**. Rolling back a suspect deploy takes seconds and may fully resolve the incident. Investigating why the deploy broke while users are still impacted trades user pain for responder curiosity.

Phase 5: Resolution

Resolution means the **root cause is fixed, verified, and deployed**. Resolution is not the same as mitigation:

- Mitigation: "We rolled back the deploy and error rates returned to normal."
- Resolution: "We identified the N+1 query in the new code path, fixed it, verified the fix in staging, deployed it, and confirmed error rates remain normal on the new code."

Many incidents have a long gap between mitigation and resolution. The service is restored quickly (good), but the underlying bug remains on a branch waiting for a proper fix. That is acceptable — mitigation buys time for a careful resolution rather than a rushed hotfix that introduces a new incident.

Phase 6: Handoff and closure

Closure includes:

- **Verification** that all metrics (not just the one that alerted) have returned to normal for a sustained period.
- **Customer communication** that the incident is resolved, with an initial summary.

- **Postmortem scheduling** (Chapter 6) for incidents above a severity threshold.
- **Handoff** if the incident spans an on-call rotation boundary — the outgoing IC briefs the incoming IC on current status, actions taken, and remaining work.

Severity levels

Severity classification determines response urgency, who is mobilized, and what communication is required. Every organization defines its own levels; the structure below is representative:

Severity	Definition	Response	Communication
Sev1 / Critical	Complete service outage or data loss/ corruption affecting all or most users	All-hands, IC assigned immediately, war room within 5 minutes	Status page, executive notification, continuous updates every 15–30 min
Sev2 / Major	Significant degradation — elevated error rates, major feature unavailable, or performance far outside SLO	On-call + backup paged, IC assigned, war room within 15 minutes	Status page, stakeholder notification, updates every 30–60 min
Sev3 / Minor	Limited impact — small subset of users, workaround available, or early warning of potential escalation	On-call paged, async coordination in Slack	Internal channel update, no external communication unless escalates
Sev4 / Low	No user impact — internal tooling issue, non-critical alert, or anomaly worth tracking	Ticket created, handled during business hours	None

Severity is assessed by **impact**, not cause. A one-line config typo that takes down the entire API is Sev1 regardless of how trivial the fix is. Conversely, a complex distributed systems bug that affects 0.1% of requests and has a workaround may be Sev3 even though the investigation is hard.

Severity can change during an incident. An incident that starts as Sev3 ("one availability zone showing elevated latency") may escalate to Sev1 ("latency is actually caused by a database failover that is now affecting all zones"). The IC owns severity assessment and re-classification.

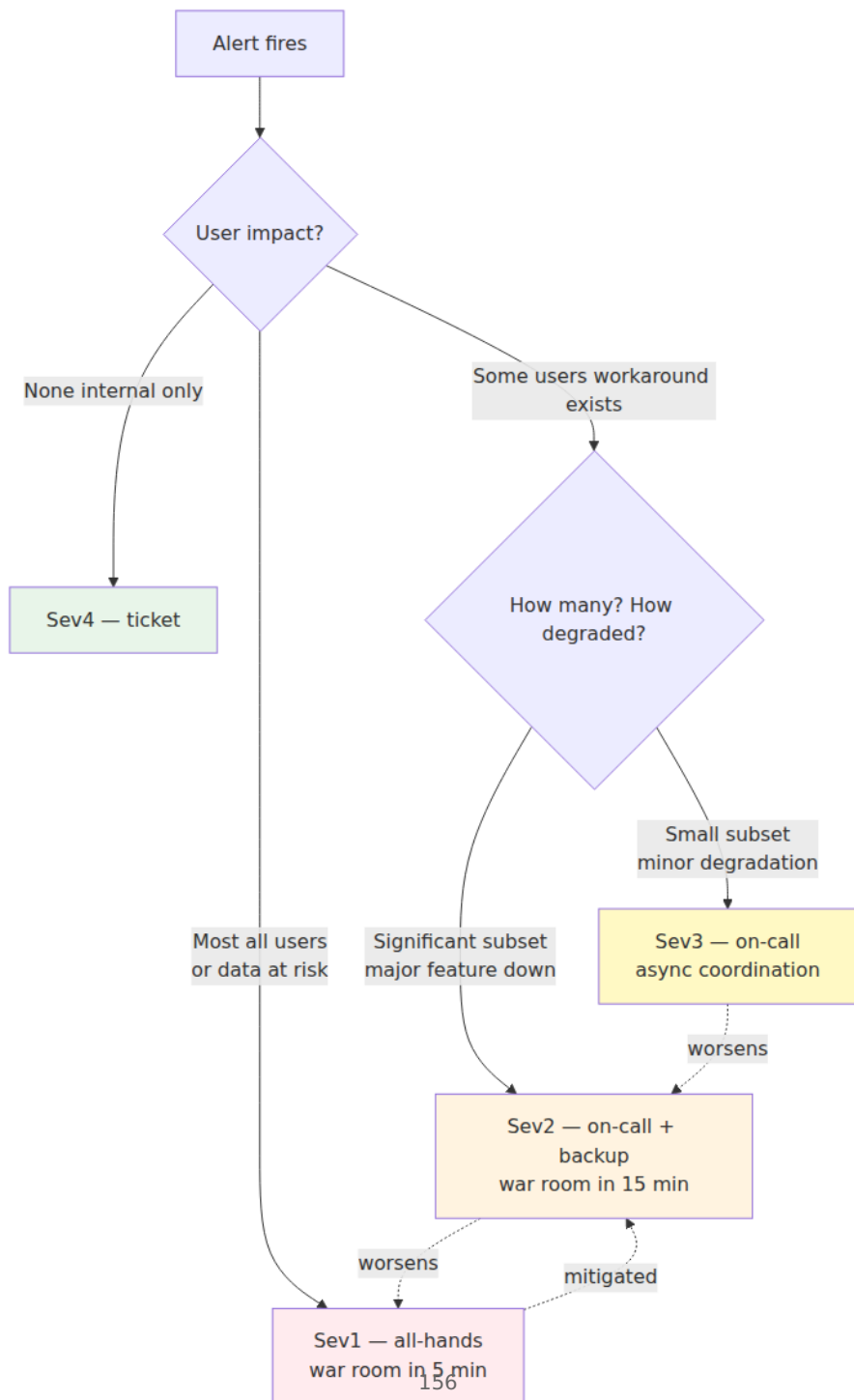


Figure 5-2: Severity classification as a decision tree. Start from user impact, not technical cause. Arrows show escalation and de-escalation paths that the IC manages.

Roles

Clear roles prevent the most common failure mode of incident response: everyone trying to do everything and nobody doing what is actually needed.

Incident Commander (IC)

The IC owns the incident. Not the fix — the *incident*. Responsibilities:

- **Owns coordination.** Decides what to try, in what order, and who does what. Is the single point of authority for mitigation decisions.
- **Maintains situational awareness.** Keeps the timeline, tracks hypotheses, ensures the war room has a shared picture.
- **Makes the hard calls.** When to escalate severity, when to page additional teams, when to try a risky mitigation because the safe one is not working.
- **Does not fix the problem directly.** The IC who is heads-down debugging a query has stopped commanding. Hands on keyboard and command cannot coexist — the IC delegates technical work.

Anyone can be IC. In many organizations the on-call who first responds becomes IC by default until someone more senior or more appropriate takes over with an explicit handoff: "I am taking IC from you — you are now Ops Lead."

Operations Lead (Ops Lead)

The Ops Lead drives the technical investigation and mitigation:

- Executes or delegates diagnostic steps (check dashboards, read logs, examine traces).
- Proposes mitigations to the IC for approval.
- Coordinates the technical contributors (SMEs) — assigns tasks, collects findings, reports status to the IC.

In small incidents the IC and Ops Lead may be the same person. In larger incidents they must be separate — the IC cannot simultaneously maintain the big picture and debug a database.

Communications Lead (Comms)

The Comms Lead owns all communication *out* of the war room:

- Posts regular stakeholder updates (internal Slack, status page, executive bridge).
- Fields incoming questions so the IC and Ops Lead are not interrupted.
- Ensures status page accuracy — the single source of truth for customers.
- Prepares the initial customer-facing summary when the incident resolves.

Without a Comms Lead, every stakeholder DM goes to the IC, who must context-switch from commanding to answering the same question five times.

Subject Matter Experts (SMEs)

SMEs are the engineers who know the affected systems deeply. They are brought in by the IC as needed:

- Diagnose within their domain (database, networking, specific service).
- Execute mitigations within their domain (run the rollback, adjust the config, scale the cluster).
- Report findings to the Ops Lead, not directly to stakeholders.

SMEs are **not** expected to manage the incident. Their job is to answer "what is happening in your system and what can we do about it" and then do what the IC decides.

Stakeholders and observers

Everyone else — managers, product, support, executives — are observers. They belong in a separate communication channel (or the status page), not in the war room. The Comms Lead bridges the two. This is not about secrecy — it is about protecting the responders' ability to focus.

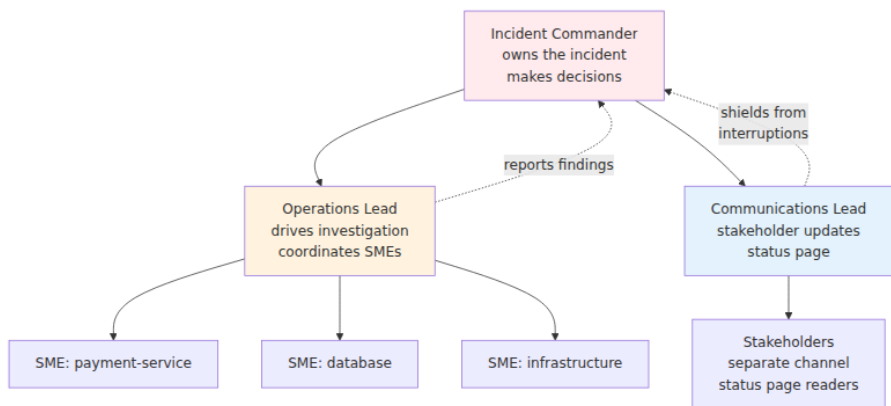


Figure 5-3: Incident role structure. The IC sits above the technical and communication tracks; SMEs report through Ops Lead; stakeholders are shielded from the war room by the Comms Lead.

On-call

Rotation design

On-call means being available to respond to pages outside normal working hours. How the rotation is structured determines whether it is sustainable or leads to burnout.

Basic rotation:

Week 1:	Alice	(primary)	Bob	(secondary / shadow)
Week 2:	Bob	(primary)	Carol	(secondary)
Week 3:	Carol	(primary)	Alice	(secondary)
Week 4:	Alice	(primary)	Bob	(secondary) ← repeats

- **Primary** is paged first. Must acknowledge within a configured window (typically 5 minutes) or the page escalates.
- **Secondary** is paged if primary does not acknowledge, or can be pulled in by primary when the incident needs more hands.

Patterns for larger organizations:

Pattern	How it works	When to use
Follow-the-sun	On-call follows business hours across regions — APAC covers APAC hours, EMEA covers EMEA hours, Americas covers Americas hours	Global teams where night pages can be eliminated entirely
Split weekday/weekend	Different people cover weekdays vs weekends	When weekend load differs from weekday
Tiered escalation	L1 (service on-call) → L2 (platform/infra) → L3 (engineering leadership)	When incidents frequently require cross-team escalation
Shadow / trainee	A new team member shadows the primary without being paged, learning the systems	Onboarding, knowledge distribution

Rotation mechanics that matter:

- **Handoff is explicit.** At rotation boundary, outgoing and incoming on-call do a live handoff: open incidents, recent deploys, known risks, pending follow-ups. A Slack message is not enough for a complex week.
- **No one is on call two weeks in a row** without explicit consent. Back-to-back rotations compound fatigue.
- **Time zone awareness.** A rotation that pages someone at 3 AM their time when a teammate in another zone is wide awake is a design failure, not an operational necessity.

Compensation and sustainability

On-call is work. Treating it as an invisible expectation rather than compensated labor is the fastest path to burnout and attrition.

- **Compensate on-call time**, not just incident time. Being tethered to a pager — unable to travel, restricted in activities, sleeping lightly — has a cost even on quiet nights. Common models: additional pay, time off in lieu, or reduced sprint load during on-call weeks.
- **Track page frequency and out-of-hours load.** If the same service pages 3 times per week at night, the problem is not on-call — it is alert quality or service reliability. Fix the cause, not the schedule.

- **Measure and limit pages per rotation.** Industry guidelines suggest no more than 2 pages per on-call shift should require waking up; more than that indicates the service or its alerts need work.
- **Post-incident recovery.** After a severe or prolonged incident (especially at night), the responders should have explicit recovery time — no expectation to be fully productive the next morning.

Paging policies

A paging policy defines how an alert reaches a human:

```
Alert fires
→ Route by labels (service, severity, environment)
→ Notify primary on-call via push + SMS + phone call
→ If no ack in 5 min → escalate to secondary
→ If no ack in 5 min → escalate to team lead / manager
→ If no ack → escalate to org-wide on-call
```

Real PagerDuty configuration:

```
# pagerduty-service.yaml - escalation policy
escalation_policy:
  name: payment-service
  num_loops: 2                # loop through escalation levels
                             twice before giving up
  rules:
    - escalation_delay_in_minutes: 5
      targets:
        - type: schedule
          id: P12345           # primary rotation schedule
    - escalation_delay_in_minutes: 5
      targets:
        - type: schedule
          id: P67890           # secondary / backup schedule
    - escalation_delay_in_minutes: 10
      targets:
        - type: user
          id: U11111           # engineering manager
        - type: schedule
          id: P99999           # org-wide fallback

# Grafana OnCall - equivalent routing
# (Grafana OnCall is the open-source alternative to PagerDuty)
```

Alertmanager routing (Prometheus ecosystem):

```

# alertmanager.yaml – route alerts to the correct receiver
route:
  group_by: [alertname, service, severity]
  group_wait: 30s          # wait before sending first notification
  (allows grouping)
  group_interval: 5m       # wait before sending additional
  notifications for same group
  repeat_interval: 4h      # re-notify if alert remains firing
  receiver: default
  routes:
    - matchers:
      - severity = "critical"
      - service = "payment-service"
      receiver: pagerduty-payment-critical
      continue: false
    - matchers:
      - severity = "critical"
      receiver: pagerduty-critical
    - matchers:
      - severity = "warning"
      receiver: slack-warnings
      group_interval: 30m  # warnings are less urgent – batch more
  aggressively
    - matchers:
      - severity = "info"
      receiver: slack-info
      group_interval: 1h

receivers:
  - name: pagerduty-payment-critical
    pagerduty_configs:
      - service_key: <pagerduty-integration-key>
        severity: critical
        details:
          firing: '{{ template "pagerduty.firing" . }}'
          client: "Prometheus Payment"
          client_url: '{{ template "pagerduty.url" . }}'
  - name: slack-warnings
    slack_configs:
      - api_url: <slack-webhook-url>
        channel: "#alerts-warnings"
        title: '{{ range .Alerts }}{{ .Annotations.summary }}{{ end }}'
        text: '{{ range .Alerts }}{{ .Annotations.description }}{{
end }}'

inhibit_rules:
  # If payment-service is fully down (critical), suppress warning
  alerts for same service
  - source_matchers: ['severity="critical"', 'service="payment-

```

```
service'']
  target_matchers: ['severity="warning"', 'service="payment-
service"']
  equal: ['service']
```

Key design principles:

- **Route by severity and service**, not by alert name. The routing tree should ensure a `critical` alert for `payment-service` pages the payment team, not a generic on-call.
- **Group related alerts** — 10 firing alerts for the same service should produce one page with 10 alert details, not 10 separate pages.
- **Inhibit lower-severity alerts** when a higher-severity alert for the same service is already firing — the responder already knows the service is down; the warning alerts add noise, not information.
- **Repeat intervals prevent silent failures** — if an alert is still firing after 4 hours, re-notify. But set them long enough that a sustained incident does not spam the on-call every 5 minutes.

Alert quality

Bad alerts are the primary driver of on-call burnout. Every alert should be evaluated against four criteria:

Criterion	Question	Bad alert	Good alert
Actionable	Does the responder know what to do?	CPU > 80% (so what?)	payment-service p99 > 1s for 10m – check fraud-check downstream latency
Accurate	Does it fire only when something is actually wrong?	Fires on every deploy due to brief latency spike	Uses for: 10m to require sustained breach
Correctly routed	Does it page someone who can fix it?	Infra alert pages application on-call	DB connection exhaustion pages DB on-call with app on-call CC'd

Criterion	Question	Bad alert	Good alert
Appropriately severe	Does the urgency match the impact?	critical for a non-user-facing batch job delay	warning for batch delay, critical for user-facing outage

Alert design patterns

SLI-based alerting (preferred):

```

# PrometheusRule – SLI burn rate alerting (see Chapter 1 for SLO math)
groups:
  - name: payment-service.slo
    rules:

# Fast-burn: 14.4× normal burn rate – page immediately (2% budget in 1
hour)
    - alert: PaymentServiceHighBurnRate
      expr: |
        (
          sum(rate(http_requests_total{service="payment-
service",status=~"5.."}[5m]))
          /
          sum(rate(http_requests_total{service="payment-service"}
[5m]))
        ) > (14.4 * 0.001) # 14.4× the 0.1% error budget
      for: 2m
      labels:
        severity: critical
        service: payment-service
      annotations:
        summary: "payment-service burning error budget at 14.4× rate"
        dashboard: "https://grafana/d/payment-service"
        runbook: "https://runbooks/runbooks/payment-service/high-
error-rate.md"

# Slow-burn: 1× burn rate sustained – ticket, not page
    - alert: PaymentServiceSlowBurn
      expr: |
        (
          sum(rate(http_requests_total{service="payment-
service",status=~"5.."}[1h]))
          /
          sum(rate(http_requests_total{service="payment-service"}
[1h]))
        ) > 0.001
      for: 30m
      labels:
        severity: warning
        service: payment-service
      annotations:
        summary: "payment-service slowly burning error budget"

```

Why SLI-based is better than resource-based: CPU at 90% is not an incident if latency and error rate are within SLO. Error rate at 5% is an incident even if CPU is at 30%. Alerting on SLIs pages people for user-visible impact; alerting on resources pages people for internal state that may not matter.

Practical thresholds:

```
# Good: alert on what the user experiences
- alert: CheckoutLatencyHigh
  expr: histogram_quantile(0.99,
sum(rate(http_request_duration_seconds_bucket{route="/checkout"}[5m]))
by (le)) > 2
  for: 10m

# Bad: alert on a resource that may not affect users
- alert: CheckoutPodCPUHigh
  expr: avg(rate(container_cpu_usage_seconds_total{pod=~"checkout-.*"
}[5m])) > 0.8
  # This pages someone whenever CPU is high, even if the service is
  healthy

# Alternative for resource alerts: route to dashboard, not pager
- alert: CheckoutPodCPUHigh
  expr: avg(rate(container_cpu_usage_seconds_total{pod=~"checkout-.*"
}[5m])) > 0.8
  for: 30m
  labels:
    severity: info      # not warning, not critical – informational
```

Alert hygiene

Every alert should have:

- A **runbook link** in its annotations — the responder should be one click from guidance.
- A **dashboard link** — one click to the relevant Grafana/Datadog panel.
- A **for duration** that prevents flapping — brief spikes that self-recover should not page.
- An **owner** — every alert has a team that is responsible for its accuracy and for tuning it when it fires incorrectly.

Periodically audit alerts:

```
# How many alerts fired in the last 30 days? Which ones paged at night?  
# Query your alertmanager / PagerDuty API for alert frequency analysis  
# Alerts that fire frequently but never lead to action are candidates  
for  
# downgrading severity or removing entirely.  
  
# Prometheus: count alert firings  
sum by (alertname) (increase(ALERTS{alertstate="firing"}[30d]))
```

Alerts that fire more than a few times per month without leading to action are noise — downgrade or delete them. An alert that has never fired in 6 months may have a threshold too high to catch real incidents.

Runbooks

A runbook is the documented procedure for diagnosing and mitigating a specific class of incident. The difference between a good runbook and tribal knowledge is the difference between a 10-minute mitigation and a 60-minute one.

What a runbook must contain

Runbook: payment-service high error rate

Symptoms

- Alert: PaymentServiceHighBurnRate fires
- Dashboard: <https://grafana/d/payment-service>
- User impact: checkout failures, payment API returning 5xx

Triage (first 5 minutes)

1. Check if a deploy just happened:

```
```bash
kubectl rollout history deployment/payment-service -n production
If a deploy is in progress or just completed → suspect the deploy
```
```

2. Check downstream dependency health:

```
```bash
fraud-check is the most common cause of payment-service errors
curl -s https://grafana/api/datasources/proxy/1/api/v1/query \
 --data-urlencode 'query=sum(rate(http_requests_total{service="fraud-check"},status=~"5..")[5m]))'
```
```

3. Check database connections:

```
```bash
kubectl exec -n production deploy/payment-service -- \
 curl -s localhost:8080/debug/stats | jq .db_connections
If connections near max (100) → connection exhaustion – go to
Mitigation B
```
```

Mitigation

Mitigation A: Rollback suspect deploy

```
```bash
kubectl rollout undo deployment/payment-service -n production
kubectl rollout status deployment/payment-service -n production --
timeout=120s
Verify: error rate returns to baseline within 2 minutes
Dashboard: https://grafana/d/payment-service?view=error-rate
```



## Mitigation B: Connection exhaustion

```
Option 1: Scale up to get more connection headroom
kubectl scale deployment payment-service -n production --replicas=20
Option 2: Kill long-running queries blocking connections
kubectl exec -n production deploy/payment-db -- \
 psql -c "SELECT pg_terminate_backend(pid) FROM pg_stat_activity
 WHERE state = 'active' AND query_start < now() - interval
 '30 seconds'
 AND datname = 'payments';"
```

## Mitigation C: Downstream failure (fraud-check down)

```
Enable degraded mode – skip fraud check for low-value transactions
curl -X PUT https://config-service/api/flags/payment.skip-fraud-check \
 -H "Authorization: Bearer $OPS_TOKEN" \
 -d '{"enabled": true, "max_amount_cents": 5000}'
Verify: checkout success rate recovers for transactions < $50
```

## Escalation

- If none of the above mitigates within 15 minutes → escalate to platform team
- PagerDuty: escalate via <https://pagerduty.com/escalate/payment-service>
- Slack: #platform-oncall
- If data corruption suspected → Sev1, page DB on-call directly

## References

- Architecture: <https://docs/architecture/payment-service.md>
- Last updated: 2026-03-15
- Owner: @payments-team

### ### Runbook principles

- **\*\*Executable, not descriptive.\*\*** Every step should be a command the responder can copy-paste, not a paragraph describing what to think about. Under stress, people follow checklists – they do not parse prose.
- **\*\*Ordered by likelihood and speed.\*\*** The most common cause and fastest mitigation come first. Rolling back a deploy is faster than investigating a database – try it first even if you are not sure it is the cause.
- **\*\*Include verification.\*\*** Every mitigation step should say how to verify it worked. Without verification, the responder does not know whether to try the next step.
- **\*\*Include escalation.\*\*** When the runbook does not help within a time bound, say who to page next. The worst outcome is a responder stuck on a runbook that does not cover the actual failure mode, not knowing who else to call.
- **\*\*Keep it current.\*\*** A runbook that references a decommissioned dashboard or a renamed service is worse than no runbook – it actively misleads. Review runbooks quarterly and after every incident where they were used.

### ### Runbook automation

At mature organizations, runbooks evolve from documents to executable automation:

```
```python
# Example: automated runbook as a ChatOps command (Slack bot /
PagerDuty automation)
# Triggered by: /incident payment-service high-error-rate

async def runbook_payment_high_error_rate():
    # Step 1: automated triage
    recent_deploy = await k8s.recent_rollout("payment-service",
window="30m")
    fraud_health = await check_service_health("fraud-check")
    db_connections = await check_db_connections("payments")

    if recent_deploy:
        await slack.post(f"Recent deploy detected:
{recent_deploy.version} – recommend rollback")
        await slack.post_button("Rollback?", action=rollback_payment)
    elif fraud_health.error_rate > 0.05:
        await slack.post(f"fraud-check error rate
{fraud_health.error_rate:.1%} – likely downstream cause")
        await slack.post_button("Enable degraded mode?",
action=enable_degraded_mode)
    elif db_connections.utilization > 0.9:
        await slack.post(f"DB connections at
```

```
{db_connections.utilization:.0%} – recommend scale-up")
    await slack.post_button("Scale to 20 replicas?",
action=scale_payment)
    else:
        await slack.post("No obvious cause – manual investigation
needed. Dashboard: https://grafana/d/payment-service")
```

Automated triage does not replace human judgment — it accelerates the first 5 minutes so the responder starts with context rather than a blank page.

Communications

The two-channel model

Incident communications must serve two audiences with different needs:

Audience	Channel	Content	Cadence
Responders (IC, Ops Lead, SMEs)	War room (Slack #incident-YYYY-MM-DD-name, Zoom bridge)	Technical detail, hypotheses, action log, timeline	Continuous
Stakeholders (support, product, executives, customers)	Status page, #incident-comms Slack, email	Impact summary, ETA, workaround if any, no internal speculation	Every 15-30 min for Sev1, every 30-60 min for Sev2

Never mix the two. Stakeholders asking "what is the status?" in the war room forces responders to stop fixing and start writing status updates. The Comms Lead bridges the gap — they sit in the war room, absorb the technical picture, and translate it for stakeholders.

Status page

The status page (e.g., Atlassian Statuspage, Instatus, or self-hosted Cachet) is the **single source of truth** for external communication:

Status page update – example progression

14:32 – Investigating

We are investigating elevated error rates on the checkout API. Some customers may experience failures when placing orders. We will provide an update within 15 minutes.

14:47 – Identified

We have identified the cause as elevated latency in our payment processing subsystem. Our team is working on a mitigation. Workaround: retrying a failed checkout after 30 seconds may succeed.

15:05 – Monitoring

A fix has been deployed and error rates have returned to normal. We are monitoring to ensure the issue is fully resolved.

15:30 – Resolved

This incident has been resolved. Checkout success rates have been nominal for 25 minutes. A postmortem will be published within 5 business days.

Status page discipline:

- **First update within 15 minutes of Sev1 declaration**, even if you have no mitigation yet. "We are investigating" is better than silence.
- **Never speculate on root cause** in external communications. "We are investigating elevated error rates" is correct; "We think the database is down" may be wrong and creates misinformation.
- **Include workaround if one exists** — even a partial workaround ("retry after 30 seconds") reduces customer impact and support load.
- **Postmortem timeline** — commit to publishing a postmortem or incident report within a stated window (typically 5 business days for Sev1).

Internal stakeholder updates

For internal stakeholders, the Comms Lead posts structured updates:

INCIDENT UPDATE — 14:47 UTC — Sev1 — payment-service elevated errors

Impact: ~40% of checkout requests failing since 14:28 UTC

Cause: Under investigation — suspect deploy at 14:25 UTC

Mitigation: Rollback in progress, ETA 5 minutes

Next update: 15:00 UTC or sooner if mitigated

War room: #incident-2026-03-15-payment-errors

Status: <https://status.example.com>

This format answers the four questions every stakeholder has: what is broken, how bad is it, what are we doing, and when will I hear more.

Incident management tooling

Tool	Role	Notes
PagerDuty / Grafana OnCall / Opsgenie	Paging, escalation, on-call schedules	Core — without this, pages do not reach humans reliably
Slack / Teams + incident bot	War room, timeline, role assignment	Most teams use ChatOps — / incident create opens channel, doc, bridge
Jira / Linear / incident tracker	Incident record, timeline, action items	Permanent record; linked to postmortem
Statuspage / Instatus / Cachet	External status communication	Single source of truth for customers
Grafana / Datadog	Dashboards, log/trace correlation	Where diagnosis happens
FireHydrant / Blameless / Rootly	Incident lifecycle automation	Auto-creates channels, docs, timelines, reminders

A minimal viable incident stack for a small team:

```
# Slack workflow: /incident command  
/incident create --severity sev2 --service payment-service --title "elevated error rate"  
# → creates #incident-2026-03-15-payment-errors  
# → creates Google Doc from template (timeline + roles)  
# → pages on-call via PagerDuty  
# → posts to #incident-comms: "Sev2 declared for payment-service – war room: #incident-..."  
# → creates Jira ticket linked to the channel
```

For larger organizations, platforms like FireHydrant or Rootly automate the full lifecycle — severity classification, role assignment, timeline capture, stakeholder notifications, and postmortem scheduling — so the IC can focus on the technical response rather than tooling.

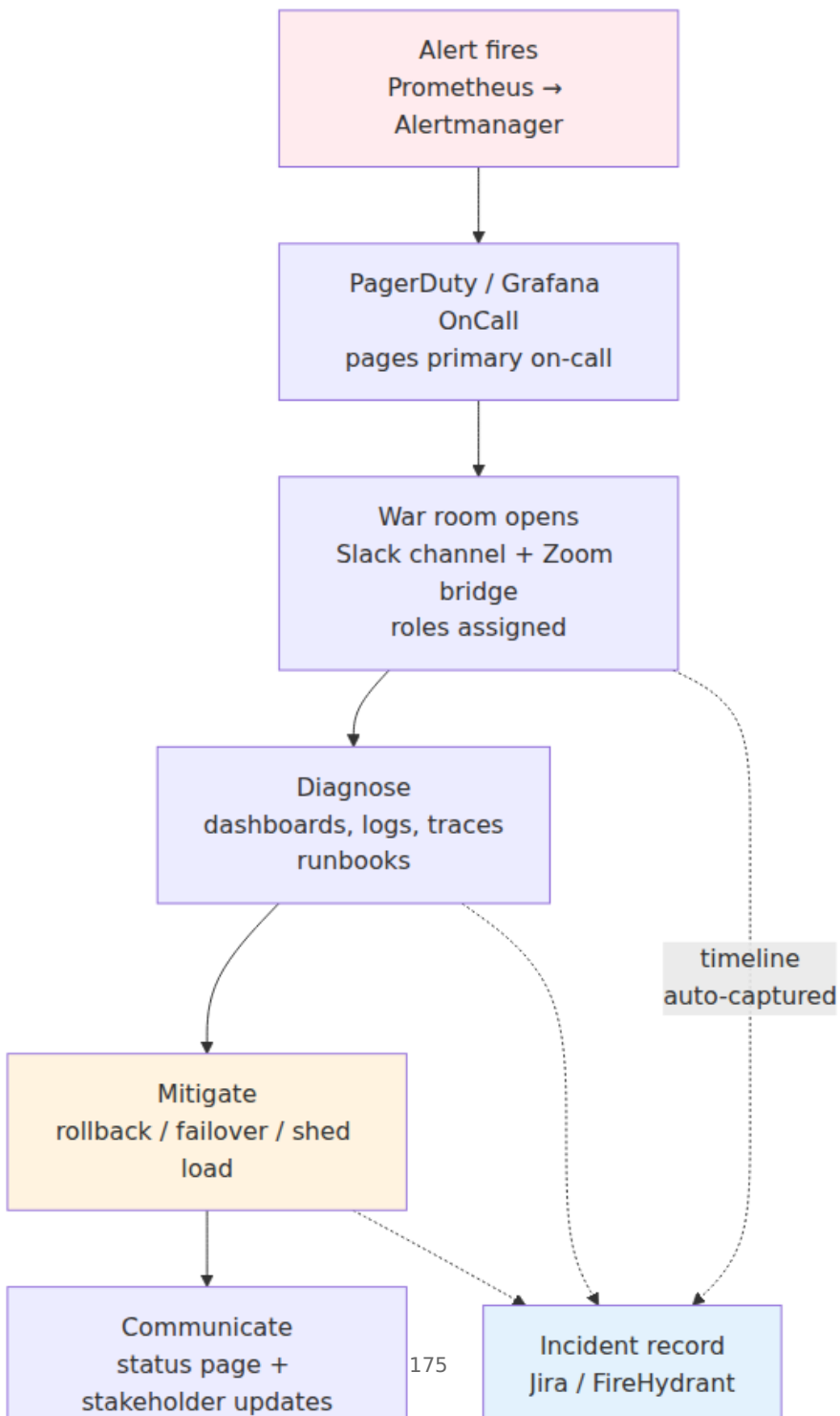


Figure 5-4: Incident tooling flow. The alert triggers the paging system, which opens the war room; diagnosis and mitigation loop until verified; the incident record captures the timeline automatically throughout.

Distributed-systems lens

Incident response for distributed backends has specific challenges that single-service operations do not:

Failure crosses team boundaries. When `checkout` fails because `payment-service` times out because `fraud-check` is overloaded because a downstream ML model is slow, the incident spans four teams. No single on-call can diagnose it alone. The IC must pull SMEs from multiple teams quickly — which requires that every service has a discoverable on-call and that escalation paths between teams are documented, not just known to managers.

Blast radius is hard to bound. A single misconfigured retry policy (Chapter 10 — Resilience Patterns) can turn a minor latency increase in one service into a cascading overload that takes down unrelated services through shared dependencies (databases, queues, thread pools). The IC must think about blast radius explicitly: which services share fate with the failing one, and should traffic be shed to protect them?

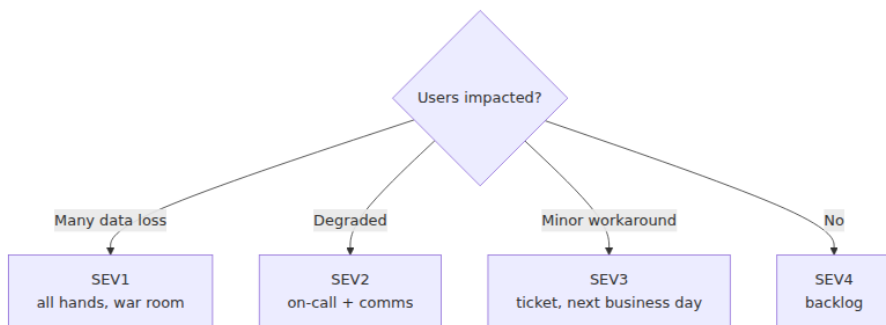
Partial failure is the common case. Total outages are dramatic but rare. Far more common is partial degradation — one availability zone slow, one tenant affected, one shard overloaded — that does not trigger a simple "is it up?" check but does violate SLOs for a subset of users. Detection must be granular enough (per-zone, per-tenant, per-shard SLIs) to catch partial failures, and triage must be nuanced enough to assess their severity correctly.

Runbooks must cover cross-service scenarios. A runbook for `payment-service` that only covers payment-service internals is incomplete. It must also cover: how to tell if the problem is actually in `fraud-check`, how to degrade gracefully when a dependency is down, and how to verify that the dependency has recovered before re-enabling the full path.

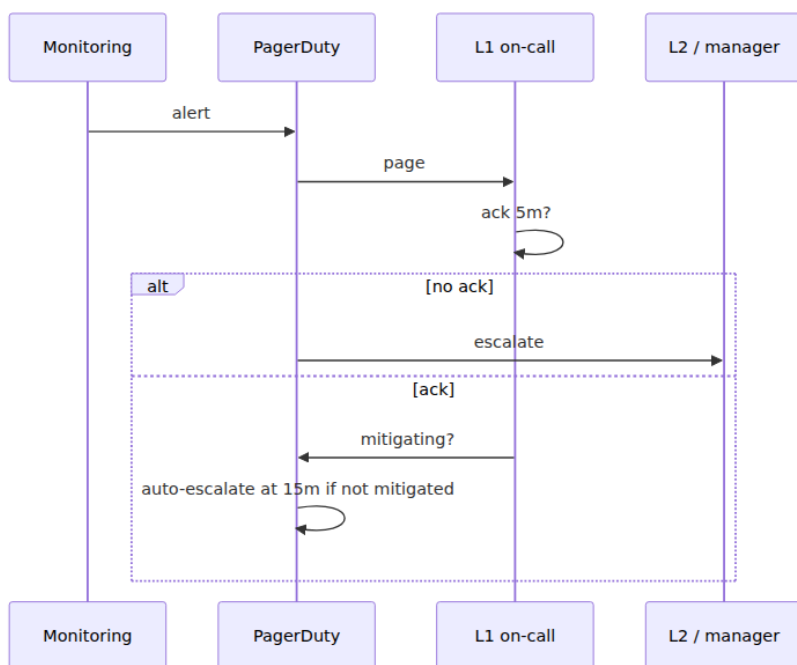
On-call load scales with service count. An organization with 100 services and one on-call rotation per service has 100 on-call slots to fill.

At 5 engineers per team, that means every engineer is on call roughly every 5 weeks — sustainable. But if each service has separate primary and secondary rotations, or if platform teams are on call for shared infrastructure plus their own services, the load concentrates on a few teams. On-call design must account for the total burden across the organization, not just per-team fairness.

Severity Decision



On-Call Escalation



War Room Comms



Key takeaways

- Incident response is about **minimizing duration and blast radius**, not preventing incidents — prevention is covered by resilience and deployment practices (Chapters 7-10), but every system will still have incidents that require practiced response.
- The **incident lifecycle** — detection → triage → coordination → mitigation → resolution → handoff — gives structure to chaos; the most critical distinction is between **mitigation** (stop the bleeding, restore service) and **resolution** (fix the root cause) — mitigate first, diagnose second.

- **Severity classification** is based on user impact, not technical cause; it determines who is mobilized and how urgently, and it can change as the incident evolves — the IC owns re-classification.
- **Four roles** — Incident Commander (owns the incident), Operations Lead (drives investigation), Communications Lead (owns stakeholder updates), and SMEs (diagnose within their domain) — prevent the failure mode where everyone tries to do everything and nobody maintains the big picture.
- **On-call rotations** must balance coverage, fairness, and sustainability: explicit handoffs, no back-to-back rotations without consent, time zone awareness, follow-the-sun where possible, and **compensated** on-call time — being tethered to a pager is work even on quiet nights.
- **Paging policies** route by severity and service, group related alerts, inhibit lower-severity noise when higher-severity alerts are firing, and escalate through primary → secondary → management → org-wide fallback — configure them in PagerDuty / Grafana OnCall and Alertmanager with `group_by`, `inhibit_rules`, and `repeat_interval`.
- **Alert quality** is the primary driver of on-call sustainability: every alert must be actionable, accurate, correctly routed, and appropriately severe — prefer SLI-based alerts (user-visible impact) over resource-based alerts (internal state that may not affect users), and audit firing frequency regularly.
- **Runbooks** must be executable (copy-paste commands, not prose), ordered by likelihood and speed, include verification for every mitigation step, include escalation when the runbook does not cover the failure, and be kept current — a stale runbook is worse than no runbook.
- **Two-channel communication** — war room for responders, status page + stakeholder channel for everyone else, bridged by the Comms Lead — protects responders' focus while keeping stakeholders informed; first external update within 15 minutes of Sev1, even if the message is just "we are investigating."
- In **distributed systems**, incidents routinely cross team boundaries, blast radius is hard to bound due to shared dependencies, partial failure is the common case, and on-call load scales with service count — design detection granularity, cross-team escalation, and on-call burden at the organizational level, not just per team.

Further reading

- Google SRE Book, Chapters 12–14 — *Practical Alerting, Being On-Call, Managing Incidents* (<https://sre.google/sre-book/practical-alerting/>, <https://sre.google/sre-book/being-on-call/>)
- Google SRE Workbook, Chapters 6–9 — *Incident Response, Postmortem Culture* (<https://sre.google/workbook/table-of-contents/>)
- PagerDuty — *Incident Response Documentation* (<https://response.pagerduty.com/>) — comprehensive, field-tested incident response guide covering roles, severity, and communications.
- Atlassian — *Incident Management Handbook* (<https://www.atlassian.com/incident-management/handbook>) — practical guide to severity levels, roles, and post-incident review.
- Grafana OnCall Documentation (<https://grafana.com/docs/oncall/latest/>) — open-source on-call scheduling and escalation.
- Prometheus Alertmanager Documentation — *Configuration, Routing, Inhibition* (<https://prometheus.io/docs/alerting/latest/configuration/>)
- Charity Majors, *On Call Shouldn't Suck* and related Honeycomb writings on alert quality and sustainable on-call.
- FireHydrant / Rootly Documentation — incident lifecycle automation platforms.
- Jeli (Ben Sigelman) — *Modern Incident Management* — evolving practices for incident analysis beyond the traditional postmortem.

Chapter 6 — Blameless Postmortems

What this chapter covers. An incident ends when service is restored — but learning from it has barely begun. Without deliberate follow-through, the same failure will recur, often with larger blast radius. The blameless postmortem is the mechanism that converts the cost of an incident into durable improvement: a structured, psychologically safe investigation that asks *how the system allowed this to happen* rather than *who caused it*, producing specific, tracked action items that make the next incident less likely or less damaging. This chapter covers when and how to write postmortems, the facilitation and cultural practices that make

blamelessness real rather than performative, the anatomy of an effective postmortem document, the action item lifecycle that turns words into fixes, and how postmortems connect to the broader reliability system — error budgets, SLIs, and organizational learning.

Learning goals — after this chapter you should be able to:

- Explain why blamelessness is a practical engineering strategy, not just a cultural nicety, and how blame suppresses the information flow that postmortems need.
- Decide when a postmortem is warranted (severity thresholds, near-miss criteria) and who should write and review it.
- Facilitate a postmortem meeting that surfaces systemic causes rather than individual fault, including the language patterns that maintain psychological safety.
- Write a postmortem document with a clear timeline, impact assessment, root cause analysis, and action items — using the template provided in this chapter.
- Apply root cause analysis techniques (Five Whys, fishbone, contributing factors) without falling into the single-root-cause trap or the counterfactual trap.
- Manage the action item lifecycle — prioritization, ownership, tracking, and verification — so postmortems produce real fixes rather than documents that are read once and forgotten.
- Describe how postmortems feed into organizational learning: sharing, trend analysis, and the relationship between postmortem culture and deployment velocity.

Why postmortems

The alternative is recurrence

Every incident is a free lesson about a failure mode that exists in your system. You already paid the cost — user impact, responder time, disrupted plans. The only question is whether you extract the value.

Without postmortems, organizations exhibit a predictable pattern:

1. Incident happens. Responders fix it under pressure, often with a quick mitigation (rollback, scale-up, manual intervention).

2. Service recovers. Everyone is relieved and returns to feature work.
3. No one writes down what happened, why, or what should change.
4. Weeks or months later, the same failure recurs — sometimes worse, because the system has grown and the blast radius is larger.
5. The same responders fix it again, slightly faster this time because they remember what they did last time — but only if they are still on the team.

Postmortems break this cycle by converting **tacit knowledge** (what the responders learned under pressure) into **explicit, shared knowledge** (a document, action items, and systemic fixes that persist beyond any individual's memory or tenure).

What postmortems are not

Misconception	Reality
A punishment mechanism	A learning mechanism. If people fear the postmortem, they will hide information — the opposite of what you need.
Only for Sev1 outages	Any incident or near-miss with learning value deserves one. The most valuable postmortems are often for Sev3 incidents that <i>could have been</i> Sev1.
A bureaucratic requirement	A technical document. The audience is the engineers who will prevent the next incident, not auditors checking a box.
A search for "the" root cause	Systems fail due to multiple contributing factors. Fixating on a single root cause misses the systemic improvements that prevent whole classes of failure.
Done when the document is written	Done when the action items are completed and verified. The document is the starting point, not the finish line.

The reliability feedback loop

Postmortems are one component of a larger reliability system:

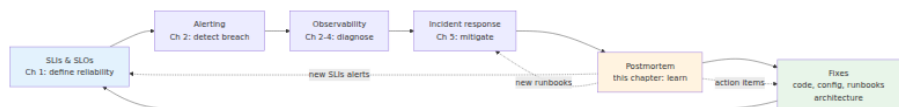


Figure 6-1: Postmortems close the reliability loop. The fixes they produce improve detection (new SLIs/alerts), response (new runbooks), and prevention (architectural changes) — making every future incident through the same path less likely.

Blameless culture

Why blame is counterproductive

Blame feels natural — someone pushed the bad config, someone missed the review, someone should have known. But blame is actively harmful to reliability for three reasons:

1. Blame hides information. If the engineer who pushed the config fears being blamed, they will minimize their role: "The config was already wrong" or "I followed the usual process." The postmortem loses the most important data — what the person actually did, saw, and thought at each step. Without that, the postmortem cannot identify the systemic factors (unclear process, missing validation, insufficient testing) that allowed the mistake.

2. Human error is never the root cause. Saying "the engineer made a mistake" explains nothing — humans always make mistakes at some rate. The useful question is: *why did the system allow that mistake to reach production?* Was there no validation? Was the deploy process confusing? Was the reviewer overloaded? Those are fixable system properties; "try harder not to make mistakes" is not.

3. Blame creates fear that slows response. If responders fear blame for actions taken during an incident, they will hesitate to make bold mitigation decisions (rolling back, shedding load, failing over) that might be wrong but are the fastest path to recovery. Hesitation during incidents directly increases MTTR and user impact.

What blameless actually means

Blameless does not mean:

- **No accountability.** People are still responsible for their work. Blameless means accountability is about *fixing the system* (writing the validation, improving the process), not about punishment.

- **No discussion of human actions.** Postmortems must describe exactly what people did — that is essential data. Blameless means describing actions without moral judgment.
- **Ignoring repeated negligence.** If someone consistently bypasses safety checks despite training and tooling, that is a management and coaching issue — but it is handled outside the postmortem, not inside it.

Blameless does mean:

- **Language matters.** Compare:
- Blameful: "Alice deployed a broken config that took down checkout."
- Blameless: "At 14:25 UTC, a config change was deployed to payment-service that set `max_connections` to 10 instead of 100. The deploy pipeline did not validate this value against the expected range. Checkout error rates rose to 40% within 3 minutes."

The blameless version contains more information (what the value was, what it should have been, what validation was missing) and no judgment. It points directly at the systemic fix (add range validation to the pipeline).

- **The "reasonable person" test.** Ask: would a reasonable, competent engineer in the same situation — with the same information, time pressure, tooling, and fatigue — plausibly have made the same decision? If yes, the system needs to change, not the person.
- **Psychological safety is built, not declared.** Saying "we are blameless" while the postmortem document subtly identifies who is at fault, or while the manager has a private conversation about performance after the postmortem, is worse than not claiming blamelessness at all. People detect the gap immediately and adjust their openness accordingly.

Building psychological safety

Concrete practices that make blamelessness real:

Practice	What it looks like
Leadership models it	Senior engineers and managers openly describe their own mistakes in postmortems: "I approved that deploy without checking the load test results."

Practice	What it looks like
Facilitator enforces language	The postmortem facilitator redirects blameful language in real time: "Let's rephrase — what about the system made that action seem correct at the time?"
No names in action items as blame	Action items are owned by teams or roles, or by the person who volunteers — not assigned as "Alice must fix the thing Alice broke."
Reward postmortem participation	Engineers who write thorough postmortems, especially for their own incidents, are recognized — not quietly penalized with extra process.
Share postmortems broadly	When postmortems are visible org-wide, everyone sees that incidents happen to everyone, including senior staff. This normalizes fallibility.

When to write a postmortem

Not every alert or minor anomaly needs a postmortem. But the threshold should be lower than most teams initially set it.

Severity-based triggers

Trigger	Postmortem required?	Notes
Sev1 (full outage / data loss)	Always, within 5 business days	Non-negotiable. The highest learning value and stakeholder visibility.
Sev2 (significant degradation)	Always, within 10 business days	Still significant impact and likely systemic causes.
Sev3 (limited impact)	If novel or with useful lessons	A Sev3 that reveals a new failure mode is more valuable than a Sev1 that repeats a known one.
Sev4 (no user impact)	Rarely, only if the near-miss is instructive	A Sev4 that <i>could have been</i> Sev1 under slightly different conditions is a near-miss worth studying.

Trigger	Postmortem required?	Notes
Near-miss (caught before impact)	When the catch was luck, not design	"We caught this in staging by accident — in production it would have been Sev1" is a critical signal.

The near-miss principle

Near-misses are incidents that did not cause user impact only because of luck or manual intervention — not because the system handled them correctly. They are disproportionately valuable because:

- They reveal the **same systemic failures** as real incidents but without the pressure and disruption of a live incident response.
- They are **more common** than actual incidents, so studying them gives more data points.
- They are **less emotionally charged**, so postmortems are easier to conduct blamelessly.

Examples of near-misses worth a postmortem:

- A bad deploy was caught by a manual check before reaching production — but no automated test would have caught it.
- A database failover succeeded but took 10× longer than expected — next time it might not succeed at all.
- An engineer noticed a misconfigured alert that would have suppressed pages during a real incident.

The heuristic: **if the only reason this was not a worse incident is something you would not want to rely on next time, write the postmortem.**

Who writes it

Role	Responsibility
Author	Usually the IC or Ops Lead from the incident — the person with the most complete picture. Sometimes a dedicated postmortem owner if the IC is unavailable.

Role	Responsibility
Contributors	Every responder and SME who participated, plus anyone whose system was involved — even if they were not paged.
Facilitator	For significant postmortems (Sev1/Sev2), a neutral facilitator who was not directly involved in the incident keeps the meeting focused and blameless.
Reviewers	Engineering leadership, affected teams, and anyone who owns action items. Postmortems are reviewed, not just published.
Stakeholders	Product, support, and executives receive the summary — not necessarily the full technical document.

The author should start the document **within 24-48 hours** of the incident while memories are fresh. Waiting a week means details are lost and the postmortem becomes an exercise in reconstructing what happened from chat logs rather than from direct recall.

The postmortem document

Template

The template below is used at many organizations (derived from the Google SRE postmortem template and widely adapted). Every section earns its place — none is filler.

Postmortem: [service] – [brief incident description]

Field	
Value	
----- -----	

Date	
2026-03-15	
Severity	
Sev1	
Duration	14:28–15:05 UTC (37
minutes)	
Impact	~40% of checkout requests failed (HTTP
500)	
Detection	Alert: PaymentServiceHighBurnRate (2 min
TTD)	
Responders	Alice (IC), Bob (Ops Lead), Carol (Comms), Dave
(SME: payments)	
Status	Draft / In Review / Approved / Action Items
Complete	
Postmortem Owner	
Alice	
Review Meeting	2026-03-18 14:00 UTC, Room/Zoom
link	

Summary

At 14:25 UTC on 2026-03-15, a configuration change to payment-service set ``max_connections`` from 100 to 10. Within 3 minutes, checkout error rates rose to 40% as payment-service exhausted its connection pool and began rejecting requests. The change was rolled back at 14:52 UTC and error rates returned to baseline by 15:05 UTC. No data loss occurred; failed checkouts were retried successfully by most customers.

Impact

- ****Users affected:**** ~12,000 checkout attempts during the 37-minute window;
~4,800 failed on first attempt. Retry success rate ~85%, so ~720 checkouts required manual retry or were abandoned.
- ****Revenue impact:**** Estimated \$18,000 in delayed/abandoned transactions
(based on average order value – finance to confirm).
- ****Duration:**** 37 minutes from first user-visible failure to full recovery.
An additional 15 minutes of elevated p99 latency (500ms → 1200ms) as connection pools warmed after rollback.
- ****Data integrity:**** No data loss or corruption. Failed payments were not charged; no duplicate charges observed.

- ****Other services:**** fraud-check and ledger-service showed elevated latency
due to retry storms from payment-service but did not independently fail.

Timeline (all times UTC)

Time	Event
14:25	Config change deployed to payment-service: <code>`max_connections: 10`</code> (intended: 100 – typo in config file)
14:28	<code>`PaymentServiceHighBurnRate`</code> alert fires (error rate 40%, threshold 5%)
14:28	PagerDuty pages Alice (primary on-call for payment-service)
14:30	Alice acknowledges, declares Sev1, opens <code>#incident-2026-03-15-payment-errors</code>
14:31	Bob joins as Ops Lead; Carol joins as Comms Lead
14:32	First status page update: "Investigating elevated error rates on checkout"
14:33	Bob identifies recent config deploy as suspect – checks rollout history
14:35	Dashboard confirms: payment-service DB connection pool exhausted (10/10), queue depth growing
14:38	Dave (SME) confirms: <code>`max_connections=10`</code> is the cause – should be 100
14:42	Alice decides: rollback config change (no code change needed)
14:52	Config rollback deployed; connection pool recovers to 12/100
14:55	Error rates dropping: 40% → 8% → 2% over 3 minutes
14:55	Second status page update: "Fix deployed, monitoring recovery"
15:05	Error rates at baseline (0.08%); p99 latency recovering
15:10	Incident declared resolved; war room closed
15:30	Final status page update: "Resolved"
15:45	Handoff note written; postmortem scheduled for 2026-03-18

Root Cause and Contributing Factors

Root cause

A configuration value (``max_connections: 10``) was set to 1/10th of its correct value due to a manual editing error. The deploy pipeline applied the config without range validation.

Contributing factors

1. ****No config validation in the deploy pipeline.**** The pipeline applies any YAML value without checking against expected ranges. A value of

10 for ``max_connections`` (valid range: 50–500) was accepted silently.

2. ****No canary or staged rollout for config changes.**** The config was deployed to 100% of payment-service instances simultaneously. A canary (5% of instances first, with automatic rollback on error rate increase) would have limited blast radius to 5% of traffic.

3. ****Connection pool metric had no alert.**** ``payment_db_connections_used`` approaching ``max_connections`` is a leading indicator that fires **before** user-visible errors. No alert existed for this signal.

4. ****Config change was not flagged as high-risk.**** The deploy was treated as a routine config update with no additional review or approval step, despite ``max_connections`` being a critical performance parameter.

5. ****Retry storm amplified impact.**** payment-service callers retried failed requests without backoff, generating 3× the normal request rate against the already-exhausted pool and spreading latency to fraud-check.

What Went Well

- Detection was fast (2 min TTD) – SLI-based alerting worked as designed.
- Rollback was quick once the cause was identified (10 min from identification to recovery).
- Comms Lead kept stakeholders informed – no duplicate status requests interrupted the war room.
- No data loss – idempotency handling (Ch 10 – Messaging) ensured failed payments were safe to retry.

What Went Poorly

- Config with no validation reached production – the most preventable link in the chain.
- Initial triage spent 5 minutes before identifying the deploy as the cause – the deploy event was not surfaced on the payment-service dashboard.
- Status page first update took 7 minutes after Sev1 declaration – should be within 5 minutes per our own policy.
-

Retry storm from callers was not anticipated and not mitigated during the incident.

Action Items

#	Action	Owner	Priority	Due	Status
1	Add range validation for all config values in deploy pipeline; reject values outside expected bounds	Platform team	P0	2026-03-25	TODO
2	Implement canary rollout for config changes (5% → 50% → 100% with auto-rollback on error rate)	Platform team	P0	2026-04-15	TODO
3	Add alert: <code>payment_db_connections_used / max_connections > 0.8 for 5m</code> → warning	Payments team	P0	2026-03-22	TODO
4	Add deploy markers to payment-service Grafana dashboard	Payments team	P1	2026-03-25	TODO
5	Implement exponential backoff + jitter for payment-service callers (see Ch 10 – Resilience Patterns)	API gateway team	P1	2026-04-10	TODO
6	Classify <code>max_connections</code> and similar params as high-risk; require second approval for changes	Payments team	P1	2026-03-30	TODO
7	Add config change integration test: deploy test config and verify service health	Payments team	P2	2026-04-30	TODO

Lessons Learned

- **Config is code** – it needs the same validation, testing, and staged rollout as application code. A single unvalidated field can have the same impact as a code defect.
- **Leading indicators prevent incidents.** The connection pool metric was available but not alerted on. An alert at 80% utilization would have paged before any user-visible failure.
- **Retry without backoff is a reliability hazard.** Every caller that retries aggressively during an outage makes the outage worse. Backoff and circuit breaking (Ch 10) are not optional for inter-service calls.

Appendices

A. Graphs

- Error rate: [Grafana link – checkout error rate 14:00–16:00 UTC]
- Connection pool: [Grafana link – db_connections_used vs max]
- Latency: [Grafana link – p50/p99 for checkout, payment-service]

B. Related Incidents

- 2026-01-20: Similar config error in inventory-service (postmortem: link)

- 2026-02-08: Retry storm during fraud-check degradation (postmortem: [link](#))

C. Detection and Response Metrics

Metric	Value
TTD (time to detect)	3 min (14:25 deploy → 14:28 alert)
TTA (time to acknowledge)	2 min (14:28 page → 14:30 ack)
TTM (time to mitigate)	24 min (14:28 alert → 14:52 rollback)
TTR (time to resolve)	37 min (14:28 alert → 15:05 baseline)

What makes this postmortem effective

- **Timeline is precise and sourced.** Every entry has a timestamp and can be verified against chat logs, deploy records, and alert history. Vague timelines ("around 2:30 we noticed...") are not useful for analysis.
- **Impact is quantified.** Not "some users were affected" but "12,000 attempts, 4,800 first-attempt failures, ~720 abandoned." Quantification drives prioritization of action items.
- **Contributing factors, not single root cause.** The postmortem identifies 5 factors, any one of which — if fixed — would have prevented or mitigated the incident. This produces 5 independent defenses (defense in depth) rather than a single fix that leaves the other holes open.
- **Action items are specific, owned, prioritized, and dated.** Not "improve validation" but "add range validation for all config values in deploy pipeline; reject outside expected bounds — Platform team, P0, due 2026-03-25."
- **What went well is included.** Recognizing what worked (fast detection, quick rollback, no data loss) reinforces good practices and balances the natural focus on what went wrong.

Root cause analysis

The single-root-cause trap

Most incidents do not have a single root cause. They have a **chain of contributing factors** where removing any one link would have

prevented the incident or reduced its impact. Insisting on "the" root cause leads to:

- Fixing only the most proximate cause (the typo) while leaving systemic causes (no validation, no canary) unaddressed.
- Premature closure — once "the" cause is found, investigation stops even though other important factors remain.

Instead, think in terms of **contributing factors** and **defense layers**:

Typo:
max_connections=10

Defense layers — each could have prevented the incident

Layer 1: Code review
catch the typo

reviewer missed it

Layer 2: Config
validation
reject out-of-range value

no validation

Layer 3: Canary deploy
limit blast radius to 5%

no canary

Layer 4: Connection
pool alert
detect before user
impact

194

no alert

Figure 6-2: Defense in depth. The incident required every layer to fail simultaneously. Fixing any single layer prevents recurrence through that path — fixing multiple layers provides resilience even when one fix is incomplete or regresses.

Each layer that failed is an independent action item. Even if one fix is imperfect (e.g., range validation has a gap for a new config param), the other layers still protect.

Five Whys

Five Whys is a simple technique for drilling past the proximate cause to systemic factors. Applied to the example incident:

Why	Answer
Why did checkout fail?	payment-service exhausted its DB connection pool and rejected requests.
Why was the pool exhausted?	<code>max_connections</code> was set to 10 instead of 100.
Why was it set to 10?	A config file was manually edited and the value was mistyped.
Why was the mistyped value deployed?	The deploy pipeline does not validate config values against expected ranges.
Why does the pipeline not validate?	Config validation was never implemented — config was assumed to be low-risk and reviewed only by visual inspection.

Five Whys is useful but has pitfalls:

- **It implies a single linear chain.** Real incidents have branching causes — Five Whys on one branch misses the others (no canary, no alert, retry storm).
- **It can become blameful.** "Why did Alice make a typo?" leads to "because she was careless" — the wrong answer. The right answer is "because the system allowed a typo to reach production without validation."
- **"Why" can be asked forever.** Stop when you reach a fixable systemic cause, not when you reach philosophy.

Use Five Whys as a starting point for one causal chain, then complement it with broader contributing-factor analysis.

Fishbone (Ishikawa) diagram

For incidents with multiple contributing factor categories, a fishbone diagram organizes them:

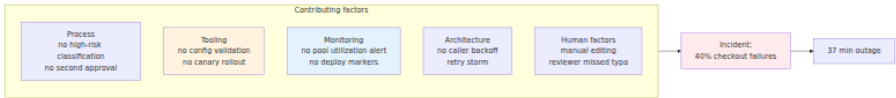


Figure 6-3: Fishbone categorization of contributing factors. Each category suggests a different type of fix — process, tooling, monitoring, architecture, or human factors (where the fix is always tooling or process, not telling humans to be more careful).

The counterfactual trap

A common postmortem anti-pattern is the counterfactual: "If Alice had been more careful, this would not have happened" or "If the reviewer had caught the typo, we would have been fine." Counterfactuals that depend on humans being more careful are not useful — they do not produce fixes that work when humans are tired, rushed, or new to the team.

The test for a useful counterfactual:

- Useless: "If the engineer had typed 100 instead of 10..." (depends on perfect human performance)
- Useful: "If the pipeline had validated that `max_connections` is between 50 and 500..." (a concrete, automatable check)
- Useful: "If the config had been canaried to 5% of instances first..." (an architectural improvement)
- Useful: "If callers had used exponential backoff..." (a resilience pattern from Chapter 10)

Every action item should pass the counterfactual test: **would this fix still prevent the incident if a different person made the same mistake under the same conditions?**

The postmortem meeting

The postmortem meeting is where the document is reviewed, debated, and refined — not where it is written for the first time. The author circulates the draft before the meeting so participants can read it in advance.

Facilitation

Role	Responsibility
Facilitator	Keeps discussion blameless, ensures all voices are heard, manages time, prevents rabbit holes. Ideally someone not directly involved in the incident.
Author	Presents the timeline and analysis, answers questions, captures feedback for revision.
Participants	All responders, SMEs, and stakeholders who can contribute context or challenge assumptions.

Facilitation techniques:

- **Start with the timeline.** Walk through events chronologically. This grounds the discussion in facts before moving to analysis and judgment.
- **Ask "how" and "what," not "who" and "why did you."** "How did the deploy process allow this value through?" not "Why did you deploy a bad config?"
- **Invite dissent.** "Does anyone see a contributing factor we missed?" or "Is there a perspective from the database team we have not heard?"
- **Time-box analysis.** Postmortem meetings should be 60-90 minutes. If root cause analysis needs more depth, schedule a follow-up rather than extending the meeting. Fatigue degrades blamelessness.
- **End with action items.** The last 15 minutes should be spent reviewing each action item for specificity, ownership, and priority. An action item that no one owns at the end of the meeting will not be completed.

Psychological safety in the meeting

The meeting is where blameless culture is most visibly tested. Warning signs that safety is eroding:

Signal	What it means	Facilitator response
Silence from junior engineers	They fear speaking up or contradicting senior staff	Directly invite them: "You were closest to the database during the incident — what did you see?"
"We already know what happened"	Premature closure, likely anchoring on the first plausible cause	"Let's make sure we've considered all contributing factors before concluding."
Naming individuals as causes	Blameful framing	Redirect: "Let's describe the action and the system context, not the person."
Defensiveness or justification	Someone feels accused	Acknowledge: "This is about how the system behaved, not about anyone's competence. What would have helped you in that moment?"
Action items assigned as punishment	Blame disguised as accountability	Ensure items are volunteered or assigned to teams, not to individuals as consequences.

Action items: from words to fixes

A postmortem without completed action items is a story, not an improvement. The action item lifecycle is where most postmortem programs fail — documents are written, action items are created, and then they sit in a backlog until the next incident reveals they were never done.

Prioritization

Priority	Meaning	Timeline	Example
P0	Would have directly prevented this incident; blocks recurrence through the primary path	Days to 2 weeks	Config range validation, connection pool alert
P1	Would have reduced blast radius or detection time; defense-in-depth	2-6 weeks	Canary deploys, deploy markers on dashboards, caller backoff
P2	General improvement surfaced by the incident; not specific to this failure mode	Backlog, next quarter	Config integration test, broader pipeline hardening

P0 items should be tracked at the same urgency as incident follow-up — not as normal sprint work that can be deprioritized for feature deadlines. If P0 postmortem items are routinely delayed, the organization is choosing to accept recurrence.

Ownership and tracking

- **Every action item has a single owner** (person or team) and a **due date**. Items without owners are wishes; items without dates are eventually forgotten.
- **Track action items in the same system as other engineering work** (Jira, Linear, GitHub issues) — not in a separate postmortem tracker that no one checks. Link each item back to the postmortem document.
- **Review action items at a regular cadence** — weekly for P0s, sprintly for P1s. Many teams have a dedicated "reliability review" meeting that covers open postmortem items alongside SLO status and error budget.
- **Verify completion**. An action item is not done when the code is merged — it is done when the fix is deployed, verified in production, and the relevant runbook or alert is updated. "Added validation" is not done until a test proves that `max_connections=10` is now rejected by the pipeline.

Measuring action item health

```
# Example queries for postmortem action item hygiene
# (adapt to your tracker – Jira JQL, Linear API, etc.)

# How many P0 action items are overdue?
# JQL: project = SRE AND labels = postmortem-action AND priority = P0
AND due < now() AND status != Done

# Postmortem completion rate: what fraction of postmortems have all P0
items completed?
# Track: (postmortems with all P0s done) / (total postmortems requiring
P0s)

# Mean time to P0 completion
# Track: avg(days between postmortem publication and last P0 item
resolved)
```

Healthy benchmarks (approximate, varies by org size):

- P0 completion within due date: > 90%
- P1 completion within due date: > 70%
- Postmortems published within SLA (5 days for Sev1): > 95%
- No postmortem with P0 items open beyond 30 days

If these numbers slip, it usually indicates that postmortem action items are competing with feature work without explicit prioritization — an organizational signal that reliability investment needs executive support.

Sharing and organizational learning

Postmortem distribution

Postmortems should be shared as broadly as is safe and useful:

Audience	What they get	Why
Responders and involved teams	Full document — timeline, root cause, all action items	They need the complete picture to implement fixes correctly.

Audience	What they get	Why
All engineering	Full document or detailed summary (depending on org size)	Every team can learn from every incident — the failure mode in payment-service may exist in their service too.
Engineering leadership	Full document + trend summary	Leadership needs to see patterns (repeated config errors, missing validation) to prioritize systemic investment.
Company-wide / executives	Executive summary — impact, cause in plain language, what is being done	Executives need to understand reliability posture without reading technical deep-dives.
Customers (for Sev1)	External incident report — impact, cause (non-technical), remediation, prevention	Customers who were affected deserve a clear account. Many companies publish these on their status page or blog.
Public (optional)	Blog post or conference talk	Sharing postmortems publicly builds industry knowledge and demonstrates engineering maturity. Companies like Cloudflare, GitHub, and AWS publish detailed public postmortems.

Storing postmortems in a searchable, centralized location (wiki, dedicated postmortem tool, or simply a well-organized document repository) ensures they are findable when a similar incident occurs. Tag postmortems by service, failure mode, and contributing factor category so trends can be analyzed.

Trend analysis

Individual postmortems fix individual incidents. Trend analysis across many postmortems reveals systemic patterns:

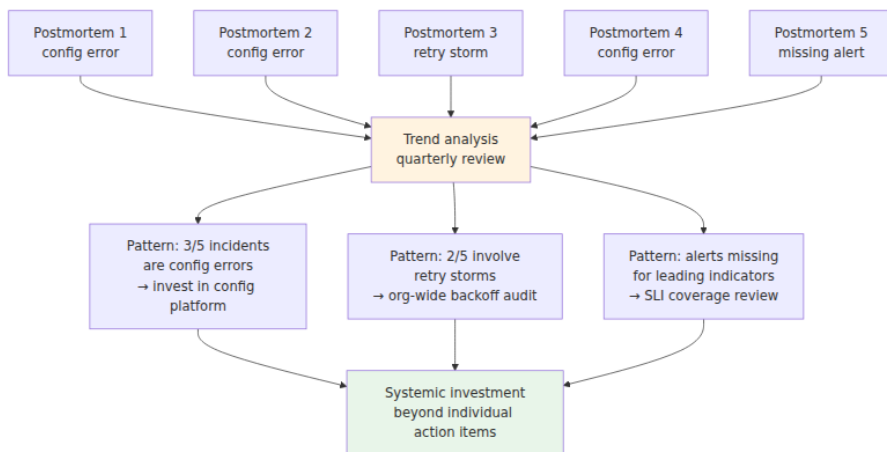


Figure 6-4: Trend analysis across postmortems. Individual action items fix individual holes; trend analysis identifies where to make systemic investments that prevent whole classes of incidents.

Quarterly, review all postmortems for:

- **Repeated failure modes.** Three config-related incidents in a quarter is a signal that config management needs systemic investment, not three independent fixes.
- **Repeated contributing factors.** If "no validation" or "missing alert" appears in multiple postmortems, the organization has a pattern of shipping without safety nets.
- **Action item completion rate.** If P0 items are frequently overdue, the reliability investment model needs adjustment.
- **TTD/TTM trends.** Are incidents being detected and mitigated faster over time? If not, observability or runbook investment may be needed.

Postmortems and error budgets

Postmortems connect directly to the error budget concept from Chapter 1:

- An incident that **burns significant error budget** creates urgency for its action items — the budget quantifies the cost and justifies the investment.

- If error budget is **exhausted**, feature work should slow and postmortem action items should be prioritized — this is the error budget doing its job as a governance mechanism.
- Postmortems that produce **new SLIs or tighter SLOs** (e.g., adding a connection pool utilization SLI after the incident above) improve the error budget's accuracy for future decisions.

Postmortems and deployment velocity

There is a common fear that postmortem culture slows teams down — more process, more approvals, more caution. In practice, the opposite is true for organizations that do it well:

- **Postmortems that produce automation** (validation, canary deploys, auto-rollback) make deploys *safer and faster*, not slower. The deploy that previously required manual checklist verification now has automated guardrails.
- **Blameless culture increases deploy frequency** because engineers are not afraid to deploy. Fear of blame for incidents leads to deploy avoidance, large batch sizes, and infrequent releases — which themselves increase incident risk (larger changes are harder to debug and roll back).
- **Shared learning reduces duplicate investigation.** When every team can read every postmortem, they can proactively fix the same failure mode in their own services before it causes an incident.

The DORA research program (Forsgren et al., *Accelerate*) consistently finds that high-performing teams have both **higher deployment frequency** and **faster MTTR** — and that blameless postmortem culture is one of the capabilities that predicts both.

Distributed-systems lens

Postmortems in distributed backends have specific characteristics that single-service postmortems do not:

Contributing factors span team boundaries. The retry storm in the example incident involved the API gateway team (callers), the payments team (service), and the platform team (deploy pipeline). The postmortem must be cross-team, with contributors from each — and action items

owned by different teams. A postmortem that only involves the team whose service failed will miss the systemic fixes in adjacent teams.

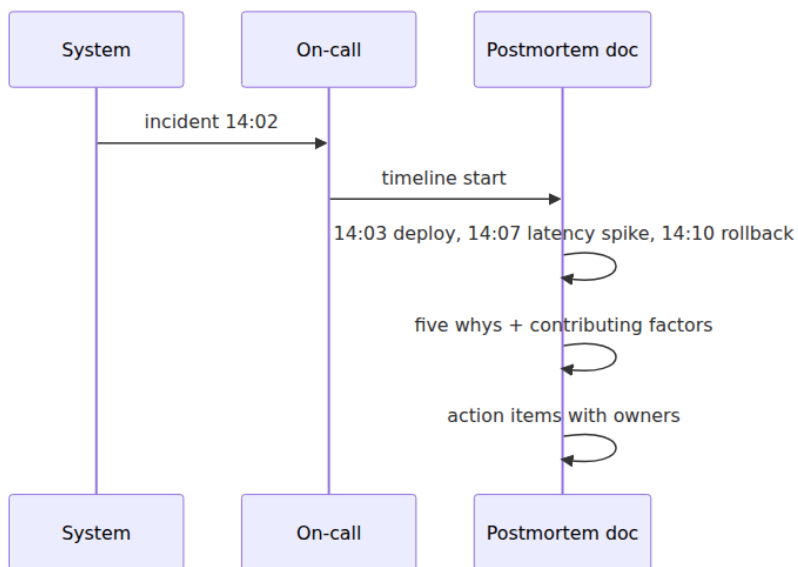
Root cause analysis must consider distributed failure modes. Was the incident caused by a consistency issue (stale cache, eventual consistency window), a coordination failure (leader election, distributed lock), a cascading overload, or a simple local error that propagated? The analysis technique must match the failure mode — Five Whys on a cascading failure may miss the architectural question of why services share fate through a single database.

Action items may require architectural investment. Some postmortem findings cannot be fixed with a config change or an alert — they require architecture: bulkheads to isolate failure domains, circuit breakers to prevent retry storms, idempotency to make retries safe (all covered in Chapter 10). These action items are larger, cross-team, and longer-term — they need explicit tracking beyond the sprint.

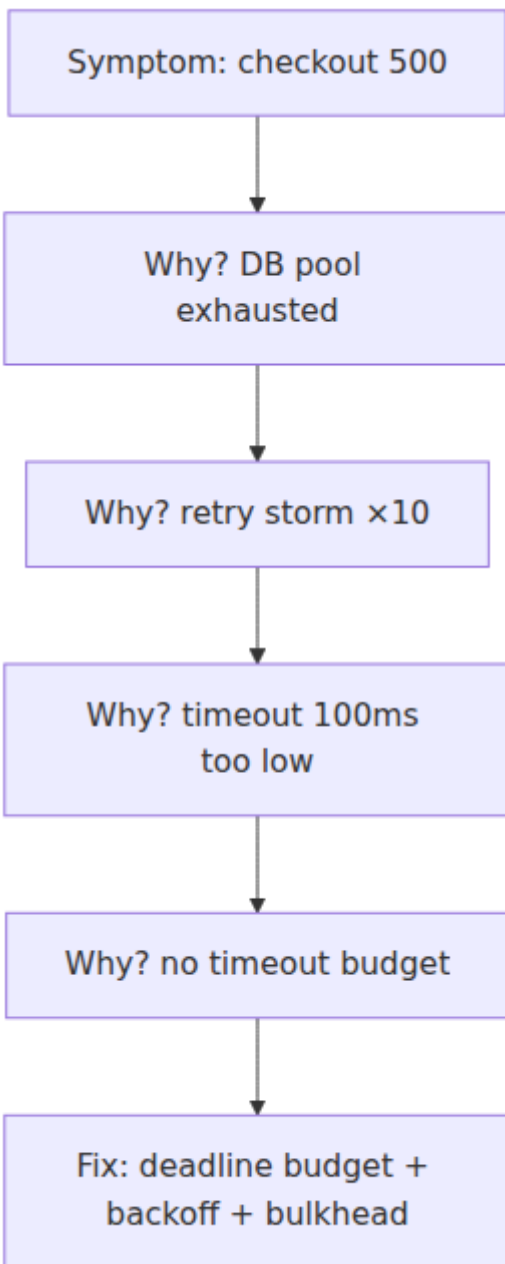
Postmortem sharing must cross service boundaries. A config validation gap found in payment-service almost certainly exists in other services that use the same deploy pipeline. The postmortem's audience is not just the payments team — it is every team that deploys through the same pipeline. Broad distribution is how one incident's learning prevents the next incident in a different service.

Near-misses in distributed systems are especially valuable. A near-miss where a network partition *almost* caused data inconsistency, or where a failover *almost* did not complete, reveals a distributed correctness gap that testing (Chapter 12, Volume 6) may not have covered. These near-misses deserve postmortems even though no user was affected — the next partition may not be so lucky.

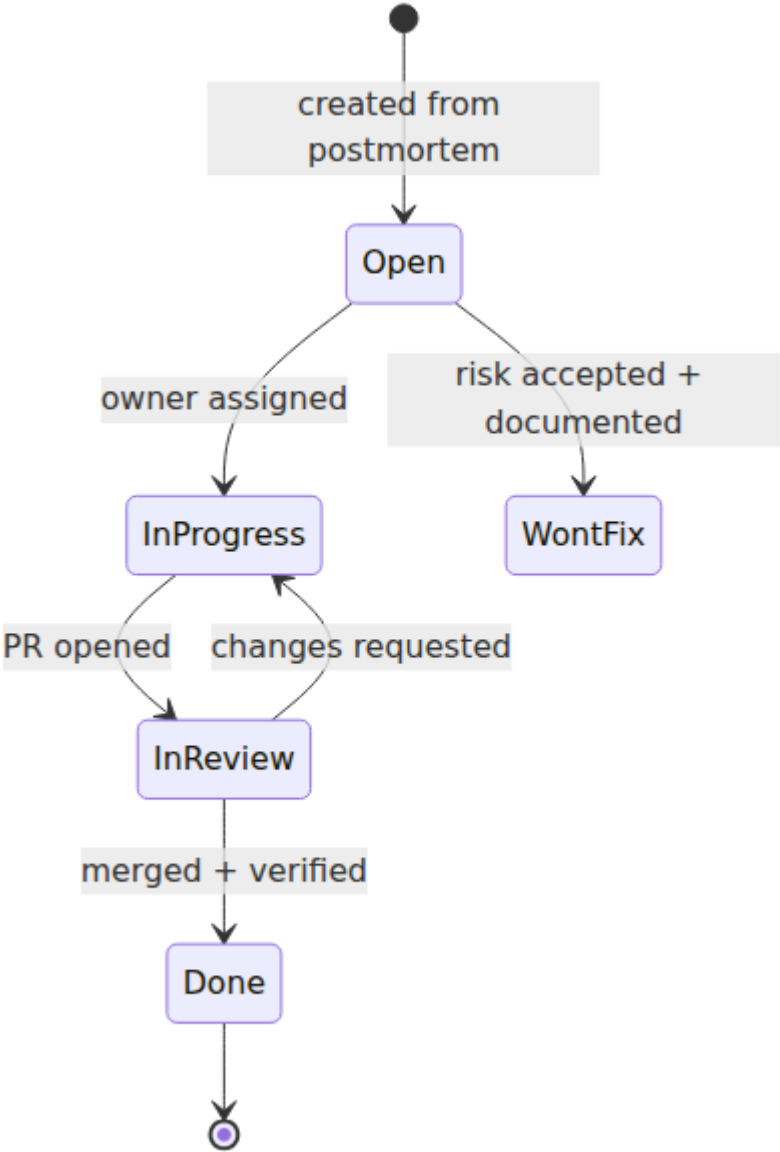
Postmortem Timeline



Five Whys



Action Item Lifecycle



Key takeaways

- Postmortems convert the **cost of an incident into durable improvement** — without them, the same failure recurs with larger blast radius as the system grows, and knowledge leaves with the responders who hold it tacitly.
- **Blamelessness is a practical strategy**, not just a cultural value: blame suppresses information flow, hides systemic causes, and makes responders hesitate during incidents — all of which increase MTTR. Blameless language describes what the system allowed, not who is at fault, and passes the "reasonable person" test.
- Write postmortems for **every Sev1 and Sev2, novel Sev3s, and instructive near-misses** — especially near-misses where the only reason impact was avoided is something you would not want to rely on next time. Start the document within 24–48 hours while memories are fresh.
- An effective postmortem document contains: **metadata** (severity, duration, impact, responders), **summary**, **quantified impact**, **precise timeline**, **root cause and contributing factors** (not single root cause), **what went well / poorly**, **specific owned action items with priorities and due dates**, **lessons learned**, and **appendices** with graphs and related incidents.
- Avoid the **single-root-cause trap** — think in contributing factors and defense layers, where fixing any one layer would have prevented the incident. Every failed layer is an independent action item providing defense in depth.
- **Five Whys** is useful for drilling one causal chain but must be complemented with broader contributing-factor analysis (fishbone diagrams) and must avoid blameful counterfactuals — every fix should pass the test: would it still work if a different person made the same mistake?
- The **postmortem meeting** reviews the pre-circulated document, not writes it from scratch; the facilitator enforces blameless language, invites dissent, ensures all voices are heard, and ends with action item review (specificity, ownership, priority).
- **Action items are not done when the document is published** — they are done when the fix is deployed, verified in production, and the relevant runbook or alert is updated. Track them in the same

system as other engineering work, review POs weekly, and measure completion rates — overdue POs are a signal that reliability investment needs executive support.

- **Share postmortems broadly** (all engineering, leadership, customers for Sev1, optionally public) and perform **quarterly trend analysis** to identify repeated failure modes and systemic investment opportunities beyond individual action items.
 - Postmortems close the **reliability feedback loop**: they produce new SLIs, new alerts, new runbooks, and architectural fixes (Chapters 7-10) that make the entire system more reliable — and **blameless culture increases deployment velocity** by replacing fear with automated safety nets.
-

Further reading

- Google SRE Book, Chapter 15 — *Postmortem Culture: Learning from Failure* (<https://sre.google/sre-book/postmortem-culture/>)
- Google SRE Workbook, Chapter 9 — *Postmortem Culture* (<https://sre.google/workbook/postmortem-culture/>)
- Etsy — *Debriefing Facilitation Guide* (<https://www.etsy.com/codeascraft/debriefing-facilitation-guide>) — the definitive guide to facilitating blameless postmortems, by John Allspaw.
- Sidney Dekker, *The Field Guide to Understanding Human Error* (Ashgate, 2014) — the intellectual foundation for blameless culture: human error as a symptom of systemic factors, not a cause.
- Nora Jones et al., *Incident Analysis: How to Learn from Incidents* — modern approaches to incident analysis beyond traditional postmortems.
- Jeli / Ben Sigelman — *Modern Incident Analysis* — evolving practices for learning from incidents as socio-technical system behavior.
- Forsgren, Humble, and Kim, *Accelerate* (IT Revolution, 2018) — DORA research linking blameless culture, deployment frequency, and reliability.
- Cloudflare, GitHub, and AWS public postmortems — real-world examples of transparent incident reporting (<https://blog.cloudflare.com/>, <https://github.blog/category/engineering/>, <https://aws.amazon.com/blogs/architecture/>).

- Lorin Hochstein, *Beyond Blameless Postmortems* — critique and extension of blameless practices for complex systems.

Chapter 7 — Load Testing and Capacity Planning

What this chapter covers. Confidence that a system will handle next week's traffic cannot be derived from dashboards alone — it must be earned by generating realistic load and observing where the system bends and breaks, then translating those observations into concrete capacity decisions. This chapter covers the full lifecycle: why load testing is distinct from functional testing, the workload models that determine whether your results mean anything, the taxonomy of tests from smoke to soak, the queueing theory that predicts saturation, the tooling that generates load at scale (k6, Vegeta, Gatling, Locust), and the capacity planning process that turns measurements into provisioning, autoscaling, and budgeting decisions. Every concept is grounded in runnable configs and production-safe practices.

Learning goals — after this chapter you should be able to:

- Distinguish smoke, load, stress, spike, soak, and breakpoint tests and choose the right type for a given question about system behavior.
- Explain open versus closed workload models, why the choice determines whether your test measures queueing or throughput, and when to use each.
- Apply Little's Law, utilization law, and the Universal Scalability Law to predict saturation points and reason about scaling limits.
- Write, run, and interpret k6 and Vegeta tests — including thresholds, checks, custom metrics, and distributed execution — and compare them to Gatling and Locust for different use cases.
- Design realistic load scenarios from production traffic data, manage test data and environment isolation, and avoid the common pitfalls that invalidate results.
- Build a capacity model that connects RPS, latency SLOs, and resource headroom to provisioning, autoscaling policies, and error budget planning.

Why load testing

Functional tests do not answer capacity questions

A service that passes every unit and integration test can still collapse the first time traffic doubles — because functional tests verify *correctness* for a single request in isolation, while production load is *concurrent, sustained, bursty, and heterogeneous*. Load testing closes that gap by asking four questions functional tests cannot:

1. **How much can we handle?** What is the maximum throughput before latency SLOs are breached or errors spike?
2. **How does it degrade?** Is degradation graceful (latency rises linearly) or catastrophic (error rate jumps to 80% at a cliff)?
3. **How long can it sustain?** Does the system leak memory, exhaust connections, or accumulate GC pressure over hours?
4. **Where is the bottleneck?** Is it CPU, memory, I/O, downstream dependency, connection pool, or lock contention?

Without answers to these, capacity planning is guesswork — provisioning by gut feel, autoscaling by hope, and incident response by surprise.

The cost of skipping it

Real incidents trace directly to untested capacity assumptions:

- A database connection pool sized for steady state but never tested under burst — when a downstream slowdown caused retries, the pool exhausted in seconds and every service sharing the database went down.
- An autoscaler configured with a 60-second scale-up delay but load tested only at steady state — during a flash sale, traffic tripled in 20 seconds; the scaler never caught up and the service shed 40% of requests.
- A cache layer that hit 99% hit rate in load tests with uniform key distribution — but production traffic was Zipfian (a few hot keys), so a single cache node became the hotspot and tail latency spiked to seconds.

Each of these would have been caught by a load test that modeled realistic traffic — not just high request volume.

Where load testing sits in the reliability lifecycle

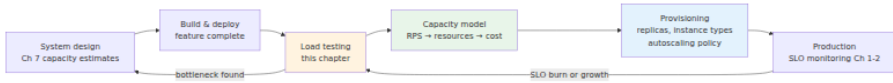


Figure 7-1: Load testing closes the loop between design-time estimates and production reality. Results feed the capacity model, which drives provisioning — and production SLO data triggers the next round of testing as traffic and architecture evolve.

Boundary note. Back-of-envelope estimation and system-level capacity math are introduced in Volume 7, Chapter 2; this chapter applies them with measured data. Autoscaling mechanics for Kubernetes workloads are covered in Volume 12, Chapter 3; this chapter covers how to *validate* that those scalers behave correctly under load. Resilience under failure is Chapter 8 (chaos) and Chapter 10 (patterns) in this volume.

Workload models: open versus closed

The single most consequential choice in load testing — and the most commonly misunderstood — is the workload model. It determines whether your test measures what production actually does.

Closed model (concurrency-limited)

A fixed number of virtual users (VUs) each loop: send request, wait for response, think time, repeat. The arrival rate is *dependent* on response time — if the system slows down, fewer requests arrive per second.

- **Analogy:** A call center with N agents — each agent handles one call at a time; if calls take longer, fewer calls are handled per hour.
- **Use when:** Modeling user sessions where a user waits for a response before acting (browser clicks, mobile app interactions with think time).
- **Risk:** Masks queueing effects. If latency doubles, arrival rate halves, so the system *appears* to handle the load — but in production with an open arrival process, the queue would grow without bound.

Open model (arrival-rate-limited)

Requests arrive at a fixed rate independent of response time — new arrivals keep coming even if the system is slow. The arrival rate is *exogenous*.

- **Analogy:** HTTP requests arriving from the internet — users do not coordinate; if your service slows from 50 ms to 500 ms, the same 1000 requests per second keep arriving.
- **Use when:** Modeling API traffic, webhook ingestion, or any system where callers do not wait for your response before sending the next request (which is most backend-to-backend traffic).
- **Risk:** Can overwhelm a system catastrophically if the arrival rate exceeds service rate — which is exactly what you want to observe in a stress test, but dangerous if run against shared environments without isolation.

Why it matters: the queueing trap

Under a closed model, the classic test pattern is "ramp VUs from 10 to 500 and measure throughput." As VUs increase, throughput rises until the system saturates — then latency increases but throughput *plateaus* because VUs are blocked waiting. The tester concludes "system handles 500 VUs, throughput is X." But under an open model with the same throughput, latency would have grown unboundedly as the queue backed up — a qualitatively different result. If your production traffic is open (and for internet-facing services it almost always is), a closed-model test *underestimates tail latency by hiding queueing*.

The fix is not to always use open — both models are valid for different scenarios — but to *match the model to production traffic* and to use open models when testing backend APIs where callers do not synchronize on responses.



Figure 7-2: Closed versus open workload models. Closed models self-throttle when latency rises, hiding queueing. Open models expose queue growth — which is why they are preferred for backend API testing.

Most modern tools support both: k6 supports open (constant arrival rate via `ramping-arrival-rate` executor) and closed (VU-based) executors;

Vegeta is inherently open (rate-driven); Gatling supports both via injection profiles.

A taxonomy of load tests

Not every load test has the same goal. Choosing the wrong type wastes time or misses the signal.

Test type	Goal	Load profile	Duration	Success signal
Smoke	Verify test harness works; catch gross regressions	Minimal (1-5 VUs or low RPS)	1-5 min	All checks pass, no errors
Load	Validate SLO compliance at expected peak	Expected peak traffic (e.g., p95 daily peak $\times$ 1.2)	10-30 min	p95 latency within SLO, error rate < threshold
Stress	Find the breaking point	Ramp beyond peak until failure	15-60 min	Identify cliff: RPS at which errors or latency breach SLO
Spike	Test elasticity under sudden surge	Instant jump (e.g., 100 $\rightarrow$ 5000 RPS in seconds)	5-15 min	Autoscaler catches up, no sustained error burst
Soak	Detect slow leaks and drift	Sustained peak (or 80% of peak)	2-12 hours	No memory leak, no GC degradation, no connection exhaustion
Breakpoint	Precisely locate capacity limit	Incremental steps with hold periods	30-90 min	Throughput vs. latency curve, knee and cliff identified

How they compose into a test suite

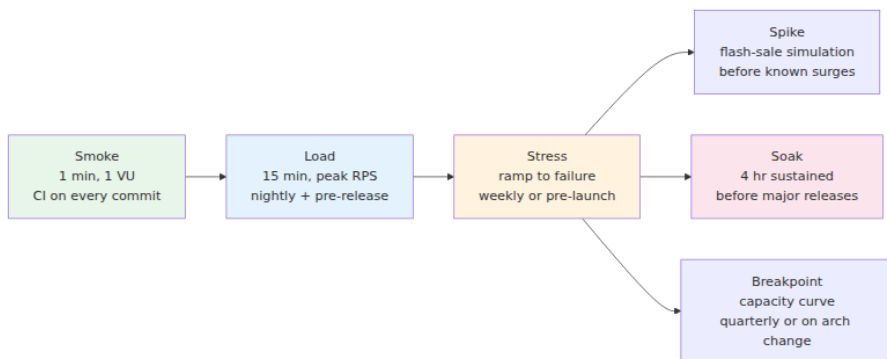


Figure 7-3: Progressive load test suite. Fast smoke tests gate every commit; heavier tests run on progressively longer cadences. Each type answers a different capacity question.

Breakpoint testing deserves special attention because it produces the most actionable output: a curve of throughput versus latency and error rate as load increases. The *knee* (where latency starts rising superlinearly) is the practical capacity limit; the *cliff* (where errors spike) is the hard failure point. Capacity planning targets the knee, not the cliff.

Capacity math: Little's Law, utilization, and the USL

Load testing produces measurements — but capacity planning requires *reasoning* about those measurements. Three laws give you the framework.

Little's Law: concurrency = throughput × latency

$$L = \lambda \times W$$

Where **L** is the average number of requests in the system (concurrency), **λ** is arrival rate (throughput), and **W** is average time in system (latency including queueing). This holds for any stable system regardless of distribution.

Practical use: If your p50 latency is 40 ms at 2000 RPS, average concurrency is $2000 \times 0.04 = 80$ concurrent requests. If your thread pool or connection pool has 100 slots, you have 20% headroom. If latency

doubles to 80 ms at the same RPS, concurrency doubles to 160 — exceeding the pool and causing queueing or rejection. Little's Law tells you that latency and concurrency are not independent — you cannot reason about one without the other.

Utilization law: utilization = throughput × service time

$$U = \lambda \times S$$

Where U is resource utilization (0 to 1), λ is throughput, and S is mean service time at that resource. For a CPU-bound service where each request needs 2 ms of CPU, at 400 RPS the CPU utilization is $400 \times 0.002 = 0.8$ (80%). Beyond 80–85% utilization, queueing theory predicts latency rises sharply — which is why targeting > 85% steady-state utilization is dangerous without careful measurement.

Universal Scalability Law (USL)

The USL models how throughput scales with concurrency, accounting for two penalties that grow with scale:

$$X(N) = N / (1 + \alpha(N-1) + \beta N(N-1))$$

Where N is concurrency (or nodes), α is the contention penalty (serial fraction, Amdahl's law), and β is the coherency penalty (cross-node coordination cost). Fitting measured throughput versus concurrency to the USL reveals:

- **α -dominated systems** hit diminishing returns but plateau gracefully — more concurrency helps less and less.
- **β -dominated systems** exhibit *retrograde* scalability — beyond some N , throughput *decreases* as concurrency increases (common with contended locks, cache coherency traffic, or distributed consensus).

Practical use: Run a breakpoint test that measures throughput at increasing concurrency levels, fit the USL, and identify whether your next scaling investment should target contention reduction (faster locks, sharding) or coherency reduction (less cross-node coordination). If β is significant, adding more replicas may make things worse, not better.

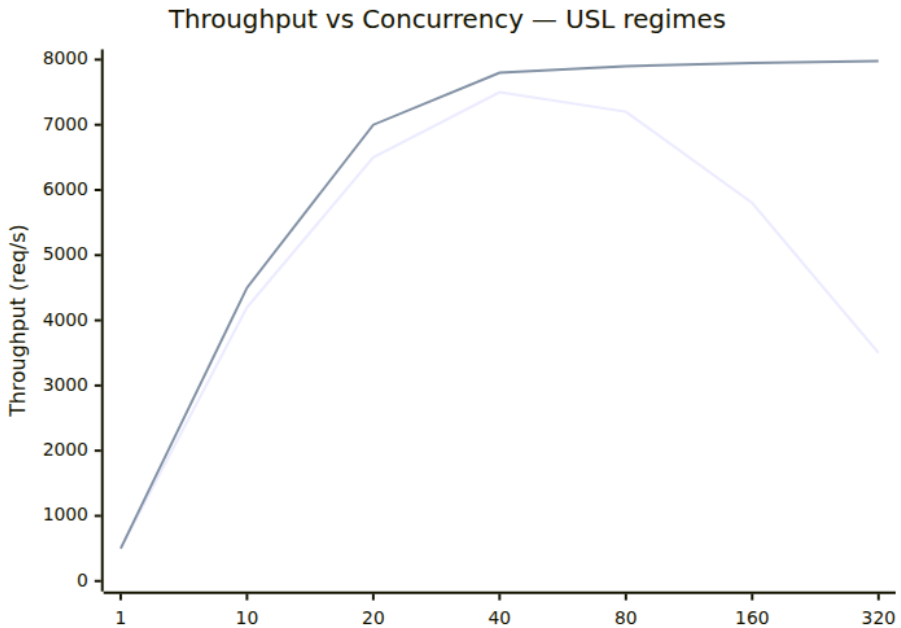


Figure 7-4: Two systems measured at increasing concurrency. Blue (high β) shows retrograde scaling — throughput collapses past 40 concurrent workers due to coherency costs. Green (low β) plateaus gracefully. Fitting the USL to these curves tells you which regime you are in and where to invest.

Queueing intuition without the heavy math

You do not need full queueing theory to make capacity decisions — one result suffices: for an M/M/1 queue (Poisson arrivals, exponential service), mean response time is:

$$R = S / (1 - U)$$

Where S is service time and U is utilization. At 50% utilization, $R = 2S$. At 80%, $R = 5S$. At 90%, $R = 10S$. The relationship is *nonlinear* — the last 10% of utilization costs 5× more latency than the first 50%. This is why capacity headroom matters and why load tests must measure latency distributions, not just throughput.

Tooling: k6, Vegeta, and the landscape

k6: scriptable, SLO-aware, developer-friendly

k6 is the default recommendation for most backend teams: JavaScript scenarios, first-class thresholds and checks, open and closed executors, and excellent Prometheus/InfluxDB output. It is written in Go, so a single binary generates significant load without the JVM overhead of Gatling.

Installation:

```
brew install k6           # macOS
# or
sudo gpg -k && sudo gpg --no-default-keyring --keyring /usr/share/
keyrings/k6-archive-keyring.gpg --keyserver hkp://
keyserver.ubuntu.com:80 --recv-keys C5AD60C380CEF53E8C86DF8424ECA9E5B91
0487D && echo "deb [signed-by=/usr/share/keyrings/k6-archive-
keyring.gpg] https://dl.k6.io/deb stable main" | sudo tee /etc/apt/
sources.list.d/k6.list && sudo apt-get update && sudo apt-get install k
6
```

Comprehensive k6 scenario — open model with thresholds, checks, and custom metrics:

```

// scenarios/checkout-load.js – open-model load test for a checkout API
import http from 'k6/http';
import { check, group, sleep } from 'k6';
import { Counter, Trend, Rate } from 'k6/metrics';
import { randomItem } from 'https://jslib.k6.io/k6-utils/1.4.0/index.js';

// Custom metrics
const checkoutErrors = new Counter('checkout_errors');
const checkoutLatency = new Trend('checkout_latency', true);
const paymentFailures = new Rate('payment_failures');

const BASE_URL = __ENV.BASE_URL || 'https://api.staging.example.com';
const PRODUCTS = ['sku-1001', 'sku-1002', 'sku-1003', 'sku-1004'];

export const options = {
  // Open-model: arrival-rate executors
  scenarios: {
    steady_state: {
      executor: 'ramping-arrival-rate',
      startRate: 50,
      timeUnit: '1s',
      preAllocatedVUs: 100,
      maxVUs: 500,
      stages: [
        { target: 200, duration: '2m' }, // ramp to expected peak
        { target: 200, duration: '5m' }, // hold at peak – measure
        { target: 500, duration: '3m' }, // stress ramp
        { target: 500, duration: '3m' }, // hold at stress
        { target: 0, duration: '1m' }, // ramp down
      ],
    },
    // Spike scenario – runs in parallel to test burst handling
    spike: {
      executor: 'ramping-arrival-rate',
      startRate: 0,
      timeUnit: '1s',
      preAllocatedVUs: 50,
      maxVUs: 200,
      startTime: '7m', // starts during
      steady_state_hold
      stages: [
        { target: 1000, duration: '10s' }, // instant spike
        { target: 1000, duration: '30s' },
        { target: 0, duration: '10s' },
      ],
    },
  },
};

```

```

// Thresholds – test fails if breached (CI gate)
thresholds: {
  http_req_failed: ['rate<0.01'], // error rate < 1%
  http_req_duration: ['p(95)<300', 'p(99)<800'], // SLO-aligned
  checkout_latency: ['p(95)<400'],
  payment_failures: ['rate<0.005'],
  checks: ['rate>0.99'], // 99% of checks pass
},
};

export function setup() {
  // Authenticate once, share token across VUs
  const res = http.post(`${BASE_URL}/auth/token`, JSON.stringify({
    client_id: __ENV.CLIENT_ID,
    client_secret: __ENV.CLIENT_SECRET,
  })), { headers: { 'Content-Type': 'application/json' } });
  check(res, { 'auth succeeded': (r) => r.status === 200 });
  return { token: res.json('access_token') };
}

export default function (data) {
  const headers = {
    'Authorization': `Bearer ${data.token}`,
    'Content-Type': 'application/json',
  };

  group('checkout flow', () => {
    // Step 1: Add to cart
    const cartRes = http.post(`${BASE_URL}/v1/cart`, JSON.stringify({
      product_id: randomItem(PRODUCTS),
      quantity: 1,
    })), { headers });

    const cartOk = check(cartRes, {
      'cart: status 201': (r) => r.status === 201,
      'cart: has cart_id': (r) => r.json('cart_id') !== undefined,
    });
    if (!cartOk) { checkoutErrors.add(1); return; }

    const cartId = cartRes.json('cart_id');

    // Step 2: Checkout – the critical path under test
    const checkoutRes = http.post(`${BASE_URL}/v1/checkout`, JSON.stringify({
      cart_id: cartId,
      payment_method: 'card_token_test',
    })), { headers });

    checkoutLatency.add(checkoutRes.timings.duration);

    const checkoutOk = check(checkoutRes, {

```

```

    'checkout: status 200': (r) => r.status === 200,
    'checkout: latency < 500ms': (r) => r.timings.duration < 500,
    'checkout: has order_id': (r) => r.json('order_id') !==
undefined,
  });

  if (!checkoutOk) {
    checkoutErrors.add(1);
    if (checkoutRes.status >= 500) paymentFailures.add(1);
    else paymentFailures.add(0);
  } else {
    paymentFailures.add(0);
  }
});

// Closed-model think time is NOT used here – open model paces via
arrival rate.
// If modeling browser users with think time, use the 'shared-
iterations' or 'per-vu-iterations' executor instead.
}

export function handleSummary(data) {
  return {
    'stdout': JSON.stringify({
      thresholds breached: Object.entries(data.metrics)
        .filter(([, m]) => m.thresholds && Object.values(m.thresholds)
          .some(t => !t.ok))
        .map([name] => name),
      p95_latency_ms: data.metrics.http_req_duration.values['p(95)'],
      p99_latency_ms: data.metrics.http_req_duration.values['p(99)'],
      error_rate: data.metrics.http_req_failed.values.rate,
      total_requests: data.metrics.http_reqs.values.count,
    }, null, 2),
    './results/summary.html': htmlReport(data),
    './results/summary.json': JSON.stringify(data, null, 2),
  };
}

```

Running it:

```

# Smoke – 1 min, low rate
k6 run --env BASE_URL=https://api.staging.example.com scenarios/
checkout-load.js --vus 1 --duration 1m

# Full scenario with Prometheus remote write (for Grafana dashboarding)
k6 run --out experimental-prometheus-rw \
--env BASE_URL=https://api.staging.example.com \
--env CLIENT_ID=test-client --env CLIENT_SECRET="$SECRET" \
scenarios/checkout-load.js

# Cloud/distributed execution (k6 Cloud or self-hosted k6-operator on
Kubernetes)
k6 cloud scenarios/checkout-load.js # or: kubectl apply -f k6-
distributed.yaml

```

Distributed k6 on Kubernetes (k6-operator):

```

# k8s/k6-distributed.yaml – 5 runners generating 2500 RPS aggregate
apiVersion: k6.io/v1alpha1
kind: TestRun
metadata:
  name: checkout-load-distributed
spec:
  parallelism: 5
  script:
    configMap:
      name: checkout-load-script
      file: checkout-load.js
  runner:
    image: grafana/k6:latest
    env:
      - name: BASE_URL
        value: "https://api.staging.example.com"
      - name: K6_PROMETHEUS_RW_SERVER_URL
        value: "http://prometheus.monitoring.svc:9090/api/v1/write"
  resources:
    requests: { cpu: "2", memory: "1Gi" }
    limits: { cpu: "4", memory: "2Gi" }

```

Vegeta: minimal, open-model, ideal for quick probes

Vegeta is a single-binary, rate-driven load generator — no scripting, just a target list and an attack rate. Perfect for quick breakpoint probes and CI smoke tests where you need to answer "can this endpoint handle X RPS?" without writing scenario logic.

```

# Install
go install github.com/tsenart/vegeta@latest

# Define targets
cat > targets.txt <<'EOF'
GET https://api.staging.example.com/v1/products
X-API-Key: test-key-123

POST https://api.staging.example.com/v1/cart
Content-Type: application/json
X-API-Key: test-key-123
@cart-payload.json
EOF

cat > cart-payload.json <<'EOF'
{"product_id": "sku-1001", "quantity": 1}
EOF

# Attack: open-model at 500 RPS for 2 minutes
vegeta attack \
  -targets=targets.txt \
  -rate=500 \
  -duration=120s \
  -timeout=5s \
  -connections=100 \
  -output=results.bin | tee >(vegeta report) | vegeta plot > plot.html

# Reports
vegeta report results.bin
# Requests      [total, rate, throughput]    60000, 500.00, 498.20
# Duration      [total, attack, wait]        2m0s, 2m0s, 1.2s
# Latencies     [mean, 50, 95, 99, max]      42ms, 31ms, 87ms, 210ms,
1.8s
# Bytes         [in, out]                    12.3MB, 4.1MB
# Success       [ratio]                      99.42%
# Status Codes  [code:count]                 200:59652  429:201  500:147
# Error Set:
# 429 Too Many Requests
# 500 Internal Server Error – connection pool exhausted

# Breakpoint: sweep rates to find the knee
for rate in 100 250 500 750 1000 1500 2000; do
  echo "=== Rate: ${rate} RPS ==="
  vegeta attack -targets=targets.txt -rate=$rate -duration=60s
  -output=/tmp/v-${rate}.bin
  vegeta report /tmp/v-${rate}.bin | grep -E "Latencies|Success|Status"
done | tee breakpoint-sweep.txt

# HdrHistogram report for tail-latency analysis
vegeta report -type="hdrplot" results.bin > latency.hdr

```

Choosing a tool

Tool	Model	Scripting	Best for	Limitation
k6	Both open & closed	JavaScript	Realistic multi-step user journeys with thresholds and CI gating	Single-node limit ~30k RPS without distribution
Vegeta	Open (rate-driven)	None (target file)	Quick probes, breakpoint sweeps, CI smoke, library use (<code>vegeta</code> as Go lib)	No scenario logic; one endpoint pattern at a time
Gatling	Both (injection DSL)	Scala DSL	Complex scenarios needing rich reports; enterprise teams on JVM	JVM memory/GC; heavier for simple probes
Locust	Closed (user-based)	Python	Teams that prefer Python; highly customizable workflows	Closed model only (until recently); Python GIL limits per-worker throughput
wrk / hey / fortio	Open	None / minimal	Microbenchmarks and quick latency checks	No scenario support; single-endpoint only

Designing realistic scenarios

A load test that does not resemble production traffic produces precise but irrelevant numbers. Realism has four dimensions.

1. Traffic mix

Production is not one endpoint at one rate — it is a distribution across endpoints, methods, and payload sizes. Derive it from access logs or tracing (Chapter 4):


```

# scripts/derive-traffic-mix.py - analyze access logs to produce k6
weights
import json
from collections import Counter

# Parse structured access logs (e.g., from Loki or CloudWatch)
endpoints = Counter()
with open("access.log") as f:
    for line in f:
        rec = json.loads(line)
        key = f"{rec['method']} {rec['route_template']}" # e.g.,
        "GET /v1/products/{id}"
        endpoints[key] += 1

total = sum(endpoints.values())
for endpoint, count in endpoints.most_common():
    pct = count / total * 100
    print(f"{endpoint:40s} {count:8d} {pct:5.1f}%")
# Output:
# GET /v1/products/{id} 452310 38.2%
# POST /v1/cart 218440 18.5%
# POST /v1/checkout 98420 8.3%
# GET /v1/cart/{id} 89310 7.5%
# ...

# Generate k6 scenario weights
print("\n// k6 weights:")
for ep, count in endpoints.most_common(5):
    print(f"// {ep}: weight {count/total:.2f}")

```

Replicate this mix in the test: 38% of VUs or arrival rate should hit `GET /v1/products/{id}`, not an equal split.

2. Data realism

- **Do not reuse the same entity.** Hitting `GET /v1/products/sku-1001` a million times tests that one cache entry — not the cache, the database, or the shard distribution. Use a realistic key distribution (often Zipfian — a few hot keys, many cold keys).
- **Generate unique payloads.** For writes, each request should create a unique entity (unique cart, unique user) or you will hit uniqueness constraint errors that mask real capacity limits.
- **Pre-seed data.** Before the test, seed the database with a realistic volume — a test against an empty database measures the wrong thing (no index depth, no compaction pressure).

```
// Zipfian key selection – a few keys get most traffic (realistic for
product catalog)
function zipfianPick(keys, skew = 1.0) {
  // Simple approximation: rank-based probability ~ 1/rank^skew
  const r = Math.random();
  const rank = Math.floor(Math.pow(r, -1 / skew)) % keys.length;
  return keys[Math.min(rank, keys.length - 1)];
}
```

3. Network and environment fidelity

- **Test in a production-like environment** — same instance types, same replica counts, same database size, same network topology. Testing against a single-node staging database and extrapolating to a 6-node production cluster is unreliable (different replication lag, different connection limits).
- **Isolate blast radius.** Never load test a shared staging environment that other teams depend on without coordination. Prefer ephemeral preview environments or dedicated load-test clusters. If testing production, use strict rate limits, off-peak windows, and explicit opt-in from on-call (see Chapter 8's blast radius controls — the same principles apply).
- **Account for downstream dependencies.** If the service under test calls 4 downstream services, those services also experience the load test's traffic. Either mock them (and document that the test does not cover downstream saturation) or coordinate so downstream teams expect the traffic.

4. Warm-up and steady state

Every system has warm-up effects: JIT compilation, connection pool filling, cache population, autoscaler delays. A test that ramps to peak instantly measures warm-up, not steady state.

- **Ramp gradually** (at least 1–2 minutes to peak) to let pools and caches warm.
- **Hold at peak** for long enough that metrics stabilize — at least 2–3× the longest cache TTL or autoscaler delay in the path.
- **Discard warm-up data** when computing SLO compliance — measure the hold period, not the ramp.

Running at scale and interpreting results

Distributed generation

A single load generator has limits: CPU for TLS handshakes, file descriptor caps, network bandwidth. When you need > 10k RPS or > 1k concurrent connections:

- **k6-operator or k6 Cloud** — shard VUs or arrival rate across N Kubernetes pods; aggregate metrics centrally.
- **Vegeta sharding** — run N Vegeta instances with `-rate=R/N` and merge `results.bin` files (sum histograms externally).
- **Gatling FrontLine / Cloud** — managed distribution with coordinated ramp.
- **Locust master/worker** — built-in distribution with Python workers.

Key principle: generators must be coordinated (same ramp profile, synchronized clocks) and metrics must be aggregated with histogram-aware merging — averaging p95 across generators is wrong; merge the raw latency histograms.

What to measure

Every load test should capture at minimum:

Signal	Metric	Collector
Throughput	Requests/sec, success rate	Load generator (k6, Vegeta)
Latency	p50, p95, p99, max — per endpoint	Load generator + server-side histogram (Prometheus)
Errors	Rate by status code / error type	Load generator + application metrics
Saturation	CPU, memory, GC pause, event loop lag, connection pool utilization, queue depth	Prometheus / CloudWatch / Datadog
Downstream	Latency and error rate of each dependency	Tracing (Chapter 4) + dependency dashboards

Signal	Metric	Collector
Autoscaler	Desired vs. actual replicas, scale events, time to scale	Kubernetes HPA metrics

Reading the results

The **throughput-latency curve** is the primary output:

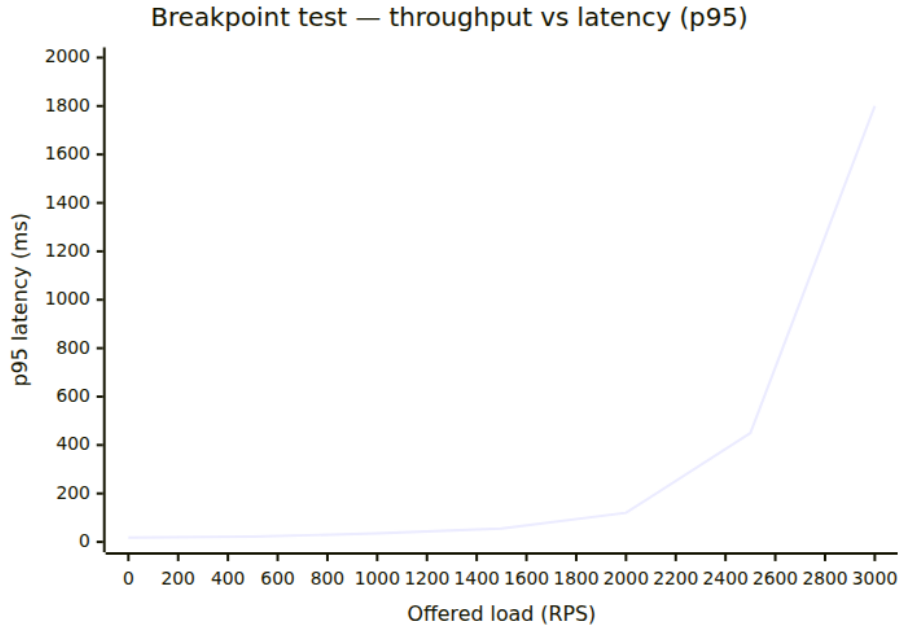


Figure 7-5: Breakpoint test result. The knee at ~1500 RPS (where p95 jumps from 55 ms to 120 ms) is the practical capacity limit. Beyond ~2000 RPS the system is saturated — queueing dominates and latency grows without bound. Provision for the knee, not the cliff.

Red flags that invalidate a test:

- Generator-side bottlenecks: generator CPU > 80%, file descriptor exhaustion, or `connect: cannot assign requested address` errors — the generator is saturated, not the system under test.
- Coordinated omission: the generator measures latency only for requests that completed, ignoring queueing time for requests that were delayed before being sent. k6 and Vegeta handle this by

measuring from scheduled arrival time (open model) — but closed-model tests with naive timing do not.

- Cache or data artifacts: hit rate 100% because the same key was reused; database empty so index scans are trivially fast.

USE and RED for bottleneck identification:

When throughput plateaus or latency spikes, drill into USE (Utilization, Saturation, Errors) per resource and RED (Rate, Errors, Duration) per service:

- CPU utilization > 85% with run queue growing → CPU-bound; profile for hot spots (Volume 13, Chapter 5).
 - Connection pool wait time rising while DB CPU is low → pool too small; increase or add pooling layer (PgBouncer).
 - Downstream p95 spiking while local CPU is idle → downstream bottleneck; the system under test is waiting — fix downstream or add caching/circuit breaking (Chapter 10).
 - GC pause time correlated with latency spikes → heap pressure; tune GC or reduce allocation rate (Volume 13).
-

Capacity planning: from measurements to decisions

The capacity model

A capacity model connects three variables: **demand** (RPS or concurrent users), **SLO** (latency and error budget), and **resources** (replicas, instance types, database capacity). The breakpoint test gives you the mapping; the model turns it into a provisioning decision.

Step 1 — Establish the demand forecast.

Combine historical growth, business events (launches, campaigns, seasonality), and SLO error budgets:

```
Peak demand estimate = current peak RPS × (1 + growth rate)^(months ahead) × burst factor
```

Example: 1200 RPS today, 8% monthly growth, 6 months ahead, 2× burst factor campaign

```
= 1200 × 1.08^6 × 2.0  
≈ 1200 × 1.587 × 2.0  
≈ 3809 RPS design target
```

Step 2 — Map demand to resources via measured data.

From the breakpoint test: the single-replica knee is 500 RPS at p95 < 300 ms. With 3 replicas and 70% target utilization (headroom for burst and rolling deploys):

```
Replicas needed = ceil(design target / (knee per replica × target utilization))  
                 = ceil(3809 / (500 × 0.70))  
                 = ceil(10.88)  
                 = 11 replicas
```

Add N+2 for zone failure tolerance in a 3-AZ deployment (lose one AZ and still handle peak): 11 → 13 replicas.

Step 3 — Validate autoscaling.

Configure the autoscaler (e.g., Kubernetes HPA on CPU or custom metric) and verify with a spike test that it scales from steady state to peak within the required time:

```

# k8s/hpa.yaml – validated by spike test
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: checkout-api
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: checko
ut-api }
  minReplicas: 6
  maxReplicas: 20
  metrics:
    - type: Pods
      pods:
        metric: { name: http_requests_per_second }
        target: { type: AverageValue, averageValue: "350" } # 70% of
500 knee
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 30
      policies:
        - type: Percent
          value: 100
          periodSeconds: 30
        - type: Pods
          value: 4
          periodSeconds: 30
      selectPolicy: Max
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Percent
          value: 25
          periodSeconds: 60

```

The spike test from the k6 scenario validates this HPA: at t=7 min, load jumps from 200 to 1200 RPS — the HPA should add replicas within 30 s and recover p95 within 60 s. If it does not, tune `stabilizationWindowSeconds` or switch from CPU to a more responsive signal (requests per second or queue depth).

Step 4 — Connect to error budget and cost.

- If current burn rate plus projected growth will exhaust the error budget before the next scaling window, capacity is a reliability risk now — escalate.
- Translate replicas to cost (instances × hours × price) and compare scaling strategies: larger instances versus more small instances

versus different instance families (Volume 12, Chapters 8-9 cover cloud cost engineering).

Headroom and the danger zone

Capacity headroom

Safe: < 60% of knee
latency flat, errors ~0



Watch: 60-80% of knee
latency rising, SLO at
risk



Danger: 80-100% of
knee
nonlinear queueing,
SLO breach imminent



Cliff: > knee
errors spike, cascading
risk

Figure 7-6: Headroom model. The target operating range is the safe zone, with autoscaling configured to keep the system there. The danger zone is where utilization law predicts superlinear latency — alert here, not at the cliff.

Continuous load testing

Capacity planning is not a one-time exercise — every significant change can shift the knee:

- **CI gate:** Smoke test on every merge (1 min, 10 RPS) — catches gross regressions (e.g., an N+1 query that triples latency) before they reach main.
- **Nightly:** Load test at peak RPS for 15 minutes — tracks the knee over time and detects gradual drift (dependency slowdown, data growth, config change).
- **Pre-release / pre-event:** Full breakpoint + spike + soak suite before launches, migrations, and known traffic events — the authoritative capacity signal for provisioning decisions.
- **Trend dashboard:** Plot knee RPS and p95 at peak over weeks — a downward trend in knee RPS is an early warning that a recent change introduced a bottleneck, even though current peak is still below the knee.

Distributed-systems lens

Load testing distributed backends has failure modes that single-service tests do not surface:

Fan-out amplification. A single API request may fan out to 8 internal services. At 1000 RPS at the edge, a downstream service with 3× fan-out sees 3000 RPS. The load test must either generate realistic fan-out (and measure downstream services) or explicitly document that downstream capacity is out of scope. Failing to account for amplification is why services pass edge load tests but downstream databases collapse.

Cache and shard hotspots. Uniform random load distributes evenly across shards and cache keys — real traffic does not. A load test that uses uniform key distribution will show healthy shard utilization while

production with Zipfian distribution has one shard at 90% and the rest at 30%. Model the real distribution or test hotspot behavior explicitly.

Retry storms under load. When downstream latency rises under load, callers retry — multiplying the effective arrival rate at the downstream. A system that handles 1000 RPS without retries may face 3000 RPS with retries (Chapter 10 covers the circuit breaker and backoff patterns that mitigate this; load tests should measure with realistic retry behavior enabled).

State and data growth. A soak test against a static dataset misses the degradation caused by data growth: larger indexes, fuller LSM trees, longer compaction pauses, growing replication lag. Seed realistic data volume and, for soak tests, include a write mix that grows the dataset during the test.

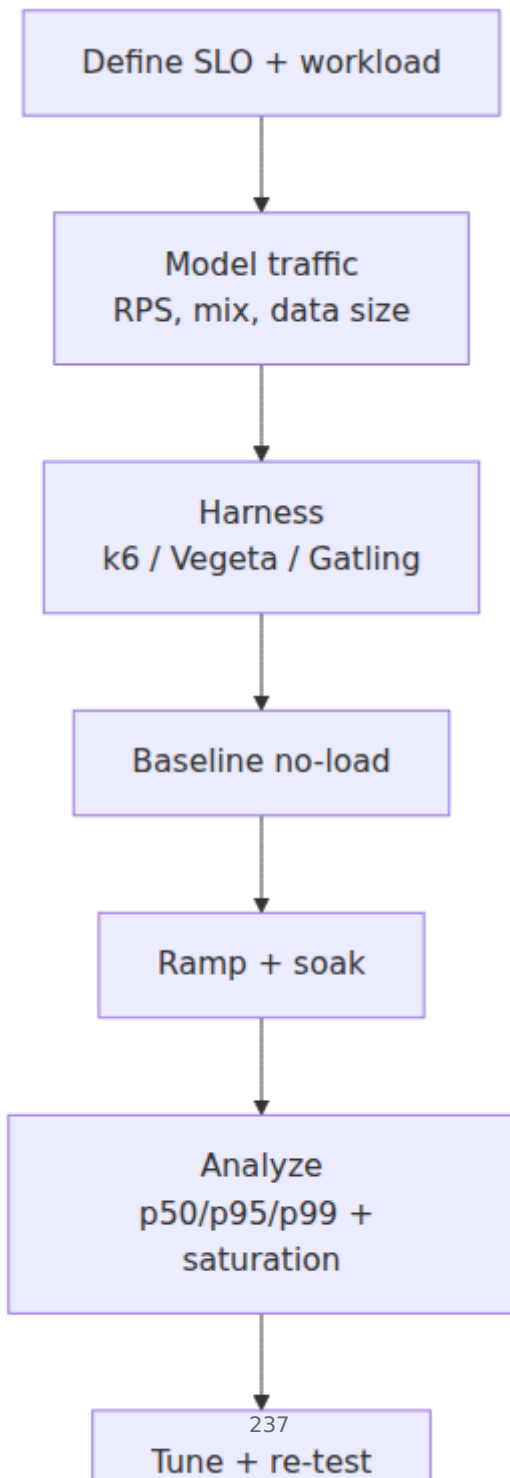
Clock and coordination pressure. At high concurrency, contention on shared coordination (leader election, distributed locks, sequence generators) can become the bottleneck — visible as increased lock wait time, not CPU. The USL's β term captures this; if present, adding replicas makes it worse.

Anti-patterns and common mistakes

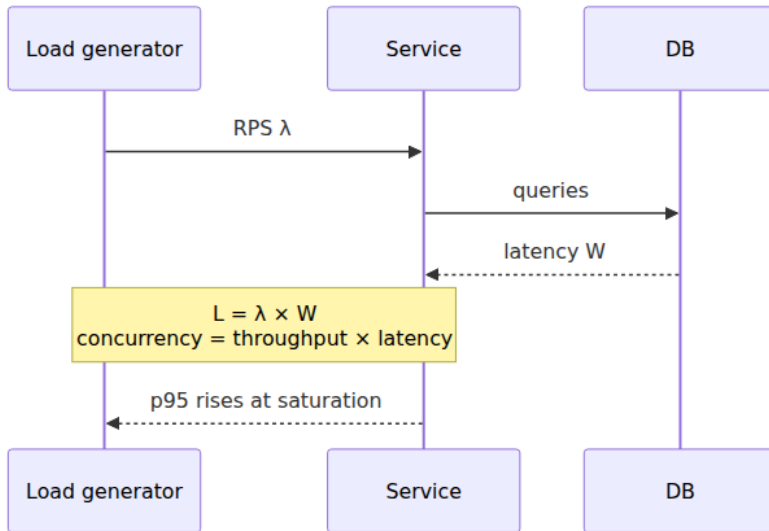
Anti-pattern	Why it hurts	Fix
Testing in a different topology	Staging with 1 DB replica and 2 app replicas extrapolates poorly to prod with 3 DB replicas and 20 app replicas — replication lag, pool limits, and shard distribution differ.	Use production-like ephemeral environments or production with strict isolation.
Averaging latencies	Mean latency hides the tail — a mean of 80 ms with p99 of 2 s means 1% of users have a terrible experience.	Report p50, p95, p99, and max — per endpoint, over the hold period.
Ignoring warm-up	Measuring ramp latency conflates JIT/pool/cache warm-up with steady-state performance.	Ramp gradually, hold, and measure the hold window.

Anti-pattern	Why it hurts	Fix
Reusing the same test data	Tests one cache entry and one shard; misses real distribution.	Unique data per request; model production key distribution.
No thresholds / CI gating	Load tests become reports that are read once and ignored.	Define thresholds (SLO-aligned) and fail the pipeline when breached.
One-and-done testing	A capacity model from 6 months ago does not reflect current code, data, or dependencies.	Automate a cadence: smoke on every commit, load nightly, full suite pre-release.
Generator is the bottleneck	Saturating the generator and attributing the limit to the system under test.	Monitor generator CPU/memory/FDs; distribute when needed; verify generator headroom.
Coordinated omission	Closed-model timing hides queueing; reported latency is lower than user-perceived.	Use open-model executors or generators that measure from scheduled arrival time.

Load Test Lifecycle



Little's Law and Saturation



Key takeaways

- Load testing answers **how much, how gracefully, and for how long** a system can handle demand — questions that functional tests cannot answer and that capacity planning depends on. Without measured data, provisioning is guesswork and autoscaling is hope.
- **Match the workload model to production traffic:** open (arrival-rate) for backend APIs where callers do not synchronize on responses (most service-to-service traffic), closed (VU-concurrency) for user-session modeling with think time. Using the wrong model hides or invents queueing and invalidates latency measurements.
- The taxonomy is purposeful: **smoke** (harness check, every commit), **load** (SLO at expected peak, nightly), **stress** (find the cliff), **spike** (elasticity under burst), **soak** (leaks over hours), **breakpoint** (precise knee and cliff). Each type answers a different capacity question — run them on different cadences, not as one monolithic test.
- **Little's Law ($L = \lambda W$) and the utilization law ($U = \lambda S$)** connect throughput, latency, concurrency, and utilization — you cannot reason about one without the others. The **Universal Scalability Law**

reveals whether scaling is contention-limited (α) or coherency-limited (β), telling you whether adding concurrency will help or hurt.

- Queueing is **nonlinear**: at 50% utilization, response time is $2\times$ service time; at 90%, it is $10\times$. Capacity headroom is not waste — it is the margin that keeps latency flat. Target the **knee** (where latency starts rising superlinearly), not the cliff, for provisioning.
- **k6** (scriptable, thresholds, open and closed executors) and **Vegeta** (minimal, open-model, ideal for quick probes and breakpoint sweeps) are the default toolkit; choose based on scenario complexity, not fashion. Distribute generators when a single node saturates, and merge histograms — not averaged percentiles.
- Realism determines relevance: replicate the **production traffic mix** (endpoint distribution from access logs), use **realistic data** (unique entities, Zipfian key distribution, seeded volume), test in a **production-like environment**, and **discard warm-up** — measure the hold period.
- A **capacity model** maps demand forecast $\rightarrow$ measured knee per replica $\rightarrow$ replicas at target utilization (typically 60–70%) plus zone-failure headroom. Validate autoscaling with **spike tests** — the scaler must catch up within the SLO window, or the policy needs tuning.
- Run load tests **continuously**: smoke on every commit, load nightly, full suite before releases and traffic events. Plot the knee over time — a downward trend is an early warning that a recent change introduced a bottleneck, even though current peak is still below it.

Further reading

- Brendan Gregg, *Systems Performance: Enterprise and the Cloud* (2nd ed., Addison-Wesley, 2020) — Chapter 7 on benchmarking and Chapter 8 on performance testing methodology.
- Baron Schwartz et al., *High Performance MySQL* (O'Reilly) — capacity reasoning for database-backed services.
- Neil Gunther, *Guerrilla Capacity Planning* (Springer, 2007) and the Universal Scalability Law paper — the definitive treatment of scalability modeling.

- Leonard Kleinrock, *Queueing Systems, Volume 1* (Wiley, 1975) — the theoretical foundation for utilization and queueing analysis used in this chapter.
- k6 documentation — executors, thresholds, and distributed execution: <https://k6.io/docs/using-k6/scenarios/executors/> and <https://k6.io/docs/using-k6/thresholds/>
- Vegeta documentation — attack, report, and histogram analysis: <https://github.com/tsenart/vegeta>
- Gatling injection profiles — open versus closed workload modeling: <https://docs.gatling.io/reference/script/core/injection/>
- Google SRE Book, Chapter 8 — *Capacity Planning* (<https://sre.google/sre-book/capacity-planning/>) — demand forecasting and resource modeling at Google scale.
- Gil Tene, *How NOT to Measure Latency* — coordinated omission and hdrhistogram: <https://www.infoq.com/articles/Response-Time-Measure>
- Philip Healy et al., *Little's Law and its applications* — practical applications in software systems.

Chapter 8 — Chaos Engineering

What this chapter covers. Distributed systems fail in ways that are difficult to predict from static analysis or code review — subtle interactions between retries, timeouts, caches, queues, and human procedures that only surface under real failure conditions. Chaos engineering is the discipline of *intentionally injecting controlled failures* into production or production-like systems to discover those weaknesses before they become incidents, and to build confidence that the system behaves as intended when the inevitable real failure occurs. This chapter covers principles and history, the full experiment lifecycle, the failure modes worth injecting, the tooling that makes it safe (Chaos Mesh, LitmusChaos, AWS Fault Injection Service, Gremlin), the blast radius and safety controls that keep experiments from becoming incidents, game days as a team practice, and how chaos connects to testing, observability, and resilience patterns. Every concept is grounded in runnable configs and production-safe practices.

Learning goals — after this chapter you should be able to:

- Explain what chaos engineering is and is not — how it differs from traditional testing, why it requires a steady-state hypothesis, and when it is appropriate versus premature.
 - Design a chaos experiment from hypothesis through steady-state metrics, fault injection, and automated abort conditions — and distinguish experiments that teach from those that merely destroy.
 - Enumerate the failure modes worth injecting — process, container, node, network, resource pressure, time, and dependency failures — and map each to the resilience pattern it validates.
 - Write and run Chaos Mesh and LitmusChaos experiments on Kubernetes, including network chaos, pod kills, stress, and I/O faults, with proper scoping and RBAC.
 - Implement blast radius controls — scoping, rate limiting, automatic rollback, and observability gating — that make chaos safe enough to run in production.
 - Organize and facilitate game days that build team muscle memory for incident response while surfacing systemic gaps.
-

What chaos engineering is

The Netflix lineage

The term was coined at Netflix around 2011, when the company was migrating from a single data center to AWS — trading a controlled physical environment for a distributed cloud where failures were more frequent and less predictable. Traditional testing had not prevented outages caused by subtle interactions: a cache node failure that triggered a retry storm that saturated a downstream database that caused a different service to time out. The insight was that *you cannot predict all failure modes from first principles in a system with dozens of services, multiple data stores, and continuous deployment* — you have to *observe* how the system actually behaves when things break.

Netflix built **Chaos Monkey** (randomly terminating production instances during business hours) as the first tool. The name reflects the idea: a monkey loose in the data center pulling cables at random. If the system is resilient, the monkey does nothing noticeable; if it is not, you learn

exactly where. Chaos Monkey was intentionally simple — it tested one thing: can the system tolerate instance loss? Later tools (Chaos Kong for AZ failure, Latency Monkey for network delay) expanded the repertory. The discipline generalized into *chaos engineering* with the publication of *Principles of Chaos Engineering* (chaos.community, 2017).

Definition

Chaos engineering is the discipline of experimenting on a system in order to build confidence in the system's capability to withstand turbulent conditions in production.

The definition has three load-bearing words:

- **Experimenting** — not random destruction. A chaos experiment has a hypothesis, a controlled scope, measured steady-state behavior, and a defined abort condition. Uncontrolled fault injection is not chaos engineering — it is just breaking things.
- **Confidence** — the goal is not to find bugs per se, but to *verify assumptions* about system behavior. "We believe the checkout service will maintain p95 < 400 ms when one payment replica is unavailable because the circuit breaker will open and fallback logic will serve cached rates." The experiment tests that belief. Whether it confirms or refutes, you have learned something precise.
- **Production** — the highest-fidelity signal comes from production, where traffic, data, and configuration are real. Experiments in staging are useful but strictly less informative — staging rarely has production's data volume, traffic shape, or cross-service interactions. The aspiration is to run in production; the practice requires earning the safety to do so (blast radius controls, gradual rollout).

What chaos engineering is not

Misconception	Reality
Randomly breaking things	Hypothesis-driven experiments with defined scope, metrics, and abort conditions. Randomness without measurement is just outages.

Misconception	Reality
A replacement for testing	Complements unit, integration, and load tests — it tests <i>emergent system behavior</i> under failure that component tests cannot capture. Volume 6, Chapter 12 covers distributed correctness testing (Jepsen, simulation); chaos covers <i>operational</i> resilience.
Only for mature hyper-scale companies	The principles apply to any distributed system with more than a few services. Start with one service and one failure mode — even a single pod-kill experiment teaches more than no experiment.
An excuse to cause incidents	Experiments must have blast radius limits, automatic abort, and stakeholder buy-in. An experiment that causes user-visible impact without prior risk assessment is an incident caused by the experiment, not a useful experiment.
A one-time activity	A continuous practice. Every architectural change, dependency addition, or scaling event can introduce new failure modes. Chaos experiments should run on a schedule, in CI/CD, and before significant releases.

Boundary note. This chapter covers chaos as *operational practice* — fault injection in running systems to validate resilience. Volume 6, Chapter 12 covers chaos as *verification of distributed correctness* (Jepsen, deterministic simulation, lineage-driven fault injection for consensus protocols). The failure modes overlap; the goals and tooling differ. Load testing (Chapter 7) measures capacity under *expected* demand; chaos measures behavior under *unexpected* failure.

Principles and the experiment lifecycle

Principles of Chaos Engineering (chaos.community)

The community principles, lightly adapted, are:

1. **Build a hypothesis around steady-state behavior.** Define what "normal" looks like in measurable terms before injecting faults. Without a steady-state definition, you cannot distinguish "the system handled the fault" from "we did not notice the impact."

2. **Vary real-world events.** Faults should resemble things that actually happen — process crashes, network partitions, slow dependencies, resource exhaustion, clock skew — not exotic conditions that would never occur in production.
3. **Run experiments in production** (or as close as safety allows). Minimize the difference between experiment and reality.
4. **Automate experiments to run continuously.** Manual experiments are run once and forgotten; automated experiments catch regressions.
5. **Minimize blast radius.** Scope experiments so the cost of an unexpected result is contained — one service, one AZ, a percentage of traffic.

The lifecycle

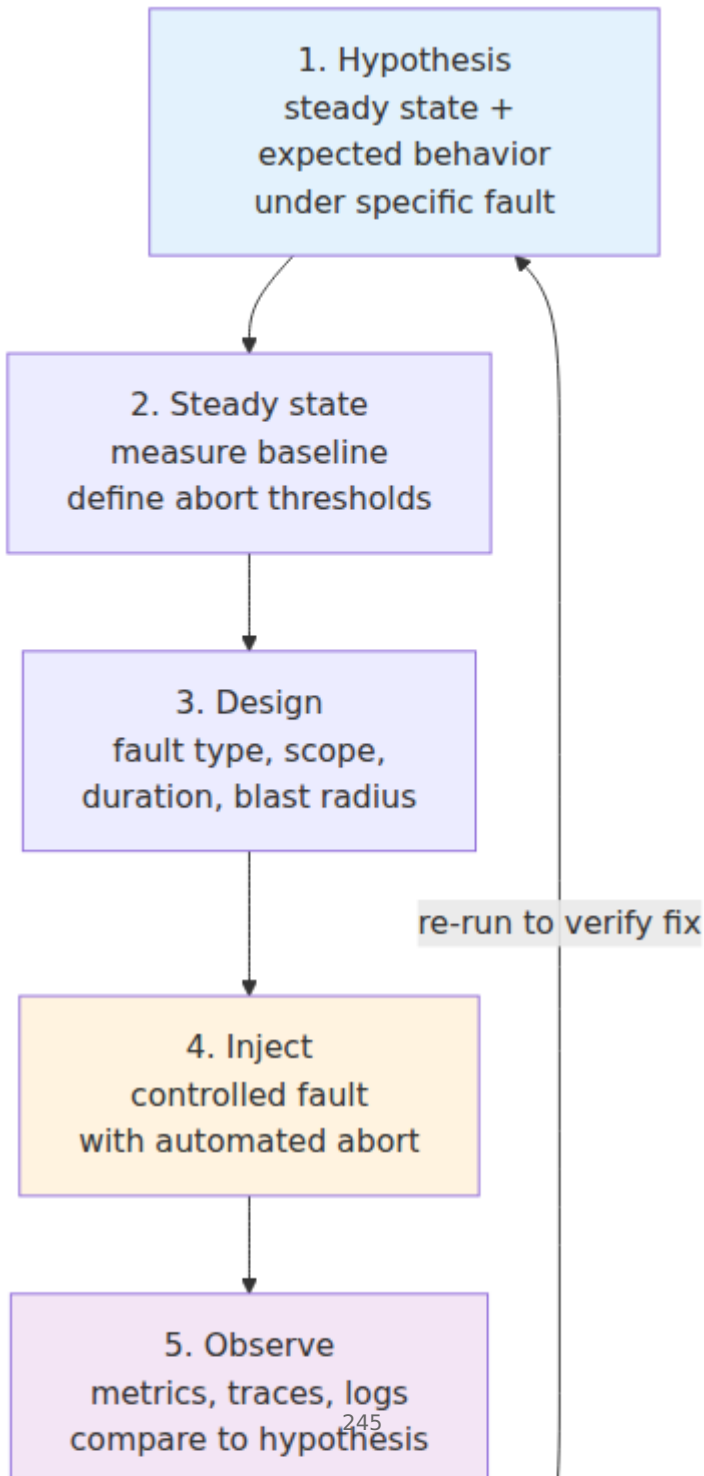


Figure 8-1: The chaos experiment lifecycle. Each experiment is a loop — hypothesis, steady state, injection, observation, learning — that either verifies resilience or surfaces a gap to fix. Verified fixes become automated regression experiments.

Each stage in detail:

1. Hypothesis — A falsifiable statement: "If we inject fault F scoped to S, then steady-state metric M will remain within bounds B because mechanism R will activate." Example: "If we kill one of three payment-service pods, checkout p95 will remain < 400 ms because the service mesh will remove the dead endpoint within 2 s and the remaining pods have headroom."

- Specific metric and bound (not "the system will be fine").
- Named resilience mechanism (circuit breaker, retry, autoscaling, failover) — the thing the experiment validates.
- Falsifiable — the experiment can prove the hypothesis wrong, and that is valuable.

2. Steady state — Measure the baseline *before* injection. Steady state is not "no errors" — it is the normal operating range of the metrics you will compare:

- SLO-derived metrics: p95 latency, error rate, throughput for the affected service and its callers.
- Resource metrics: CPU, memory, queue depth, connection pool utilization.
- Business metrics (when relevant): checkout conversion rate, search result freshness.

Record for long enough that you know what normal variance looks like — a 30-second baseline is not steady state if the metric naturally oscillates over minutes. Define **abort thresholds**: if error rate exceeds X or p95 exceeds Y during the experiment, abort automatically — do not wait for a human to notice.

3. Design — Choose fault, scope, and duration:

- Fault type — one of the catalog in the next section.
- Scope — which service / namespace / AZ / percentage of pods or traffic.

- Duration — long enough to observe steady-state impact (including autoscaler and circuit breaker reaction time), short enough that blast radius is bounded. Typically 1-10 minutes for pod/network faults, longer for soak-style resource pressure.
- Scheduling — business hours with on-call present is the default for production experiments. Off-hours chaos is higher risk and lower observability (fewer eyes).

4. Inject — Execute the fault through tooling with:

- Automated abort on threshold breach.
- Manual abort (single command or button) accessible to on-call without needing the experiment owner's involvement.
- Full audit trail: what was injected, when, by whom, with what scope.

5. Observe — Compare metrics during injection to:

- Steady-state baseline (did the metric leave the normal range?).
- Hypothesis (did the expected resilience mechanism activate? Did fallback logic serve correct results, not just fast ones?).
- Caller impact (did *callers* of the affected service see degradation that the affected service's own metrics hid?).

Use the observability stack from Chapters 2-4: metrics for latency and errors, tracing for dependency impact and fallback path verification, logs for error details.

6. Learn and fix — Whether the hypothesis held or not, produce a short write-up:

- What was expected, what was observed, what was the gap.
- Root cause of any gap (missing circuit breaker, wrong timeout, insufficient headroom, misconfigured retry).
- Action items with owners — the same lifecycle as a postmortem (Chapter 6). A chaos experiment that finds a gap but does not produce a tracked fix has wasted its value.
- Re-run the experiment after the fix to verify.

7. Automate — Experiments that passed become regression guards: schedule them periodically (daily, weekly) or gate deployments (run before promoting to production). Experiments that failed and were fixed are re-run to verify — then promoted to regression guards as well.

Hypothesis quality matters

Compare:

- Weak: "Killing a pod will not break the service." — Unfalsifiable in any useful sense (what does "break" mean? which metric?).
- Weak: "The system is resilient to network partitions." — Too broad; resilience to which partition, between which components, at what traffic level?
- Strong: "Partitioning payment-service from postgres-primary for 30 s will cause checkout p95 to rise to at most 600 ms (from 120 ms baseline) and error rate to at most 2% because the read replica will serve stale-but-acceptable data via the fallback path, and no checkout will be double-charged." — Specific fault, specific metrics, specific bounds, specific mechanism, specific correctness property.

Strong hypotheses force you to articulate what you believe about the system — which is where the learning lives. If you cannot articulate a strong hypothesis, you do not yet understand the system well enough for the experiment to be interpretable. That is itself a signal to invest in observability before chaos.

Failure modes worth injecting

The catalog below maps fault types to the resilience patterns they validate (Chapter 10) and the historical incidents they resemble. Not every system needs every fault — prioritize by the failure modes that are most likely and most damaging for your architecture.

Fault class	Specific faults	Validates	Resembles
Process / pod	Kill pod, kill container, crash loop, OOMKill, SIGTERM vs SIGKILL	Replica redundancy, graceful shutdown, readiness probes, PodDisruptionBudget	Any instance failure — the most common prod fault

Fault class	Specific faults	Validates	Resembles
Node	Drain node, kernel panic, disk full, kubelet failure	Node spread, anti-affinity, cluster autoscaler, PDB	AZ or hardware failure
Network	Latency injection (50-1000 ms), packet loss (1-30%), partition (drop 100%), bandwidth limit, DNS failure	Timeouts, retries with backoff, circuit breakers, hedged requests, fallback	Network partition, congested link, DNS outage
Resource pressure	CPU stress, memory stress, I/O stress, file descriptor exhaustion, network I/O saturation	Resource limits, QoS classes, backpressure, load shedding	Noisy neighbor, memory leak, GC pressure
Time	Clock skew (NTP drift), clock jump	Lease expiry, TTL correctness, distributed lock safety	NTP misconfiguration, VM live migration
Dependency	Downstream 500s, downstream latency, downstream unavailability, message queue stall	Timeouts, circuit breakers, bulkheads, fallbacks, idempotency	Downstream incident, dependency deploy gone wrong

Fault class	Specific faults	Validates	Resembles
State	Database failover, cache eviction/flush, queue depth spike, disk corruption	Failover, cache warming, DLQ handling (Volume 10)	DB primary failure, cache cluster loss
Configuration	Bad config push, feature flag flip, secret rotation failure	Config validation, flag defaults, secret reload	The most common cause of self-inflicted incidents

Why network faults are the most revealing

Network faults — latency, loss, and partitions — are disproportionately valuable because they exercise the *interaction* between services, not just the health of a single service. A pod kill tests whether replicas exist; a 500 ms latency injection tests whether timeouts, retries, and circuit breakers are correct across the call graph. The most damaging distributed systems incidents (retry storms, cascading timeouts, split-brain writes) are triggered by network behavior that is degraded but not fully failed — exactly the conditions that network chaos reproduces.

Tooling

Chaos Mesh (CNCF, Kubernetes-native)

Chaos Mesh is the most widely used Kubernetes chaos tool — a CNCF incubating project with CRD-based experiments, fine-grained scoping, and a dashboard. It covers pod, network, stress, I/O, time, kernel, DNS, HTTP, and JVM faults.

Installation:

```
# Helm install (requires Kubernetes 1.22+) – pin the chart version for reproducibility
helm repo add chaos-mesh https://charts.chaos-mesh.org
helm install chaos-mesh chaos-mesh/chaos-mesh --version 2.6.3 --atomic --wait \
  --namespace chaos-mesh --create-namespace \
  --set chaosDaemon.runtime=containerd \
  --set chaosDaemon.socketPath=/run/containerd/containerd.sock \
  --set dashboard.create=true

# Verify
kubectl get pods -n chaos-mesh
# chaos-controller-manager-... Running
# chaos-daemon-... Running (one per node)
# chaos-dashboard-... Running
```

Pod kill — validate replica redundancy:

```
# experiments/pod-kill.yaml – kill one payment-service pod, verify checkout SLO holds
apiVersion: chaos-mesh.org/v1alpha1
kind: PodChaos
metadata:
  name: payment-pod-kill
  namespace: chaos-mesh
spec:
  action: pod-kill
  mode: one # one pod; alternatives: all,
fixed: 2, fixed-percent: 50, random-max-percent: 30
  selector:
    labelSelectors:
      app: payment-service
    namespaces:
      - production
  # Optional: graceful vs immediate
  # gracePeriod: 30 # seconds; 0 = SIGKILL, >0 =
SIGTERM then SIGKILL
  duration: "60s" # Chaos Mesh restores
automatically after duration
  scheduler:
    cron: "@every 10m" # for continuous regression
(optional)
```

```
kubectl apply -f experiments/pod-kill.yaml
# Watch: checkout p95 and error rate during the 60s window
# Verify: remaining payment pods handle load; no checkout errors
# Abort early if needed:
kubectl delete -f experiments/pod-kill.yaml
```

Network latency — validate timeout and circuit breaker behavior:

```
# experiments/network-latency.yaml – inject 500ms latency to payment-
service calls
apiVersion: chaos-mesh.org/v1alpha1
kind: NetworkChaos
metadata:
  name: payment-latency
  namespace: chaos-mesh
spec:
  action: delay
  mode: all                                # affect all pods matching
  selector
    selector:
      labelSelectors:
        app: checkout-service            # fault at the caller – egress to
payment
      namespaces:
        - production
  direction: to                            # egress only
  target:
    selector:
      labelSelectors:
        app: payment-service
      namespaces:
        - production
  mode: all
  delay:
    latency: "500ms"
    jitter: "100ms"
  # 400-600ms range – more realistic than fixed
  correlation: "25"                        # 25% correlation between
successive packets
  duration: "120s"
```

Network partition — validate failover and fallback:

```

# experiments/network-partition.yaml – isolate one AZ from the primary
database
apiVersion: chaos-mesh.org/v1alpha1
kind: NetworkChaos
metadata:
  name: db-partition
  namespace: chaos-mesh
spec:
  action: partition
  mode: all
  selector:
    labelSelectors:
      app: checkout-service
    namespaces:
      - production
  direction: both # bidirectional partition
  target:
    selector:
      labelSelectors:
        app: postgres
        role: primary
      namespaces:
        - production
    mode: all
  duration: "60s"

```

Stress — CPU, memory, and I/O pressure:

```

# experiments/stress-cpu.yaml – saturate CPU on one replica to test QoS
and load shedding
apiVersion: chaos-mesh.org/v1alpha1
kind: StressChaos
metadata:
  name: checkout-cpu-stress
  namespace: chaos-mesh
spec:
  mode: one
  selector:
    labelSelectors:
      app: checkout-service
    namespaces:
      - production
  stressors:
    cpu:
      workers: 2 # number of stressor threads
      load: 80 # target CPU load percentage per
worker
      options: ["--timeout 120s"]
      duration: "120s"
---
# experiments/stress-memory.yaml – memory pressure on a cache node
apiVersion: chaos-mesh.org/v1alpha1
kind: StressChaos
metadata:
  name: redis-memory-stress
  namespace: chaos-mesh
spec:
  mode: one
  selector:
    labelSelectors:
      app: redis
    namespaces:
      - production
  stressors:
    memory:
      workers: 1
      size: "512MB"
# allocate 512MB – tune to trigger eviction without OOMKill
      options: ["--timeout 90s"]
      duration: "90s"

```

I/O fault — validate disk failure handling:

```
# experiments/io-fault.yaml – I/O latency and errors on a stateful
service
apiVersion: chaos-mesh.org/v1alpha1
kind: IOChaos
metadata:
  name: postgres-io-delay
  namespace: chaos-mesh
spec:
  action: latency
  mode: one
  selector:
    labelSelectors:
      app: postgres
    namespaces:
      - production
  volumePath: /var/lib/postgresql/data
  path: /var/lib/postgresql/data/**/*
  delay: "100ms"
  percent: 50 # 50% of I/O operations delayed
  duration: "60s"
```

Time skew — validate lease and TTL correctness:

```
# experiments/time-skew.yaml – skew clock on one replica by 5 seconds
apiVersion: chaos-mesh.org/v1alpha1
kind: TimeChaos
metadata:
  name: checkout-time-skew
  namespace: chaos-mesh
spec:
  mode: one
  selector:
    labelSelectors:
      app: checkout-service
    namespaces:
      - production
  timeOffset: "5s" # forward skew; use "-5s" for
backward
  clockIds: ["CLOCK_REALTIME"]
  containerNames: ["checkout"] # target specific container in
multi-container pod
  duration: "60s"
```

LitmusChaos (CNCF, Kubernetes-native, experiment-as-code)

Litmus takes an experiment-as-code approach with a central control plane (ChaosCenter), reusable experiments from ChaosHub, and explicit steady-state probes that gate fault injection and verify recovery.

Installation:

```
# Install Litmus control plane
kubectl create namespace litmus
kubectl apply -f https://raw.githubusercontent.com/litmuschaos/litmus/
master/mkdocs/docs/litmus-3.0.0.yaml

# Install ChaosHub experiments
kubectl apply -f https://hub.litmuschaos.io/api/chaos/master?file=chart
s/generic/experiments.yaml
```

Litmus experiment — pod delete with probes:


```

# experiments/litmus-pod-delete.yaml – delete a pod, verify via probes
that SLO holds
apiVersion: litmuschaos.io/v1alpha1
kind: ChaosEngine
metadata:
  name: payment-chaos
  namespace: production
spec:
  appinfo:
    appns: production
    applabel: app=payment-service
    appkind: deployment
  engineState: active
  chaosServiceAccount: litmus-admin
  jobCleanupPolicy: delete
  experiments:
    - name: pod-delete
      spec:
        components:
          env:
            - name: TOTAL_CHAOS_DURATION
              value: "60"
            - name: CHAOS_INTERVAL
              value: "10"
            - name: FORCE
              value: "false"
# false = graceful SIGTERM; true = SIGKILL
            - name: PODS_AFFECTED_PERC
              value: "50" # kill 50% of matching pods
# Probes: steady-state checks that gate the experiment
        probe:
# Pre-chaos: verify checkout SLO before injecting
          - name: checkout-p95-probe
            type: httpProbe
            mode: SOT # Start of Test – must pass
before fault injection
            httpProbe/inputs:
              url: "http://checkout-service.production.svc:8080/
health"
              expectedResponseCode: "200"
              method: get
# Continuous: abort if error rate exceeds threshold during
experiment
          - name: error-rate-probe
            type: cmdProbe
            mode: Continuous # evaluated throughout the
experiment
        runProperties:
          probeTimeout: "5s"
          interval: "5s"

```

```

        retry: 1
cmdProbe/inputs:
  command: |
    curl -s http://prometheus.monitoring.svc:9090/api/v1/
query \
    --data-urlencode
'query=sum(rate(http_requests_total{service="checkout",code=~"5.."}
[1m])) / sum(rate(http_requests_total{service="checkout"}[1m]))' \
    | python3 -c "import sys,json;
d=json.load(sys.stdin); v=float(d['data']['result'][0]['value'][1]) if
d['data']['result'] else 0; sys.exit(0 if v < 0.05 else 1)"
    # Post-chaos: verify recovery
- name: recovery-probe
  type: httpProbe
  mode: EOT                                # End of Test – must pass after
fault is removed
  httpProbe/inputs:
    url: "http://checkout-service.production.svc:8080/
health"
    expectedResponseCode: "200"
    method: get

```

Key difference from Chaos Mesh: Litmus probes are first-class — the experiment *cannot start* if SOT probes fail and *aborts* if continuous probes fail. This makes Litmus well-suited for automated pipelines where you need gating without human judgment.

Choosing a tool

Tool	Scope	Strengths	Trade-offs
Chaos Mesh	Kubernetes	Widest fault catalog (pod, network, stress, I/O, time, kernel, DNS, HTTP, JVM); dashboard; fine-grained scoping	Kubernetes only; daemonset per node
LitmusChaos	Kubernetes	Probe-gated experiments; ChaosHub reusable experiments; ChaosCenter UI; GitOps-friendly	Slightly more setup (control plane + CRDs + hub)

Tool	Scope	Strengths	Trade-offs
AWS Fault Injection Service (FIS)	AWS (EC2, EKS, ECS, RDS, AZ)	Managed; AZ power interruption, instance termination, throttling, network disruption; IAM-scoped; no agent required on EC2	AWS only; narrower fault catalog than Chaos Mesh
Gremlin	Multi-platform (K8s, VMs, bare metal)	Host and container attacks; broadest platform coverage; enterprise controls (RBAC, audit, halt)	Commercial; agent required

For Kubernetes teams, **Chaos Mesh** is the default for fault breadth and **Litmus** for probe-gated automation — many teams run both (Chaos Mesh for network/time/kernel faults, Litmus for pod/process with strict probe gating).

AWS FIS example — AZ disruption:

```
{
  "description": "Disrupt AZ us-east-1-az2 - validate multi-AZ failover",
  "targets": {
    "InstancesInAZ": {
      "resourceType": "aws:ec2:instance",
      "resourceTags": { "AZ": "us-east-1-az2" },
      "selectionMode": "ALL"
    }
  },
  "actions": {
    "RebootInstances": {
      "actionId": "aws:ec2:reboot-instances",
      "parameters": { "duration": "PT2M" },
      "targets": { "Instances": "InstancesInAZ" }
    }
  },
  "stopConditions": [
    { "source": "aws:cloudwatch:alarm", "value": "arn:aws:cloudwatch:us-east-1:123456789:alarm:checkout-p95-high" },
    { "source": "aws:cloudwatch:alarm", "value": "arn:aws:cloudwatch:us-east-1:123456789:alarm:checkout-error-rate-high" }
  ],
  "roleArn": "arn:aws:iam::123456789:role/FISExperimentRole"
}
```

```
aws fis create-experiment-template --cli-input-json file:///fis-az-  
disruption.json  
aws fis start-experiment --experiment-template-id EXTxxxxxxx
```

FIS stop conditions are the equivalent of abort thresholds — CloudWatch alarms that automatically halt the experiment if breached.

Safety: blast radius, abort conditions, and guardrails

Chaos without safety controls is just risky fault injection. The following controls are non-negotiable for production experiments.

Scoping (blast radius limits)

Control	How	Example
Namespace / label scoping	Experiment targets only pods matching label selector in specific namespaces	<code>app=payment-service, namespaces=[production]</code> — not <code>all</code>
Mode limits	Affect one or a fixed percentage, not all	<code>mode: one</code> or <code>fixed-percent: 25</code> — never <code>all</code> for destructive faults in prod
AZ / zone scoping	Limit to single AZ to validate zone isolation without full region impact	Chaos Mesh <code>selector</code> with <code>topology.kubernetes.io/zone</code> label
Traffic scoping	Use header or percentage-based routing to affect only a fraction of requests	Service mesh fault injection on 5% of traffic (Istio <code>fault</code> with <code>percentage</code>)
Time scoping	Short duration with automatic restoration	<code>duration: "60s"</code> — faults self-heal; no experiment runs unbounded

Control	How	Example
Opt-in scoping	Annotate services that are chaos-ready; experiments skip non-annotated services	<code>chaos.alpha.kubernetes.io/enabled: "true"</code> label check via OPA/Kyverno policy

Abort conditions

Every experiment must have at least one automated abort path:

- **Metric-based abort.** Prometheus or CloudWatch query evaluated continuously: if error rate > threshold or p95 > threshold, abort. In Litmus, this is a `Continuous` probe; in Chaos Mesh, use an external watcher (a small controller or a `Workflow` with a `StatusCheck` node that deletes the experiment CR on breach).
- **Manual abort.** A single `kubectl delete` or dashboard button that any on-call engineer can execute without needing experiment context. Document it in the experiment announcement.
- **Timeout abort.** Every experiment has a hard duration after which the fault is removed automatically — even if observation is still in progress.

Guardrails

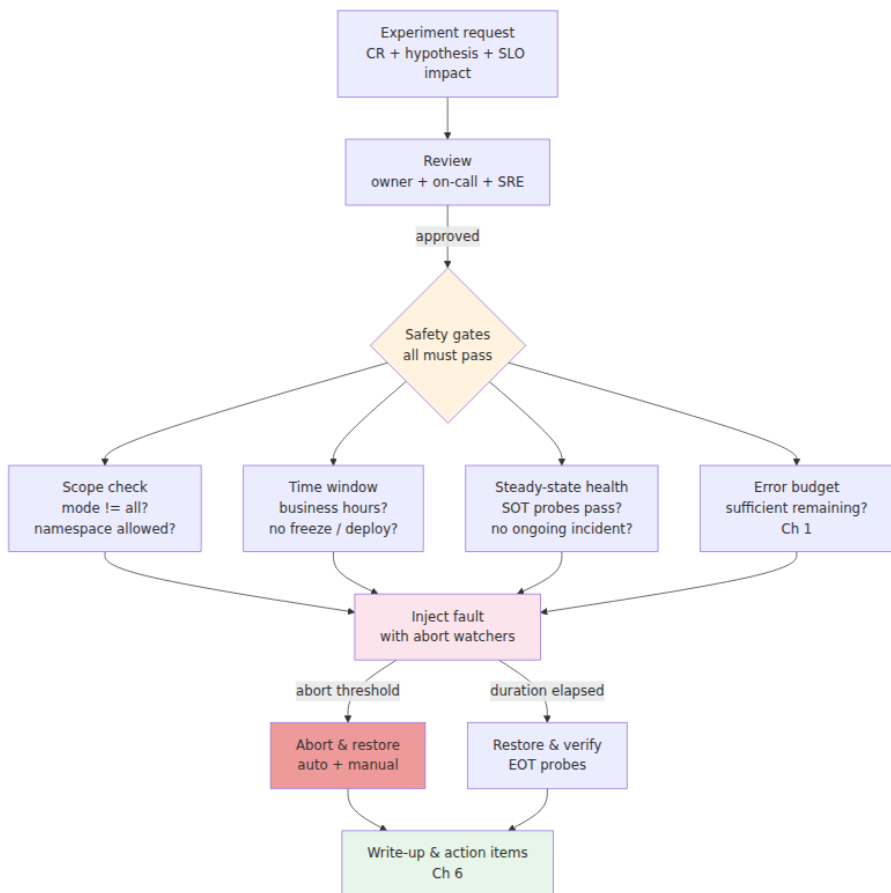


Figure 8-2: Safety gates for a production chaos experiment. No experiment runs without passing scope, window, health, and budget checks — and every experiment has both automated and manual abort paths.

Additional guardrails:

- **Change freeze awareness.** Do not run chaos during deploys, migrations, or active incidents — the signal is confounded and the combined blast radius is unpredictable.
- **Error budget gating.** If the service's error budget is already significantly burned (Chapter 1), skip the experiment — even a small

additional error rate may breach the SLO. This makes error budgets a natural throttle for chaos aggressiveness.

- **Progressive rollout.** Start with staging, then production with `mode: one` and short duration, then expand scope only after the narrow experiment passes. Never go from "never run chaos" to "partition the primary database in production."
- **RBAC.** Restrict who can create chaos CRs. Chaos Mesh and Litmus both support Kubernetes RBAC on their CRDs — only the chaos service account and designated SREs should have `create` on `PodChaos` / `NetworkChaos` / `ChaosEngine`.

OPA/Kyverno policy to enforce blast radius limits:

```

# policies/chaos-guardrails.yaml – Kyverno: block dangerous chaos
experiments
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: chaos-guardrails
spec:
  validationFailureAction: Enforce
  background: true
  rules:
    - name: block-all-mode-in-production
      match:
        any:
          - resources:
              kinds: ["PodChaos", "NetworkChaos", "StressChaos", "IOCha
os"]
            validate:
              message: "Chaos experiments with mode=all are forbidden in prod
uction namespaces"
              deny:
                conditions:
                  any:
                    - key: "{{ request.object.spec.selector.namespaces }}"
                      operator: AnyIn
                      value: ["production"]
                    - key: "{{ request.object.spec.mode }}"
                      operator: Equals
                      value: "all"
          - name: require-duration-limit
            match:
              any:
                - resources:
                    kinds: ["PodChaos", "NetworkChaos", "StressChaos", "IOCha
os"]
              validate:
                message: "Chaos experiments must declare duration <= 300s in pr
oduction"
            pattern:
              spec:
                duration: "?*"
                # Duration parsing is string-based; enforce via CEL or
webhook for numeric comparison.
                # Kyverno CEL variant:
                # cel:
                #   expressions:
                #     - expression: "duration(object.spec.duration) <=
duration('300s')"
                #       message: "Duration exceeds 300s limit for
production"

```

Detection and observability during chaos

A chaos experiment is only as good as its observability — if you cannot measure the impact, you cannot verify the hypothesis. Before any experiment, confirm that the following are in place (Chapters 2–4):

- **Metrics:** Latency histograms (not averages), error rates, throughput, and resource saturation per service — with dashboards that show the affected service *and its callers*. A downstream failure may keep the downstream's own metrics healthy (it is just slow) while the caller's error rate spikes.
- **Tracing:** Distributed traces that show whether fallback paths were taken, whether retries fired, and where time was spent. After a network latency injection, traces should show the caller's timeout firing and the fallback span — if they show retries instead, the circuit breaker may not be opening as expected.
- **Logs:** Structured logs from the affected service and its callers at a level that captures circuit breaker state transitions, retry attempts, and fallback invocations — not just error logs.
- **SLO burn rate:** Multi-window burn rate alerts (Chapter 1) that fire proportionally to error budget consumption — these are the natural abort thresholds for chaos experiments because they already encode "how much error is too much."

Chaos-specific observability:

- Annotate chaos experiment windows in dashboards (Grafana annotations, Datadog events) so impact is visually correlated with fault injection — without this, on-call may investigate a chaos-induced anomaly as a real incident.
- Record which pods / nodes / network paths were affected and for how long — this is the experiment's ground truth for post-experiment analysis.
- Measure *correctness*, not just latency and errors. A fallback that returns stale data quickly (low latency, no errors) but serves incorrect results (wrong prices, duplicate charges) passes latency checks but fails correctness. Include correctness probes: compare fallback responses to expected behavior, verify idempotency, check for duplicate side effects.

Game days

What they are

A **game day** (sometimes called *fire drill*) is a facilitated, team-based exercise where a failure scenario is injected — via chaos tooling or manual simulation — and the team practices detection, diagnosis, and recovery as if it were a real incident. Unlike automated chaos experiments that test *system* resilience, game days test *team* resilience: does on-call know how to detect the failure, does the runbook work, does the escalation path function, can the team coordinate under time pressure?

Format

Before:

1. Choose a scenario with a plausible real-world trigger: database primary failure, AZ loss, downstream dependency outage, config push gone wrong.
2. Define learning objectives: "Verify that the failover runbook completes in < 10 minutes" or "Validate that on-call can identify the root cause from dashboards alone within 5 minutes."
3. Prepare the injection: chaos experiment CR, manual steps (scale a deployment to zero, revoke a credential, introduce latency via service mesh), and abort conditions.
4. Brief participants on scope and safety — but not on the specific fault or timing if the goal is to test detection. For first game days, transparency is better than surprise — the goal is learning, not trickery.
5. Schedule during business hours with full team present, including on-call, service owner, and optionally observers from adjacent teams.

During:

1. Inject the fault. Start the clock.
2. Observe without intervening (unless abort threshold is breached).
Note: who detected it, how long to detection, what signals were used, what actions were taken, where the runbook was unclear or wrong.

3. Facilitate lightly — answer questions about scope and safety, but do not guide diagnosis. The point is to see how the team performs without the facilitator's help.
4. After recovery (or abort), hold a brief debrief while memories are fresh: what went well, what was confusing, what took longer than expected.

After:

1. Write a short postmortem-style document (Chapter 6): scenario, timeline, observations, gaps, action items.
2. Track action items: fix the runbook, add the missing alert, correct the dashboard, practice again.
3. Schedule the next game day — monthly for critical services, quarterly for others. Vary scenarios to cover different failure modes over time.

Example game day scenarios

Scenario	Injection	Tests
Database primary failure	Kill postgres primary pod; or <code>kubectl scale statefulset postgres --replicas=0</code> for the primary	Failover automation, read-replica promotion, connection pool recovery, data loss window
AZ outage	<code>NetworkChaos</code> partition isolating one AZ; or FIS <code>RebootInstances</code> for all instances in one AZ	Zone redundancy, PDB, cross-AZ replication, load balancer draining
Downstream dependency slow	<code>NetworkChaos</code> delay 800 ms to downstream service	Timeouts, circuit breakers, fallbacks, user-facing degradation vs. full outage
Credential expiry	Revoke or rotate a service credential; or mount an expired cert	Secret reload, cert rotation automation, graceful degradation vs. crash loop

Scenario	Injection	Tests
Config push failure	Apply a bad config via the normal CD pipeline	Config validation, automated rollback, blast radius of bad config
Cache cluster loss	PodChaos killing all redis pods; or IOChaos on cache volumes	Cache miss storm, database overload under cache loss, warming behavior

Game days and chaos experiments

They are complementary, not competing:

- **Chaos experiments** are automated, narrow, hypothesis-driven, and run continuously — they verify that a specific resilience mechanism works.
- **Game days** are manual, broad, scenario-driven, and run periodically — they verify that the *team* can handle a realistic incident end to end.

A mature program runs both: automated chaos experiments as regression guards (Litmus on a schedule, Chaos Mesh workflows in CI) and game days as team practice (monthly for tier-0 services). The chaos experiments build confidence in the system; the game days build confidence in the people.

Distributed-systems lens

Chaos engineering is particularly essential — and particularly subtle — in distributed backends:

Emergent failures. The most interesting failures are not single-component failures but *interactions*: a slow downstream causes retries that saturate a different downstream that causes a different service to time out. Single-service chaos (kill one pod) will not surface these — you need *multi-service* chaos: inject latency in service A and observe the cascade through services B, C, and D. Design experiments that span the call graph, not just one hop.

Consistency under partition. Network partitions are the canonical distributed systems fault (Volume 6, Chapter 1-6). Chaos experiments that partition a consensus group (etcd, ZooKeeper, database replication) validate that the system maintains consistency (no split-brain writes) and availability (reads still served from the majority partition). These experiments are higher risk and should start in staging with Jepsen-style correctness checks before moving to production with read-only partitions.

Coordination and leader election. Killing the leader of a consensus group or the holder of a distributed lock tests failover correctness — but the interesting signal is not whether failover happens (it will) but *how long it takes* and *whether there is a window of unavailability or duplicate leadership*. Measure leader election latency and verify that no two nodes believe they are leader simultaneously (fencing correctness).

Idempotency and exactly-once. Chaos that causes retries and redelivery (network partition followed by heal, pod kill during request processing) tests whether the system's idempotency and deduplication logic is correct. Without idempotency, retried requests cause duplicate charges, duplicate messages, or inconsistent state. Verify correctness (no duplicates), not just availability (no errors) — see Volume 6, Chapter 9 and Volume 10, Chapter 2 for the theory, and test it with chaos.

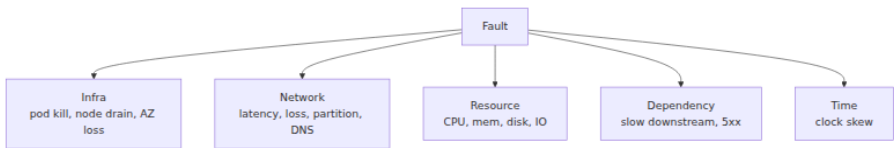
Backpressure and load shedding under induced load. Chaos that degrades a downstream (latency injection, resource stress) increases queue depth and latency upstream — testing whether backpressure and load shedding (Chapter 10, Volume 10 Chapter 7) activate correctly. Without them, the upstream accumulates unbounded work and eventually OOMs or times out globally.

Anti-patterns and common mistakes

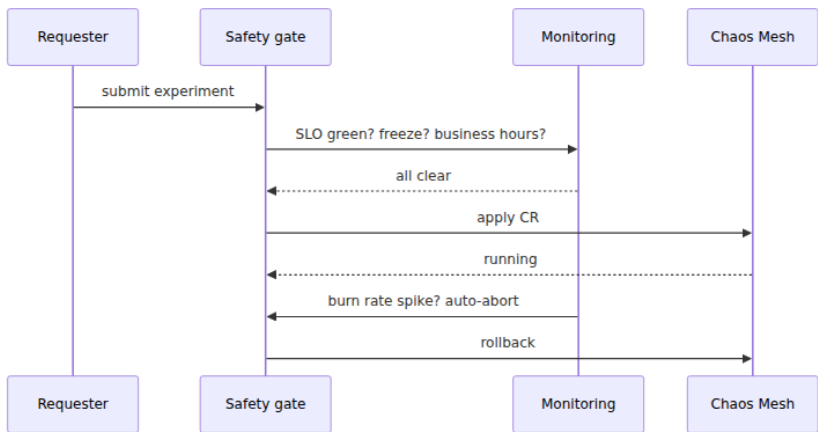
Anti-pattern	Why it hurts	Fix
No hypothesis	Injecting faults without a falsifiable hypothesis produces observations that cannot be interpreted — was the system supposed to do that?	Write a hypothesis with specific metric, bound, and mechanism before every experiment.

Anti-pattern	Why it hurts	Fix
Too broad, too early	<code>mode: all</code> on the first experiment in production — if the system is not resilient, you cause the incident you were trying to prevent.	Progressive scoping: <code>mode: one</code> , short duration, staging first, then narrow prod, then expand.
Measuring only the affected service	The affected service may appear healthy (it is just slow) while its callers are failing — the real SLO impact is at the caller.	Dashboard the affected service <i>and its callers</i> ; include caller error rate in abort conditions.
No abort conditions	An experiment that degrades more than expected continues until a human notices — turning an experiment into an incident.	Automated metric-based abort (Litmus continuous probes, FIS stop conditions, or external watcher) on every experiment.
Chaos without observability	Injecting faults when dashboards and alerts cannot distinguish chaos-induced anomalies from normal variance — results are uninterpretable.	Invest in observability (Chapters 2–4) before chaos; annotate experiment windows in dashboards.
One-time chaos	Running chaos once, finding nothing, and declaring the system resilient — missing regressions introduced by the next deploy.	Automate: schedule experiments periodically and gate deployments.
Blaming the chaos	"The chaos experiment caused the incident" — technically true, but the experiment <i>revealed</i> a pre-existing weakness; the same failure would have happened from a real fault.	Blameless write-up (Chapter 6): the experiment is the messenger, not the cause.
Chaos as a gate without SLOs	Running chaos without SLO-derived abort thresholds — abort decisions are subjective and inconsistent.	Define abort thresholds from SLO burn rates (Chapter 1); chaos respects the same error budget as production traffic.

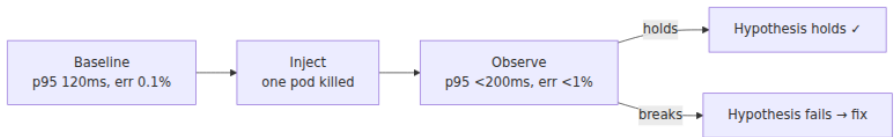
Fault Injection Taxonomy



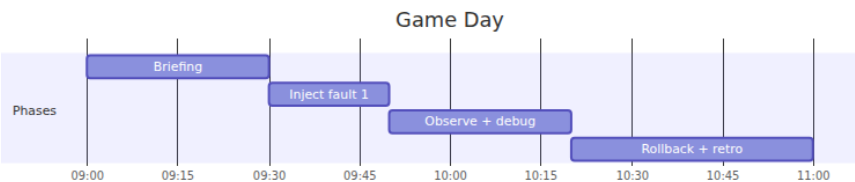
Safety Gates Sequence



Steady-State Hypothesis



Game Day Timeline



Key takeaways

- Chaos engineering is **hypothesis-driven experimentation on the system's ability to withstand realistic failures** — not random destruction. Every experiment has a falsifiable hypothesis with a specific metric, bound, and named resilience mechanism, plus defined steady state, scope, duration, and abort conditions.
- The lifecycle is **hypothesis → steady state → design → inject → observe → learn → automate**. Experiments that verify resilience become automated regression guards; experiments that find gaps produce tracked action items (postmortem lifecycle, Chapter 6) and are re-run after fixes.
- **Strong hypotheses** are specific and falsifiable: "Partitioning payment from postgres-primary for 30 s will cause p95 to rise to at most 600 ms because the read replica fallback will activate." Weak hypotheses ("the system is resilient") are uninterpretable.
- The **failure catalog** spans process/pod, node, network (latency, loss, partition), resource pressure, time skew, dependency degradation, state loss, and config errors. **Network faults** are disproportionately valuable because they test interactions across the call graph — the source of cascading failures, retry storms, and split-brain.
- **Chaos Mesh** (widest fault catalog, Kubernetes-native CRDs) and **LitmusChaos** (probe-gated, experiment-as-code, ChaosCenter) are the defaults for Kubernetes; **AWS FIS** for managed AWS faults with IAM scoping and CloudWatch stop conditions; **Gremlin** for multi-platform coverage. Many teams combine Chaos Mesh for fault breadth with Litmus for probe-gated automation.
- **Safety is non-negotiable**: scope narrowly (mode: one, single namespace/AZ, short duration), abort automatically on metric thresholds (Litmus continuous probes, FIS stop conditions), allow single-command manual abort, respect change freezes and error budget state, and enforce blast radius limits via RBAC and admission policy (Kyverno/OPA). Never go from "no chaos" to broad production faults — **progressive rollout** from staging to narrow prod to expanded scope.
- **Observability determines experiment value** (Chapters 2–4): measure the affected service *and its callers*, use latency histograms and correctness probes (not just error rates), annotate experiment

windows in dashboards, and set abort thresholds from **SLO burn rates** (Chapter 1).

- **Game days** test team resilience (detection, diagnosis, recovery, coordination) while **automated chaos experiments** test system resilience (specific mechanisms). A mature program runs both: automated experiments continuously as regression guards and game days periodically (monthly for tier-0) as team practice.
 - In distributed systems, the most revealing chaos spans **multiple services and the call graph**, tests **consistency under partition** and **leader failover correctness**, validates **idempotency under retry**, and verifies that **backpressure and load shedding** activate when downstream degradation increases upstream queue depth.
-

Further reading

- Principles of Chaos Engineering — <https://principlesofchaos.org/> — the foundational community principles (2017).
- Casey Rosenthal et al., *Chaos Engineering: System Resiliency in Practice* (O'Reilly, 2020) — the definitive book, including the experiment lifecycle and case studies.
- Netflix — *Chaos Monkey, Chaos Kong, and the Simian Army* — <https://netflixtechblog.com/tagged/chaos-engineering> — the origin story and evolution.
- Chaos Mesh documentation — <https://chaos-mesh.org/docs/> — fault catalog, scoping, and workflow orchestration.
- LitmusChaos documentation — <https://docs.litmuschaos.io/> — ChaosCenter, probes, and ChaosHub.
- AWS Fault Injection Service — <https://docs.aws.amazon.com/fis/> — managed chaos for AWS with IAM and CloudWatch integration.
- Google — *DiRT (Disaster Recovery Testing) at Google* — <https://sre.google/sre-book/disaster-recovery-testing/> — Google's game day practice at scale.
- Nora Jones et al., *GameDay: The Resilience Exercise Playbook* — facilitation guide for team-based resilience exercises.

- Jepsen analyses — <https://jepsen.io/analyses> — distributed correctness testing under partition; complementary to operational chaos (see Volume 6, Chapter 12).
- Adrian Cockcroft, *Learning from Chaos* — early articulation of chaos as confidence building for cloud migration.
- Bruce Schneier on blameless culture and learning from failure — see Chapter 6 (postmortems) for the cultural foundation that makes chaos safe to practice.

Chapter 9 — Deployment Strategies: Blue/Green, Canary, and Feature Flags

What this chapter covers. Every deployment is a bet that the new version is better than the old one — but the cost of losing that bet depends entirely on how you deploy. A strategy that shifts 100% of traffic to an untested version can turn a subtle bug into a full outage in seconds; a strategy that exposes the new version to 1% of traffic with automated rollback can contain the same bug to a handful of requests. This chapter covers the deployment strategies that make deploys safe: rolling, blue/green, canary (including automated progressive delivery with Argo Rollouts and Flagger), and feature flags (including OpenFeature and flag evaluation at scale), plus the traffic management, metrics analysis, and rollback automation that make each strategy trustworthy. Every pattern is grounded in runnable Kubernetes configs and real flag evaluation code.

Learning goals — after this chapter you should be able to:

- Compare rolling, recreate, blue/green, canary, shadow (dark launch), and A/B strategies on blast radius, rollback speed, resource cost, and operational complexity — and choose the right strategy for a given service tier and risk profile.
- Explain how canary analysis works — metric templates, success criteria, and automated promotion or rollback — and why a canary without automated analysis is just a slow rolling deploy.
- Write and operate Argo Rollouts and Flagger canary pipelines on Kubernetes, including traffic management via Istio, NGINX, and

Gateway API, metric analysis via Prometheus, and webhook-based custom checks.

- Design a feature flag system — flag types, evaluation models, targeting rules, and lifecycle — and implement flags with OpenFeature, including local evaluation, percentage rollouts, and kill switches.
 - Implement automated rollback that triggers on SLO burn, error rate, or custom health signals — and connect rollback policy to error budgets (Chapter 1).
 - Reason about the distributed-systems dimensions: database migration compatibility, cross-service deploy ordering, and the interplay between deployment strategy and capacity headroom.
-

Why deployment strategy matters

Deployments are the leading cause of incidents

Across industry surveys and postmortem corpora, a consistent finding is that a large fraction of Sev1 and Sev2 incidents are triggered by deployments — new code, new config, new infrastructure. The code may be correct in isolation but interact badly with production traffic, data, downstream behavior, or the previous version's state. The question is not whether deploys will sometimes be bad — they will — but whether the deployment strategy *contains* the blast radius and *recovers* quickly when they are.

A deployment strategy answers three questions:

1. **How much traffic sees the new version at each step?** (blast radius control)
2. **How is the new version evaluated?** (signal — metrics, checks, human judgment)
3. **How quickly and automatically is a bad version removed?** (rollback)

Strategies that answer these well make it safe to deploy frequently (which itself reduces risk — smaller changes are easier to reason about and roll back). Strategies that answer them poorly incentivize infrequent, large, high-risk deploys.

The DORA connection

The DORA research program (*Accelerate*, Forsgren et al.) finds that elite teams deploy *more frequently* and have *lower change failure rates* simultaneously — and that deployment automation (including progressive delivery and feature flags) is one of the capabilities that predicts both. The mechanism is straightforward: safe deployment strategies reduce the cost of a bad deploy, which reduces the fear of deploying, which enables smaller batch sizes, which reduces the chance that any single deploy is bad. It is a virtuous cycle — but only if the strategy actually contains blast radius and automates rollback.

What "safe" means for different tiers

Not every service needs the same strategy:

Tier	Example	Acceptable blast radius	Strategy
Tier 0 — critical path	Checkout, payments, auth	< 5% of traffic at any step; automated rollback in < 2 min	Canary with automated analysis + feature flags for risky changes
Tier 1 — important, not critical	Search, recommendations, notifications	< 25% per step; rollback in < 5 min	Canary or blue/green with metric gating
Tier 2 — internal / batch	Admin tools, analytics pipeline, internal dashboards	50-100% acceptable; rollback in < 15 min	Rolling or blue/green; canary is often overkill

Tier 0 services justify the complexity of automated canary analysis and flag infrastructure; tier 2 services often do not. Choose per service, not per organization.

The strategy catalog

Recreate

Delete all old pods, then create new pods. The simplest strategy — and the most disruptive: there is a window with zero replicas serving traffic. Only appropriate for non-availability-sensitive workloads (batch jobs, development environments) or single-replica services that cannot run two versions simultaneously due to resource constraints.

```
# Recreate – downtime is inherent
spec:
  strategy:
    type: Recreate
```

Rolling (Kubernetes default)

Replace pods incrementally: create one new pod, wait for it to become ready, delete one old pod, repeat. Controlled by `maxSurge` (how many extra pods above desired) and `maxUnavailable` (how many pods can be unavailable during the rollout).

```
spec:
  replicas: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 2           # up to 12 pods during rollout
      maxUnavailable: 1    # at least 9 pods available throughout
```

- **Blast radius:** Gradual — one pod at a time by default. But there is no traffic splitting: once a new pod is ready, it receives its share of load immediately via the Service endpoints. New code is exercised proportionally as it rolls out.
- **Rollback:** `kubectl rollout undo` — but it is itself a rolling update in reverse, so rollback is not instant. And there is no automated signal — a human must notice the problem and trigger it.
- **When to use:** Default for most tier 1–2 services. Simple, built-in, no extra tooling. Insufficient for tier 0 where you need traffic control and automated analysis.

Blue/Green

Run two full environments — blue (current) and green (new) — and switch traffic atomically from one to the other. Both environments run simultaneously during verification; rollback is an instant traffic switch back to blue.

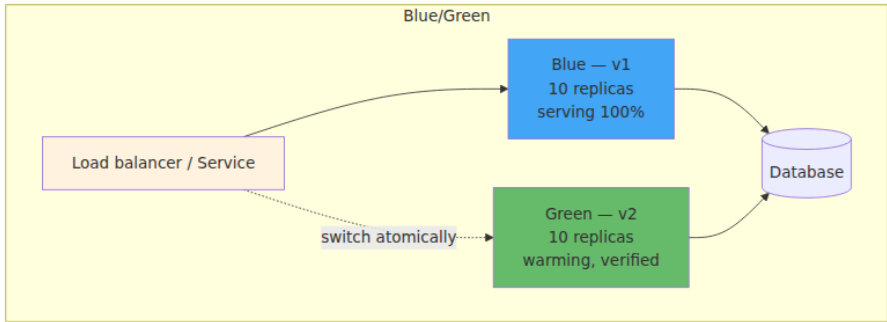


Figure 9-1: Blue/green. Both versions run at full scale; traffic switches atomically via label selector or load balancer. Rollback is an instant switch back — but the resource cost is 2× during the transition window.

- **Blast radius:** All-or-nothing at the switch point — 0% then 100%. Verification happens before the switch (smoke tests against green), but green has not served production traffic, so load-related bugs are not caught until the switch.
- **Rollback:** Near-instant — flip the Service selector or load balancer target back. No pod churn.
- **Resource cost:** 2× during the transition (both versions at full scale). For large services this can be expensive, and the cost scales with the number of services deployed blue/green simultaneously.
- **When to use:** Services where version coexistence is problematic (incompatible state, singleton constraints), or where instant rollback is worth the 2× resource cost. Common for database-adjacent services and infrastructure components.

Kubernetes blue/green with Argo Rollouts:

```

# rollout-bluegreen.yaml – Argo Rollouts blue/green with preview
service and auto-promotion
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: checkout-api
  namespace: production
spec:
  replicas: 10
  strategy:
    blueGreen:
      activeService: checkout-active           # Service serving
production traffic
      previewService: checkout-preview         # Service for verifying
green before promotion
      autoPromotionEnabled: false             # manual promotion gate –
human verifies preview
      scaleDownDelaySeconds: 600              # keep old ReplicaSet for
10 min after promotion (rollback window)
      prePromotionAnalysis:
# analysis before promotion – must pass
        templates:
          - templateName: checkout-smoke
        args:
          - name: service-name
            value: checkout-preview
      postPromotionAnalysis:                   # analysis after promotion
– auto-rollback if fails
        templates:
          - templateName: checkout-slo
        args:
          - name: service-name
            value: checkout-active
# Anti-affinity: never co-locate blue and green on same node
during transition
# (set via pod template affinity)
      selector:
        matchLabels: { app: checkout-api }
      template:
        metadata:
          labels: { app: checkout-api }
        spec:
          affinity:
            podAntiAffinity:
              preferredDuringSchedulingIgnoredDuringExecution:
                - weight: 100
                  podAffinityTerm:
                    labelSelector:
                      matchLabels: { app: checkout-api }
                    topologyKey: kubernetes.io/hostname

```

```

    containers:
      - name: checkout
        image: registry.example.com/checkout:v2.1.0
        ports: [{ containerPort: 8080 }]
        readinessProbe:
          httpGet: { path: /health, port: 8080 }
          initialDelaySeconds: 5
          periodSeconds: 5
        resources:
          requests: { cpu: "500m", memory: "512Mi" }
          limits: { cpu: "1000m", memory: "1Gi" }
---
apiVersion: v1
kind: Service
metadata:
  name: checkout-active
  namespace: production
spec:
  selector: { app: checkout-api }
  ports: [{ port: 80, targetPort: 8080 }]
---
apiVersion: v1
kind: Service
metadata:
  name: checkout-preview
  namespace: production
spec:
  selector: { app: checkout-api }
  ports: [{ port: 80, targetPort: 8080 }]

```

Promotion is manual (`kubectl argo rollouts promote checkout-api`) or triggered by CI after preview verification. Rollback is `kubectl argo rollouts undo checkout-api` — flips the active service back to the previous ReplicaSet instantly without pod churn.

Canary

Shift a small percentage of traffic to the new version, evaluate metrics, and progressively increase the percentage — aborting and rolling back automatically if metrics breach thresholds. The canonical safe-deploy strategy for tier 0.

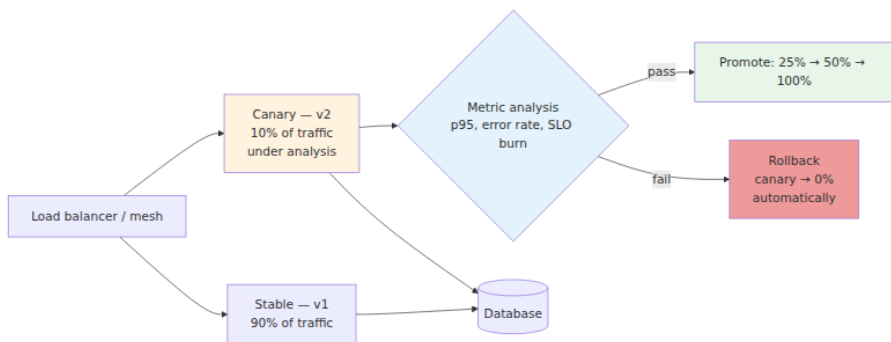


Figure 9-2: Canary. Traffic splits by weight; the canary is evaluated at each step. Automated analysis gates promotion — if metrics breach, the canary is removed without human intervention. The blast radius at any point is the canary weight.

- **Blast radius:** Proportional to canary weight at each step — 5% canary means at most 5% of users see a bad version.
- **Rollback:** Automatic on metric breach; manual via single command otherwise. Rollback is traffic-weight removal (canary weight → 0), not pod churn — fast.
- **Resource cost:** Canary replicas only — e.g., 1 canary pod per 10 stable pods for a 10% canary is ~10% overhead, not 100% like blue/green.
- **When to use:** Tier 0 and any service where bad deploys have user-visible impact. Requires traffic management (service mesh, ingress, or Gateway API) and metric analysis infrastructure — the tooling complexity is the main cost.

Shadow (dark launch) and A/B

- **Shadow (mirror):** Duplicate production traffic to the new version *without returning its responses to users* — the new version processes real requests but its responses are discarded. Tests the new code path under production load without any user impact. Useful for validating performance, correctness (compare shadow vs. stable responses), and downstream load — but doubles downstream traffic during the shadow period, so downstream must be prepared.
- **A/B (experiment):** Split traffic to measure *business* impact (conversion rate, engagement) rather than just technical metrics.

Requires longer holds (days to weeks) and statistical significance.
Typically implemented via feature flags with experiment assignment, not deployment strategy per se.

Progressive delivery: automated canary analysis

A canary without automated analysis is just a slow manual deploy — a human watches dashboards and decides whether to promote. Progressive delivery automates that loop: metric queries, success criteria, and promotion/rollback decisions.

How it works

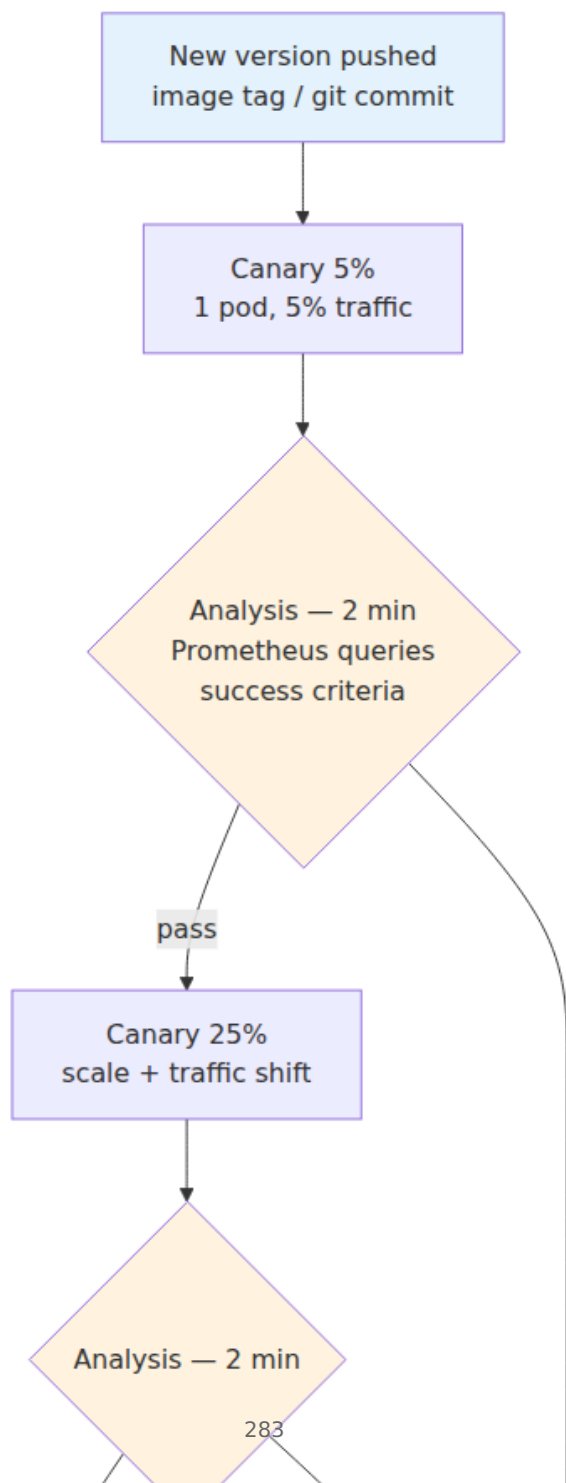


Figure 9-3: Automated canary progression. Each step runs a fixed analysis window of metric queries against the canary. Any failure aborts the rollout and removes canary traffic — no human in the loop for the failure path.

What to analyze

Canary analysis queries fall into three categories:

Category	Query
Error rate	<code>sum(rate(http_requests_total{service="checkout",pod=~"checkout-canary.*",code=~"5.."}[2m])) / sum(rate(http_requests_total{service="checkout",pod=~"checkout-canary.*"}[2m]))</code>
Latency	<code>histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket{service="checkout",pod=~"checkout-canary.*"}[2m])) by (le))</code>
SLO burn	Multi-window burn rate for the canary slice
Custom health	Webhook that checks business metrics (checkout conversion, queue depth, down rate)

The comparison strategy matters: comparing canary to *stable* (the current version serving 95% of traffic) is more robust than comparing to an absolute threshold, because it automatically accounts for production

variance — if the whole system is degraded, both canary and stable will show elevated errors and the canary will still pass relative comparison.

Argo Rollouts — canary with Prometheus analysis

Argo Rollouts is the most widely adopted Kubernetes progressive delivery controller. It replaces the Deployment with a Rollout CRD, manages ReplicaSets for stable and canary, controls traffic via service mesh or ingress, and runs analysis from metric templates.

Analysis templates (reusable metric queries):

```

# analysis-templates.yaml - Prometheus-backed canary analysis
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: error-rate
  namespace: production
spec:
  args:
    - name: service-name
    - name: canary-hash           # injected by Rollouts - identifies
canary pods
  metrics:
    - name: error-rate
      interval: 30s
      count: 4                   # 4 × 30s = 2 min analysis window per
step
      successCondition: result[0] <= 0.05
# error rate ≤ 5% (generous canary threshold)
      failureLimit: 2           # 2 consecutive failures → analysis
failed → abort
    provider:
      prometheus:
        address: http://prometheus.monitoring.svc:9090
        query: |
          sum(
            rate(http_requests_total{
              service="{{args.service-name}}",
              pod=~"{{args.canary-hash}}-.*",
              code=~"5.."
            })[2m]
          )
          /
          sum(
            rate(http_requests_total{
              service="{{args.service-name}}",
              pod=~"{{args.canary-hash}}-.*"
            })[2m]
          )
      ---
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: p95-latency
  namespace: production
spec:
  args:
    - name: service-name
    - name: canary-hash
  metrics:
    - name: p95

```

```

interval: 30s
count: 4
successCondition: result[0] <= 0.4    # p95 ≤ 400ms
failureLimit: 2
provider:
  prometheus:
    address: http://prometheus.monitoring.svc:9090
    query: |
      histogram_quantile(0.95,
        sum(
          rate(http_request_duration_seconds_bucket{
            service="{{args.service-name}}",
            pod=~"{{args.canary-hash}}-.*"
          }[2m])
        ) by (le)
      )
    ---
# Comparison template – canary vs stable (more robust than absolute threshold)
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: error-rate-comparison
  namespace: production
spec:
  args:
    - name: service-name
    - name: canary-hash
    - name: stable-hash
  metrics:
    - name: error-rate-delta
      interval: 30s
      count: 4
      # Canary error rate must not exceed stable by more than 2 percentage points
      successCondition: result[0] <= 0.02
      failureLimit: 2
      provider:
        prometheus:
          address: http://prometheus.monitoring.svc:9090
          query: |
            (
              sum(rate(http_requests_total{service="{{args.service-name}}",pod=~"{{args.canary-hash}}-.*",code=~"5.."}[2m]))
              / sum(rate(http_requests_total{service="{{args.service-name}}",pod=~"{{args.canary-hash}}-.*"}[2m]))
            )
            -
            (
              sum(rate(http_requests_total{service="{{args.service-name}}",pod=~"{{args.stable-hash}}-.*",code=~"5.."}[2m]))

```

```

        / sum(rate(http_requests_total{service="{{args.service-
name}}",pod=~"{{args.stable-hash}}-.*"}[2m]))
    )
---
# Webhook template - custom business health check
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: checkout-conversion
  namespace: production
spec:
  args:
    - name: service-name
  metrics:
    - name: conversion-rate
      interval: 60s
      count: 3
      successCondition: result == true
      failureLimit: 1
      provider:
        web:
          url: "http://health-checker.production.svc:8080/check?
service={{args.service-name}}&metric=conversion"
          timeoutSeconds: 10
          jsonPath: "${$.healthy}"

```

Canary Rollout with Istio traffic management:


```

# rollout-canary-istio.yaml – Argo Rollouts canary via Istio
VirtualService
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: checkout-api
  namespace: production
spec:
  replicas: 10
  strategy:
    canary:
      # Traffic management – Istio VirtualService + DestinationRule
      canaryService: checkout-canary
      stableService: checkout-stable
      trafficRouting:
        istio:
          virtualService:
            name: checkout-vs
            routes: ["primary"]          # named route in the
VirtualService
          destinationRule:
            name: checkout-dr
            canarySubsetName: canary
            stableSubsetName: stable
      steps:
        - setWeight: 5                    # 5% to canary
        - analysis:
            templates:
              - templateName: error-rate
              - templateName: p95-latency
            args:
              - name: service-name
                value: checkout-api
        - setWeight: 25                    # 25% to canary
        - analysis:
            templates:
              - templateName: error-rate-comparison
              - templateName: p95-latency
              - templateName: checkout-conversion
            args:
              - name: service-name
                value: checkout-api
        - setWeight: 50
        - pause: { duration: 5m }        # manual observation window
before 100%
      # Final step is implicit: 100% to stable (new version becomes
stable)

# Automatic rollback is built in – any analysis failure aborts to 0%
analysis:

```

```

    templates:
      - templateName: error-rate
    args:
      - name: service-name
        value: checkout-api
    # Background analysis runs throughout the entire rollout
  selector:
    matchLabels: { app: checkout-api }
  template:
    metadata:
      labels: { app: checkout-api }
    spec:
      containers:
        - name: checkout
          image: registry.example.com/checkout:v2.1.0
          ports: [{ containerPort: 8080 }]
          readinessProbe:
            httpGet: { path: /health, port: 8080 }
            periodSeconds: 5
          resources:
            requests: { cpu: "500m", memory: "512Mi" }
            limits: { cpu: "1000m", memory: "1Gi" }
---
# Istio VirtualService – Rollouts will patch weights
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: checkout-vs
  namespace: production
spec:
  hosts: ["checkout.example.com"]
  gateways: ["production/checkout-gateway"]
  http:
    - name: primary
      route:
        - destination: { host: checkout-
          stable.production.svc.cluster.local, subset: stable }
          weight: 100
        - destination: { host: checkout-
          canary.production.svc.cluster.local, subset: canary }
          weight: 0
---
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: checkout-dr
  namespace: production
spec:
  host: checkout-stable.production.svc.cluster.local
  subsets:
    - name: stable

```

```
labels: { app: checkout-api }  
- name: canary  
labels: { app: checkout-api }
```

NGINX Ingress variant (for clusters without a service mesh):

```
# Traffic routing via NGINX Ingress – no mesh required  
strategy:  
  canary:  
    canaryService: checkout-canary  
    stableService: checkout-stable  
    trafficRouting:  
      nginx:  
        stableIngress: checkout-stable-ingress  
        # Rollouts patches the canary ingress weight annotation  
steps:  
  - setWeight: 10  
  - analysis:  
    templates: [{ templateName: error-rate }]  
  - setWeight: 50  
  - pause: { duration: 3m }
```

```

# NGINX ingresses
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: checkout-stable-ingress
  annotations:
    kubernetes.io/ingress.class: nginx
spec:
  rules:
    - host: checkout.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend: { service: { name: checkout-stable, port: {
number: 80 } } }
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: checkout-canary-ingress
  annotations:
    kubernetes.io/ingress.class: nginx
    nginx.ingress.kubernetes.io/canary: "true"
    nginx.ingress.kubernetes.io/canary-weight: "0" # patched by
Rollouts
spec:
  rules:
    - host: checkout.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend: { service: { name: checkout-canary, port: {
number: 80 } } }

```

Flagger (Weaveworks, now FluxCD ecosystem)

Flagger is the alternative progressive delivery operator — originally tied to FluxCD but usable standalone. It automates canary analysis with a simpler model and supports more traffic providers (Istio, Linkerd, NGINX, Gateway API, App Mesh, Gloo).

Flagger canary:

```

# flagger-canary.yaml – Flagger canary with Gateway API + Prometheus
analysis
apiVersion: flagger.app/v1beta1
kind: Canary
metadata:
  name: checkout-api
  namespace: production
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: checkout-api
  service:
    port: 80
    targetPort: 8080
    gateways: ["production/checkout-gateway"]
    # Flagger creates primary, canary, and canary-Gateway HTTPRoute
    # automatically
  analysis:
    interval: 30s
    threshold: 3 # consecutive failures before
    rollback
    maxWeight: 50
    stepWeight: 10 # 10% → 20% → ... → 50% →
    promotion
  metrics:
    - name: request-success-rate
      thresholdRange: { min: 99 } # success rate ≥ 99%
      interval: 30s
    - name: request-duration
      thresholdRange: { max: 400 } # p99 ≤ 400ms
      interval: 30s
    templateRef:
      name: latency-p99
      namespace: flagger-system
    # Custom metric – business health
    - name: checkout-conversion
      thresholdRange: { min: 2.5 } # conversion rate ≥ 2.5%
      interval: 60s
    templateRef:
      name: conversion-rate
      namespace: flagger-system
  webhooks:
    - name: load-test
      type: rollout # run once when canary is created
      url: http://flagger-loadtester.flagger-system/loadtest
      metadata:
        cmd: "hey -z 1m -q 10 -c 2 http://checkout-api-
        canary.production:80/health"
    - name: acceptance-test

```

```

    type: pre-rollout           # must pass before any traffic
shift
    url: http://test-runner.production.svc:8080/acceptance
    timeout: 30s
    metadata:
      type: bash
      cmd: "curl -sd '{"test\":" checkout-flow\'}' http://
checkout-api-canary.production:80/test"
    - name: slack-notify
      type: event
      url: http://event-receiver.flagger-system/notify
      metadata:
        channel: "#deploys"
    # Gateway API HTTPRoute for traffic splitting – Flagger patches
weights
    # Flagger query templates (MetricTemplate CRDs)
---
apiVersion: flagger.app/v1beta1
kind: MetricTemplate
metadata:
  name: latency-p99
  namespace: flagger-system
spec:
  provider:
    type: prometheus
    address: http://prometheus.monitoring.svc:9090
  query: |
    histogram_quantile(0.99,
      sum(
        rate(http_request_duration_seconds_bucket{
          app="{{ target }}",
          route="{{ route }}"
        }[2m])
      ) by (le)
    )
---
apiVersion: flagger.app/v1beta1
kind: MetricTemplate
metadata:
  name: conversion-rate
  namespace: flagger-system
spec:
  provider:
    type: prometheus
    address: http://prometheus.monitoring.svc:9090
  query: |
    sum(rate(checkout_completed_total{service="{{ target }}"}[2m]))
    / sum(rate(checkout_started_total{service="{{ target }}"}[2m]))
    * 100

```

Choosing between Argo Rollouts and Flagger:

Dimension	Argo Rollouts	Flagger
Control model	Rollout CRD replaces Deployment; explicit step definitions	Canary CRD wraps a Deployment; convention-based steps (<code>stepWeight</code> , <code>maxWeight</code>)
Traffic providers	Istio, NGINX, ALB, Gateway API, SMI, App Mesh	Istio, Linkerd, NGINX, Gateway API, App Mesh, Gloo, Contour
Analysis	AnalysisTemplate + AnalysisRun; flexible success conditions (CEL)	MetricTemplate + threshold ranges; simpler but less expressive
Ecosystem	ArgoCD native; <code>kubectl argo rollouts</code> CLI; dashboard	FluxCD native; works standalone; Grafana dashboards included
Maturity	CNCF, widely adopted with ArgoCD	CNCF, broad traffic provider support
When to choose	Already on ArgoCD; need expressive analysis with custom success conditions	Already on FluxCD; want simpler canary config; need Linkerd or Gloo

Both support the same core flow: create canary → shift weight → analyze → promote or rollback. Choose based on your GitOps platform and traffic provider, not on feature comparison — they converge on the same capability.

Feature flags: decoupling deploy from release

Deployment strategies control *which version serves traffic*; feature flags control *which code path executes* within a version. Together they enable the most powerful safety pattern: deploy code to production with the new path disabled, then enable it progressively via flags — with instant kill-switch rollback that does not require a redeploy.

Flag types

Type	Purpose	Example	Lifetime
Release flag	Gate a new feature until ready; decouple deploy from launch	<code>checkout_v2_enabled</code> — new checkout flow behind flag	Weeks to months; removed after full rollout
Experiment flag	A/B test with controlled assignment and metrics	<code>search_ranking_v3</code> — 50/50 split, measure conversion	Duration of experiment (days to weeks)
Ops / kill switch	Disable a feature or code path under incident	<code>payments_use_new_processor</code> — flip to <code>false</code> if new processor degrades	Permanent; always present as circuit breaker
Permission flag	Gate features by plan, role, or entitlement	<code>enterprise_sso_enabled</code> — only for enterprise tier	Permanent
Config flag	Dynamic configuration via flag system	<code>rate_limit_checkout_rps</code> — tune without redeploy	Permanent or until replaced by proper config

The critical distinction: **release and experiment flags are temporary** (they accumulate debt if not removed); **ops and permission flags are permanent** (they are part of the system's control plane).

Flag lifecycle

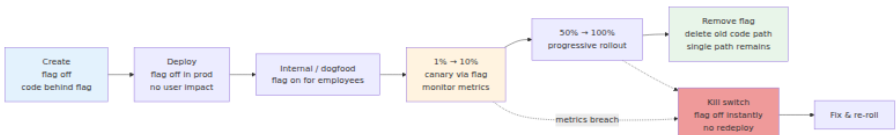


Figure 9-4: Feature flag lifecycle. Code ships with the flag off (no risk), is validated internally, rolls out progressively by flag targeting, and the flag is removed after full rollout. The kill switch provides instant rollback at any stage without a redeploy.

Flag debt is real — every flag adds a code branch that must be tested, reasoned about, and eventually removed. Teams that do not enforce flag removal accumulate combinatorial complexity: 10 flags means 1024 potential code path combinations, most untested. Track flag age, alert on flags older than their intended lifetime, and make flag removal part of the definition of done for the feature.

OpenFeature: the vendor-neutral flag standard

OpenFeature is a CNCF standard for feature flag evaluation — a single API that works with any flag provider (LaunchDarkly, Flagsmith, Unleash, Flipt, ConfigCat, or a homegrown system). It prevents vendor lock-in and standardizes flag evaluation across services.

OpenFeature evaluation — Go example with local flag file provider:

```

// flags/evaluator.go – OpenFeature evaluation with local + remote
provider
package flags

import (
    "context"
    "log/slog"

    "github.com/open-feature/go-sdk/openfeature"
    "github.com/open-feature/go-sdk/openfeature/memprovider"
)

// Flags used by the checkout service
const (
    FlagCheckoutV2      = "checkout_v2_enabled"
    FlagNewProcessor    = "payments_use_new_processor"
    FlagSearchRankingV3 = "search_ranking_v3"
)

func InitFlags() {
    // In production, replace memprovider with your flag provider:
    // LaunchDarkly: github.com/launchdarkly/go-sdk-common
    // Unleash:      github.com/Unleash/unleash-go-sdk
    // Flipt:        github.com/flipt-io/flipt-client-sdks/go
    // Flagsmith:    github.com/Flagsmith/flagsmith-go-client
    provider :=
memprovider.NewInMemoryProvider(map[string]memprovider.Flag{
    FlagCheckoutV2: {
        Variants:    map[string]any{"on": true, "off": false},
        DefaultVariant: "off",
    },
    FlagNewProcessor: {
        Variants:    map[string]any{"on": true, "off": false},
        DefaultVariant: "on", // kill switch – new processor is
default, flag disables it
    },
})
    openfeature.SetProvider(provider)
}

// Evaluate a boolean flag for a request context
func IsCheckoutV2Enabled(ctx context.Context, userID, region string) bool {
    client := openfeature.NewClient("checkout-service")
    enabled, err := client.BooleanValue(ctx, FlagCheckoutV2, false, openfeature.EvaluationContext{
        TargetingKey: userID,
        Attributes: map[string]any{
            "region": region,
        },
    },

```

```

    })
    if err != nil {
        slog.Warn("flag evaluation failed, using default", "flag", Flag
CheckoutV2, "error", err)
        return false // safe default – old path
    }
    return enabled
}

```

```

// handler/checkout.go – flag-gated code path with kill switch
func (h *Handler) Checkout(w http.ResponseWriter, r *http.Request) {
    ctx := r.Context()

    // Kill switch: if new processor is flagged off, use fallback
    immediately
    useNewProcessor := flags.IsNewProcessorEnabled(ctx)

    // Release flag: new checkout flow vs. legacy
    if flags.IsCheckoutV2Enabled(ctx, currentUser(r), region(r)) {
        h.checkoutV2(w, r, useNewProcessor)
        return
    }
    h.checkoutV1(w, r, useNewProcessor)
}

func (h *Handler) checkoutV2(w http.ResponseWriter, r *http.Request, us
eNewProcessor bool) {
    // New flow – includes new processor path gated by ops flag
    var err error
    if useNewProcessor {
        err = h.processorV2.Charge(r.Context(), chargeReq)
        if err != nil {
            // Fallback: flag evaluation already decided, but processor
failure
            // can also trigger local fallback – defense in depth
            slog.Error("v2 processor failed, falling back", "error", er
r)

            err = h.processorV1.Charge(r.Context(), chargeReq)
        }
    } else {
        err = h.processorV1.Charge(r.Context(), chargeReq)
    }
    // ...
}

```

Flag evaluation architecture — local vs. remote:

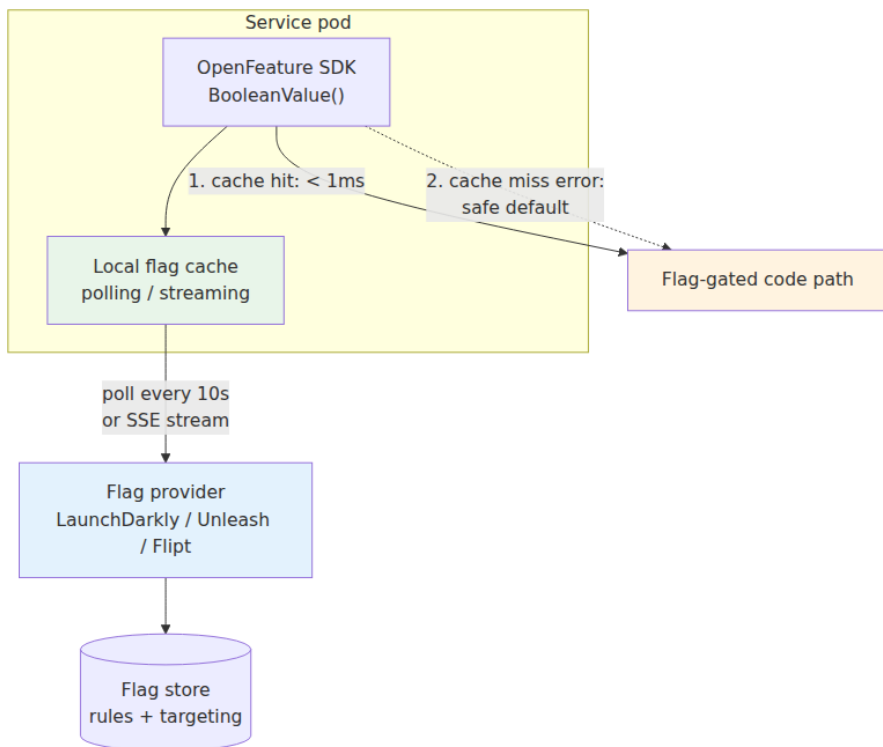


Figure 9-5: Flag evaluation architecture. The SDK evaluates against a local cache that syncs from the provider — evaluation is local and fast (< 1 ms), not a network call per request. On cache miss or error, the safe default is returned.

Key design decisions for flag infrastructure:

- **Local evaluation is mandatory for latency-sensitive paths.** A flag check that makes a network call per request adds latency and a new failure mode. The SDK must cache flags locally and evaluate without I/O. Almost all production flag systems work this way — LaunchDarkly streaming, Unleash polling, Flipt local evaluation engine.
- **Targeting rules are evaluated locally.** Percentage rollouts (1% → 10% → 100%), attribute-based targeting (region, plan, user ID hash), and prerequisite flags (enable flag B only if flag A is on) are all evaluated from the cached flag set.

- **Flag changes propagate within seconds**, not per-request. A kill switch flip takes effect within the cache poll interval (typically 5–30 s) — fast enough for incident response, not instant. For sub-second kill switches, combine flags with a separate fast-path mechanism (e.g., a config value watched via `fsnotify` or an in-memory circuit breaker).
- **Flags must have safe defaults**. If the flag provider is unreachable, the SDK returns the default value — which should be the *safe* code path (old version, not new). Never default to the new path.

Percentage rollout with stickiness:

A naive percentage rollout hashes the user ID and enables the flag if `hash(userID) % 100 < percentage`. This is sticky — the same user always gets the same variant — which is essential for user experience (a user should not flip between old and new checkout on every request). Most flag providers implement this as consistent hashing on the targeting key.

```

# Flipt flag definition – percentage rollout with stickiness
# flags/checkout_v2.yaml
namespace: production
flags:
  - key: checkout_v2_enabled
    name: "Checkout V2"
    type: BOOLEAN
    description: "New checkout flow – progressive rollout"
    enabled: true
    rollouts:
      - description: "10% of users – sticky on user_id"
        segment:
          segmentKeys: ["beta_users"]           # explicit segment
          (internal team, beta group)
          rank: 1
          threshold:
            percentage: 100                     # 100% of this segment
            value: true
      - description: "10% of remaining users"
        rank: 2
        threshold:
          percentage: 10
          value: true
    # Default: false (old checkout)
  variants:
    - key: "on"
      attachment: { enabled: true }
    - key: "off"
      attachment: { enabled: false }

```

Flags versus canary: when to use which

Dimension	Canary (traffic split)	Feature flag (code path)
Granularity	Per-replica / per-pod traffic weight	Per-request targeting (user, region, plan)
Rollback	Traffic weight → 0 (seconds)	Flag flip (seconds, via cache poll)
Testing	New binary / image	Same binary, different code path
Scope	Entire service version	Single feature within a service
Use for	Validating a new build end-to-end	Gating a specific feature, experiment, or kill switch

They compose: deploy a new version via canary (validate the build), then enable a feature within that version via flag (validate the feature). The canary protects against build-level regressions; the flag protects against feature-level regressions — and the flag's kill switch works even after the canary has fully promoted.

Traffic management: the substrate

All deployment strategies except recreate and rolling require traffic splitting — which requires a traffic management layer.

Layer	Mechanism	Weight granularity	When to use
Kubernetes Service	Label selector switching (blue/green only)	All-or-nothing (no weighted split)	Blue/green without weighted canary
Istio / Linkerd	VirtualService / TrafficSplit with weighted destinations	Per-request, any percentage	Full canary with fine-grained weights; already running a mesh
NGINX Ingress	<code>canary-weight</code> annotation	Per-request, any percentage	Canary without a mesh; simple ingress-based splitting
Gateway API	HTTPRoute with weighted <code>backendRefs</code>	Per-request, any percentage	Modern replacement for Ingress; works with any Gateway controller
Application-level	Flag-based or header-based routing in app code	Per-request, any logic	Shadow, A/B, and header-based canary (e.g., <code>x-canary: true</code> routes to new version)

Gateway API (the modern standard):

```
# Gateway API – weighted canary via HTTPRoute (Flagger or Rollouts with
Gateway API provider)
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: checkout-route
  namespace: production
spec:
  parentRefs:
    - name: production-gateway
  hostnames: ["checkout.example.com"]
  rules:
    - matches:
        - path: { type: PathPrefix, value: / }
      backendRefs:
        - name: checkout-stable
          port: 80
          weight: 90
        - name: checkout-canary
          port: 80
          weight: 10
# Progressive delivery controllers patch the weights: 90/10 → 75/25 →
50/50 → 100/0
```

Gateway API is the recommended choice for new clusters — it is vendor-neutral (works with Istio, Cilium, NGINX Gateway, Envoy Gateway, and others), supports weighted splitting natively, and replaces the Ingress API which is effectively frozen.

Database migrations and cross-service ordering

Deployment strategy is not just about pods — it is about the *entire system state* that the new version depends on.

The expand/contract pattern for database migrations

A canary that runs two versions simultaneously against the same database must handle the case where both versions' schemas coexist. The expand/contract (parallel change) pattern makes this safe:

Phase 1  **Expand:** Add new column/table (both versions can read; new version writes new shape)
 Deploy new version via canary (both versions serve traffic, both schemas present)
Phase 2  **Migrate:** Backfill data from old shape to new shape
Phase 3  **Contract:** Remove old column/table after all replicas run the new version

Each phase is a separate deploy — never combine a breaking schema change with a code deploy in one step. Tools that enforce this: `golang-migrate`, `Flyway`, `Alembic`, `Atlas`, or managed solutions (PlanetScale branching, Neon branching).

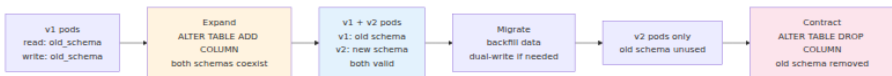


Figure 9-6: Expand/contract for zero-downtime schema changes. Both versions coexist during the canary window — the schema must be compatible with both. Contract only after the old version is fully drained.

Cross-service deploy ordering

When service A depends on a new API in service B, the deploy order matters:

- **Additive changes** (new field, new endpoint): Deploy B first (new API is available but unused), then A (starts calling new API). Safe with any deployment strategy — the new API is backward compatible.
- **Breaking changes** (removed field, changed semantics): Requires coordinated expand/contract — B deploys a version that supports both old and new API shapes, A migrates to the new shape, B removes the old shape. This is an API versioning problem (Volume 8, Chapter 5) — deployment strategy alone cannot solve it.

Rollback: automated, fast, and connected to SLOs

Rollback is not a manual `kubectl` command run after someone notices an alert — it is an automated response to SLO breach that happens without human intervention.

Rollback triggers

Trigger	Signal	Latency to rollback
Canary analysis failure	Prometheus metric breach during canary steps	Seconds (next analysis interval)
SLO burn rate	Multi-window burn rate alert (Chapter 1)	1–5 min (alert evaluation window)
Error budget exhaustion	Error budget policy (Chapter 1)	Policy-defined (e.g., freeze deploys when budget < 25%)
Webhook failure	Custom health check (business metric, synthetic probe)	Webhook timeout (10–30 s)
Manual	Human judgment via CLI or dashboard	Human reaction time (minutes)

Connecting rollback to error budgets

The error budget (Chapter 1) is the natural governor for deployment aggressiveness:

- If the service has ample error budget, canary analysis can be lenient (higher error tolerance) and promotion can be faster.
- If the budget is nearly exhausted, canary analysis should be strict and deploys should be gated — or blocked entirely until the budget recovers.
- A canary-induced error rate that burns budget faster than the SLO allows should abort even if absolute error rates are low — the *rate of burn* matters, not just the absolute value.

```

# Argo Rollouts – error budget-aware analysis via webhook
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: error-budget-burn
  namespace: production
spec:
  metrics:
    - name: burn-rate
      interval: 30s
      count: 4
      # Multi-window burn rate query – abort if canary burn rate
      # exceeds 5x normal
      successCondition: result[0] < 5.0
      failureLimit: 1
      provider:
        prometheus:
          address: http://prometheus.monitoring.svc:9090
          query: |
            (
              sum(rate(http_requests_total{service="checkout",code=~"5.
              ."}[5m]))
              / sum(rate(http_requests_total{service="checkout"}[5m]))
            )
            /
            (1 - 0.999) # 1 - SLO target (99.9%): normalized burn rate
            # Burn rate 1.0 = exactly at SLO; 5.0 = burning 5x the
            budget rate

```

Choosing a strategy: decision framework

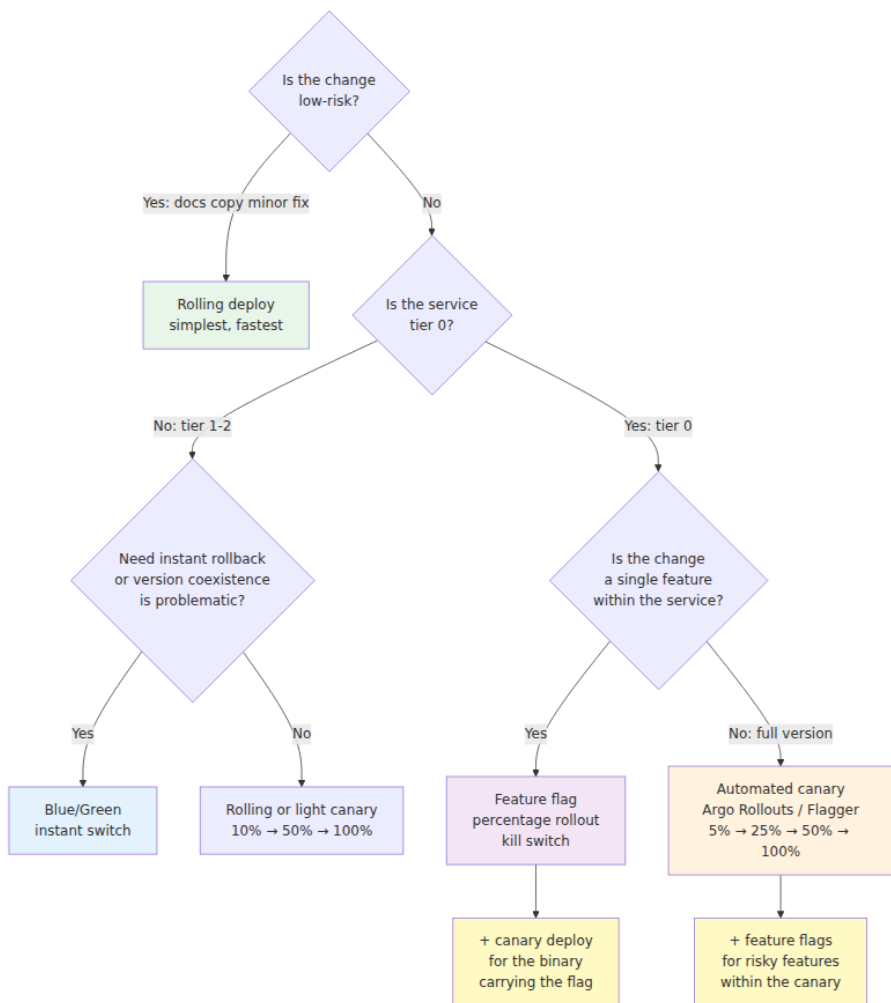


Figure 9-7: Deployment strategy decision framework. Risk and tier determine the strategy; feature flags and canary compose — the safest tier-0 deploys use both.

Distributed-systems lens

Deployment strategy in distributed backends has failure modes that single-service reasoning misses:

Version skew during rollout. During a rolling or canary deploy, two versions serve traffic simultaneously. If v2 writes data in a new format that v1 cannot read, requests that land on v1 after v2 has written will fail — even though each version is correct in isolation. The fix is expand/contract (above) and forward-compatible serialization (Volume 8, Chapter 8) — never assume all replicas run the same version at any instant.

Cross-service version dependencies. If services A and B are deployed simultaneously and A v2 depends on B v2's new API, there is a window where A v2 calls B v1 (which lacks the API) or A v1 calls B v2 (which may have removed the old API). Sequence deploys (B before A for additive changes) or use expand/contract for breaking changes. Avoid simultaneous cross-service deploys that create version skew in the call graph.

Capacity during deployment. A rolling deploy temporarily reduces available replicas (if `maxUnavailable > 0`) or requires extra capacity (if `maxSurge > 0`). A blue/green deploy requires $2\times$ capacity. Under high load, the reduced headroom during a rolling deploy can push the system into the danger zone (Chapter 7, Figure 7-6) — a deploy that is safe at low traffic can cause SLO breach at peak. Schedule tier-0 deploys outside peak or ensure headroom before deploying.

Flag evaluation under partition. If flag evaluation requires a network call (remote evaluation), a network partition makes flags unavailable — and the safe default determines whether the system degrades to the old or new path. Local evaluation (cached flags) avoids this but introduces staleness — a kill switch flip during a partition will not propagate to partitioned pods until the partition heals. Design for both: local evaluation for steady state, with a separate fast-path kill mechanism for partitions if sub-second response is required.

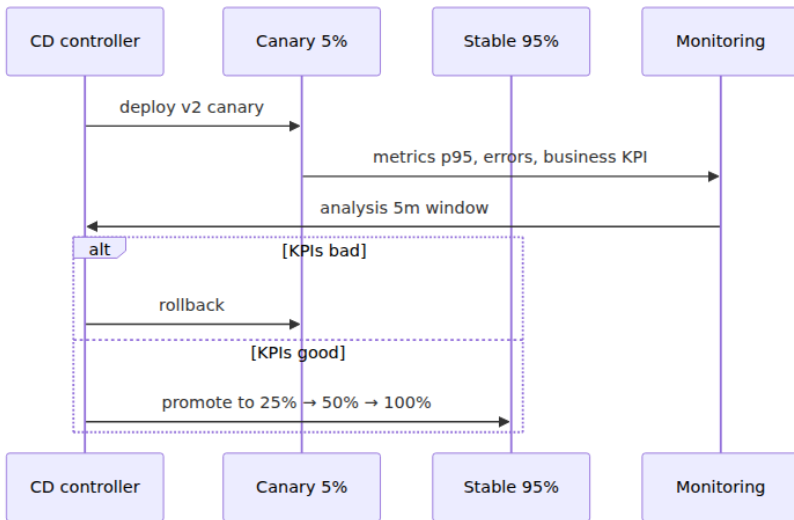
Canary signal validity. A 5% canary that receives 5% of traffic may not exercise the same code paths as 100% traffic — especially for features with low trigger rates (e.g., a checkout path that 2% of users hit: $5\% \text{ canary} \times 2\% \text{ trigger} = 0.1\%$ of total traffic exercises the new checkout code — too little for statistical significance in a 2-minute analysis window). For low-frequency paths, either increase canary weight, extend analysis duration, or use flag-based targeting that routes 100% of the relevant user segment to the canary.

Anti-patterns and common mistakes

Anti-pattern	Why it hurts	Fix
Canary without automated analysis	A canary that requires a human to watch dashboards is just a slow rolling deploy — the human may not notice the signal in time, especially at low canary weights.	Automate analysis with metric templates and automatic rollback; human approval only for promotion after analysis passes.
Absolute thresholds instead of relative comparison	Absolute thresholds (error rate < 5%) pass when the whole system is degraded (stable is at 6%) and fail when the canary is actually fine but the threshold is too strict.	Compare canary to stable (or to pre-deploy baseline) with a delta tolerance.
Flag without safe default	If the flag provider is unreachable, the service crashes or takes the wrong path — the flag system becomes a new failure mode.	Every flag evaluation must have a safe default (old path) on error; test the provider-down scenario.
Permanent flags that are never removed	Accumulated flags create combinatorial code paths, most untested — 10 flags = 1024 combinations.	Track flag age; alert on flags older than intended lifetime; make removal part of the feature's definition of done.
Breaking schema change in one deploy	Two versions cannot coexist with an incompatible schema — the canary breaks the stable version's reads.	Expand/contract: additive migration first, code deploy second, cleanup third — three separate deploys.
Deploying multiple tier-0 services simultaneously	Simultaneous deploys create cross-service version skew and combined blast radius that is hard to attribute.	Sequence tier-0 deploys; never deploy two critical-path services concurrently.

Anti-pattern	Why it hurts	Fix
No load testing of the canary path	Canary analysis at 5% traffic does not validate performance under full load — a canary that passes at 5% may collapse at 100%.	Complement canary with load testing (Chapter 7) and shadow traffic for performance validation.
Rollback that requires a new deploy	Rolling back by deploying the old image is itself a deploy — with its own risk and delay.	Blue/green and canary rollback is a traffic switch (weight $\rightarrow 0$), not a redeploy — instant and safe. Feature flag rollback is a flag flip — no binary change.

Canary Analysis and Auto-Rollback



Key takeaways

- Deployment strategy determines **blast radius** (how much traffic sees the new version), **signal** (how the new version is evaluated), and **rollback** (how quickly a bad version is removed). The strategy is the

primary lever for deployment safety — more impactful than code review or testing alone for containing bad deploys.

- **Rolling** (Kubernetes default) is simple and sufficient for tier 1-2 services but has no traffic control or automated analysis. **Blue/green** provides instant rollback via traffic switch at $2\times$ resource cost — best when version coexistence is problematic. **Canary** provides progressive traffic shifting with automated analysis — the default for tier 0 where blast radius must be minimized.
- A **canary without automated analysis is just a slow rolling deploy**. Progressive delivery (Argo Rollouts, Flagger) automates the loop: shift weight → query Prometheus → check success criteria → promote or abort. Analysis should **compare canary to stable** (relative), not to an absolute threshold, to account for production variance.
- **Argo Rollouts** (explicit steps, expressive CEL-based analysis, ArgoCD-native) and **Flagger** (convention-based steps, broad traffic provider support, FluxCD-native) both implement the same core flow — choose based on your GitOps platform and traffic provider (Istio, NGINX, Gateway API, Linkerd). Both support the modern **Gateway API** for vendor-neutral traffic splitting.
- **Feature flags** decouple deploy from release: code ships with the flag off (no risk), is validated internally, rolls out progressively by targeting rules (percentage, attributes, segments), and is removed after full rollout. The **kill switch** (flag flip, no redeploy) is the fastest rollback path — seconds via cache poll.
- **OpenFeature** provides a vendor-neutral flag evaluation API; the SDK evaluates against a **local cache** (polling or streaming from the provider) so flag checks are < 1 ms and do not add a network call per request. Every flag must have a **safe default** (old path) for provider-down scenarios.
- **Flags and canaries compose**: canary validates the build end-to-end, flags gate individual features within the build. The safest tier-0 deploys use both — canary for the binary, flags for risky features, with flag kill switches available even after the canary has fully promoted.
- **Database migrations require expand/contract** — additive schema change first, code deploy second, cleanup third — because canary and rolling deploys run two versions simultaneously against the same

database. Breaking schema changes in one deploy will break the stable version.

- **Rollback must be automated and connected to SLOs** — canary analysis failure, SLO burn rate alerts, and error budget policy should all trigger rollback without human intervention. Manual rollback is the fallback, not the primary path. The **error budget** (Chapter 1) naturally governs deployment aggressiveness.
-

Further reading

- Argo Rollouts documentation — canary, blue/green, analysis, and traffic routing: <https://argoproj.github.io/argo-rollouts/>
- Flagger documentation — canary analysis, metric templates, and Gateway API: <https://docs.flagger.app/>
- Gateway API documentation — HTTPRoute weighted splitting: <https://gateway-api.sigs.k8s.io/>
- OpenFeature specification — <https://openfeature.dev/docs/reference/concepts/evaluation-api> — vendor-neutral flag evaluation.
- LaunchDarkly, Unleash, Flipt, Flagsmith documentation — production flag providers with local evaluation SDKs.
- Martin Fowler — *Feature Toggles (aka Feature Flags)* — <https://martinfowler.com/articles/feature-toggles.html> — the foundational taxonomy of flag types.
- Pete Hodgson — *Feature Toggle Patterns* and flag debt discussion.
- Forsgren, Humble, and Kim, *Accelerate* (IT Revolution, 2018) — DORA research on deployment frequency, change failure rate, and the practices that predict both.
- Google SRE Book, Chapter 16 — *Release Engineering* (<https://sre.google/sre-book/release-engineering/>) — safe release practices at Google scale.

- Sam Newman, *Monolith to Microservices* (O'Reilly) — expand/contract and deployment ordering for distributed systems.

Chapter 10 — Resilience Patterns in Production

What this chapter covers. A single slow downstream can cascade into a full outage if every caller blocks waiting for it — threads exhaust, queues fill, retries amplify, and suddenly the entire mesh is down because one dependency stuttered. Resilience patterns are the structural defenses that prevent local failures from becoming systemic: timeouts that bound waiting, retries that heal transient faults without amplifying them, circuit breakers that stop calling a failing dependency, bulkheads that isolate failure domains, load shedding that protects the server from its callers, and fallbacks that degrade gracefully when the happy path is unavailable. This chapter covers every pattern in the depth needed to configure it correctly in production — with real Resilience4j, Istio, and Envoy configs, the tuning trade-offs that determine whether the pattern helps or hurts, and the observability and chaos practices that prove it works before an incident does.

Learning goals — after this chapter you should be able to:

- Configure timeouts, retries with jittered backoff, hedged requests, and deadline propagation so that transient faults are healed without creating retry storms or unbounded tail latency.
- Implement circuit breakers (Resilience4j, Istio `DestinationRule`, Envoy) and bulkheads (thread-pool and semaphore) — explain the state machine, tune thresholds, and avoid the common misconfigurations that make breakers flap or never open.
- Apply concurrency limiting and adaptive load shedding (Netflix `concurrency-limits-java`, Envoy, K8s) to protect a service from overload driven by callers, and choose between shedding at the edge, at the service, and at the downstream.
- Design graceful degradation and fallback strategies — cached, stale, default, and partial responses — and decide when a fallback is safer than a failure.

- Reason about the interactions between patterns — how retries and circuit breakers compose, why a timeout is a prerequisite for every other pattern, and how bulkheads prevent breaker-induced thread starvation.
- Validate resilience with fault injection and chaos experiments, measure it with SLI/SLO burn, and operate it with dashboards and alerts that distinguish expected degradation from real incidents.

Boundary note. Volume 3, Chapter 11 introduced the wire-level primitives — timeouts, retries, exponential backoff, and hedged requests — as network reliability mechanisms. This chapter is the *production* treatment: how those primitives compose with circuit breakers, bulkheads, and load shedding into a coherent resilience architecture; how to configure them in the frameworks and meshes you actually run; and how to tune and verify them under real traffic. Volume 7, Chapter 11 approached bulkheads and breakers from the system-design perspective; here we go deep on implementation, configuration, and operational reality. Volume 10, Chapter 7 covered backpressure and flow control inside messaging systems; this chapter covers request-path resilience between synchronous services.

Why resilience patterns exist

The anatomy of a cascade

Consider a service `api` that calls `catalog` that calls `inventory`. `inventory` degrades — p95 latency climbs from 20 ms to 4 s due to a bad index. Without resilience patterns:

1. `catalog` threads block for 4 s each waiting for `inventory`. Its thread pool saturates.
2. `catalog` stops responding to `api` within `api`'s timeout — or `api` also has no timeout, so `api` threads block too.
3. `api` thread pool saturates. Now every endpoint on `api` is down — even those that do not touch `inventory`.
4. Callers of `api` retry. Retry load amplifies the pressure. The incident widens.
5. Autoscaling adds pods, but new pods immediately saturate too — scaling does not help when the bottleneck is a downstream.

Total time from degraded downstream to full outage: often under 60 seconds. Every pattern in this chapter exists to interrupt this chain at a different link.



Figure 10-1: A slow downstream without resilience cascades through thread pools and is amplified by retries. With timeouts, bulkheads, breakers, and fallbacks the failure is contained and partially degraded rather than total.

The prerequisites

Two mechanisms are prerequisites for every other pattern:

- **Timeouts** bound how long a caller waits. Without a timeout, no other pattern triggers — the thread simply blocks until the downstream eventually responds or the TCP connection breaks minutes later.
- **Deadlines** propagate the caller's remaining budget downstream so that work that cannot possibly be returned in time is not started.

If you take one rule from this chapter: *every outbound call must have an explicit timeout and participate in deadline propagation*. Everything else is layered on top.

Timeouts, deadlines, and hedged requests

Timeouts

A timeout is a contract: "if I have not received a response in N milliseconds, I will abandon this attempt." Correct timeout values are derived from the downstream's latency distribution and the caller's SLO — not guessed.

Timeout strategy	How to set	When to use
p99 + headroom	Downstream p99 × 1.5-2, capped by caller SLO	Default for latency-sensitive paths

Timeout strategy	How to set	When to use
SLO-derived	Caller SLO budget minus local processing time	When the caller has a tight SLO (eimplies checkout)
Tiered	Short timeout for the first attempt, longer for a retry	When the first attempt targets a fast replica

Timeouts must be set at every layer: client library, service mesh proxy, and gateway. A common failure is setting a timeout in application code but leaving the Envoy/NGINX proxy at its default (often 15 s or infinite), so the proxy holds the connection long after the application gave up — wasting resources for no reason.

```
# Istio: per-route timeout – must be tighter than the caller's timeout
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: catalog
spec:
  hosts: [catalog.prod.svc.cluster.local]
  http:
    - match: [{ uri: { prefix: /api/catalog } }]
      timeout: 500ms # hard deadline for this route
      retries:
        attempts: 2
        perTryTimeout: 250ms # each attempt bounded
        retryOn: 5xx,gateway-error,connect-failure,retriable-4xx
        retryRemoteLocalities: true
      route:
        - destination: { host: catalog.prod.svc.cluster.local }
```

```
// Resilience4j: TimeLimiter + CompletableFuture – application-layer
// timeout
TimeLimiterConfig tlConfig = TimeLimiterConfig.custom()
    .timeoutDuration(Duration.ofMillis(500))
    .cancelRunningFuture(true) // interrupt the thread on timeout
    .build();
TimeLimiter timeLimiter = TimeLimiter.of("catalog", tlConfig);

// Compose with other decorators (order matters – TimeLimiter
// outermost)
Supplier<CompletableFuture<Product>> supplier =
    TimeLimiter.decorateFutureSupplier(timeLimiter,
        () -> CompletableFuture.supplyAsync(() -> catalogClient.fetch(i
            d)));

Supplier<Product> retrying = Retry.decorateSupplier(retry,
    CircuitBreaker.decorateSupplier(circuitBreaker,
        () -> supplier.get().join()));
```

Anti-pattern: timeout longer than the caller's timeout. If `api` has a 1 s timeout on its inbound request but allows 2 s for its call to `catalog`, `api` will always return 504 via its own timeout — the downstream result, even if successful at 1.5 s, is discarded. Timeouts must nest: each downstream timeout strictly less than the remaining inbound deadline.

Deadline propagation

A deadline is an absolute timestamp ("respond by T") propagated through the call graph via headers or gRPC context. Each hop computes its remaining budget as `deadline - now()` and sets its downstream timeout accordingly. This prevents wasted work — if the caller has 40 ms left, there is no point starting a 200 ms downstream call.

gRPC deadlines are the canonical implementation — the `grpc-timeout` header is honored by every conforming implementation without application code:

```

// Server: respect the inbound deadline propagated via context
func (s *CatalogServer) GetProduct(ctx context.Context, req *pb.GetRequest) (*pb.Product, error) {
    // ctx already carries the deadline from grpc-timeout header
    if deadline, ok := ctx.Deadline(); ok {
        remaining := time.Until(deadline)
        if remaining < 50*time.Millisecond {
            return nil, status.Error(codes.DeadlineExceeded, "insufficient budget")
        }
    }
    // Pass the same context downstream – deadline propagates automatically
    inv, err := s.inventory.Get(ctx, &invpb.GetRequest{Sku: req.Sku})
    if err != nil {
        return nil, err
    }
    // ... build product response
    return product, nil
}

// Client: set a deadline on every outbound call
ctx, cancel := context.WithTimeout(parentCtx, 500*time.Millisecond)
defer cancel()
product, err := catalogClient.GetProduct(ctx, req)

```

For HTTP/REST, propagate via a header (commonly `X-Deadline` or `X-Timeout-Ms`) and enforce in middleware:

```

@Component
public class DeadlineFilter implements Filter {
    static final String DEADLINE_HEADER = "X-Deadline-Ms";
    @Override public void doFilter(ServletRequest req, ServletResponse
res, FilterChain chain)
        throws IOException, ServletException {
        HttpServletRequest http = (HttpServletRequest) req;
        String header = http.getHeader(DEADLINE_HEADER);
        long deadlineMs = header != null ? Long.parseLong(header)
            : System.currentTimeMillis() + 1000; // default budget
        long remaining = deadlineMs - System.currentTimeMillis();
        if (remaining <= 0) {
            ((HttpServletRequest) res).sendError(504, "deadline
already exceeded");
            return;
        }
        // Store remaining so downstream clients read it
        MDC.put("deadlineMs", String.valueOf(deadlineMs));
        chain.doFilter(req, res);
    }
}

```

Hedged requests

A hedged request sends the same request to multiple replicas and uses the first response, canceling the others. It trades extra load for reduced tail latency — valuable when p99 latency is dominated by occasional slow replicas (GC pauses, noisy neighbors) rather than systemic overload.

Hedging must be used sparingly: at 10% hedging, you add 10% load; at 100% hedging (send to two replicas every time), you double load — which under overload makes the problem worse. Adaptive hedging (hedge only when p50 latency indicates the first attempt is slow) is the safe default.


```
// Resilience4j-style hedged execution (simplified)
public <T> CompletableFuture<T> hedged(
    Supplier<CompletableFuture<T>> primary,
    Supplier<CompletableFuture<T>> hedge,
    Duration hedgeDelay,
    ScheduledExecutorService scheduler) {

    CompletableFuture<T> result = new CompletableFuture<>();
    CompletableFuture<T> p = primary.get();

    // Schedule the hedge only if primary hasn't completed within
    // hedgeDelay
    ScheduledFuture<?> hedgeTask = scheduler.schedule(() -> {
        if (!p.isDone()) {
            hedge.get().whenComplete((v, ex) -> {
                if (ex == null) result.complete(v);
            });
        }
    }, hedgeDelay.toMillis(), TimeUnit.MILLISECONDS);

    p.whenComplete((v, ex) -> {
        hedgeTask.cancel(false);
        if (ex == null) result.complete(v);
        else if (!result.isDone()) result.completeExceptionally(ex);
    });
    return result;
}
```

Use hedging only when: the downstream is replicated, requests are idempotent and cheap to cancel, and you hedge a small fraction or adaptively. Never hedge writes or non-idempotent operations.

Retries — healing without harming

When to retry

Retry only when the failure is likely transient and the request is safe to repeat:

Retryable	Not retryable
503, 429, 504, TCP <code>connect-failure</code> , <code>reset</code>	400, 401, 403, 404, 422 — caller error

Retryable	Not retryable
Idempotent GET/PUT/DELETE	Non-idempotent POST without idempotency key
Timeout where downstream effect is unknown — retry only with idempotency	Business-logic errors (insufficient balance, invalid state)

The Resilience4j and Istio configs below encode this explicitly — `retryOn` lists the retryable conditions rather than retrying on any failure.

Backoff with jitter

Without backoff, retries from many callers synchronize into a thundering herd at the instant the downstream recovers — immediately overloading it again. Exponential backoff with jitter decorrelates retries:

```
delay = min(base * 2^attempt + jitter, maxDelay)
jitter ∈ [0, base * 2^attempt) // full jitter (AWS best practice)
```

```

RetryConfig retryConfig = RetryConfig.<Product>custom()
    .maxAttempts(3)
    .waitDuration(Duration.ofMillis(100))
    .enableExponentialBackoff()
    .exponentialBackoffMultiplier(2.0)
    .enableRandomizedWait()           // full jitter
    .randomizedWaitFactor(0.5)       // jitter up to 50% of
    computed delay
    .retryOnResult(p -> p == null)    // retry on null (e.g., not
    found due to replication lag)
    .retryOnException(e -> e instanceof TimeoutException
        || e instanceof ConnectException
        || (e instanceof HttpStatusException he && he.g
    etStatusCode() >= 500))
    .retryExceptions(TimeoutException.class, ConnectException.class)
    .ignoreExceptions(BadRequestException.class, NotFoundException.clas
    s)
    .build();
Retry retry = Retry.of("catalog", retryConfig);

// Observe retry events – critical for dashboards
retry.getEventPublisher()
    .onRetry(e -> log.warn("retry catalog attempt={ } wait={ }ms
    lastError={ }",
        e.getNumberOfRetryAttempts(), e.getWaitInterval().toMillis(),
    e.getLastThrowable().toString()))
    .onError(e -> meterRegistry.counter("catalog.retry.exhausted").incr
    ement());

```

```

# Istio retry – note perTryTimeout prevents a slow retry from consuming
the whole budget
retries:
  attempts: 3
  perTryTimeout: 300ms
  retryOn: 5xx,reset,connect-failure,retriable-4xx
  retryRemoteLocalities: true # prefer different locality on retry

```

Retry budgets

Even with correct per-request retry logic, the aggregate retry rate can overwhelm a recovering downstream. A retry budget caps retries as a fraction of total request volume — e.g., allow retries up to 20% of successful requests. When the budget is exhausted, further failures are returned immediately without retrying.

Envoy and Finagle implement retry budgets natively. In application code, a token-bucket approximates it:

```
// Simple retry budget: allow retries up to ratio of successes
public class RetryBudget {
    private final AtomicLong success = new AtomicLong();
    private final AtomicLong retries = new AtomicLong();
    private final double maxRetryRatio; // e.g. 0.2

    public boolean tryAcquire() {
        long s = success.get();
        long r = retries.get();
        // Allow retry if retries / successes < ratio. Bootstrap: allow
        // first N retries.
        if (s < 10) { retries.incrementAndGet(); return true; }
        if ((double) r / s < maxRetryRatio) { retries.incrementAndGet()
; return true; }
        return false;
    }
    public void recordSuccess() { success.incrementAndGet(); }
}
```

Rule of thumb: set `maxAttempts` low (2–3 including the initial attempt), keep `perTryTimeout` short, and add a retry budget at the layer that has the broadest view of traffic (mesh or gateway rather than per-pod application code).

Circuit breakers

A circuit breaker prevents a caller from hammering a downstream that is clearly failing. It tracks recent failure rate and, when a threshold is breached, *opens* — failing fast without making the downstream call at all — for a cooldown period. After the cooldown, it allows a probe request (*half-open*); success closes the breaker, failure re-opens it.

The state machine

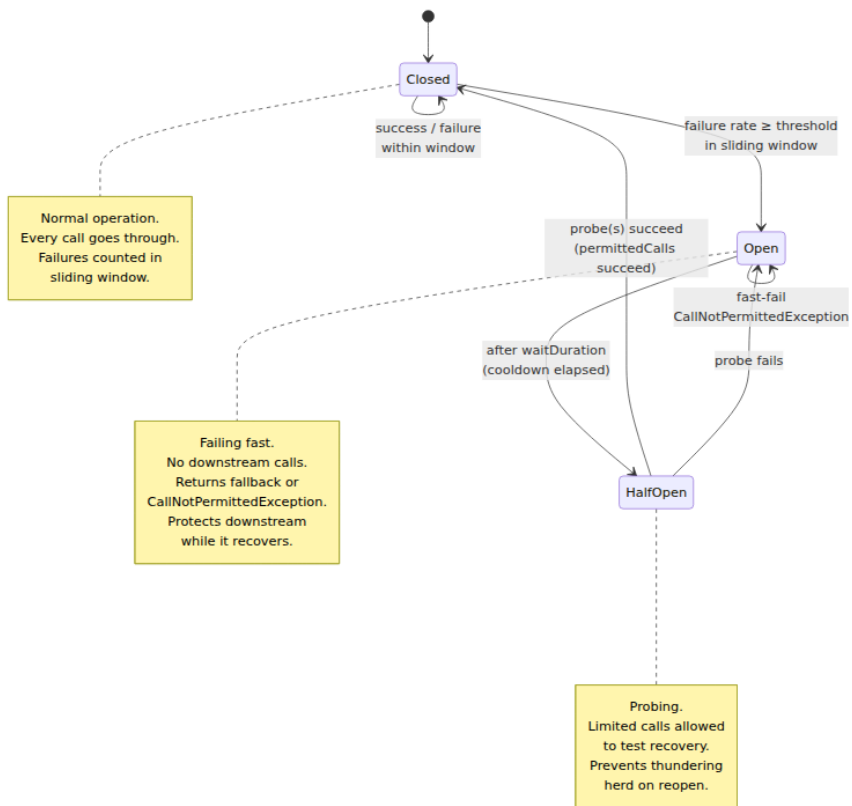


Figure 10-2: Circuit breaker state machine. The breaker opens when failure rate in the sliding window exceeds the threshold, fast-fails for a cooldown, then probes in half-open before closing.

Resilience4j circuit breaker

```
CircuitBreakerConfig cbConfig = CircuitBreakerConfig.custom()
    // Sliding window – count-based (last N calls) or time-based
    .slidingWindowType(CircuitBreakerConfig.SlidingWindowType.COUNT_BASED)
    .slidingWindowSize(100)                // evaluate last 100 calls
    .minimumNumberOfCalls(20)              // need 20 calls before
    evaluating (avoid flapping on low traffic)
    .failureRateThreshold(50.0f)           // open if ≥50% failures in
    window
    .slowCallRateThreshold(60.0f)          // also consider slow calls
    as failures
    .slowCallDurationThreshold(Duration.ofMillis(800))
    .waitDurationInOpenState(Duration.ofSeconds(30)) // cooldown
    before half-open
    .permittedNumberOfCallsInHalfOpenState(5) // probe with 5
    calls
    .automaticTransitionFromOpenToHalfOpenEnabled(true)
    .recordExceptions(TimeoutException.class, ConnectException.class,
        HttpStatusException.class)
    .ignoreExceptions(BadRequestException.class, NotFoundException.class)
    .build();

CircuitBreaker breaker = CircuitBreaker.of("catalog", cbConfig);

// Decorate – order: TimeLimiter outermost, then Retry, then
// CircuitBreaker, then Bulkhead innermost
Supplier<Product> guarded = Decorators.ofSupplier(() -> catalogClient.fetch(id))
    .withCircuitBreaker(breaker)
    .withBulkhead(bulkhead)
    .withFallback(throwable -> Product.cachedOrDefault(id))
    .decorate();

breaker.getEventPublisher()
    .onStateTransition(e -> log.warn("breaker {} {} -> {}",
        e.getCircuitBreakerName(), e.getStateTransition().getFromState()),
    ),
    e.getStateTransition().getToState()))
    .onCallNotPermitted(e -> meterRegistry.counter("breaker.catalog.not
        _permitted").increment());
```

Tuning guidance:

- `slidingWindowSize` 50–100 for high-traffic services, 10–20 for low-traffic (pair with `minimumNumberOfCalls` to avoid premature opening).

- `failureRateThreshold` 40–60% for most services. Lower thresholds open aggressively — good for critical-path protection, bad for noisy services that normally see some failures.
- `waitDurationInOpenState` 15–60 s. Too short and the downstream has not recovered; too long and you delay recovery detection.
- Always `ignoreExceptions` for caller errors (4xx) — they are not downstream failures.

Istio / Envoy outlier detection (mesh-level breaker)

In a service mesh the breaker is configured per destination and enforced by the sidecar — no application code needed. Envoy calls it *outlier detection*.

```

apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: catalog-circuit-breaker
spec:
  host: catalog.prod.svc.cluster.local
  trafficPolicy:
    # Connection pool / circuit breaker – Envoy enforced
    connectionPool:
      tcp: { maxConnections: 200 }
    # total TCP connections to upstream
    http:
      http1MaxPendingRequests: 100          # queue length before
      shedding (HTTP/1.1)
      http2MaxRequests: 200                # max concurrent requests
      (HTTP/2)
      maxRequestsPerConnection: 0          # 0 = unlimited
      maxRetries: 3
      outlierDetection:
        consecutive5xxErrors: 5
      # eject after 5 consecutive 5xx
        interval: 10s                      # scan interval
        baseEjectionTime: 30s              # how long to eject (×
      # never eject more than 50% of pods
      # ejection factor on consecutive)
        maxEjectionPercent: 50
      # if <50% healthy, stop
      # ejecting (avoid total blackout)
        splitExternalLocalOriginErrors: true # distinguish upstream 5xx
      # from gateway errors
      subsets:
        - name: v1
          labels: { version: v1 }
        - name: v2
          labels: { version: v2 }
      trafficPolicy:
        connectionPool:
          http: { http2MaxRequests: 50 }    # tighter limit for canary
subset

```

Key differences from Resilience4j:

- **Granularity:** Resilience4j breaks per caller instance; Envoy breaks per sidecar per upstream cluster — failure on one pod does not open the breaker on another pod (but outlier ejection removes that upstream pod from the pool for *all* sidecars).
- **Signal:** Resilience4j uses failure *rate*; Envoy outlier detection uses consecutive 5xx or gateway errors by default — configure

`consecutiveGatewayErrors` and `enforcingConsecutive5xx` to tune sensitivity.

- **Composition:** Use both. Mesh-level breakers protect infrastructure; application-level breakers protect with richer signals (latency, business errors) and execute fallbacks.

Breaker pitfalls

- **Breaker on every hop without fallback** just converts a downstream failure into an immediate failure — useful for shedding load but not for graceful degradation. Pair breakers with fallbacks where the product allows it.
- **Breaker per endpoint vs. per downstream.** A single breaker per downstream service conflates endpoints with different failure modes. Prefer per-operation breakers when one downstream serves both critical and non-critical paths.
- **Breaker flap** — threshold too low or window too small causes rapid open/close cycling. Fix with larger windows and `minimumNumberOfCalls`.

Bulkheads

A bulkhead isolates failure domains so that a slow or failing downstream cannot exhaust the resources needed to serve other requests. The name comes from ship compartments: one flooded compartment does not sink the ship.

Two isolation strategies dominate in backend services:



Figure 10-3: Without bulkheads a slow downstream exhausts the shared thread pool. Thread-pool bulkheads isolate but add thread-hopping overhead. Semaphore bulkheads (preferred) limit concurrency without extra threads and fail fast.

Semaphore bulkhead (preferred)

Modern services on non-blocking runtimes (Netty, virtual threads) should prefer semaphore bulkheads — they limit concurrency without dedicating threads, avoid context-switch overhead, and compose naturally with circuit breakers.

```
BulkheadConfig bhConfig = BulkheadConfig.custom()
    .maxConcurrentCalls(30)           // max concurrent calls
    through this bulkhead
    .maxWaitDuration(Duration.ofMillis(50)) // how long to wait for a
    permit before failing
    .build();
Bulkhead bulkhead = Bulkhead.of("catalog", bhConfig);

// ThreadPoolBulkhead variant – only when caller threads must not block
ThreadPoolBulkheadConfig tpConfig = ThreadPoolBulkheadConfig.custom()
    .maxThreadPoolSize(30)
    .coreThreadPoolSize(20)
    .queueCapacity(50)
    .keepAliveDuration(Duration.ofSeconds(30))
    .build();
ThreadPoolBulkhead tpBulkhead = ThreadPoolBulkhead.of("catalog-tp", tpC
onfig);

// Usage – semaphore bulkhead decorates the call directly
Supplier<Product> isolated = Bulkhead.decorateSupplier(bulkhead,
    CircuitBreaker.decorateSupplier(breaker,
        () -> catalogClient.fetch(id)));
```

For Kubernetes, bulkheads also exist at the infrastructure layer — `ResourceQuota` and `LimitRange` per namespace, and per-tenant node pools — covered in Volume 12, Chapter 7.

Choosing bulkhead boundaries

Bulkheads should follow failure domains, not just downstream names:

- **Per downstream** when downstreams fail independently — `inventory` slowness should not block `pricing`.
- **Per criticality** when one downstream serves both critical and best-effort paths — isolate checkout's call to `catalog` from the recommendation engine's call to the same service.
- **Per tenant** in multi-tenant systems — one tenant's burst should not starve others (see Volume 12, Chapter 7).

Load shedding and concurrency limiting

When every caller is well-behaved but aggregate load exceeds capacity (flash sale, thundering herd after recovery), the server must protect itself by *shedding* excess work — returning 503 or 429 immediately rather than queueing until it collapses.

Concurrency limiting (Netflix model)

The most effective server-side defense is limiting *inflight* requests rather than queueing them. Netflix's `concurrency-limits-java` implements the Vegas and Gradient algorithms that adapt the limit based on observed latency — tightening under pressure, relaxing when healthy.

```

// Gradient limiter – adapts limit based on RTT gradient
Limit limit = GradientLimit.newBuilder()
    .initialLimit(100)
    .minLimit(20)
    .maxLimit(500)
    .rttTolerance(1.5)           // allow 1.5× baseline RTT before
tightening
    .build();

// Vegas limiter – alternative, tracks queuing delay
Limit vegas = VegasLimit.newBuilder()
    .initialLimit(100)
    .maxConcurrency(500)
    .build();

// gRPC server interceptor that enforces the limit
public class ConcurrencyLimitInterceptor implements ServerInterceptor {
    private final Limiter<ServerRequest> limiter;
    @Override public <ReqT, RespT> ServerCall.Listener<ReqT> interceptC
all(
        ServerCall<ReqT, RespT> call, Metadata headers, ServerCallH
andler<ReqT, RespT> next) {
        Optional<Limiter.Listener> listener = limiter.acquire(call);
        if (listener.isEmpty()) {
            call.close(Status.RESOURCE_EXHAUSTED
                .withDescription("concurrency limit exceeded")
                .augmentDescription("retry-after: 50ms"), new Metadata(
));
            return new ServerCall.Listener<>() {};
        }
        ServerCall<ReqT, RespT> wrapped = new ForwardingServerCall.Simp
leForwardingServerCall<>(call) {
            @Override public void close(Status status, Metadata trailer
s) {
                listener.get().onSuccess();
                super.close(status, trailers);
            }
        };
        return next.startCall(wrapped, headers);
    }
}

```

Envoy enforces concurrency limiting at the proxy without application code:

```
# Envoy circuit breaker as concurrency limiter (per upstream cluster)
circuit_breakers:
  thresholds:
    - priority: DEFAULT
      max_connections: 1000
      max_pending_requests: 500
      max_requests: 500          # hard cap on inflight requests to
upstream
      max_retries: 3
      track_remaining: true
      retry_budget:
        budget_percent: 20      # retries capped at 20% of active
requests
      min_retry_concurrency: 5
```

Shedding strategy

Layer	Mechanism	Signal	Response
Edge / gateway	Global rate limiting (Redis/ Envoy RLS)	Requests/sec per tenant or global	429 + Retry-After
Service	Concurrency limiter (Vegas/ Gradient)	Inflight count + RTT gradient	503 + Retry-After (or 429 if caller is tenant-scoped)
Downstream	Circuit breaker + bulkhead	Failure rate, slow-call rate	Fast-fail + fallback

Shedding at the service is more accurate than at the edge because the service knows its actual capacity; shedding at the edge is broader and protects the entire fleet. Use both in layers — edge for coarse global protection, service for precise local protection.

Always include `Retry-After` on 429/503 so well-behaved clients back off rather than retrying immediately.

Fallbacks and graceful degradation

A fallback is what the caller does when the downstream is unavailable. Not every failure deserves a fallback — failing loudly is sometimes correct — but for read-heavy, user-facing paths a degraded response is far better than an error page.

Fallback catalog

Fallback	Freshness	Use when	Example
Cached	Stale but recent	Reads with tolerance for staleness	Return last-known product catalog from Redis/local cache
Default / static	Always available	Degradation is acceptable	Return generic recommendations instead of personalized
Partial	Incomplete	Some fields are optional	Return product without reviews when review service is down
Empty / no-op	N/A	Best-effort side effects	Drop analytics event, enqueue for later
Fail	N/A	Correctness requires the downstream	Payment authorization — never fallback, fail the checkout

```

// Resilience4j fallback – ordered by preference
Supplier<ProductView> withFallback = Decorators.ofSupplier(() -> catalogClient.fetchView(id))
    .withCircuitBreaker(breaker)
    .withBulkhead(bulkhead)
    .withFallback(List.of(
        // 1. Try stale cache
        throwable -> ProductView.fromCache(id),
        // 2. Try degraded (partial) view
        throwable -> ProductView.degraded(id)))
    .decorate();

// Distinguish fallback invocation in metrics – degraded is not success
breaker.getEventPublisher().onCallNotPermitted(e ->
    meterRegistry.counter("catalog.fallback.cache", "reason", "breaker_
open").increment());

// HTTP layer – return 200 with degraded signal vs 503
@GetMapping("/products/{id}")
public ResponseEntity<ProductView> get(@PathVariable String id) {
    try {
        ProductView view = withFallback.get();
        if (view.isDegraded()) {
            return ResponseEntity.ok()
                .header("X-Degraded", "true")
                .header("Cache-Control", "no-cache")
                .body(view);
        }
        return ResponseEntity.ok(view);
    } catch (CallNotPermittedException e) {
        // No fallback available – fail fast with Retry-After
        return ResponseEntity.status(503)
            .header("Retry-After", "5")
            .build();
    }
}

```

Rules for safe fallbacks:

- Never fallback on writes — retry with idempotency or fail; a fallback that pretends a write succeeded causes data loss or double-charge.
- Cache fallbacks need a bounded staleness policy — `Cache-Control: stale-if-error` with explicit `stale-while-revalidate` semantics rather than unbounded stale.
- Degraded responses must be marked (`X-Degraded: true`, or a field in the envelope) so the caller and observability know the response is degraded — otherwise degraded looks like success in dashboards.

Composing patterns — order and interaction

Patterns compose, but order matters. The decorator stack from outermost to innermost should be:

```
TimeLimiter → Retry → CircuitBreaker → Bulkhead → (actual call) → Fallback
```

Rationale:

- **TimeLimiter outermost** so the total time including retries is bounded.
- **Retry outside breaker** so retries are counted as breaker failures (if every retry fails, the breaker should see one logical failure, not N). Some teams place breaker outside retry so each retry attempt is individually counted — choose one and be consistent. Resilience4j recommends breaker inside retry by default.
- **Bulkhead innermost** so permits are held for the shortest time — only during the actual downstream call, not during backoff sleeps.

```
// Canonical composition – Resilience4j Decorators helper enforces correct order
```

```
Supplier<Product> resilient = Decorators.ofSupplier(() -> catalogClient.fetch(id))  
    .withTimeLimiter(timeLimiter, executor) // outermost  
    .withRetry(retry)  
    .withCircuitBreaker(breaker)  
    .withBulkhead(bulkhead) // innermost  
    .withFallback(e -> Product.cachedOrDefault(id))  
    .decorate();
```

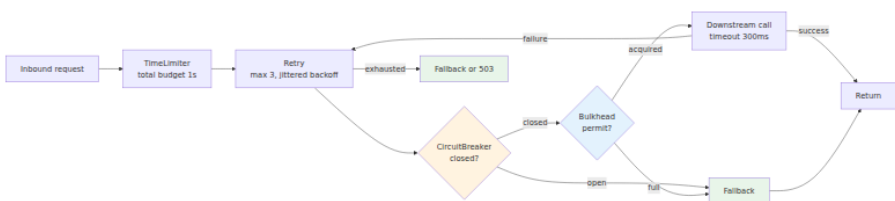


Figure 10-4: Decorator ordering for composing resilience patterns. *TimeLimiter* bounds total time, *Retry* heals transient faults, *CircuitBreaker* fast-fails when the downstream is clearly unhealthy,

Bulkhead isolates concurrency, and Fallback provides degraded responses.

Timeout is the foundation — without it, bulkhead permits are held indefinitely, breakers never see failures promptly, and retries wait forever. Verify every layer has a timeout before adding any other pattern.

Observability — proving resilience

Resilience patterns that are not observed are indistinguishable from absent.

Metrics

Metric	Breaker	Bulkhead
State	<code>breaker_state{state=open}</code> gauge	<code>bulkhead_available_permits</code>
Rate	<code>breaker_failures_total</code> , <code>breaker_not_permitted_total</code>	<code>bulkhead_rejected_total</code>
Latency	<code>breaker_slow_calls_total</code>	<code>bulkhead_wait_duration_seconds</code>
Fallback	<code>fallback_invocations_total{reason}</code>	—

Expose breaker state as a gauge (0=closed, 1=open, 2=half-open) so dashboards show at a glance which downstreams are currently broken. Alert on state transitions, not just on open — flapping is as important as sustained open.

Dashboard and alerts

```
# Prometheus alerting – breaker open and bulkhead shedding are warning,
# not page,
# unless the fallback is also failing or degraded rate is high.
groups:
- name: resilience
  rules:
  - alert: CircuitBreakerOpen
    expr: resilience_circuitbreaker_state{state="open"} == 1
    for: 2m
    labels: { severity: warning }
    annotations:
      summary: "Breaker {{ $labels.name }} open for >2m"
      runbook: "https://runbooks.prod/breaker-open"

  - alert: CircuitBreakerFlapping
    expr: increase(resilience_circuitbreaker_state_transitions_total[10m]) > 6
    labels: { severity: warning }
    annotations:
      summary: "Breaker {{ $labels.name }} flapping – threshold or
window misconfigured?"

  - alert: BulkheadShedding
    expr: rate(resilience_bulkhead_rejected_total[5m]) > 10
    labels: { severity: warning }
    annotations:
      summary: "Bulkhead {{ $labels.name }} rejecting >10/s – downstream
slow or limit too tight"

  - alert: FallbackFailing
    expr: rate(fallback_invocations_total{result="error"}[5m]) > 5
    labels: { severity: critical }
    annotations:
      summary: "Fallback for {{ $labels.name }} failing – degraded
path is broken"

  - alert: RetryExhaustedSpike
    expr: rate(resilience_retry_exhausted_total[5m]) > 20
    labels: { severity: warning }
    annotations:
      summary: "Retries exhausted for {{ $labels.name }} – downstream
am not recovering with retries"
```

Chaos validation

Every resilience pattern should have a corresponding fault-injection test that proves it activates correctly. Use Chaos Mesh, Litmus, or Toxiproxy in staging — and controlled fault injection in production (see Chapter 8).

```
# Chaos Mesh – inject 2s latency into inventory to trigger catalog's
breaker and bulkhead
apiVersion: chaos-mesh.org/v1alpha1
kind: NetworkChaos
metadata: { name: inventory-latency }
spec:
  action: delay
  direction: to
  selector:
    labelSelectors: { app: inventory }
  delay:
    latency: 2000ms
    jitter: 500ms
    duration: 5m
---
# Verify: catalog breaker should open within 30s, bulkhead rejections
rise,
# fallback served, catalog's other endpoints (profile, health) remain
healthy,
# api p95 degrades gracefully rather than spiking.
```

Test matrix for resilience patterns:

Fault	Expected activation	Verify
Downstream 2 s latency	Timeout → retry → breaker open → fallback	Breaker state, fallback rate, p95 bounded
Downstream 100% 500	Breaker open within window, bulkhead not saturated	Breaker transitions, no thread exhaustion
Downstream 0% failure but slow ramp	Bulkhead permits exhaust, concurrency limiter sheds	<code>bulkhead_rejected_total</code> rises, 503 with <code>Retry-After</code>

Fault	Expected activation	Verify
Caller burst 10×	Concurrency limiter sheds, edge rate limiter 429s	429/503 rate, no OOM, recovery within seconds after burst

Putting it together — a resilient service template

```

# Kubernetes Deployment with resilience-relevant tuning
apiVersion: apps/v1
kind: Deployment
metadata: { name: catalog }
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: catalog
          image: catalog:1.42.0
          ports: [{ containerPort: 8080 }]
          env:
            - name: CATALOG_INVENTORY_TIMEOUT_MS
              value: "300"
            - name: CATALOG_INVENTORY_BREAKER_FAILURE_RATE
              value: "50"
            - name: CATALOG_BULKHEAD_MAX_CONCURRENT
              value: "30"
          resources:
            requests: { cpu: "500m", memory: "512Mi" }
            limits: { cpu: "1000m", memory: "1Gi" }
          readinessProbe:
            httpGet: { path: /health/ready, port: 8080 }
            periodSeconds: 5
            failureThreshold: 2
          livenessProbe:
            httpGet: { path: /health/live, port: 8080 }
            periodSeconds: 10
          # Graceful shutdown – finish inflight, respect bulkhead
          permits

      lifecycle:
        preStop:
          exec: { command: ["/bin/sh", "-c", "sleep 15"] }
    ---
apiVersion: v1
kind: Service
metadata: { name: catalog }
spec:
  selector: { app: catalog }
  ports: [{ port: 80, targetPort: 8080 }]
  ---
# Istio policies – timeout, retry, breaker, outlier detection
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata: { name: catalog }
spec:
  hosts: [catalog.prod.svc.cluster.local]
  http:
    - timeout: 500ms

```

```

    retries:
      attempts: 2
      perTryTimeout: 250ms
      retryOn: 5xx,gateway-error,connect-failure,retriable-4xx
    route: [{ destination: { host: catalog.prod.svc.cluster.local } } ]
  ---
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata: { name: catalog }
spec:
  host: catalog.prod.svc.cluster.local
  trafficPolicy:
    connectionPool:
      tcp: { maxConnections: 200 }
      http: { http2MaxRequests: 200, maxRetries: 3 }
    outlierDetection:
      consecutive5xxErrors: 5
      interval: 10s
      baseEjectionTime: 30s
      maxEjectionPercent: 50

```

Application wiring (Spring Boot + Resilience4j) in `application.yml`:

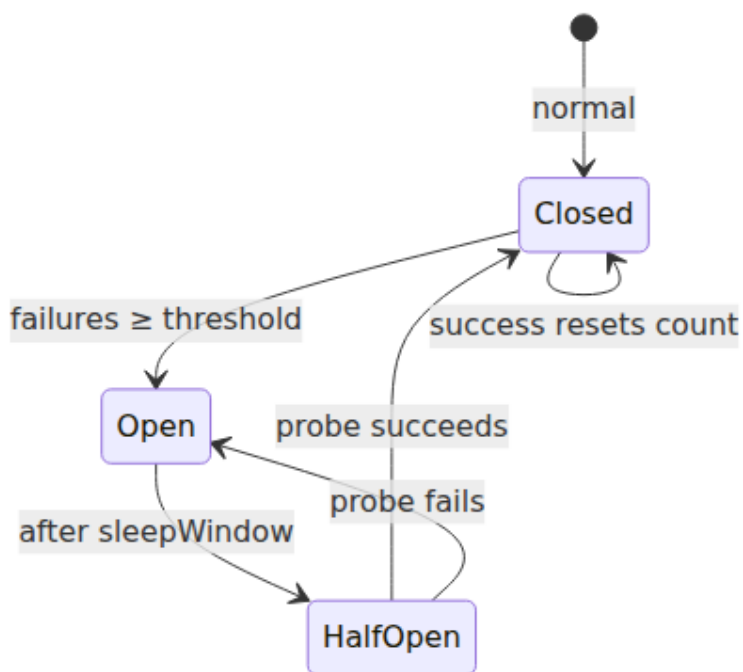
```

resilience4j:
  timelimiter:
    configs:
      default: { timeout-duration: 500ms, cancel-running-future: true }
  retry:
    configs:
      default:
        max-attempts: 3
        wait-duration: 100ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2.0
        enable-randomized-wait: true
        randomized-wait-factor: 0.5
        retry-exceptions: [java.util.concurrent.TimeoutException, java.
net.ConnectException]
        ignore-exceptions: [com.example.BadRequestException]
  circuitbreaker:
    configs:
      default:
        sliding-window-type: COUNT_BASED
        sliding-window-size: 100
        minimum-number-of-calls: 20
        failure-rate-threshold: 50
        slow-call-rate-threshold: 60
        slow-call-duration-threshold: 800ms
        wait-duration-in-open-state: 30s
        permitted-number-of-calls-in-half-open-state: 5
        automatic-transition-from-open-to-half-open-enabled: true
  bulkhead:
    configs:
      default: { max-concurrent-calls: 30, max-wait-duration: 50ms }
  bulkhead-instances:
    inventory: { max-concurrent-calls: 20, max-wait-duration: 20ms }
# tighter for known-slow downstream

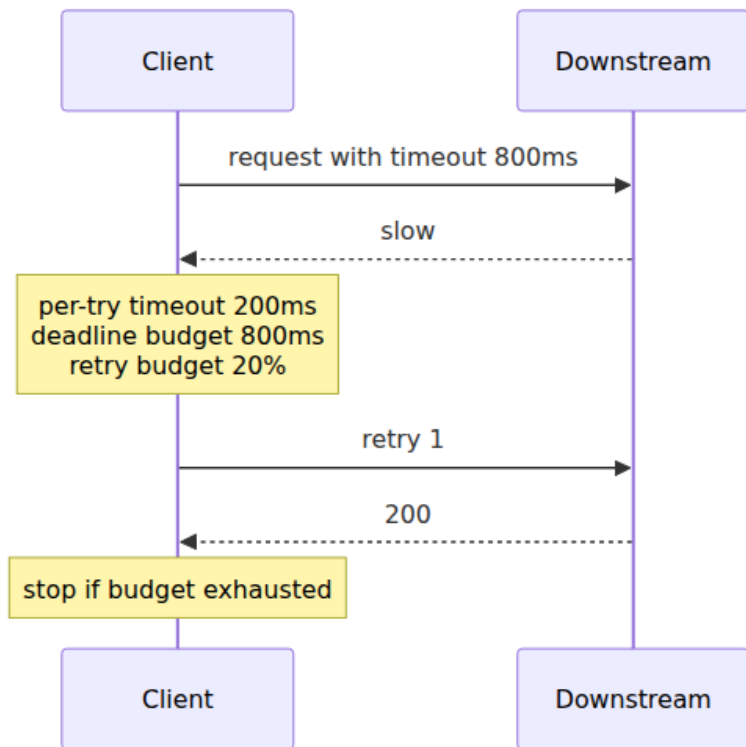
management:
  endpoints.web.exposure.include: health,prometheus,circuitbreakers,bul
kheads,retries
  health.circuitbreakers.enabled: true

```

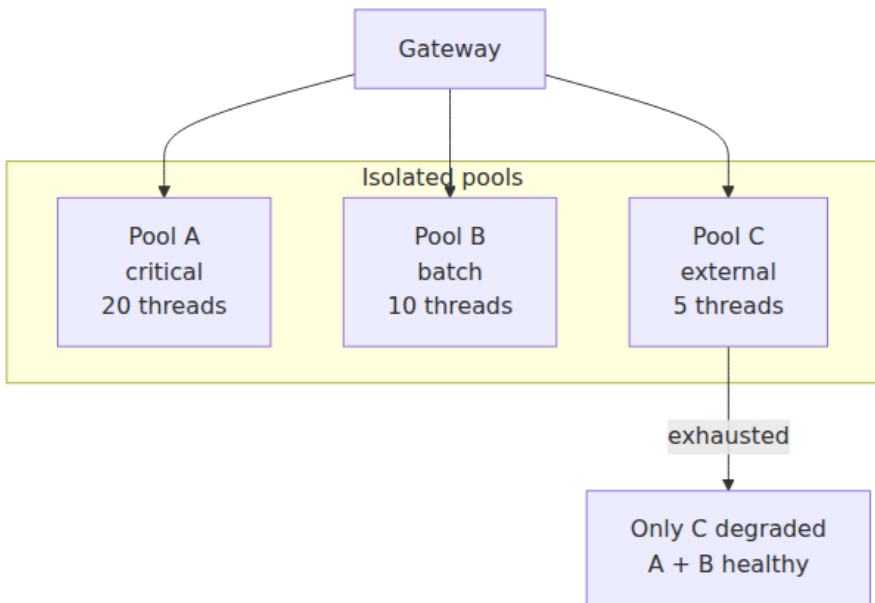
Circuit Breaker States



Timeout and Retry Budget



Bulkhead Isolation



Key takeaways

- Every outbound call needs an explicit timeout strictly less than the remaining inbound deadline — without this, no other resilience pattern can function. Propagate deadlines via gRPC context or HTTP headers so downstream work that cannot be returned in time is never started.
- Retries heal transient faults but amplify overload — retry only idempotent operations on retryable errors (503/429/504/connect-failure), use exponential backoff with full jitter, keep `maxAttempts` at 2-3, and enforce a retry budget ($\leq 20\%$ of success volume) at the broadest layer.
- Circuit breakers fast-fail when a downstream is clearly unhealthy, giving it time to recover and callers a chance to fallback. Tune `slidingWindowSize` (50-100), `failureRateThreshold` (40-60%), `waitDurationInOpenState` (15-60 s), and always ignore caller errors (4xx). Use both application-level (Resilience4j) and mesh-level (Istio/Envoy outlier detection) breakers — they protect at different granularities.

- Bulkheads isolate failure domains — a slow `inventory` should not exhaust the threads needed for `profile` or `health`. Prefer semaphore bulkheads over thread-pool bulkheads on modern non-blocking runtimes; size limits per downstream and per criticality, not just per service.
- Concurrency limiting with adaptive algorithms (Vegas/Gradient) is the most effective server-side defense against aggregate overload — it sheds excess work with `503/429 + Retry-After` instead of queueing until collapse. Layer shedding at edge (global) and service (local) for defense in depth.
- Fallbacks provide degraded but useful responses — cached, stale, default, or partial — but never fallback on writes. Mark degraded responses explicitly (`X-Degraded: true`) so observability distinguishes degraded from success.
- Composition order matters: `TimeLimiter` → `Retry` → `CircuitBreaker` → `Bulkhead` → `call` → `Fallback`. Validate every pattern with fault injection — inject latency and errors in staging and (carefully) in production, and verify the expected breaker/bulkhead/fallback activation and bounded p95.
- Observe resilience: export breaker state as a gauge, alert on transitions and flapping, dashboard fallback rates, and treat fallback failure as critical — a broken fallback turns degradation into outage.

Further reading

- Nygard, M. — *Release It!* (2nd ed., Pragmatic Bookshelf, 2018). The original catalog of stability patterns — circuit breaker, bulkhead, timeout, and their production consequences.
- Netflix — `concurrency-limits-java` (GitHub: `Netflix/concurrency-limits-java`) — Vegas and Gradient limiter implementations and the theory behind adaptive concurrency limiting.
- Resilience4j documentation — <https://resilience4j.readme.io> — configuration reference for `CircuitBreaker`, `Retry`, `Bulkhead`, `TimeLimiter`, and decorator composition.
- Istio — *Circuit Breaking* and *Outlier Detection* — <https://istio.io/latest/docs/tasks/traffic-management/circuit-breaking/> and `DestinationRule` reference.

- Envoy — *Circuit Breakers and Retry Budgets* — https://www.envoyproxy.io/docs/envoy/latest/configuration/upstream/circuit_breakers and https://www.envoyproxy.io/docs/envoy/latest/configuration/http/http_filters/router_filter#retry-budget
- gRPC — *Deadlines* — <https://grpc.io/docs/guides/deadlines/> — propagation semantics and language-specific handling.
- AWS Builders' Library — *Timeouts, retries and backoff with jitter* — <https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/>
- Google SRE Book — Chapter 22, *Addressing Cascading Failures* — <https://sre.google/sre-book/addressing-cascading-failures/>